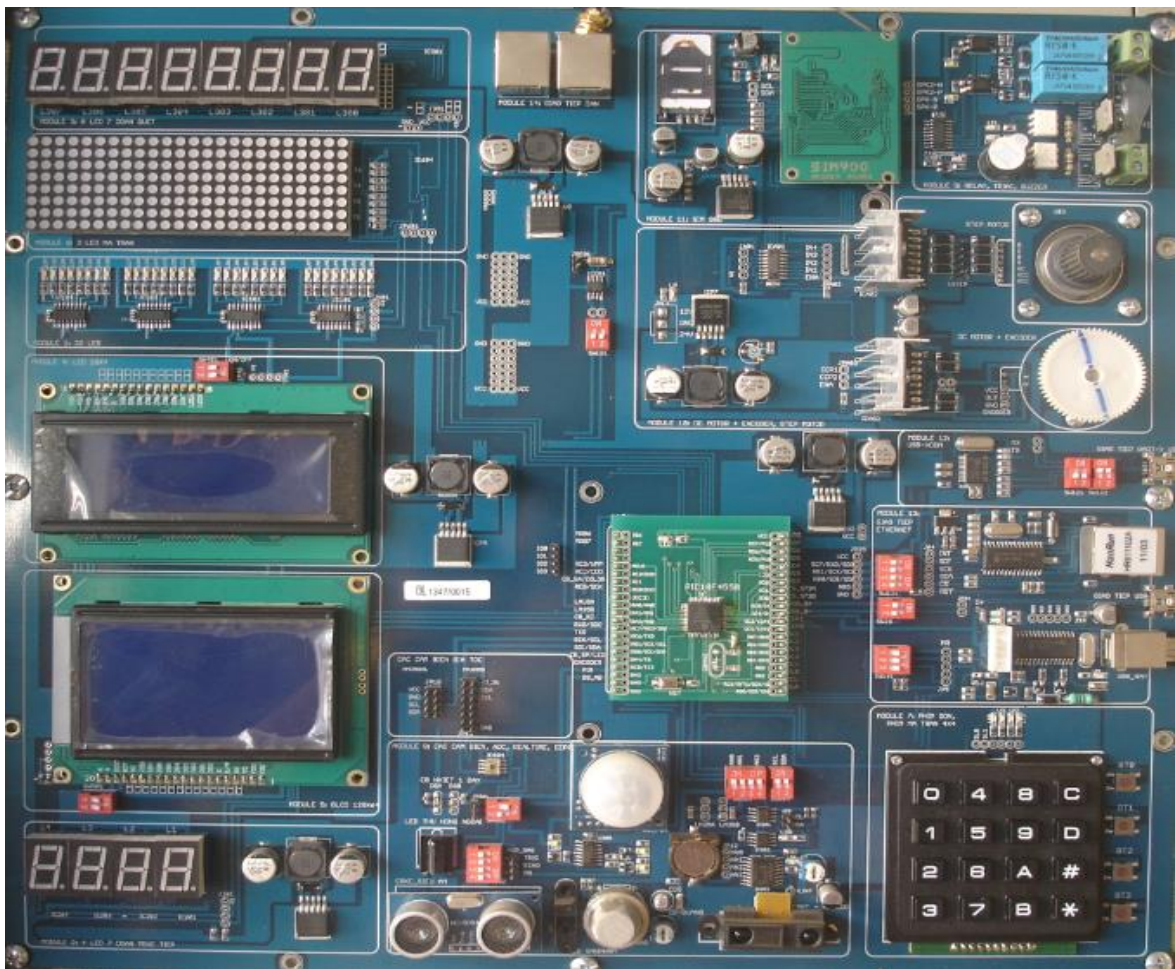


ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ – CÔNG NGHIỆP



GIÁO TRÌNH THỰC HÀNH **VI ĐIỀU KHIỂN PIC**



NGUYỄN ĐÌNH PHÚ
PHAN VÂN HOÀN
TRƯƠNG NGỌC ANH
(Lớp Thầy Hoàn)

TP HCM, 1/2023

LỜI NÓI ĐẦU

Thực hành Vi xử lý hoặc vi điều khiển là môn học cơ sở cho tất cả các ngành Kỹ Thuật Điện – Điện Tử, Điện Tử - Truyền Thông, Tự Động Điều Khiển, Kỹ Thuật Máy Tính và Cơ Điện Tử, sau khi đã học xong lý thuyết với mục đích tiếp cận thực tế lập trình điều khiển và ứng dụng của vi điều khiển từ cơ bản đến nâng cao nhằm ôn tập các kiến thức đã học, rèn luyện các kỹ năng lập trình, kỹ năng gỡ rối, kiểm tra, quan sát, phân tích, suy luận, đánh giá.

Các bài thực hành được biên soạn theo kit thực hành vi điều khiển PIC 18F4550 với cấu hình phần cứng giao tiếp với nhiều ngoại vi như led đơn dùng thanh ghi dịch, led 7 dùng thanh ghi dịch, led 7 đoạn quét, phím đơn, phím ma trận, LCD, GLCD, led ma trận 2 màu xanh đỏ, động cơ bước và động cơ DC, cùng nhiều loại cảm biến khác nhau như cảm biến nhiệt độ LM35, cảm biến nhiệt 1 dây, cảm biến khoảng cách dùng siêu âm, dùng hồng ngoại, cảm biến chuyển động, ...

Các bài thực hành được biên soạn từ dễ đến khó, từ đơn giản đến phức tạp về phần cứng lẫn phần mềm. Qua từng chương thì mức độ khó về phần cứng và phần mềm sẽ tăng dần do tính chất kết hợp. Mỗi chương có rất nhiều bài thực hành và bài tập, mục đích của từng bài rõ ràng, cùng một vấn đề nhưng được viết ở nhiều dạng khác nhau nhằm giúp người học có thể hiểu dễ dàng và nhanh chóng.

Các phần cứng có liên quan đều được trình bày lại chi tiết, rõ ràng, có lưu đồ giải thuật, có tính toán và giải thích rõ ràng.

Tài liệu được biên soạn thành 10 chương:

Chương 1: Cấu hình kit thực hành vi điều khiển.

Chương 2: Sử dụng phần mềm CCS và PICKIT.

Chương 3: Thực hành module 1: 32 led đơn dùng thanh ghi dịch 74HC595, nút nhấn, bàn phím ma trận.

Chương 4: Thực hành module 2: 4 led 7 đoạn anode chung dùng thanh ghi dịch 74HC595, timer/counter.

Chương 5: Thực hành module 3: 8 led 7 đoạn anode chung kết nối theo phương pháp quét dùng thanh ghi dịch MBI5026, timer/counter.

Chương 6: Thực hành module 4: LCD 20X4, GLCD 128X64 dùng thanh ghi dịch MBI5026.

Chương 7: Thực hành chuyển đổi tương tự sang số ADC, các cảm biến.

Chương 8: thực hành module 5: Real time DS13B07, ADC-DAC PCF 8591, Eeprom nối tiếp AT24C256 theo chuẩn I2C và các cảm biến.

Chương 9: Thực hành module 6: động cơ bước, động cơ DC và encoder, PWM, điều khiển PID.

Chương 10: Thực hành module 7: điều khiển led ma trận 2 màu.

Nội dung chương 1 trình bày chi tiết về cấu hình kết nối phần cứng của kit thực hành vi điều khiển PIC 18F4550. Do kit được thiết kế theo tiêu chí không còn dùng dây bus để kết nối giữa port của vi điều khiển với các đối tượng điều khiển nên có dùng thêm nhiều IC mở rộng port như IC chốt 74HC573, thanh ghi dịch nối tiếp sang song song 8 bit 74HC595 và 16 bit MBI5026, người học cần phải hiểu rõ vai trò, chức năng của các IC này. Mỗi module đều có trình bày sơ đồ kết nối phần cứng và có giải thích. Người học cần phải hiểu rõ kết nối phần cứng mới khai thác hết được các tính năng độc lập điều khiển đơn giản cũng như kết hợp nhiều module để điều khiển cho một chức năng lớn. Đây là chương quan trọng trình bày hết mọi nguyên lý kết nối phần cứng, phải hiểu rõ chúng thì mới hiểu được các thư viện và chương trình.

Nội dung chương 2 trình bày chi tiết về cách sử dụng phần mềm để lập trình các chương trình cho vi điều khiển PIC, cách biên dịch, cách tìm lỗi và phần mềm để nạp code file HEX vào bộ nhớ của vi điều khiển PIC.

Nội dung chương 3 thực hành điều khiển module 32 led đơn, nút nhấn, bàn phím ma trận, nội dung trình bày rất chi tiết, từng bước, như giới thiệu các hàm đã viết trong thư viện và giải thích chức năng các hàm, các biến và các tên đã định nghĩa, ... có rất nhiều hàm đã viết sẵn nhằm giúp bạn thực hiện các ứng dụng điều khiển một cách nhanh chóng. Sau đó bắt đầu viết các chương trình mẫu điều khiển 32 led từ đơn giản đến nâng cao, có nhiều cách viết và có nhiều bài tập để luyện tập dựa vào các bài mẫu. Người học cần hiểu các bài mẫu để làm các bài tập vì đó là các bài mở rộng nhằm ứng dụng các kiến thức đã học. Tiếp theo là các chương trình giao tiếp với nút nhấn đơn, bài đơn giản, bài cho thấy hiện tượng dội của phím nhấn và cách chống dội. Cuối cùng là giao tiếp với bàn phím ma trận.

Nội dung chương 4 thực hành điều khiển module 4 led 7 đoạn dùng 4 IC thanh ghi dịch mở rộng. Nội dung được trình bày rất chi tiết, từng bước như giới thiệu các hàm đã viết trong thư viện và giải thích chức năng các hàm, các biến và các tên đã định nghĩa liên quan đến module 4 led 7 đoạn, ... có rất nhiều hàm đã viết sẵn nhằm giúp bạn thực hiện các ứng dụng điều khiển một cách nhanh chóng.

Sau đó bắt đầu viết các chương trình mẫu điều khiển 4 led 7 đoạn từ đơn giản đến nâng cao. Kết hợp với các bộ định thời timer và counter và nguồn phát hiện xung khi có sản phẩm đi qua để thực hành các chương trình đếm xung ngoại. Kết hợp với module 32 led đơn, nút nhấn và bàn phím ma trận để thực hiện yêu cầu điều khiển phức tạp hơn.

Nội dung chương 5 thực hành điều khiển module 8 led 7 đoạn kết nối theo phương pháp quét, dùng IC thanh ghi dịch mở rộng. Nội dung trình bày rất chi tiết, từng bước như giới thiệu các hàm đã viết trong thư viện và giải thích chức năng các hàm, các biến và các tên đã định nghĩa liên quan đến module 8 led 7 đoạn, ... Có rất nhiều hàm đã viết sẵn nhằm giúp bạn thực hiện các ứng dụng điều khiển một cách nhanh chóng.

Sau đó bắt đầu viết các chương trình mẫu điều khiển 8 led 7 đoạn từ đơn giản đến nâng cao, từ 2 led mở rộng dần đến 8 led. Kết hợp với các bộ định thời timer và counter để đếm thời gian làm động hồ hiển thị giờ phút giây, kết hợp với nút nhấn để chỉnh thời gian. Kết hợp với module 32 led đơn, 4 led 7 đoạn, nút nhấn và bàn phím ma trận để thực hiện yêu cầu điều khiển phức tạp hơn.

Nội dung chương 6 thực hành điều khiển module LCD 20x4, GLCD 128x64 dùng IC thanh ghi dịch mở rộng. Nội dung trình bày đầy đủ phần lý thuyết LCD, GLCD, tập lệnh của LCD, GLCD. Nguyên lý khởi tạo và hiển thị text, hiển thị hình ảnh. Từng bước giới thiệu các hàm đã viết trong thư viện và giải thích chức năng các hàm, các biến và các tên đã định nghĩa liên quan đến module LCD và GLCD. Có rất nhiều hàm đã viết sẵn nhằm giúp bạn thực hiện các ứng dụng điều khiển một cách nhanh chóng.

Tiếp theo phần viết các chương trình mẫu điều khiển hiển thị trên LCD, GLCD từ đơn giản đến nâng cao. Kết hợp với module 32 led đơn, 4 led 7 đoạn, nút nhấn và bàn phím ma trận để thực hiện yêu cầu điều khiển phức tạp hơn.

Nội dung chương 7 thực hành phần chuyển đổi ADC tích hợp trong vi điều khiển PIC 18F4550. Trình bày đầy đủ nguyên lý hoạt động của ADC, các lệnh liên quan đến ADC. Trình bày lý thuyết cảm biến LM35, mạch giao tiếp các cảm biến với vi điều khiển, cách tính toán độ phân giải.

Tiếp theo là phần viết các chương trình mẫu đọc và chuyển đổi nhiệt độ hiển thị trên led 7 đoạn, trên LCD, trên GLCD từ đơn giản đến nâng cao.

Thực hiện tương tự cho cảm biến đo khoảng cách dùng sóng hồng ngoại và cảm biến nhiệt một dây và cảm biến đo khoảng cách dùng sóng siêu âm.

Kết hợp với module 32 led đơn, 4 led 7 đoạn, nút nhấn và bàn phím ma trận để thực hiện yêu cầu điều khiển phức tạp hơn, tổng hợp nhiều chương trình điều khiển với nhiều chức năng.

Nội dung chương 8 thực hành phân giao tiếp vi điều khiển PIC 18F4550 với các thiết bị ngoại vi qua chuẩn I2C. Trình bày đầy đủ nguyên lý hoạt động của các thiết bị ngoại vi như IC thời gian thực DS13B07, IC ADC-DAC PCF8591, IC nhớ nối tiếp AT24C256.

Tiếp theo là phần viết các chương trình mẫu đọc khởi tạo, cài đặt thời gian các và chuyển đổi nhiệt độ hiển thị trên led 7 đoạn, trên LCD, trên GLCD từ đơn giản đến nâng cao.

Kết hợp với module 32 led đơn, 4 led 7 đoạn, nút nhấn và bàn phím ma trận để thực hiện yêu cầu điều khiển phức tạp hơn, tổng hợp nhiều chương trình điều khiển với nhiều chức năng.

Nội dung chương 9 thực hành phân giao tiếp vi điều khiển PIC 18F4550 với động cơ bước và động cơ DC thông qua IC giao tiếp công suất L298. Trình bày đầy đủ nguyên lý hoạt động của động cơ bước, động cơ DC, sơ đồ mạch giao tiếp, nguyên lý điều khiển cơ bản và nguyên lý điều khiển điều chế xung PWM.

Tiếp theo là phần viết các chương trình mẫu điều khiển động cơ từ cơ bản đến nâng cao, điều khiển ổn định tốc độ theo giải thuật PID.

Kết hợp với các module khác để thực hiện yêu cầu điều khiển phức tạp hơn, tổng hợp nhiều chương trình điều khiển với nhiều chức năng.

Nội dung chương 10 thực hành điều khiển module led ma trận 2 màu dùng IC thanh ghi dịch mở rộng. Nội dung trình bày đầy đủ phần lý thuyết mạch giao tiếp giữa vi điều khiển với 3 led ma trận. Nguyên lý quét cột để hiển thị kí tự, nguyên lý quét hàng để đáp ứng yêu cầu mở rộng, cách tìm mã kí tự.

Tiếp theo là phần viết các chương trình mẫu điều khiển hiển thị từng kí tự trên led ma trận từ đơn giản đến nâng cao.

Kết hợp với các module khác để thực hiện yêu cầu điều khiển phức tạp hơn, tổng hợp nhiều chương trình điều khiển với nhiều chức năng.

Tài liệu và bộ thực hành này sẽ giúp người học thực hành rất nhanh, hiệu quả cao, không mất nhiều thời gian lãng phí như các bộ thực hành trước đây. Sau khi kết thúc các bạn sẽ có kỹ năng tay nghề cao, kỹ năng làm việc thành thạo và nắm vững kiến thức, giúp bạn tự tin cho việc làm trong tương lai.

Trong quá trình biên soạn không thể tránh được các sai sót nên rất mong các bạn đọc đóng góp xây dựng . Mọi ý kiến đóng góp xin gửi về nhóm tác giả đại diện theo địa chỉ phund@hcmute.edu.vn hoặc phu_nd@yahoo.com.

Nhóm tác giả xin cảm ơn các bạn bè đồng nghiệp đã đóng góp nhiều ý kiến, xin cảm ơn người thân trong gia đình cho phép nhóm tác giả có nhiều thời gian biên soạn tài liệu này.

Nguyễn Đình Phú

Phan Văn Hoàn

Trương Ngọc Anh

DANH SÁCH HÌNH

Hình 1- 1. Bộ thực hành vi điều khiển.	2
Hình 1- 2. Sơ đồ khối kit vi điều khiển PIC18F4550 giao tiếp với 6 module ngoại vi.	3
Hình 1- 3. Sơ đồ chân và sơ đồ khối của IC chốt 74573.....	4
Hình 1- 4. Sơ đồ khối thanh ghi 74HC595.....	7
Hình 1- 5. Sơ đồ chi tiết bên trong thanh ghi 74HC595.	8
Hình 1- 6. Sơ đồ khối thanh ghi MBI5026.....	9
Hình 1- 7. Sơ đồ giao tiếp port D của PIC 18F4550 với 2 IC chốt mở rộng.	10
Hình 1- 8. Sơ đồ nguyên lý module 32 led đơn.	11
Hình 1- 9. Sơ đồ nguyên lý module 4 led 7 đoạn.....	12
Hình 1- 10. Sơ đồ nguyên lý module 8 led 7 đoạn quét.....	13
Hình 1- 11. Sơ đồ nguyên lý module LCD 20×4.	15
Hình 1- 12. Sơ đồ nguyên lý module GLCD 128×64.	16
Hình 1- 13. Sơ đồ nguyên lý module phím nhấn đơn và phím ma trận.	17
Hình 1- 14. Sơ đồ nguyên lý module thu phát hồng ngoại tạo xung.....	17
Hình 1- 15. Sơ đồ nguyên lý module điều khiển động cơ bước và động cơ DC.	18
Hình 1- 16. Sơ đồ nguyên lý module điều khiển Relay, Triac, tải 220V AC.	18
Hình 1- 17. Sơ đồ nguyên lý module giao tiếp I2C và các cảm biến tương tự.	20
Hình 1- 18. Sơ đồ nguyên lý module cảm biến hồng ngoại PIR.....	20
Hình 1- 19. Sơ đồ nguyên lý module 2 cảm biến nhiệt DS18B20.	21
Hình 1- 20. Sơ đồ nguyên lý module kết nối Encoder của motor.	21
Hình 1- 21. Sơ đồ nguyên lý module cảm biến led thu, siêu âm, khí ga, màu.....	21
Hình 1- 22. Sơ đồ nguyên lý mạch điều khiển 3 led ma trận.	22
Hình 1- 23. Sơ đồ nguyên lý mạch giao tiếp máy tính bằng cổng USB qua IC chuyển đổi.....	23
Hình 1- 24. Sơ đồ nguyên lý mạch giao tiếp SIM900.....	24
Hình 2- 1. Biểu tượng phần mềm CCS.	26
Hình 2- 2. Giao diện phần mềm CCS.....	26
Hình 2- 3. Tạo file mới.....	27
Hình 2- 4. Đặt tên file mới và đường dẫn.	27
Hình 2- 5. Màn hình soạn thảo.....	28
Hình 2- 6. Màn hình thay đổi font.....	28
Hình 2- 7. Màn hình của chương trình chớp tắt 4 led.	29

Hình 2- 8. Màn hình thông báo kết quả biên dịch thành công.	30
Hình 2- 9. Màn hình phần mềm nạp PICKIT2.....	31
 Hình 3- 1. Sơ đồ nguyên lý giao tiếp vi điều khiển với module 32 led đơn.	34
Hình 3- 5. Sơ đồ nguyên lý giao tiếp vi điều khiển với 4 nút nhấn đơn.	44
Hình 3- 9. Sơ đồ nguyên lý giao tiếp vi điều khiển với ma trận 16 phím.	50
Hình 3- 10. Lưu đồ quét ma trận 16 phím.....	51
 Hình 4- 1. Sơ đồ nguyên lý giao tiếp vi điều khiển với module 4 led 7 đoạn.....	56
Hình 4- 2. Lưu đồ điều khiển module 4 led 7 đoạn hiển thị 4 số 0, 1, 2, 3.....	58
Hình 4- 3. Lưu đồ điều khiển đếm từ 0 đến 9.	59
Hình 4- 4. Sơ đồ khối timer T0: chế độ 8 bit.	62
Hình 4- 5. Sơ đồ khối timer T0: chế độ 16 bit.	62
Hình 4- 6. Hình ảnh led thu phát HY860F và sơ đồ nguyên lý.....	63
Hình 4- 7. Sơ đồ nguyên lý module thu phát hồng ngoại tạo xung.....	63
 Hình 5- 1. Sơ đồ nguyên lý giao tiếp vi điều khiển với module 8 led 7 đoạn.....	72
Hình 5- 2. Lưu đồ điều khiển 8 led sáng từ 0 đến 7 trên module 8 led 7 đoạn.	74
Hình 5- 3. Sơ đồ khối của timer T1.....	77
 Hình 6- 1. Hình ảnh mặt trước của LCD.....	81
Hình 6- 2. Hình ảnh mặt sau của LCD.....	82
Hình 6- 3. Sơ đồ khối của bộ điều khiển LCD.....	83
Hình 6- 4. Dạng sóng điều khiển của LCD.	87
Hình 6- 5. Hiển thị 32 kí tự ở LCD 16×2.....	90
Hình 6- 6. Hiển thị 80 kí tự ở LCD 20×4.....	91
Hình 6- 7. Trình tự khởi tạo LCD.	92
Hình 6- 8. Giao tiếp vi điều khiển PIC với LCD qua thanh ghi dịch MBI5026.	93
Hình 6- 9. Lưu đồ điều khiển LCD hiển thị 4 chuỗi kí tự.....	98
Hình 6- 12. Lưu đồ quét phím ma trận hiển thị trên LCD.	104
Hình 6- 13. Lưu đồ hiển thị mã tự tạo trái tim trên LCD.....	108
Hình 6- 14. Hình ảnh các đoạn cho số lớn trên LCD.....	110
Hình 6- 15. Tìm mã của các đoạn cho số lớn trên LCD.....	111

Hình 6- 16. Hình ảnh các số lớn từ 0 đến 9 trên LCD.	111
Hình 6- 17. LCD tính theo địa chỉ hàng và cột.	112
Hình 6- 18. Hình ảnh của GLCD.	117
Hình 6- 19. Cấu trúc IC ST7920.	118
Hình 6- 20. Tung độ hiển thị GDRAM và địa chỉ tương ứng của GLCD.....	122
Hình 6- 21. Vẽ lại hình GDRAM theo khối bộ nhớ.....	123
Hình 6- 22. Bộ nhớ GDRAM của GLCD có kích thước 32×256.	123
Hình 6- 23. Bộ nhớ GDRAM của GLCD 32×256 bố trí theo kích thước 64×128.	123
Hình 6- 24. Bộ nhớ GDRAM của GLCD 32×256 chi tiết.	124
Hình 6- 25. Sơ đồ nguyên lý module GLCD 128×64.	127
Hình 7- 1. Sơ đồ khối của ADC PIC18F4550.....	147
Hình 7- 2. Hình ảnh cảm biến LM35CZ.	149
Hình 7- 3. Sơ đồ nguyên lý module ADC và các cảm biến tương tự.....	150
Hình 7- 4. Sơ đồ nguyên lý module điều khiển Relay, Triac, tải 220V AC.	151
Hình 7- 6. Lưu đồ chuyển đổi ADC kênh thứ 0 để đọc nhiệt độ và so sánh điều khiển buzzer.	153
Hình 7- 7. Hình cảm biến khoảng cách GP2D12.....	157
Hình 7- 8. Sơ đồ khối bên trong cảm biến khoảng cách GP2D12.	157
Hình 7- 9. Dạng sóng của cảm biến khoảng cách GP2D12.	158
Hình 7- 10. Đồ thị chuyển đổi của cảm biến khoảng cách GP2D12.....	159
Hình 7- 11. Hình cảm biến siêu âm và góc phát thu sóng.....	161
Hình 7- 12. Giảm đồ thời gian của cảm biến HC-SR04.....	162
Hình 7- 13. Sơ đồ nguyên lý cảm biến khoảng cách siêu âm.	163
Hình 7- 14. Các dạng cảm biến DS18B20.	167
Hình 7- 15. Sơ đồ khối của cảm biến DS18B20.	167
Hình 7- 16. Cấu trúc 2 thanh ghi lưu nhiệt độ của cảm biến DS18B20.....	168
Hình 7- 17. Cấu trúc 2 thanh ghi lưu nhiệt độ báo động của cảm biến DS18B20.....	169
Hình 7- 18. Sơ đồ cấp nguồn cho cảm biến DS18B20.....	170
Hình 7- 19. Cấu trúc mã 64 bit của cảm biến DS18B20.....	170
Hình 7- 20. Tổ chức bộ nhớ ROM của cảm biến DS18B20.	170
Hình 7- 21. Byte thanh ghi điều khiển của cảm biến DS18B20.	171
Hình 7- 22. Lưu đồ các lệnh liên quan đến ROM của cảm biến DS18B20.	173

Hình 7- 23. Lưu đồ của các lệnh chức năng của cảm biến DS18B20.....	175
Hình 7- 24. Dạng sóng của xung reset và xung hiện diện.....	177
Hình 7- 25. Dạng sóng khe thời gian đọc/ghi.	178
Hình 7- 26. Chi tiết dạng sóng khe thời gian đọc mức 1 của thiết bị chủ.	179
Hình 7- 27. Thu ngắn các khoảng thời gian.	179
Hình 7- 28. Mạch đo nhiệt độ dùng 2 cảm biến DS18B20.	180
Hình 8- 1. Hệ thống các thiết bị giao tiếp theo chuẩn I2C.	188
Hình 8- 2. Quá trình chủ ghi dữ liệu vào tớ.	190
Hình 8- 3. Quá trình chủ đọc dữ liệu từ tớ.	190
Hình 8- 4. Sơ đồ chân DS1307.....	190
Hình 8- 5. Sơ đồ kết nối vi điều khiển với DS1307.	191
Hình 8- 6. Tổ chức bộ nhớ của DS1307.....	192
Hình 8- 7. Tổ chức các thanh ghi thời gian.	192
Hình 8- 8. Cấu trúc bên trong DS1307.....	193
Hình 8- 9. Giao tiếp vi điều khiển các thiết bị theo chuẩn I2C.	194
Hình 8- 11. Sơ đồ khối IC PCF8591.	201
Hình 8- 12. Sơ đồ khối IC PCF8591.	202
Hình 8- 13. Cấu trúc thanh ghi điều khiển của IC PCF8591.	204
Hình 8- 14. Cấu trúc khối DAC của IC PCF8591.....	205
Hình 8- 15. Công thức điện áp ra và đặc tính chuyển đổi.....	206
Hình 8- 16. Dạng sóng chuyển đổi D/A.....	206
Hình 8- 17. Dạng sóng truyền dữ liệu của IC PCF8591.	207
Hình 8- 18. Dạng sóng chuyển đổi A/D ở chế độ đơn cực.	207
Hình 8- 19. Dạng sóng chuyển đổi A/D ở chế độ ngõ vào vi sai.....	208
Hình 8- 20. Địa chỉ của IC PCF8591.	208
Hình 8- 21. Tín hiệu đọc ghi.	209
Hình 8- 22. Chế độ ghi dữ liệu vào IC PCF8591.	209
Hình 8- 23. Chế độ đọc dữ liệu từ IC PCF8591.....	209
Hình 8- 24. Mối quan hệ giữa sự thay đổi ánh sáng và điện trở.	212
Hình 9- 1. Động cơ bước.....	215
Hình 9- 2. Sơ đồ nguyên lý động cơ bước.	215

Hình 9- 3. Hình dạng IC L298.	216
Hình 9- 4. Sơ đồ nguyên lý IC L298.	216
Hình 9- 5. IC L298 điều khiển 1 động cơ bước.	217
Hình 9- 6. Mạch giao tiếp động cơ bước và động cơ DC.	218
Hình 9- 7. Lưu đồ điều khiển động cơ bước quay thuận bằng 2 nút ON, OFF.....	223
Hình 9- 8 Động cơ DC trong BTN.....	226
Hình 9- 9. Encoder gắn trong sau động cơ.....	226
Hình 9- 10. IC L298 điều khiển 1 động cơ DC dùng 1 cầu H.	227
Hình 9- 11. IC L298 điều khiển 1 động cơ DC dùng 2 cầu H.	227
Hình 9- 12. Mạch giao tiếp động cơ bước và động cơ DC.	228
Hình 9- 13. Dạng sóng điều chế độ rộng xung.....	233
Hình 9- 14. Sơ đồ khối của PWM PIC.....	234
Hình 9- 15. Dạng sóng PWM.....	235
Hình 10- 1. Sơ đồ nguyên lý giao tiếp vi điều khiển với 3 led ma trận.	244
Hình 10- 2. Lưu đồ điều khiển hiển thị 4 kí tự trên 3 led ma trận.	252

DANH SÁCH BẢNG

Bảng 1- 1. Tên các chân của IC 74HC573.....	4
Bảng 1- 2. Bảng trạng thái IC 74HC573.....	5
Bảng 1- 3. Bảng trạng thái IC 74HC595.....	7
Bảng 1- 4. Bảng trạng thái IC MBI5026.....	9
Bảng 1- 5. Các tín hiệu mở rộng của IC chốt 74HC573_A.....	10
Bảng 1- 6. Các tín hiệu mở rộng của IC chốt 74HC573_B.....	10
Bảng 1- 7. Bảng mã led 7 đoạn loại anode chung.....	12
Bảng 1- 8. Bảng dữ liệu điều khiển quét lần lượt các transistor.....	14
Bảng 4- 1. Các thanh ghi có liên quan đến timer T0.....	62
Bảng 5- 1. Bảng dữ liệu điều khiển quét lần lượt các transistor.....	73
Bảng 6- 1. Các chân của LCD.....	81
Bảng 6- 2. Các lệnh điều khiển LCD.....	83
Bảng 6- 3. Mã lệnh Function set.....	85
Bảng 6- 4. Mã lệnh Display control.....	85
Bảng 6- 5. Mã lệnh Clear Display.....	86
Bảng 6- 6. Mã lệnh entry mode.....	86
Bảng 6- 7. Mã lệnh entry mode.....	86
Bảng 6- 8. Cho biết các thông số thời gian của LCD.....	88
Bảng 6- 9. Bảng mã ASCII.....	88
Bảng 6- 10. Địa chỉ vùng nhớ hiển thị DDRAM.....	89
Bảng 6- 11. Địa chỉ từng kí tự của LCD 20x4.....	91
Bảng 6- 12. Thiết lập địa chỉ CGRAM.....	106
Bảng 6- 13. Vùng nhớ CGRAM.....	106
Bảng 6- 14. Mã của trái tim.....	107
Bảng 6- 15. Các chân của GLCD.....	117
Bảng 6- 16. Các font chữ 8x16 của GLCD.....	121
Bảng 6- 17. Địa chỉ gởi mã của font 16x16.....	121
Bảng 6- 18. Tập lệnh điều khiển cơ bản của GLCD.....	124
Bảng 6- 19. Tập lệnh điều khiển mở rộng của GLCD.....	126

Bảng 7- 1. Giá trị tại các tọa độ chính.....	159
Bảng 7- 2. Độ phân giải và thời gian chuyển đổi.....	171
 Bảng 8- 1. Tên và chức năng từng chân của IC PCF8591.	201
 Bảng 9- 1. Các trạng thái điều khiển động cơ bước theo bước đủ.	218
Bảng 9- 2. Các trạng thái điều khiển động cơ bước dạng nửa bước.	219
Bảng 9- 3. Các trạng thái điều khiển động cơ DC.	228
 Bảng 10- 1. Mã quét cột cho led màu đỏ sáng, xanh tắt.	245
Bảng 10- 2. Mã quét cột cho led màu xanh sáng, đỏ tắt.	245
Bảng 10- 3. Mã quét cột cho led màu xanh và đỏ đều sáng thành màu cam.	246
Bảng 10- 4. Mã các kí tự mẫu A, B, C, D.	246
Bảng 10- 5. Dùng phần mềm tìm mã ta được bảng mã ma trận của các kí tự.	246
Bảng 10- 6. Mã quét hàng.	255
Bảng 10- 7. Bảng thứ tự các cột màu đỏ.	255
Bảng 10- 8. Bảng thứ tự các cột màu xanh.	256

MỤC LỤC

GIÁO TRÌNH THỰC HÀNH	i
LỜI NÓI ĐẦU	ii
DANH SÁCH HÌNH	vi
DANH SÁCH BẢNG	xi
MỤC LỤC	xiii
Chương 1: CẤU HÌNH KIT THỰC HÀNH VI ĐIỀU KHIỂN	1
1.1 GIỚI THIỆU BỘ KIT THỰC HÀNH	2
1.2 SƠ ĐỒ NGUYÊN LÝ KIT VI ĐIỀU KHIỂN	4
1.2.1 Hệ thống vi điều khiển PIC18F4550 giao tiếp với các module ngoại vi	4
1.2.2 Module 1: Giao tiếp với Led đơn	10
1.2.3 Module 2: 4 led 7 đoạn trực tiếp	11
1.2.4 Module 3: 8 led 7 đoạn quét	13
1.2.5 Module 4: LCD 20×4	14
1.2.6 Module 5: GLCD 128×64	15
1.2.7 Module 6: phím đơn và phím ma trận 4×4	16
1.2.8 Module 7: thu phát hồng ngoại tạo xung	17
1.2.9 Module 8: giao tiếp relay, buzzer và động cơ	17
1.2.10 Module 9: giao tiếp ADC, thời gian thực, bộ nhớ Eeprom và các cảm biến	19
1.2.11 Module 10: giao tiếp các cảm biến nhiệt, cảm biến khoảng cách, led thu	20
1.2.12 Module 11: giao tiếp led ma trận	22
1.2.13 Module 12: giao tiếp máy tính bằng USB qua IC chuyển đổi RS232	23
1.2.14 Module 13: giao tiếp SIM900-TE-C-V1.02	23
Chương 2: SỬ DỤNG PHẦN MỀM CCS VÀ PICKIT	25
2.1 GIỚI THIỆU	26
2.2 HƯỚNG DẪN SỬ DỤNG PHẦN MỀM CCS	26
2.2.1 Biên soạn chương trình cơ bản	26
2.2.2 Nạp chương trình vào bộ nhớ vi điều khiển	30
Chương 3: THỰC HÀNH MODULE 1– 32 LED ĐƠN DÙNG THANH GHI DỊCH 74HC595, NÚT NHẤN, BÀN PHÍM MA TRẬN	33
3.1 CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN 32 LED ĐƠN	34
3.1.1 Mạch giao tiếp vi điều khiển với module 32 led đơn	34
3.1.2 Các hàm điều khiển module 32 led đơn	35

3.1.3	Các chương trình điều khiển 32 led đơn dùng lệnh for	37
3.1.4	Các chương trình điều khiển 32 led đơn dùng lệnh if	41
3.2	CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN 32 LED ĐƠN VÀ NÚT NHẤN	43
3.2.1	Giới thiệu.....	43
3.2.2	Mạch điện giao tiếp vi điều khiển với 4 nút nhấn đơn	44
3.2.3	Định nghĩa tên các nút nhấn trong thư viện	44
3.2.4	Các chương trình điều khiển dùng các nút nhấn đơn.....	45
3.2.5	Mạch điện giao tiếp vi điều khiển với ma trận phím.....	49
3.2.6	Các chương trình điều khiển dùng ma trận phím.....	51
3.3	CÁC CÂU HỎI ÔN TẬP	54
<i>Chương 4: THỰC HÀNH MODULE 2 – 4 LED 7 ĐOẠN ANODE DÙNG THANH GHI DỊCH 74HC595, COUNTER/TIMER</i>		
		55
4.1	CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN 4 LED 7 ĐOẠN	56
4.1.1	Mạch điện giao tiếp vi điều khiển với module 4 led 7 đoạn	56
4.1.2	Hàm điều khiển module 4 led 7 đoạn.....	57
4.1.3	Các chương trình điều khiển 4 led 7 đoạn.....	57
4.2.1	Tóm tắt timer/counter T0	61
4.2.2	Cảm biến thu phát hồng ngoại HY860F.....	62
4.2.3	Mạch điện dùng T0 đếm xung ngoại.....	63
4.2.4	Các ứng dụng đếm xung ngoại dùng counter T0.....	64
4.3	KẾT HỢP ĐIỀU KHIỂN 32 LED VÀ ĐẾM XUNG DÙNG COUNTER T0	66
4.4	CÁC CÂU HỎI ÔN TẬP	69
<i>Chương 5: THỰC HÀNH MODULE 3 – 8 LED 7 ĐOẠN ANODE CHUNG KẾT NỐI THEO PP QUÉT DÙNG THANH GHI DỊCH MBI5026</i>		
		70
5.1	MẠCH ĐIỆN GIAO TIẾP VI ĐIỀU KHIỂN VỚI 8 LED 7 ĐOẠN QUÉT	71
5.1.1	Mạch điện giao tiếp vi điều khiển với module 8 led 7 đoạn	71
5.1.2	Các hàm điều khiển module 8 led 7 đoạn quét	73
5.1.3	Các chương trình hiển thị 8 led 7 đoạn quét.....	74
5.2	CÁC CHƯƠNG TRÌNH ĐẾM THỜI GIAN HIỂN THỊ 8 LED	75
5.2.1	Đếm giây dùng delay	76
5.2.2	Đếm giây dùng timer T1 định thời.....	76
5.3	KẾT HỢP 3 MODULE	79
5.4	CÁC CÂU HỎI ÔN TẬP	79
<i>Chương 6: THỰC HÀNH MODULE 4 – LCD 20×4, GLCD 128×64</i>		
		80
6.1	LÝ THUYẾT LCD	81
6.1.1	Giới thiệu LCD	81

6.1.2	Sơ đồ chân của LCD.....	81
6.1.3	Bộ điều khiển LCD và các vùng nhớ	82
6.1.4	Các lệnh điều khiển LCD.....	83
6.1.5	Các hoạt động đọc ghi LCD.....	87
6.1.1	Mã ASCII	88
6.1.2	Vùng nhớ hiển thị DDRAM.....	89
6.1.3	Lưu đồ khởi tạo LCD	91
6.2	LCD TRONG KIT THỰC HÀNH.....	92
6.2.1	Mạch giao tiếp vi điều khiển với LCD	92
6.2.2	Dữ liệu và hàm điều khiển LCD và GLCD	94
6.3	CÁC CHƯƠNG TRÌNH SỬ DỤNG VÙNG NHỚ DDRAM CỦA LCD.....	95
6.3.1	Các ứng dụng hiển thị kí tự, dịch chuỗi trên LCD.....	95
6.3.2	Đếm thời gian và đếm xung ngoại hiển thị trên LCD	102
6.3.3	Các ứng dụng kết hợp bàn phím ma trận và LCD.....	103
6.4	CÁC ỨNG DỤNG VÙNG NHỚ CGRAM CỦA LCD	106
6.4.1	Vùng nhớ CGRAM	106
6.4.2	Cách tìm mã kí tự mong muốn.....	107
6.4.3	Các chương trình ứng dụng các kí tự - tự tạo trên LCD	107
6.4.4	Tạo mã 7 đoạn kích thước lớn trên LCD	110
6.5	LÝ THUYẾT GLCD	116
6.5.1	Giới thiệu GLCD	116
6.5.2	Sơ đồ chân của GLCD.....	117
6.5.3	IC điều khiển GLCD ST7920	118
6.5.4	Mô tả chức năng ic điều khiển GLCD ST7920	119
6.5.5	Các lệnh điều khiển GLCD.....	124
6.6	CÁC BÀI THỰC HÀNH DÙNG GLCD	127
6.6.1	Sơ đồ mạch của GLCD.....	127
6.6.2	Các bài thực hành hiển thị kí tự trên GLCD	127
6.6.3	Các bài thực hành hiển thị hình ảnh trên GLCD.....	143
6.7	KẾT HỢP NHIỀU MODULE.....	145
	<i>Chương 7: THỰC HÀNH CHUYỂN ĐỔI TƯƠNG TỰ SANG SỐ - ADC, CÁC CẢM BIẾN</i>	<i>146</i>
7.1	CHUYỂN ĐỔI TƯƠNG TỰ SANG SỐ CỦA PIC	147
7.1.1	Khảo sát ADC của PIC 18F4550.....	147
7.1.2	Tập lệnh c cho khối ADC của PIC	148
7.2	ĐO NHIỆT ĐỘ DÙNG CẢM BIẾN LM35.....	149

7.2.1	Cảm biến LM35	149
7.2.2	Mạch điện giao tiếp vi điều khiển PIC với cảm biến LM35	150
7.2.3	Đo nhiệt dùng cảm biến LM35 hiển thị trên 4 led 7 đoạn	151
7.3	ĐO KHOẢNG CÁCH DÙNG CẢM BIẾN GP2D12.....	156
7.3.1	Cảm biến GP2D12	156
7.3.2	Đo khoảng cách hiển thị trên 4 led 7 đoạn	160
7.4	ĐO KHOẢNG CÁCH DÙNG CẢM BIẾN SIÊU ÂM HC-SR 04.....	161
7.4.1	Cảm biến HC-SR 04	161
7.4.2	Cảm biến HC-SR 04 trong kit thực hành.....	162
7.4.3	Đo khoảng cách dùng HCSR-04 hiển thị trên 4 led 7 đoạn	165
7.5	CẢM BIẾN NHIỆT 1 DÂY	166
7.5.1	Cảm biến nhiệt DS18B20	166
7.5.2	Hình ảnh và tên các chân của cảm biến	166
7.5.3	Sơ đồ khối của cảm biến DS18B20.....	167
7.5.4	Hoạt động của cảm biến DS18B20	168
7.5.5	Hoạt động cảnh báo quá nhiệt của cảm biến DS18B20.....	169
7.5.6	Cấp nguồn cho cảm biến DS18B20	169
7.5.7	Mã 64 bit của cảm biến DS18B20	170
7.5.8	Bộ nhớ rom của cảm biến DS18B20	170
7.5.9	Thanh ghi định cấu hình của cảm biến DS18B20.....	171
7.5.10	Trình tự hoạt động của cảm biến DS18B20	171
7.5.11	Các lệnh hoạt động của cảm biến DS18B20.....	172
7.5.12	Các dạng tín hiệu của chuẩn 1 dây	176
7.5.13	Trình tự khởi động – xung reset và xung hiện diện	176
7.6	CÁC CHƯƠNG TRÌNH ĐO NHIỆT ĐỘ DÙNG CẢM BIẾN DS18B20	179
7.6.1	Mạch điện giao tiếp vi điều khiển với cảm biến DS18B20	179
7.6.2	Đo nhiệt độ dùng cảm biến DS18B20 hiển thị trên led 7 đoạn.....	180
Chương 8: THỰC HÀNH MODULE 5 – REAL TIME DS13B07, ADC–DAC PCF8591, EEPROM NỐI TIẾP AT24C256 THEO CHUẨN I2C VÀ CÁC CẢM BIẾN.....		187
8.1	LÝ THUYẾT I2C	188
8.1.1	Giới thiệu.....	188
8.1.2	Tổng quan về truyền dữ liệu chuẩn I2C.....	188
8.2	KHẢO SÁT IC REALTIME DS13B07	190
8.2.1	Khảo sát IC DS1307	190
8.2.2	Sơ đồ giao tiếp vi điều khiển với DS1307	193

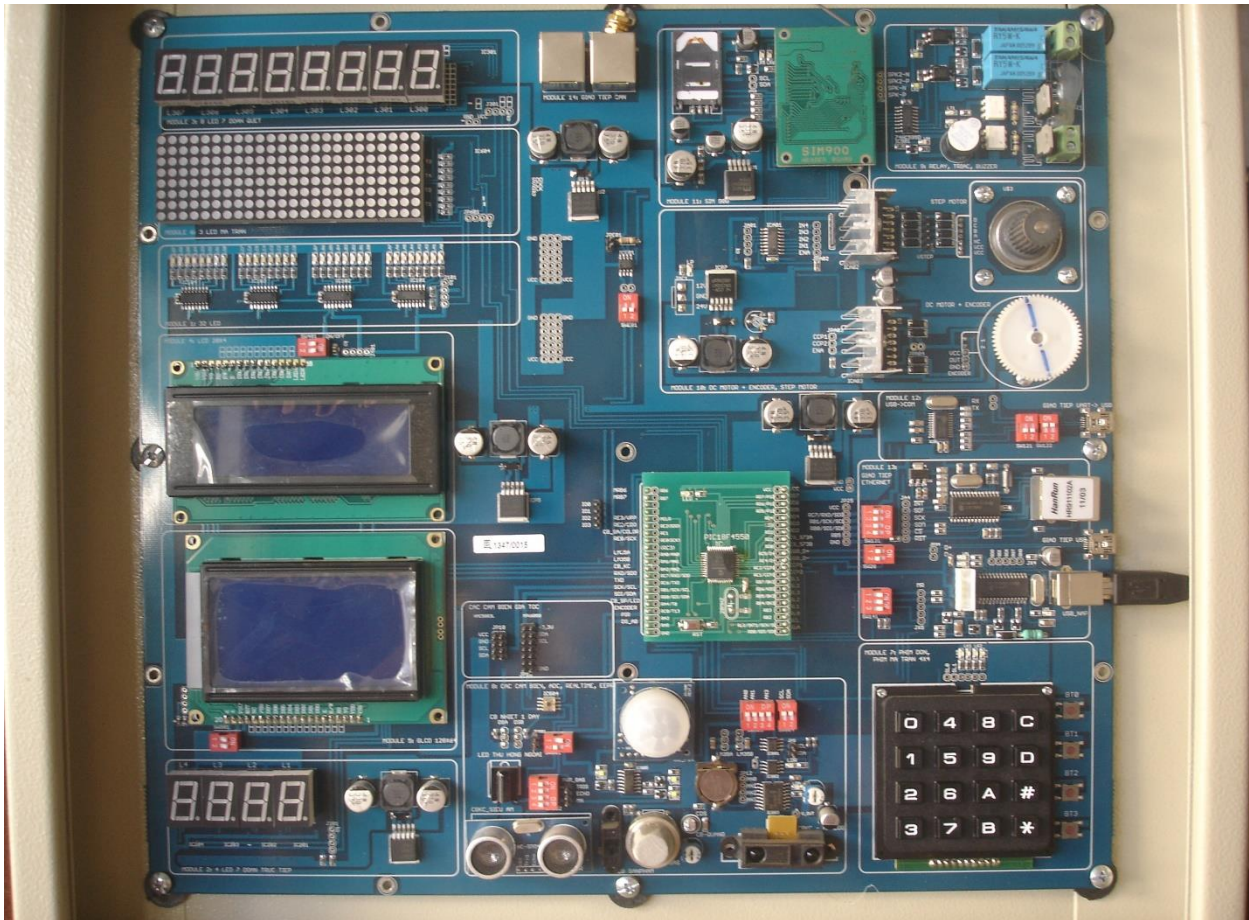
8.2.3	Các chương trình giao tiếp DS13B07.....	194
8.3	KHẢO SÁT IC PCF8591.....	199
8.3.1	Giới thiệu IC PCF8591.....	199
8.3.2	Cấu hình IC PCF8591.....	200
8.3.3	Ứng dụng IC PCF8591.....	200
8.3.4	Sơ đồ chân IC PCF8591.....	200
8.3.5	Sơ đồ khối IC PCF8591.....	202
8.3.6	Địa chỉ ngoài IC PCF8591	202
8.3.7	Thanh ghi điều khiển của IC PCF8591	203
8.3.8	Chuyển đổi D/A của IC PCF8591	205
8.3.9	Chuyển đổi A/D của IC PCF8591	206
8.3.10	Điện áp tham chiếu của IC PCF8591.....	208
8.3.11	Bộ dao động của IC PCF8591.....	208
8.3.12	Địa chỉ thanh ghi của IC PCF8591	208
8.3.13	Giao tiếp vi điều khiển với IC PCF8591.....	209
8.3.14	Các chương trình giao tiếp IC PCF 8591	209
8.4	CẢM BIẾN QUANG TRỞ.....	212
8.4.1	Khảo sát quang trở.....	212
8.4.2	Vi điều khiển giao tiếp với quang trở	212
8.4.3	Các chương trình điều khiển dùng quang trở	212
Chương 9: THỰC HÀNH MODULE 6 – ĐỘNG CƠ BƯỚC, ĐỘNG CƠ DC VÀ ENCODER, PWM, ĐIỀU KHIỂN PID		214
9.1	ĐIỀU KHIỂN ĐỘNG CƠ BƯỚC.....	215
9.1.1	Giới thiệu động cơ bước.....	215
9.1.2	IC giao tiếp công suất điều khiển L298	215
9.1.3	Mạch giao tiếp điều khiển động cơ bước.....	217
9.1.4	Mã điều khiển động cơ bước	218
9.1.5	Các biến và hàm điều khiển Relay, Triac, Buzzer, động cơ	219
9.1.6	Các bài thực hành điều khiển động cơ bước.....	221
9.2	ĐIỀU KHIỂN ĐỘNG CƠ DC, ENCODER.....	225
9.2.1	Giới thiệu động cơ DC, ENCODER.....	225
9.2.2	Mạch giao tiếp điều khiển động cơ DC.....	227
9.2.3	Dữ liệu điều khiển động cơ DC	228
9.2.4	Các bài thực hành điều khiển động cơ DC.....	228
9.3	ĐIỀU KHIỂN THAY ĐỔI TỐC ĐỘ ĐỘNG CƠ DC PWM, ENCODER	233

9.3.1	Giới thiệu PWM (PULSE WIDTH MODULATION).....	233
9.3.2	Cấu trúc khối điều chế độ rộng xung PWM của PIC.....	234
9.3.3	Tính chu kỳ xung PWM.....	235
9.3.4	Tính hệ số chu kỳ xung PWM	235
9.3.5	Các lệnh điều khiển PWM.....	235
9.3.6	Tính toán điều khiển tốc độ động cơ bằng PWM	236
9.3.7	Các bài thực hành thay đổi tốc độ động cơ dc bằng PWM.....	237
9.4	ĐIỀU KHIỂN KẾT HỢP 2 ĐỘNG CƠ BƯỚC VÀ DC	240
9.5	ĐIỀU KHIỂN ỔN ĐỊNH TỐC ĐỘ ĐỘNG CƠ DC	240
9.3.1	Giới thiệu PID.....	240
9.3.2	Điều khiển động cơ dùng PID.....	240
Chương 10: CÁC BÀI THỰC HÀNH MODULE 7 – ĐIỀU KHIỂN LED MA TRẬN 2 MÀU		243
10.1	SƠ ĐỒ MẠCH GIAO TIẾP VI ĐIỀU KHIỂN VỚI LED MA TRẬN.....	243
10.2	MÃ KÍ TỰ CHO LED MA TRẬN DẠNG QUÉT CỘT.....	244
10.3	CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN LED MA TRẬN QUÉT CỘT.....	248
10.4	MÃ KÍ TỰ CHO LED MA TRẬN DẠNG QUÉT HÀNG.....	254
10.5	CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN LED MA TRẬN QUÉT HÀNG	264
TÀI LIỆU THAM KHẢO		267

Chương 1: CẤU HÌNH KIT THỰC HÀNH VI ĐIỀU KHIỂN

1.1 GIỚI THIỆU BỘ KIT THỰC HÀNH

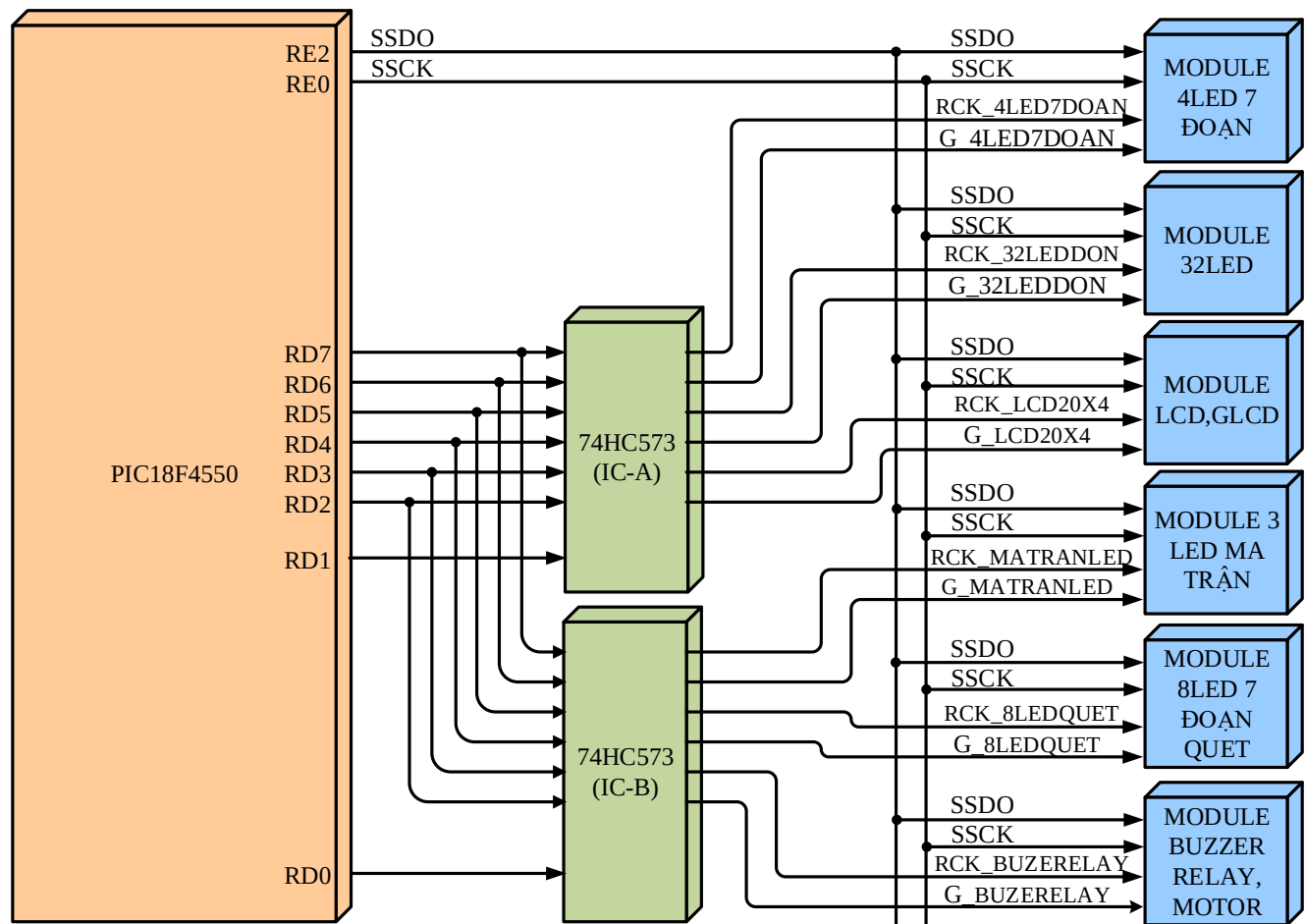
Chương này giới thiệu bộ kit thực hành vi điều khiển tương đối đầy đủ các yêu cầu phần cứng như hình 1-1, có thể giúp bạn thực hành và tự nghiên cứu, tự học.



Hình 1-1. Bộ thực hành vi điều khiển.

- ✓ Bộ thực hành vi điều khiển có thể sử dụng cho nhiều loại vi điều khiển bằng cách thay đổi socket.
- ✓ Các module thực hành gồm có:
 - Có 32 led đơn giao tiếp dùng IC mở rộng 74HC595.
 - Có 4 led 7 đoạn kết nối trực tiếp giao tiếp dùng IC mở rộng 74HC595.
 - Có 8 led 7 đoạn kết nối theo phương pháp quét thông qua IC MBI5026.
 - Có 3 led ma trận 8×8 hai màu xanh và đỏ dùng IC MBI5026.
 - Có 1 LCD 20x4 kết nối thông qua IC MBI5026.
 - Có 1 GLCD 128x64 kết nối thông qua IC MBI5026.
 - Có 4 phím đơn kết nối trực tiếp.
 - Bàn phím ma trận gồm 16 phím.
 - Có 2 relay và 2 triac.
 - Có 1 buzzer.
 - Có mạch công suất điều khiển động cơ bước và 1 động cơ bước.
 - Có mạch công suất điều khiển động cơ DC và 1 động cơ DC, có encoder.

- Chuyển đổi tín hiệu tương tự sang số (ADC) đa hợp 4 kênh.
- Chuyển đổi tín hiệu số sang tín hiệu tương tự (DAC).
- Có 2 cảm biến LM35.
- Có 2 cảm biến nhiệt theo chuẩn 1 dây DS18B20.
- Có 1 cảm biến PIR nhận biết chuyển động.
- Có 1 cảm biến khí gas/khói.
- Có 2 cảm biến khoảng cách.
- Có giao tiếp đồng hồ thời gian thực nối tiếp DS13B07.
- Có 1 bộ nhớ EEPROM giao tiếp theo chuẩn I2C.
- Có 1 mạch thu phát xung.
- Có cảm biến gia tốc.
- Có 1 cảm biến quang trở.
- Có mạch nạp PIC.
- Có mạch chuyển đổi từ USB sang cổng COM.
- Có giao tiếp mạng CAN.
- Có module giao tiếp Ethernet.
- Có module giao tiếp SIM900-TEC-V102



Hình 1-2. Sơ đồ khối kit vi điều khiển PIC18F4550 giao tiếp với 6 module ngoại vi.

1.2 SƠ ĐỒ NGUYÊN LÝ KIT VI ĐIỀU KHIỂN

Bộ thực hành vi điều khiển có thể sử dụng nhiều loại vi điều khiển khác nhau, tuy nhiên, tài liệu chỉ trình bày cho loại vi điều khiển PIC18F4550.

1.2.1 Hệ thống vi điều khiển PIC18F4550 giao tiếp với các module ngoại vi

Sơ đồ khối của kit dùng vi điều khiển PIC18F4550 giao tiếp với 6 module ngoại vi như hình 1-2.

Kit vi điều khiển có nhiều module ngoại vi. Để có thể giao tiếp hết các module ngoại vi thì vi điều khiển dùng thêm 2 IC mở rộng port là 74HC573, được phân biệt là IC-A và IC-B.

Sơ đồ khối giúp bạn đọc dễ tiếp cận, sau đó bạn đọc có thể xem sơ đồ nguyên lý chi tiết ở hình tiếp theo.

Trong sơ đồ khối hình 1-2 thì port D có 6 bit nối song song với 2 IC chốt 74HC573 để tạo 12 ngõ ra độc lập để điều khiển 6 module giao tiếp.

Hai bit RD1 và RD0 dùng để điều khiển ngõ vào chốt dữ liệu của 2 IC chốt.

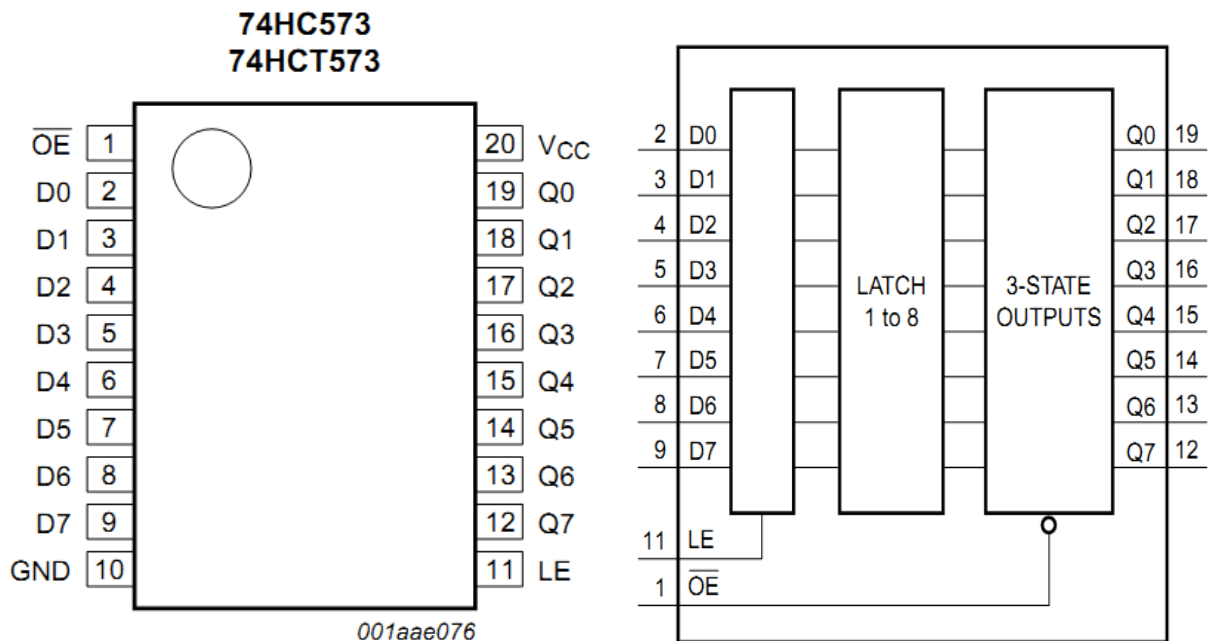
Để hiểu rõ cách thức chốt dữ liệu thì ta phải tìm hiểu nguyên lý hoạt động của IC chốt.

a. IC chốt 74HC573:

Sơ đồ kí hiệu và bảng trạng thái IC chốt như hình 1-3.

IC chốt 74HC573 có 8 ngõ vào nhận dữ liệu D₇-D₀ và 8 ngõ ra Q₇-Q₀.

Tín hiệu điều khiển xuất dữ liệu (OE: output enable) tích cực mức thấp cho phép đưa dữ liệu từ mạch chốt ở bên trong ra ngoài, nếu không cho phép thì các ngõ ra sẽ ở trạng thái tổng trở cao.



Hình 1-3. Sơ đồ chân và sơ đồ khối của IC chốt 74573.

Bảng 1-1. Tên các chân của IC 74HC573.

Symbol	Pin	Description
$\overline{OE}$	1	3-state output enable input (active LOW)
D[0:7]	2, 3, 4, 5, 6, 7, 8, 9	data input
GND	10	ground (0 V)
LE	11	latch enable input (active HIGH)
Q[0:7]	19, 18, 17, 16, 15, 14, 13, 12	3-state latch output
V _{CC}	20	supply voltage

Tín hiệu điều khiển chốt (LE: latch enable):

Khi ở mức 1 thì cho phép thông dữ liệu: dữ liệu đưa đến 8 ngõ vào D₇-D₀ sẽ được lưu vào mạch chốt bên trong và nếu được phép thì đưa ra bên ngoài Q₇-Q₀.

Khi ở mức 0 thì không cho phép thông dữ liệu ngõ vào D₇-D₀ lưu vào mạch chốt, dữ liệu lưu trong mạch chốt chính là dữ liệu trước khi hạ tín hiệu chốt về mức 0.

Bảng 1-2. Bảng trạng thái IC 74HC573.

Table 3. Function table^[1]

Operating mode	Control		Input	Internal latches	Output
	$\overline{OE}$	LE	D _n		Q _n
Enable and read register (transparent mode)	L	H	L	L	L
			H	H	H
Latch and read register	L	L	l	L	L
			h	H	H
Latch register and disable outputs	H	L	l	L	Z
			h	H	Z

Có thể nói ngắn gọn là khi tín hiệu LE ở mức 1 thì cho phép thông dữ liệu từ ngõ vào đến ngõ ra, khi LE ở mức 0 thì tín hiệu bị chặn lại và các ngõ ra lưu lại dữ liệu trước khi hạ chốt.

b. Hoạt động của 2 IC chốt 74HC573 trong hệ thống:

Trong sơ đồ khối hình 1-2 thì hai tín hiệu OE đều được nối GND để luôn cho phép xuất dữ liệu ra, hai bit RD0 và RD1 dùng để điều khiển 2 tín hiệu LE của 2 IC chốt.

Bình thường thì 2 tín hiệu chốt ở mức logic 0, khi đó không IC nào được phép nhận dữ liệu.

Khi gửi dữ liệu ra 6 bit từ RD2 đến RD7 thì cả 2 IC chốt đều có dữ liệu ở đầu vào.

Muốn IC chốt nào nhận dữ liệu 6 bit này thì ta tiến hành mở chốt cho IC đó, IC còn lại không bị ảnh hưởng.

Tương tự bạn tiến hành xuất dữ liệu ra chốt dữ liệu lại cho IC còn lại.

Kết quả ta có thể có 12 đường tín hiệu khác nhau ở ngõ ra của 2 IC chốt.

c. Chức năng của 12 đường trong hệ thống:

12 tín hiệu ngõ ra của 2 IC chốt chia thành 6 cặp, mỗi cặp 2 tín hiệu có tên chung là RCK và G, còn phần tiếp theo của tên chỉ cho biết đối tượng điều khiển.

Vi điều khiển PIC dùng 2 tín hiệu được đặt tên là SSDO và SSCK giao tiếp song song với tất cả 6 module và 2 tín hiệu này có chức năng dịch chuyển dữ liệu nối tiếp từ vi điều khiển đến các module.

Trong sơ đồ khối hình 1-2 thì mỗi module giao tiếp sử dụng 4 tín hiệu: SSDO, SSCK, RCK_x và G_x, với x là tên gắn với chức năng của từng module tương ứng.

Các module ngoại vi sử dụng các thanh ghi dịch 74HC595 và MBI5026 có chức năng chuyển dữ liệu từ nối tiếp sang song – song để điều khiển các thiết bị ở ngõ ra.

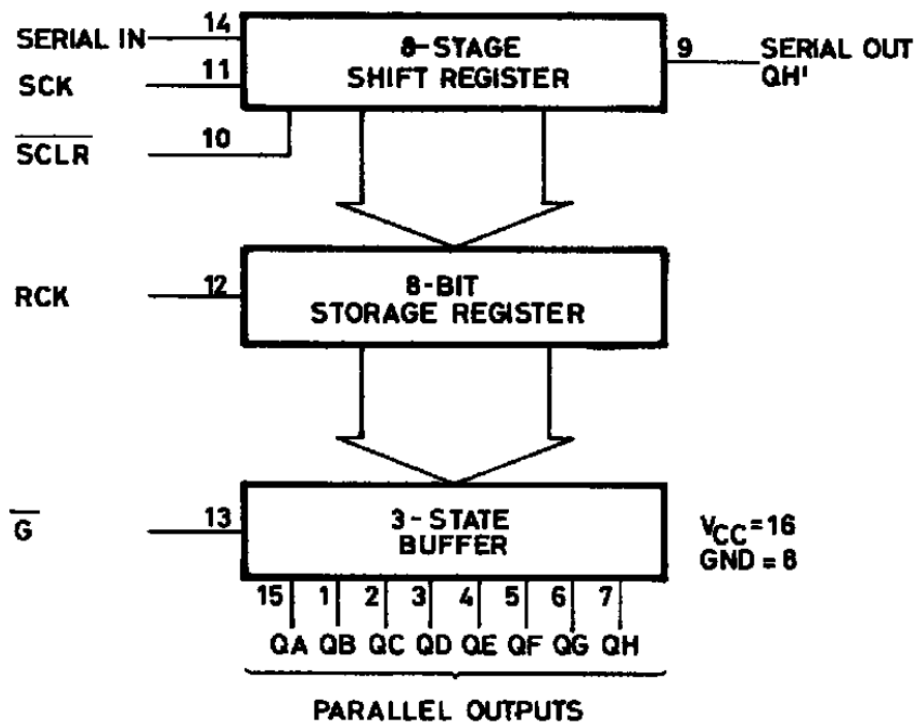
Kit vi điều khiển được thiết kế theo kiểu giao tiếp này với mục đích mở rộng nhiều tín hiệu giao tiếp để đủ tín hiệu điều khiển tất cả các ngoại vi, không sử dụng dây bus nối giữa các port với các module như các kit vi điều khiển trước đây, nhờ đó khắc phục được các nhược điểm đứt dây, kết nối sai port, sai thứ tự tín hiệu, ...

Ta cần phải tìm hiểu nguyên lý hoạt động của các IC thanh ghi dịch 74HC595 và MBI5026.

d. Hoạt động của IC 74HC595:

Sơ đồ kí hiệu và bảng trạng thái IC 74HC595 như hình 1-4. IC 74HC595 có 2 tầng thanh ghi 8 bit: tầng thanh ghi dịch (8-bit Stage Shift Register) và tầng thanh ghi lưu trữ (8-bit Store Register).

- Chân tín hiệu SERIAL_IN (SI) là ngõ vào nhận dữ liệu nối tiếp.
- Chân tín hiệu SCK dùng để nhận xung clock để đẩy dữ liệu vào thanh ghi dịch.
- Chân tín hiệu SCLR dùng để xóa dữ liệu trong thanh ghi dịch nối tiếp. Tín hiệu này tích cực mức 0 và không dùng thì nối với mức 1.
- Chân tín hiệu QH' dùng để kết nối với thanh ghi dịch tiếp theo.
- Tín hiệu RCK dùng để nạp dữ liệu từ thanh ghi dịch bên trong sang thanh ghi lưu trữ và nếu cho phép thì xuất dữ liệu ra ngoài.
- Chân tín hiệu G dùng để mở bộ đệm 3 trạng thái xuất tín hiệu ra ngoài.



Hình 1-4. Sơ đồ khối thanh ghi 74HC595.

Bảng 1-3. Bảng trạng thái IC 74HC595.

INPUTS					OUTPUT
SI	SCK	SCLR	RCK	G	
X	X	X	X	H	QA THRU QH OUTPUTS DISABLE
X	X	X	X	L	QA THRU QH OUTPUTS ENABLE
X	X	L	X	X	SHIFT REGISTER IS CLEARED
L		H	X	X	FIRST STAGE OF S.R. BECOMES "L" OTHER STAGES STORE THE DATA OF PREVIOUS STAGE, RESPECTIVELY
H		H	X	X	FIRST STAGE OF S.R. BECOMES "H" OTHER STAGES STORE THE DATA OF PREVIOUS STAGE, RESPECTIVELY
X		H	X	X	STATE OF S.R IS NOT CHANGED
X	X	X		X	S.R. DATA IS STORED INTO STORAGE REGISTER
X	X	X		X	STORAGE REGISTER STATE IS NOT CHANGED

X: DON'T CARE

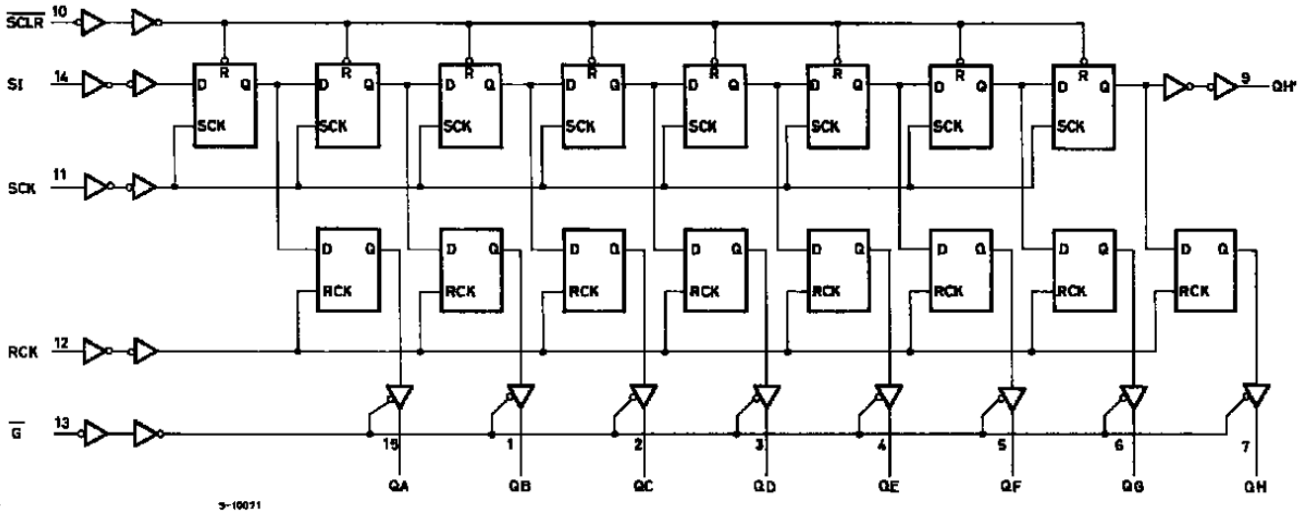
Chức năng 2 tầng thanh ghi là khi dịch chuyển dữ liệu sẽ xảy ra ở tầng thứ nhất không làm ảnh hưởng tầng thứ 2 và không ảnh hưởng đến ngõ ra, chờ cho dịch chuyển xong dữ liệu mới cho phép xuất dữ liệu ra ngoài.

Sơ đồ bên trong của IC thanh ghi dịch 74HC595 như hình 1-5.

Trong sơ đồ trên ta thấy có 2 tầng thanh ghi dùng flip flop D, tầng thứ nhất được điều khiển bởi 3 tín hiệu SCLR, SI và SCK.

Dữ liệu ngõ ra của 8 flip flop D được nối đến 8 ngõ vào của 8 flip flop D ở tầng thứ 2. 8 bit này thay đổi ở bên trong không làm ảnh hưởng đến 8 ngõ ra q khi chưa có xung RCK.

Khi có xung RCK thì 8 bit dữ liệu được đẩy đến 8 ngõ ra Q tương ứng của 8 flip flop ở tầng thứ 2 và nếu được phép bởi tín hiệu G thì dữ liệu sẽ xuất ra ngoài.



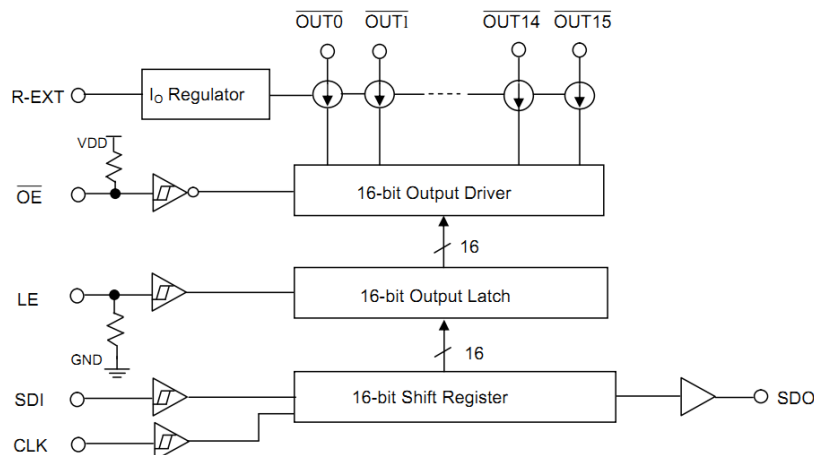
Hình 1-5. Sơ đồ chi tiết bên trong thanh ghi 74HC595.

e. Hoạt động của IC MBI5026:

Sơ đồ khối của vi mạch MBI5026 và bảng trạng thái hoạt động của IC như hình 1-6:

MBI5026 là thanh ghi dịch 16 bit có thể nhận dòng từ 5mA đến 90mA có thể thiết lập được bằng điện trở gắn thêm bên ngoài. Áp có thể hoạt động lên đến 36V phù hợp với các tải công suất như relay, solenoid, ...

- Chân tín hiệu SDI là ngõ vào nhận dữ liệu nối tiếp.
- Chân tín hiệu CLK dùng để nhận xung clock để đẩy dữ liệu vào thanh ghi dịch.
- Chân tín hiệu LE dùng để giống như RCK để nạp dữ liệu từ tầng 1 sang tầng 2.
- Chân tín hiệu SDO dùng để kết nối với thanh ghi dịch tiếp theo.
- Chân tín hiệu OE dùng để mở bộ đệm 3 trạng thái xuất tín hiệu ra ngoài.
- Chân tín hiệu R-EXT dùng để giới hạn dòng tải chạy vào, khi dùng thì nối với một đầu của điện trở và đầu còn lại của điện trở nối mass.
- 16 tín hiệu ngõ ra tích cực mức thấp, chỉ nhận dòng vào chứ không cấp dòng ra.



Hình 1-6. Sơ đồ khối thanh ghi MBI5026.**Bảng 1-4. Bảng trạng thái IC MBI5026.**

CLK	LE	$\overline{OE}$	SDI	$\overline{OUT0} \dots \overline{OUT7} \dots \overline{OUT15}$	SDO
	H	L	D_n	$\overline{D_n} \dots \overline{D_{n-7}} \dots \overline{D_{n-15}}$	D_{n-15}
	L	L	D_{n+1}	No Change	D_{n-14}
	H	L	D_{n+2}	$\overline{D_{n+2}} \dots \overline{D_{n-5}} \dots \overline{D_{n-13}}$	D_{n-13}
	X	L	D_{n+3}	$\overline{D_{n+2}} \dots \overline{D_{n-5}} \dots \overline{D_{n-13}}$	D_{n-13}
	X	H	D_{n+3}	Off	D_{n-13}

f. Hoạt động của vi điều khiển với các thanh ghi và IC chốt:

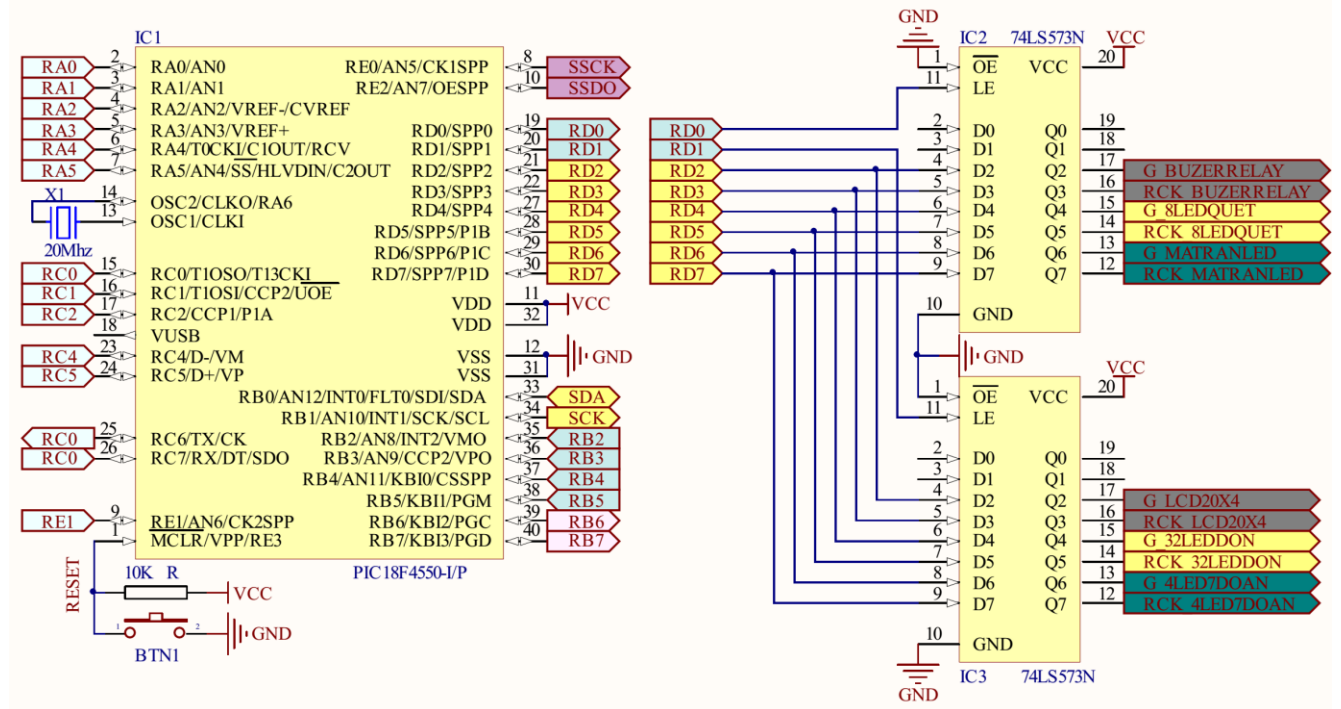
Qua khảo sát 2 loại IC thanh ghi dịch thì khi giao tiếp với vi điều khiển chỉ cần dùng 4 tín hiệu như đã nêu là: SDI, SCK, RCK, G đối với IC 74HC595 và SDI, CLK, LE, OE đối với IC MBI5026, các tín hiệu theo thứ tự thì chỉ khác tên, còn chức năng thì giống nhau.

Hoạt động của vi điều khiển đẩy dữ liệu ra thanh ghi dịch như sau:

- Vi điều khiển dịch dữ liệu nối tiếp ra chân RE2 và được đặt tên SSDO và xuất xung nhịp ở chân RE0 và được đặt tên SSCK để đẩy dữ liệu nối tiếp vào các thanh ghi dịch 74HC595 và MBI5026. Hai tên SSDI và SSCK có thêm chữ S cho khác với 2 tên SDO là SCK là do phần mềm đã định nghĩa cho chuẩn truyền dữ liệu SPI.
- Sau khi dịch xong toàn bộ dữ liệu thì vi điều khiển tạo 1 xung RCK_x để nạp dữ liệu từ thanh ghi dịch sang thanh ghi lưu trữ trong các thanh ghi dịch.
- Vi điều khiển tiến hành điều khiển chân G_x sang mức tích cực để cho phép xuất dữ liệu từ thanh ghi lưu trữ ra ngoài để điều khiển thiết bị.

Sơ đồ nguyên lý giao tiếp vi điều khiển với 2 IC chốt như hình 1-7:

Trong sơ đồ thì tín hiệu OE của IC chốt không dùng nên nối mức 0 để luôn cho phép xuất dữ liệu từ mạch chốt bên trong đến các ngõ ra Q.



Hình 1-7. Sơ đồ giao tiếp port D của PIC 18F4550 với 2 IC chốt mở rộng.

Bảng 1-5. Các tín hiệu mở rộng của IC chốt 74HC573_A.

TT	TÊN TÍN HIỆU	TÍN HIỆU ĐIỀU KHIỂN	MODULE NGOẠI VI
1	RCK_4LED7DOAN	Điều khiển tín hiệu RCK các thanh ghi dịch 74595	4 led 7 đoạn trực tiếp
2	G_4LED7DOAN	Điều khiển tín hiệu G các thanh ghi dịch 74595	
3	RCK_32LEDDON	Điều khiển tín hiệu RCK các thanh ghi dịch 74595	32 led đơn
4	G_32LEDDON	Điều khiển tín hiệu G các thanh ghi dịch 74595	
5	RCK_LCD20X4	Điều khiển tín hiệu RCK thanh ghi dịch MBI5026	LCD 20X4
6	G_LCD20X4	Điều khiển tín hiệu G thanh ghi dịch MBI5026	

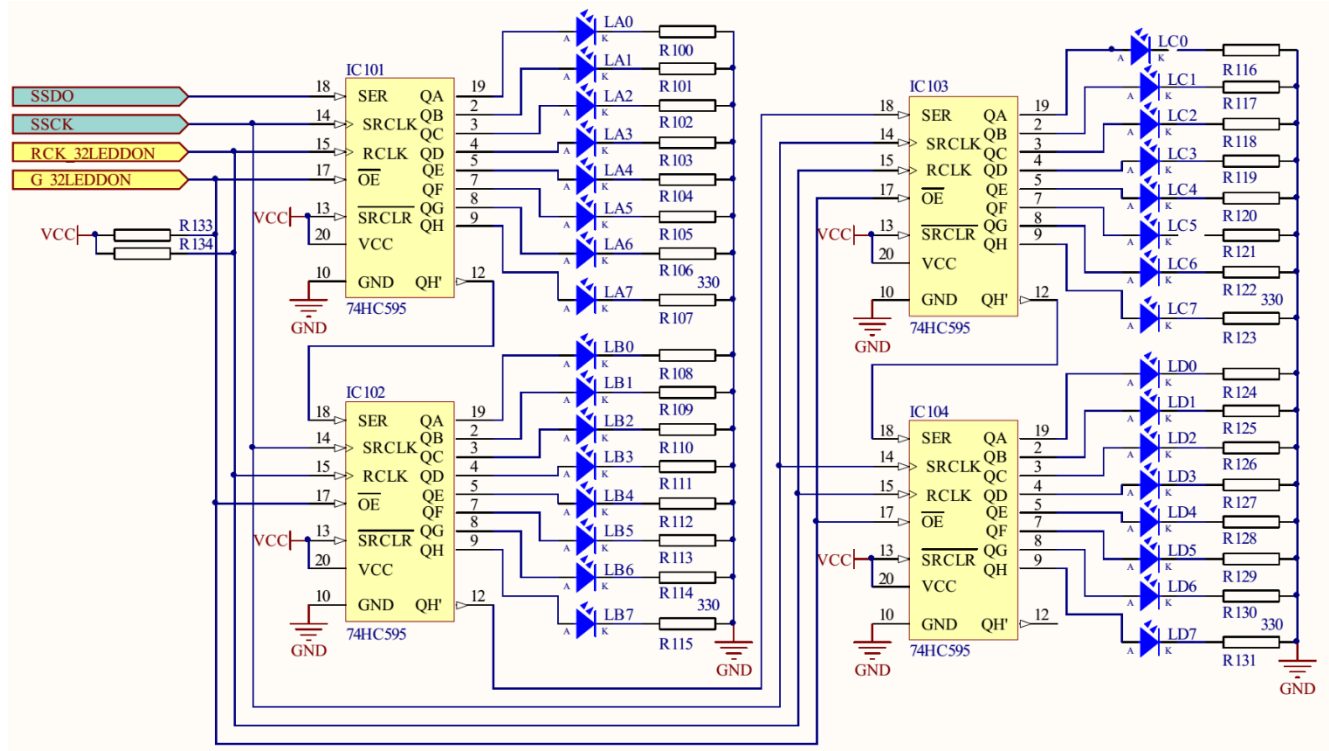
Bảng 1-6. Các tín hiệu mở rộng của IC chốt 74HC573_B.

TT	TÊN TÍN HIỆU	TÍN HIỆU ĐIỀU KHIỂN	MODULE NGOẠI VI
1	RCK_MATRANLED	Điều khiển tín hiệu RCK các thanh ghi dịch MBI5026	3 Led ma trận
2	G_MATRANLED	Điều khiển tín hiệu G các thanh ghi dịch MBI5026	
3	RCK_8LEDQUET	Điều khiển tín hiệu RCK thanh ghi dịch MBI5026	8 led 7 đoạn quét
4	G_8LEDQUET	Điều khiển tín hiệu G thanh ghi dịch MBI5026	
5	RCK_BUZERRELAY	Điều khiển tín hiệu RCK thanh ghi dịch 74595	Relay, buzzer, motor
6	G_BUZERRELAY	Điều khiển tín hiệu G thanh ghi dịch 74595	

1.2.2 Module 1: Giao tiếp với Led đơn

Một trong những ứng dụng đơn giản làm quen với lập trình là điều khiển 32 led đơn với các hiệu ứng quảng cáo từ đơn giản đến phức tạp. Để có thể điều khiển nhiều led sáng thì kit sử dụng IC thanh ghi dịch dữ liệu từ nối tiếp sang song song để điều khiển led như hình 1-8.

Vi điều khiển giao tiếp với module 32 led thông qua 4 IC thanh ghi dịch 74HC595 bằng 4 tín hiệu SSDO, SSCK, RCK_32LEDDON và G_32LEDDON.



Hình 1-8. Sơ đồ nguyên lý module 32 led đơn.

Mức 1 led sáng, mức 0 led tắt.

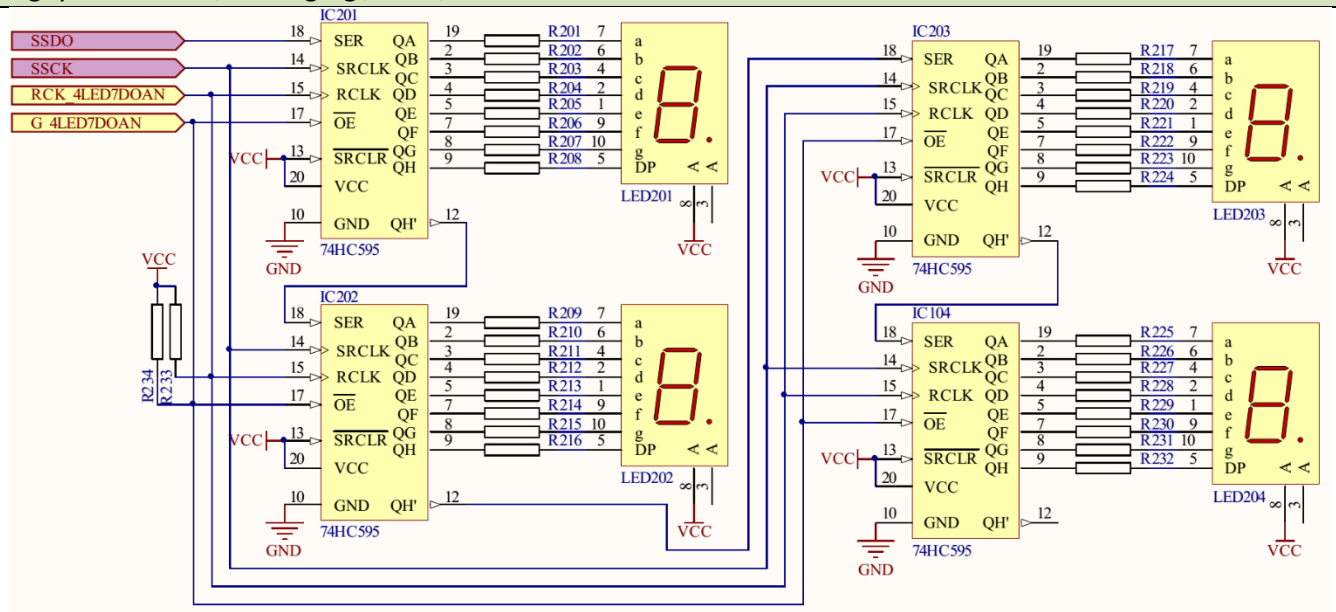
1.2.3 Module 2: 4 led 7 đoạn trực tiếp

Led 7 đoạn dùng để hiển thị số trong nhiều ứng dụng như máy đếm sản phẩm, máy đếm tiền, đèn giao thông, ...

Trong kit này có giao tiếp với 4 led 7 đoạn trực tiếp dùng 4 IC thanh ghi dịch 74HC595 có sơ đồ như hình 1-9.

Vi điều khiển giao tiếp với module 4 led 7 đoạn thông qua 4 IC thanh ghi dịch 74HC595 bằng 4 tín hiệu SSDO, SSCK, RCK_4LED7DOAN và G_4LED7DOAN.

Mức 0 led sáng, mức 1 led tắt.



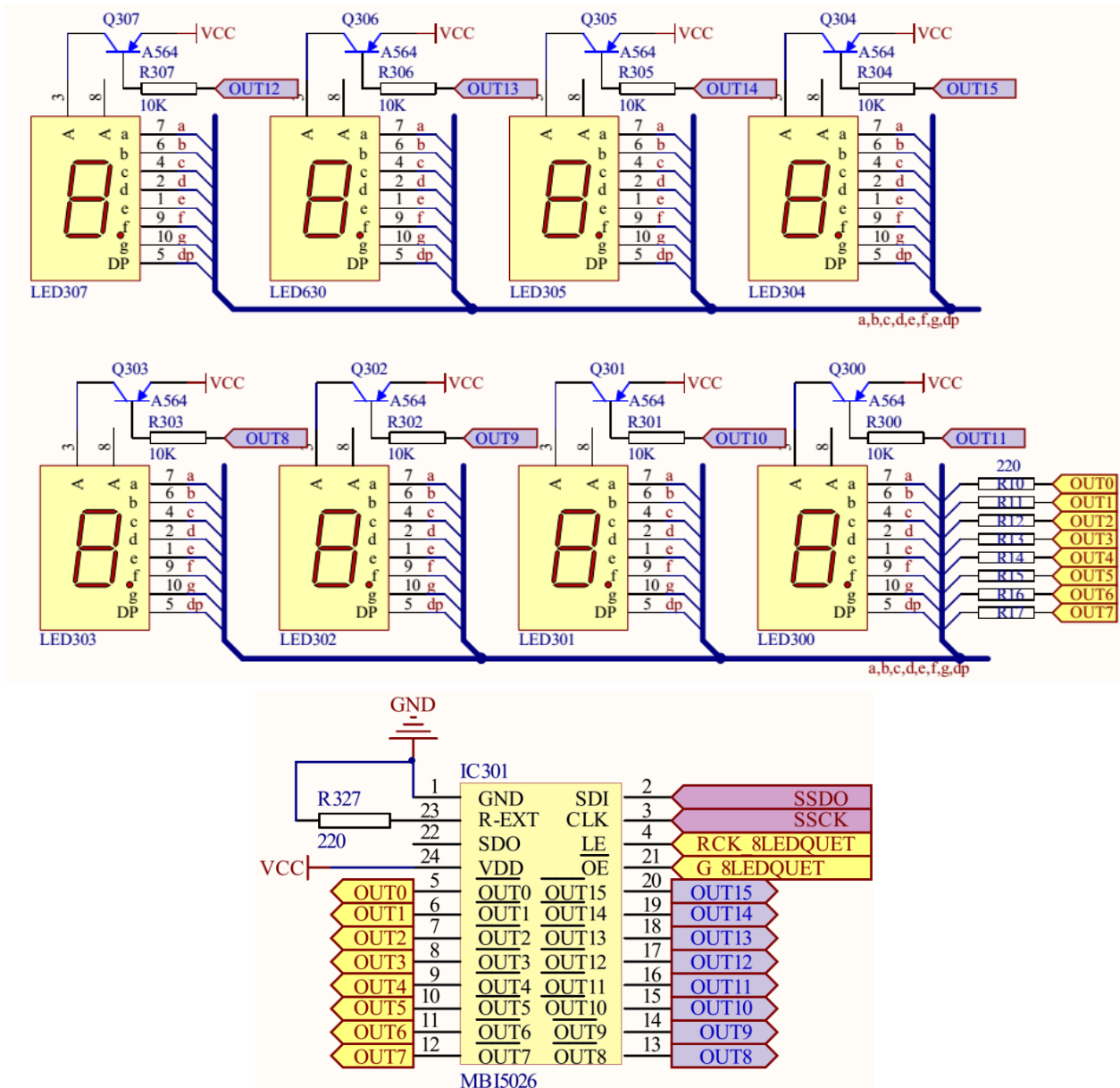
Hình 1-9. Sơ đồ nguyên lý module 4 led 7 đoạn.

Bảng 1-7. Bảng mã led 7 đoạn loại anode chung.

Số hex	dp	g	f	e	d	c	b	a	Mã số hex
0	1	1	0	0	0	0	0	0	C0
1	1	1	1	1	1	0	0	1	F9
2	1	0	1	0	0	1	0	0	A4
3	1	0	1	1	0	0	0	0	B0
4	1	0	0	1	1	0	0	1	99
5	1	0	0	1	0	0	1	0	92
6	1	0	0	0	0	0	1	0	82
7	1	1	1	1	1	0	0	0	F8
8	1	0	0	0	0	0	0	0	80
9	1	0	0	1	0	0	0	0	90
A	1	0	0	0	1	0	0	0	88
B	1	0	0	0	0	0	1	1	83
C	1	1	0	0	0	1	1	0	C6
D	1	0	1	0	0	0	0	1	A1
E	1	0	0	0	0	1	1	0	86

1.2.4 Module 3: 8 led 7 đoạn quét

Phần này trình bày phương pháp điều khiển 8 led bằng phương pháp quét. Trong kit này có giao tiếp với 8 led 7 đoạn trực tiếp dùng 1 IC thanh ghi dịch 16 bit MBI 5026 có sơ đồ như hình 1-10.



Hình 1-10. Sơ đồ nguyên lý module 8 led 7 đoạn quét.

Vì điều khiển giao tiếp với module 8 led 7 đoạn quét thông qua IC thanh ghi dịch MBI 5026 bằng 4 tín hiệu SSDO, SSCK, RCK_8LEDQUET và G_8LEDQUET.

Mức 0 led sáng, mức 1 led tắt.

Trong dữ liệu 16 bit thì 8 bit xuất ra đầu tiên sẽ điều khiển transistor, bit thứ 0 điều khiển led thứ 0 bên phải, bit thứ 1, điều khiển led 1, tương tự bit thứ 7 điều khiển led 7 tận cùng bên trái.

Bit bằng 0 thì transistor dẫn led sáng, bit bằng 1 thì transistor tắt làm led tắt.

8 bit xuất ra sau cùng là mã 7 đoạn hiển thị trên led.

Khi điều khiển 8 led 7 đoạn quét thì phải gọi 2 byte: 1 byte quét led và 1 byte mã 7 đoạn.

IC thanh ghi dịch MBI 5026 dạng cực thu để hở nên phải dùng điện trở kéo lên để điều khiển transistor, các đoạn của led cùng với điện trở hạn dòng đóng vai trò là điện trở kéo lên.

Bảng 1-8. Bảng dữ liệu điều khiển quét lần lượt các transistor.

TT led sáng	Mã quét 8 led								Mã số hex
	Chỉ cho 1 transistor dẫn để 1 led sáng								
	7	6	5	4	3	2	1	0	
7	1	1	1	1	1	1	1	0	FE
6	1	1	1	1	1	1	0	1	FD
5	1	1	1	1	1	0	1	1	FB
4	1	1	1	1	0	1	1	1	F7
3	1	1	1	0	1	1	1	1	EF
2	1	1	0	1	1	1	1	1	DF
1	1	0	1	1	1	1	1	1	BF
0	0	1	1	1	1	1	1	1	7F

1.2.5 Module 4: LCD 20×4

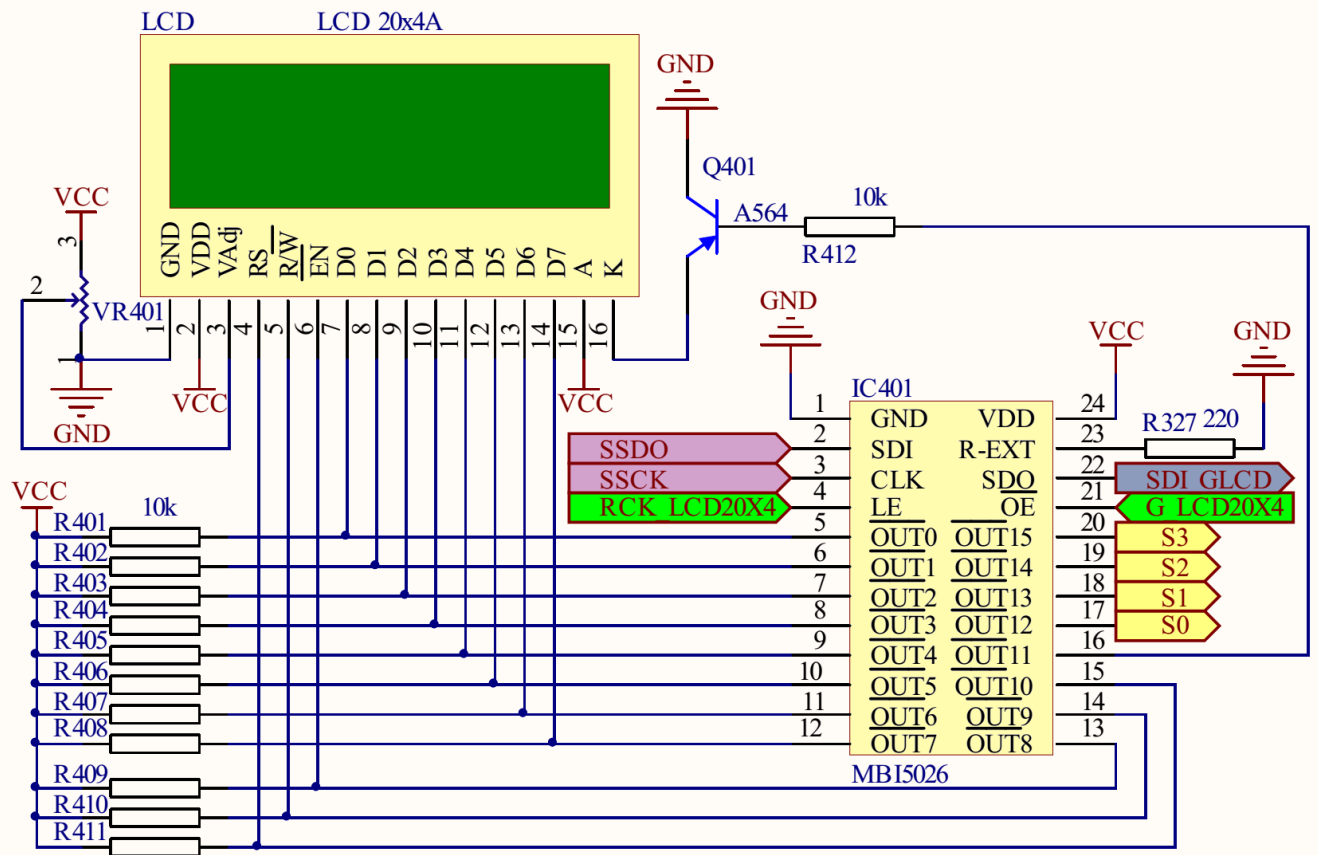
Module LCD dùng để hiển thị nhiều kí tự mã ASCII, kit vi điều khiển có tích hợp LCD 20×4 để có thể hiển thị được 4 hàng và mỗi hàng có 20 kí tự, sơ đồ mạch giao tiếp vi điều khiển với LCD qua IC thanh ghi dịch MBI5026 có sơ đồ như hình 1-11.

MBI có 16 ngõ ra kết nối điều khiển LCD như sau:

- Ngõ ra thứ 0 đến bit thứ 7 dùng để giao tiếp dữ liệu với LCD.
- Ngõ ra thứ 8, 9, 10, 11 là các tín hiệu E, RW, RS và điều khiển transistor nguồn của đèn nền. Mức 0 làm transistor dẫn cho đèn nền sáng, mức 1 hoặc trạng thái tổng trở cao làm transistor tắt, đèn nền tắt.
- Bit thứ 12 đến 15 giao tiếp với cảm biến màu sắc.

Trong thư viện đã định nghĩa sẵn các bit để điều khiển LCD.

Ngõ ra dữ liệu của IC MBI 5026 sẽ nối đến ngõ vào của IC MBI 5026 điều khiển GLCD.



Hình 1-11. Sơ đồ nguyên lý module LCD 20×4.

1.2.6 Module 5: GLCD 128×64

Module GLCD dùng để hiển thị hình ảnh và kí tự, sơ đồ mạch giao tiếp vi điều khiển với GLCD qua IC MBI5026 có sơ đồ như hình 1-12.

MBI có 16 ngõ ra kết nối điều khiển GLCD như sau:

Ngõ ra từ thứ 0 đến thứ 7 làm 8 bit giao tiếp dữ liệu.

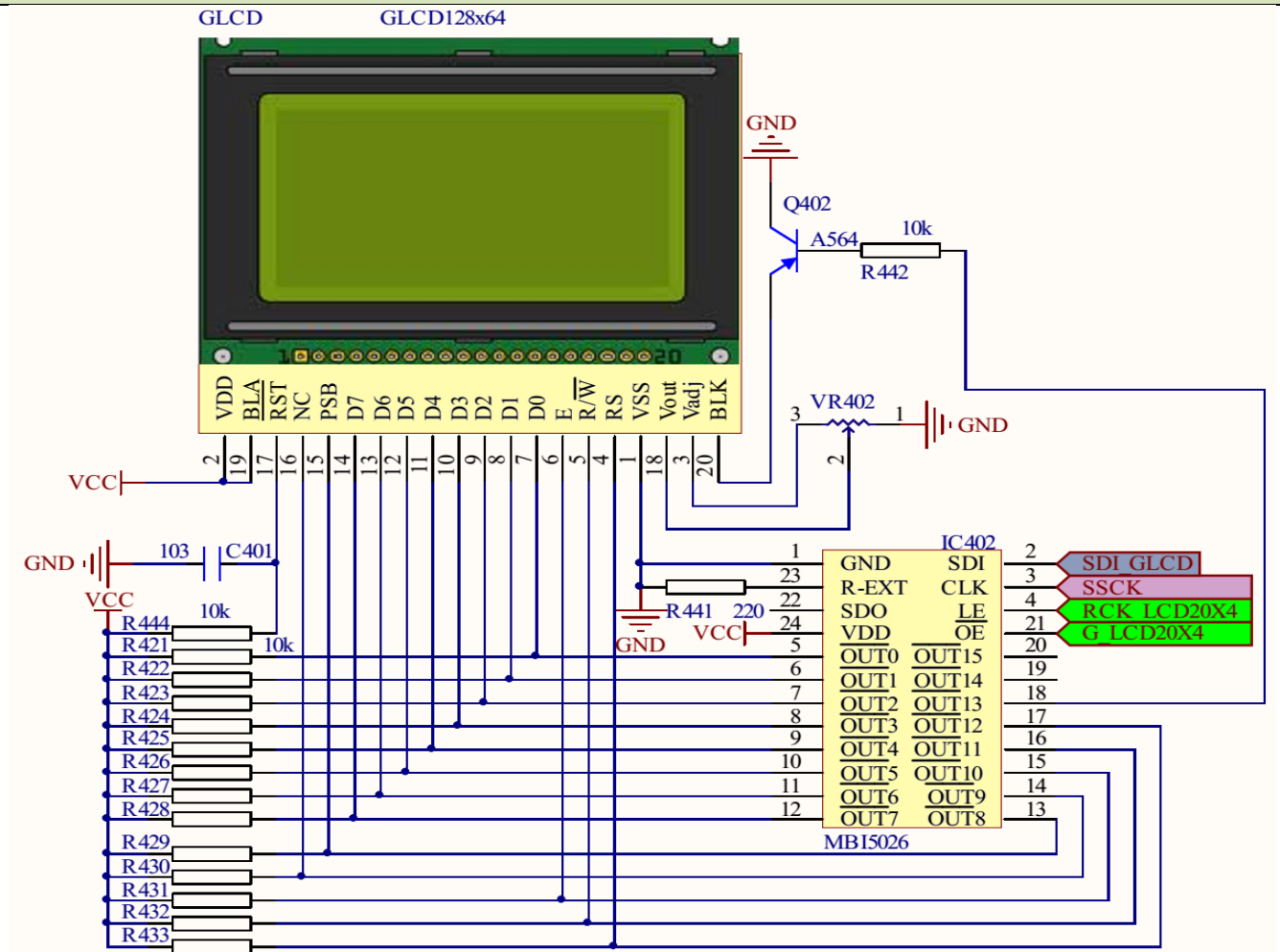
Hai ngõ ra thứ 8, 9 làm 2 tín hiệu chọn chip CS1, CS2.

Ba ngõ ra tiếp theo 10, 11, 12 làm 3 bit điều khiển E, R/W, RS.

Ngõ ra thứ 13 làm bit điều khiển transistor mở nguồn cho GLCD khi cần sử dụng. Mức 0 làm transistor dẫn, đèn anode sáng, mức 1 làm led tắt.

Do LCD và GLCD nối tiếp nhau nên có 3 chế độ sử dụng;

- Chỉ điều khiển LCD
- Chỉ điều khiển GLCD
- Điều khiển cả 2 LCD và GLCD.



Hình 1-12. Sơ đồ nguyên lý module GLCD 128×64.

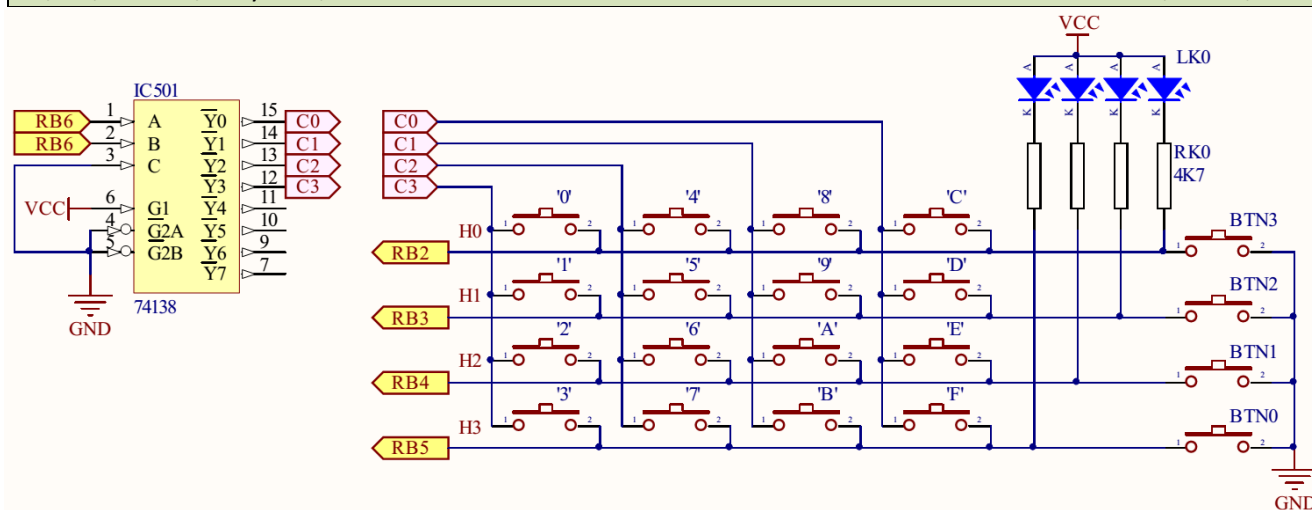
1.2.7 Module 6: phím đơn và phím ma trận 4×4

Để làm quen với cách thức giao tiếp bàn phím thì kit có thiết kế giao tiếp 4 phím nhấn đơn và phím ma trận 4×4 được 16 phím có sơ đồ nguyên lý như hình 1-13.

Vi điều khiển giao tiếp với module 4 nút nhấn đơn BT0, BT1, BT2, BT3 sử dụng 4 bit RB2, RB3, RB4 và RB5. Các tín hiệu đều có điện trở kéo lên nguồn và có led, khi nhấn thì led sáng mức logic là 0, khi không nhấn thì mức logic là 1, led tắt.

Phím ma trận 4×4 sử dụng 4 bit RB2, RB3, RB4 và RB5 đóng vai trò là hàng (H0, H1, H2, H3) có dùng điện trở và led kéo lên nguồn. Hai bit RB6, RB7 qua IC giải mã tạo thành 4 ngõ ra làm 4 cột C0, C1, C2 và C3.

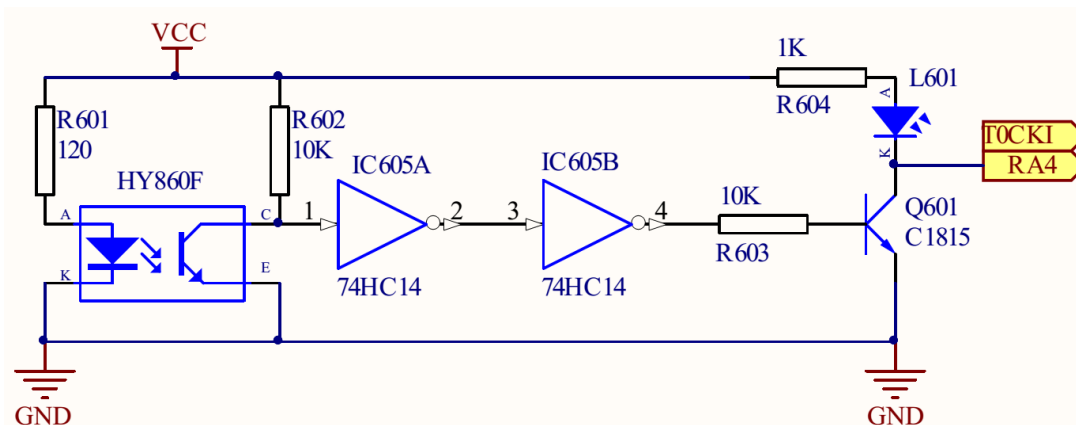
Tổ hợp 4 hàng và 4 cột dùng để quét bàn phím ma trận.



Hình 1-13. Sơ đồ nguyên lý module phím nhấn đơn và phím ma trận.

1.2.8 Module 7: thu phát hồng ngoại tạo xung

Kit có tích hợp module thu phát hồng ngoại tạo xung khi có sản phẩm che chắn giữa led phát và led thu, có led sáng tắt để báo hiệu, ngõ ra của xung đưa đến timer/counter để thực hành các bài đếm xung ngoại dùng counter. Sơ đồ mạch như hình 1-14.



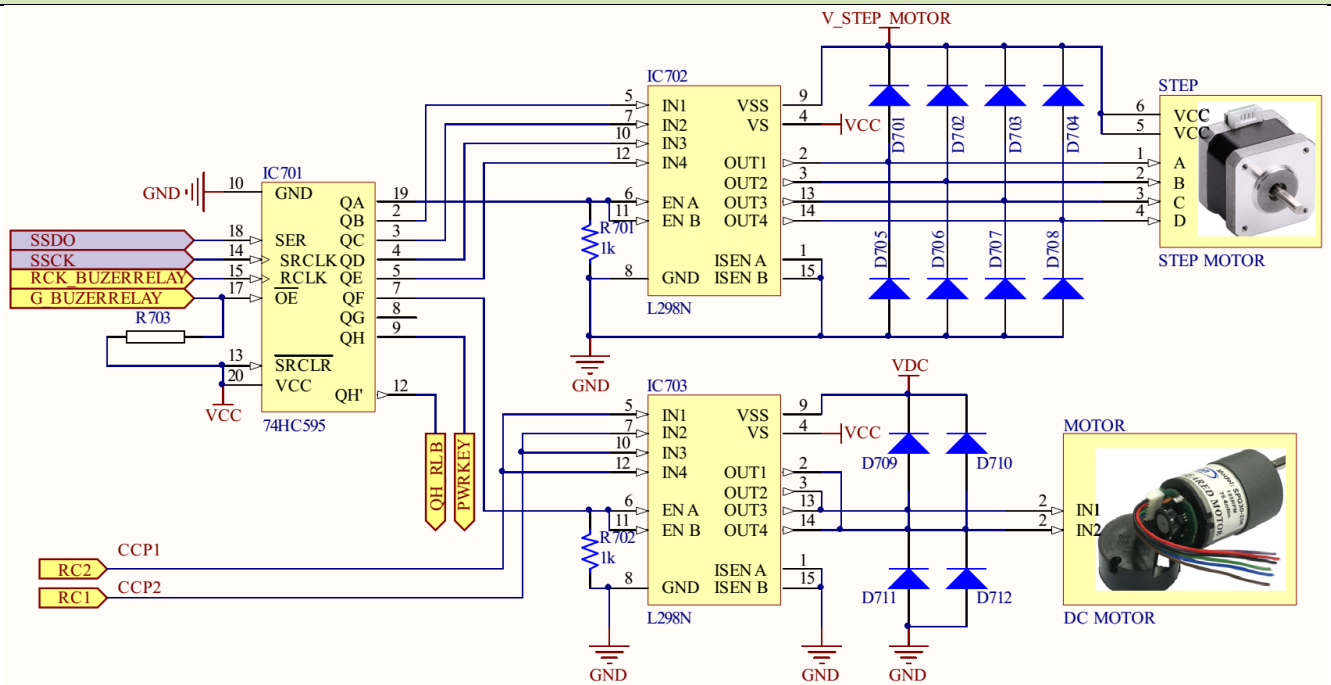
Hình 1-14. Sơ đồ nguyên lý module thu phát hồng ngoại tạo xung.

1.2.9 Module 8: giao tiếp relay, buzzer và động cơ

Kit có tích hợp module giao tiếp điều khiển động cơ bước và động cơ DC, Triac, Relay, Buzzer thông qua 2 thanh ghi dịch 74HC595.

IC thanh ghi dịch 74HC595 thứ nhất có sơ đồ mạch như hình 1-15.

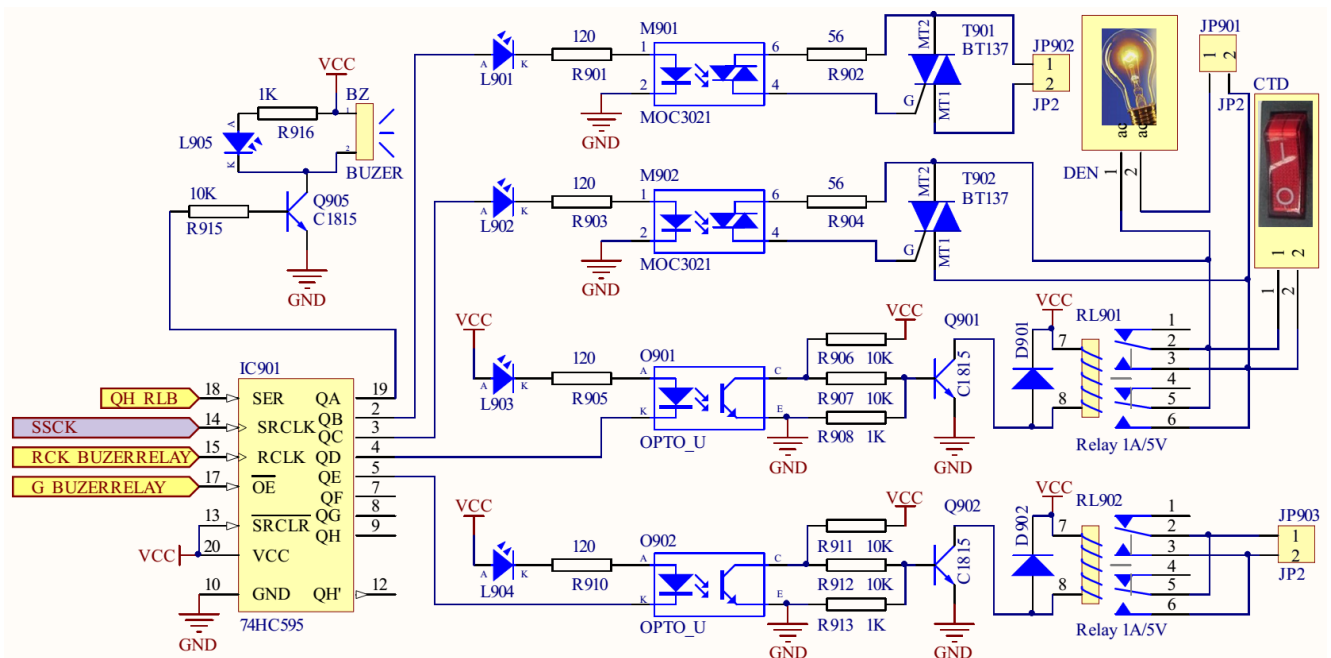
- Ngõ ra từ QA đến QE giao tiếp với IC công suất L298 để điều khiển động cơ bước.
- Ngõ ra QF để giao tiếp với IC công suất L298 để điều khiển cho phép động cơ DC cùng với 2 tín hiệu CCP1 và CCP2 của vi điều khiển để điều khiển động cơ: chạy ngừng, thay đổi tốc độ.
- Ngõ ra QH để giao tiếp điều khiển nguồn của SIM900.
- Ngõ ra QH* để chuyển dữ liệu sang IC 74HC595 thứ 2.



Hình 1-15. Sơ đồ nguyên lý module điều khiển động cơ bước và động cơ DC.

Động cơ DC sử dụng nguồn 24V, động cơ bước dùng nguồn 6.7V nên sử dụng nguồn điều chỉnh bằng LM2576-ADJ, áp được điều chỉnh cho phù hợp với động cơ bước.

IC thanh ghi dịch 74HC595 thứ hai có sơ đồ mạch như hình 1-16.



Hình 1-16. Sơ đồ nguyên lý module điều khiển Relay, Triac, tải 220V AC.

- Ngõ ra QA điều khiển tắt mở Buzzer, mức 1 làm Buzzer kêu, mức 0 làm Buzzer tắt.
- Ngõ ra QB, QC điều khiển tắt mở 2 opto triac MOC3021 để điều khiển triac công suất, ngõ ra triac nối với domino.

- Ngõ ra QD, QE điều khiển tắt mở 2 relay để điều khiển relay công suất, ngõ ra relay nối với domino.

Chú ý: trong sơ đồ thì có dùng 1 Relay và 1 Triac nối với tải là bóng đèn 25W qua một contact (CTD) để điều khiển đèn sáng hay tắt làm gia tăng nhiệt độ cho các cảm biến nhiệt.

Ở các bài đo nhiệt độ dùng cảm biến nếu muốn thấy sự thay đổi nhiệt độ thì mở công tắt đèn (CTD) làm đèn sáng để tăng nhiệt cho đến nhiệt độ mong muốn, 4 cảm biến nhiệt được đặt gần bóng đèn để hấp thụ nhiệt.

Ở các bài đo nhiệt độ dùng cảm biến tự động tắt khi quá nhiệt và tự động mở khi nhiệt độ thấp hơn thì ta sử dụng Relay để điều khiển bóng đèn.

Động cơ DC có gắn Encoder để tạo xung và xung đưa về counter T1.

1.2.10 Module 9: giao tiếp ADC, thời gian thực, bộ nhớ Eeprom và các cảm biến

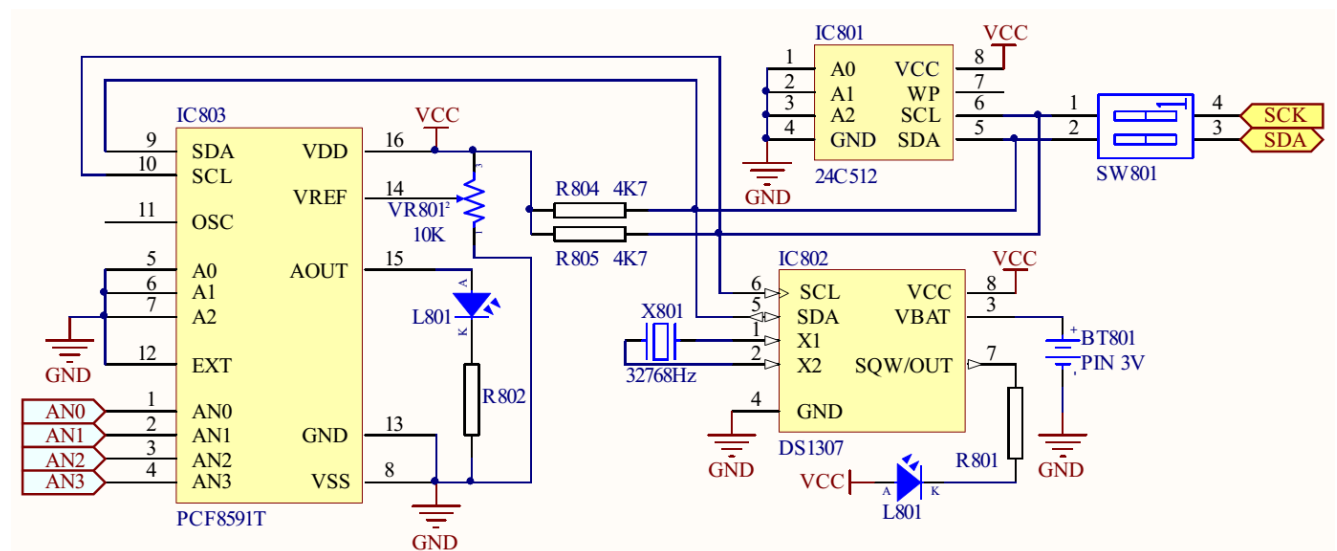
Kit có tích hợp các thiết bị giao tiếp với vi điều khiển theo chuẩn I2C bao gồm: ADC 8 bit PCF8591T, IC thời gian thực DS1307 và IC nhớ Eeprom AT24C512 có sơ đồ mạch như hình 1-17. Các tín hiệu của I2C là SDA và SCL sẽ nối với SDA và SCL của vi điều khiển thông qua switch SW801 và có 2 điện trở kéo lên nguồn.

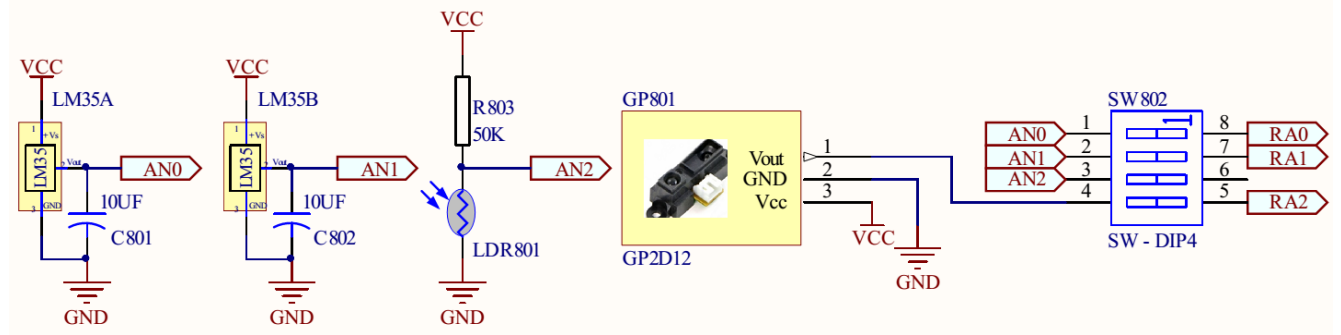
IC thời gian thực có nguồn pin để khi mất điện IC vẫn hoạt động. Ngõ ra SQW/OUT dạng cực thu để hở và được nối với led.

IC PCF8591T có 4 kênh ADC từ AN0 đến AN3. Có 2 cảm biến LM35 nối với kênh AN0 và AN1, cảm biến quang nối với kênh AN2 và cảm biến khí ga ở phần sau thì nối với kênh AN3.

Hai cảm biến nhiệt LM35 và cảm biến khoảng cách GP2D12 còn nối với ADC của vi điều khiển PIC 18F4550 thông qua switch SW802.

PCF8591T còn có 1 kênh DAC với ngõ ra là AOUT nối với led và có biến trở VR801 để chỉnh độ phân giải cho khối ADC và DAC.





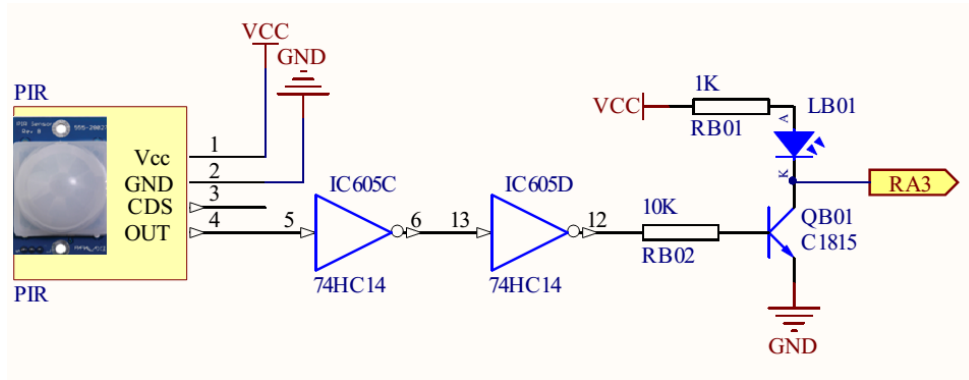
Hình 1-17. Sơ đồ nguyên lý module giao tiếp I2C và các cảm biến tương tự.

Cảm biến nhiệt LM35 có thêm 1 tụ điện ở ngõ ra để giảm bớt độ nhạy để đo cho ổn định hơn. Cảm biến ánh sáng có điện trở nối lên nguồn có giá trị từ 10kΩ trở lên.

Các cảm biến trên đều là cảm biến có điện áp ra dạng tương tự.

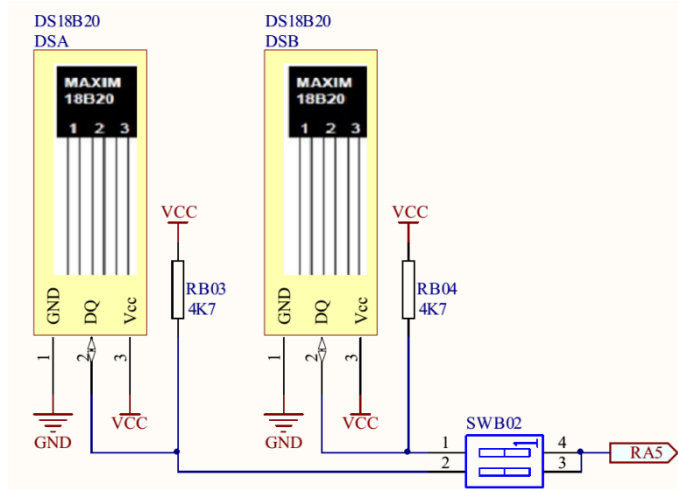
1.2.11 Module 10: giao tiếp các cảm biến nhiệt, cảm biến khoảng cách, led thu

Kit có tích hợp cảm biến phát hiện người chuyển động PIR và có led báo hiệu dùng để làm mạch báo trộm nối với chân tín hiệu AN3/RA3.



Hình 1-18. Sơ đồ nguyên lý module cảm biến hồng ngoại PIR.

Có 2 cảm biến nhiệt tích hợp ADC bên trong xuất nháp tín hiệu bằng 1 dây còn gọi là cảm biến 1 dây (one wire) DS18B20. Do có 2 cảm biến nên phân biệt là A và B, có pin-header 2 chân và có switch SWB02 để có thể nối 1 hoặc 2 cảm biến với 1 chân RA5. Sơ đồ như hình 1-19.



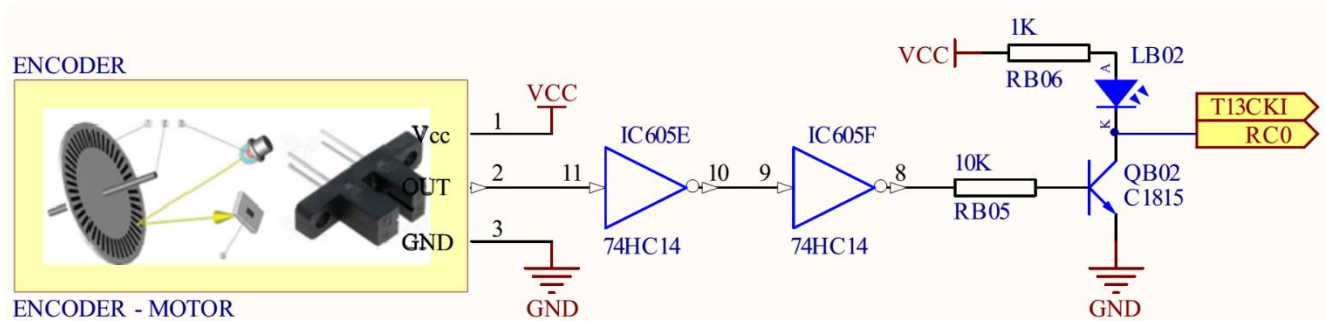
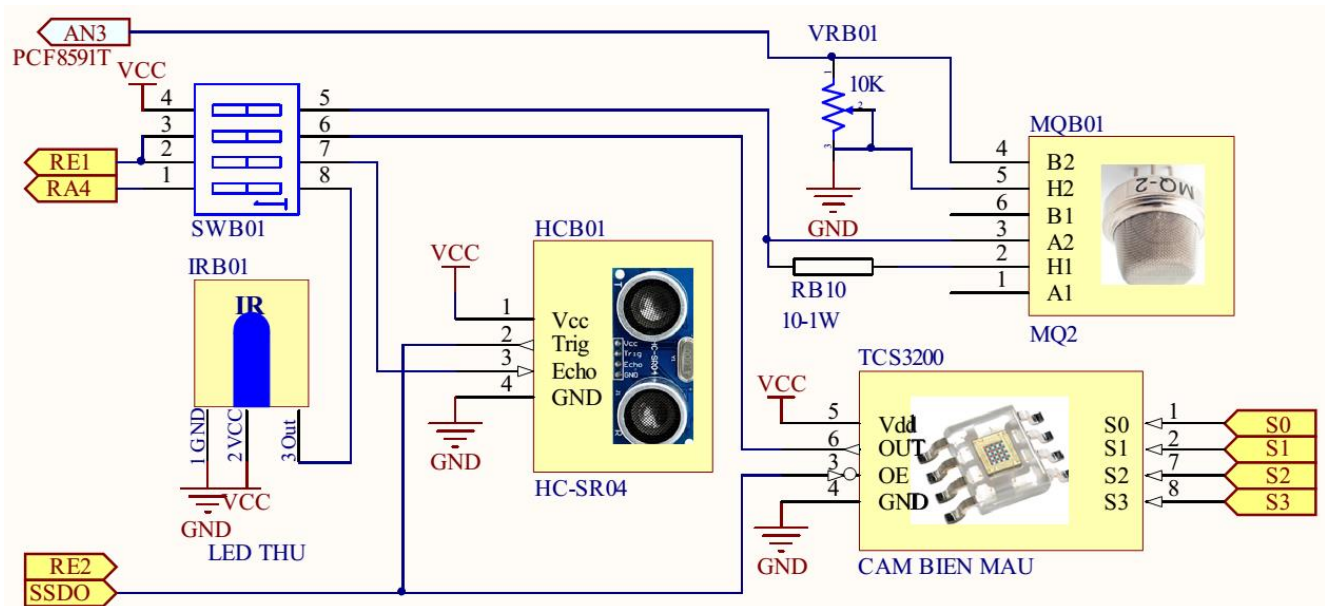
Hình 1-19. Sơ đồ nguyên lý module 2 cảm biến nhiệt DS18B20.

Có các bài thực hành cho 2 cảm biến như sau:

Bài thực hành chỉ dùng 1 trong 2 cảm biến: Với bài dạng này thì không cần biết và không cần đọc mã bộ nhớ của cảm biến vì chỉ có 1 cảm biến. Khi thực hành ở dạng này thì chuyển 1 switch bất kỳ của SWB02 sang vị trí ON, cái còn lại ở vị trí OFF.

Bài thực hành dùng cả 2 cảm biến: với bài dạng này thì cần phải biết code của bộ nhớ của 2 cảm biến, khi truy xuất cảm biến nào thì phải gởi địa chỉ của cảm biến đó mới truy xuất được. Khi thực hành ở dạng này thì chuyển cả 2 switch của SWB02 sang vị trí ON.

Motor DC của kit có gắn sẵn Encoder và được đưa vào mạch Schmitt trigger để hiệu chỉnh tín hiệu rồi đưa vào chân RC0/T1CK13 như hình 1-20.

**Hình 1-20. Sơ đồ nguyên lý module kết nối Encoder của motor.****Hình 1-21. Sơ đồ nguyên lý module cảm biến led thu, siêu âm, khí ga, màu.**

Kit có cảm biến Gas, khi dùng thì chuyển SWB01(4) sang ON để mở nguồn, khi không dùng thì chuyển sang OFF vì dòng tiêu thụ của cảm biến lớn. Ngõ ra của cảm biến được đưa đến khối ADC PCF8591T chân AN3.

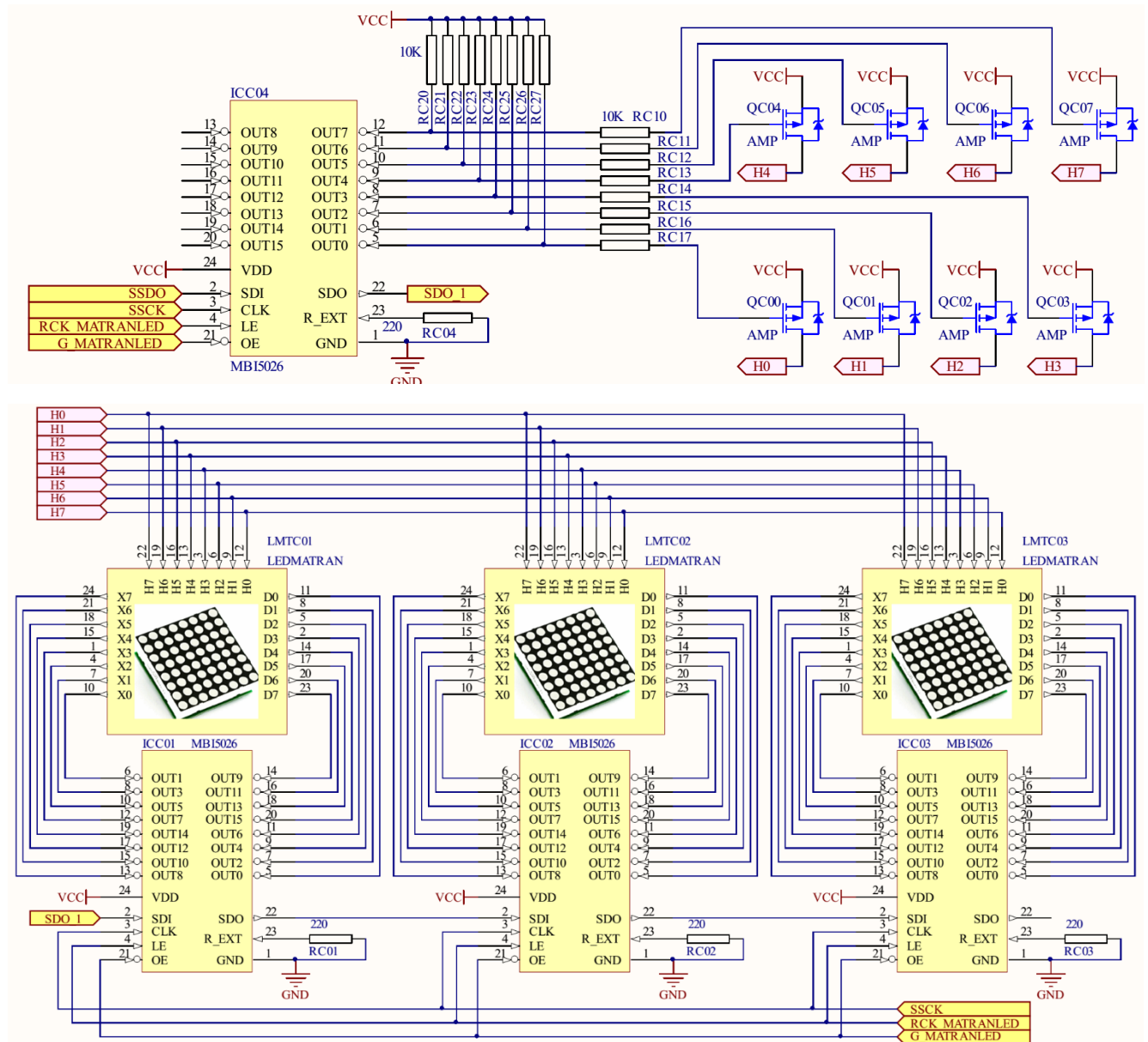
Có led thu hồng ngoại có thể dùng remote để điều khiển từ xa. Khi dùng thì chuyển SWB01(1) sang ON, khi không dùng thì chuyển sang OFF.

Có 1 cảm biến khoảng cách dùng sóng siêu âm cho kết quả khoảng cách tỉ lệ với tần số xung. Khi dùng thì chuyển SWB802(2) sang ON, SWB02(3) sang OFF, khi không dùng thì chuyển sang OFF.

Có 1 cảm màu sắc. Khi dùng thì chuyển SWB02(3) sang ON, SWB02(1) sang OFF, khi không dùng thì chuyển sang OFF.

1.2.12 Module 11: giao tiếp led ma trận

Một trong những ứng dụng phổ biến trong quảng cáo là thông tin được hiển thị trên led ma trận, để giúp người học hiểu được nguyên lý điều khiển led ma trận như thế nào thì trong hệ thống có thiết kế giao tiếp với 3 led ma trận 8x8 hai màu xanh và đỏ. Sơ đồ nguyên lý trình bày ở hình 1-22.



Hình 1-22. Sơ đồ nguyên lý mạch điều khiển 3 led ma trận.

Mỗi led ma trận 8x8 2 màu có tổng cộng 24 tín hiệu điều khiển gồm 8 tín hiệu anode chung và 8 tín hiệu cathode chung cho màu đỏ và 8 tín hiệu cathode chung cho màu xanh.

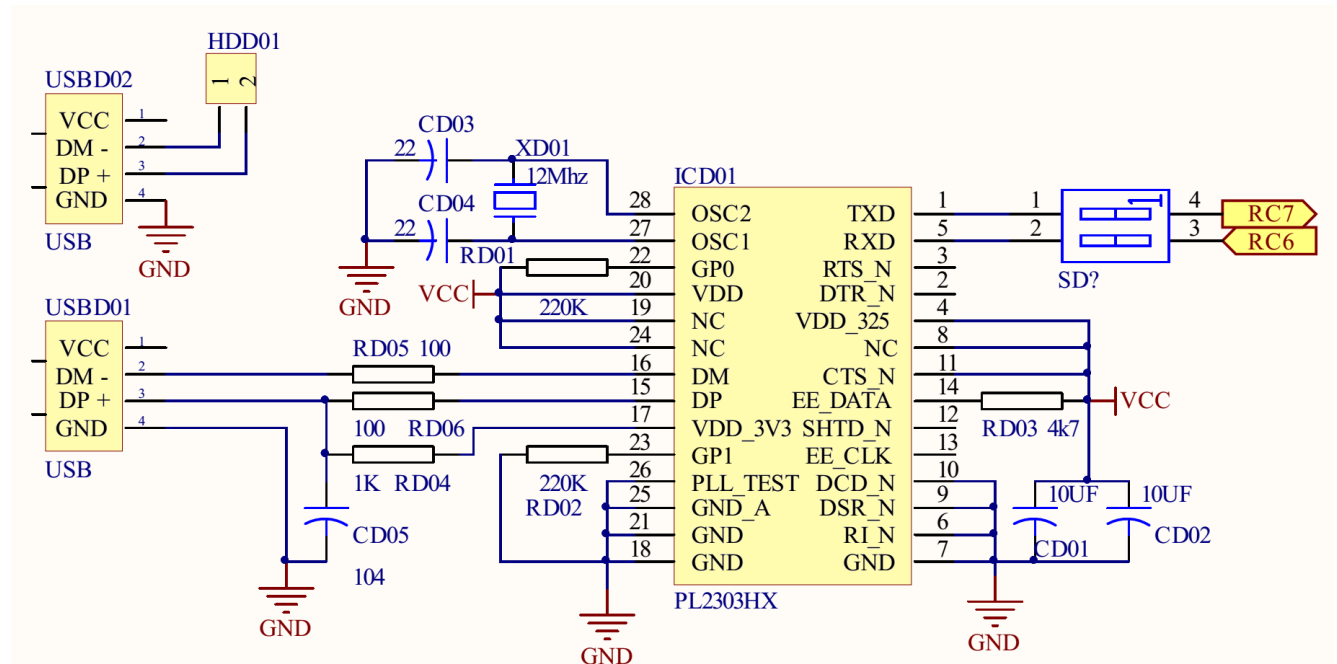
Tín hiệu cathode màu xanh và màu đỏ của mỗi led sẽ do một IC MBI 5026 điều khiển, 3 led sẽ dùng 3 IC MBI 5026.

Tín hiệu anode chung của 3 led được nối chung với nhau và trong sơ đồ được đặt tên là H0 đến H7. Do sử dụng phương pháp quét nên phải khuếch đại dòng bằng transistor Mosfet AMP 4935, tín hiệu từ vi điều khiển được chuyển từ nối tiếp sang song song lấy ra ở 8 ngõ ra từ out0 đến out7 điều khiển 8 transistor để điều khiển anode của 3 led ma trận.

4 IC MBI 5026 được mắc nối tiếp đường dữ liệu SSDO, SSDO của vi điều khiển sẽ kết nối với SDI của IC C04, ngõ ra dữ liệu nối tiếp SDO của IC C01 kết nối với ngõ vào SDI của IC C02, tương tự cho các IC còn lại. Các tín hiệu còn lại SSCK, RCK_MATRANLED, G_MATRANLED thì nối song song.

1.2.13 Module 12: giao tiếp máy tính bằng USB qua IC chuyển đổi RS232

Kit có tích hợp mạch chuyển đổi chuẩn giao tiếp USB thành RS232 do IC PL2303HX để thực hành các bài truyền dữ liệu giữa máy tính và vi điều khiển, sơ đồ nguyên lý trình bày ở hình 1-23.

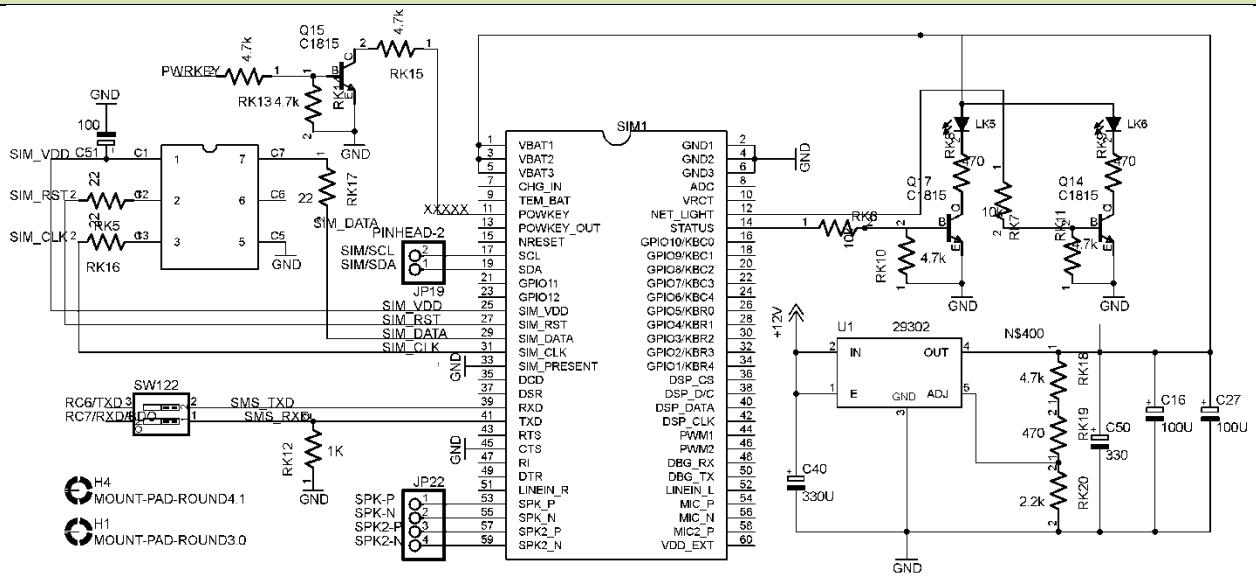


Hình 1-23. Sơ đồ nguyên lý mạch giao tiếp máy tính bằng cổng USB qua IC chuyển đổi.

Cổng USBD01 dùng để giao tiếp với máy tính theo chuẩn RS232, cổng USBD02 nối với pin-header HDD01 để thực hành các bài giao tiếp máy tính với vi điều khiển có tích hợp chuẩn giao tiếp USB như PIC18F4550.

1.2.14 Module 13: giao tiếp SIM900-TE-C-V1.02

Kit có tích hợp mạch giao tiếp module SIM900-TE-C-V1.02, sơ đồ nguyên lý trình bày ở hình 1-24.



Hình 1-24. Sơ đồ nguyên lý mạch giao tiếp SIM900.

Vi điều khiển giao tiếp với SIM thông qua 3 tín hiệu, 1 tín hiệu điều khiển khởi tạo SIM có tên là PWRKEY, 2 tín hiệu giao tiếp truyền dữ liệu theo chuẩn UART là RXD, TXD thông qua SW122 nối đến 2 tín hiệu RC6/TXD và RC7/RXD.

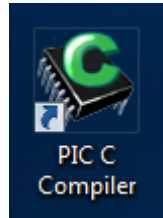
Chương 2: SỬ DỤNG PHẦN MỀM CCS VÀ PICKIT

2.1 GIỚI THIỆU

Phần mềm lập trình cho vi điều khiển PIC có nhiều chương trình khác nhau, tài liệu này trình bày cách sử dụng phần mềm CCS.

Nếu máy tính bạn sử dụng chưa cài phần mềm CCS thì tiến hành copy chương trình CCS vào máy tính và cài đặt theo hướng dẫn của phần mềm.

Biểu tượng phần mềm CCS sau khi cài xong xuất hiện trên desktop như hình sau:



Hình 2-1. Biểu tượng phần mềm CCS.

2.2 HƯỚNG DẪN SỬ DỤNG PHẦN MỀM CCS

Sau khi cài đặt xong thì tiến hành soạn thảo và biên dịch chương trình theo trình tự sau:

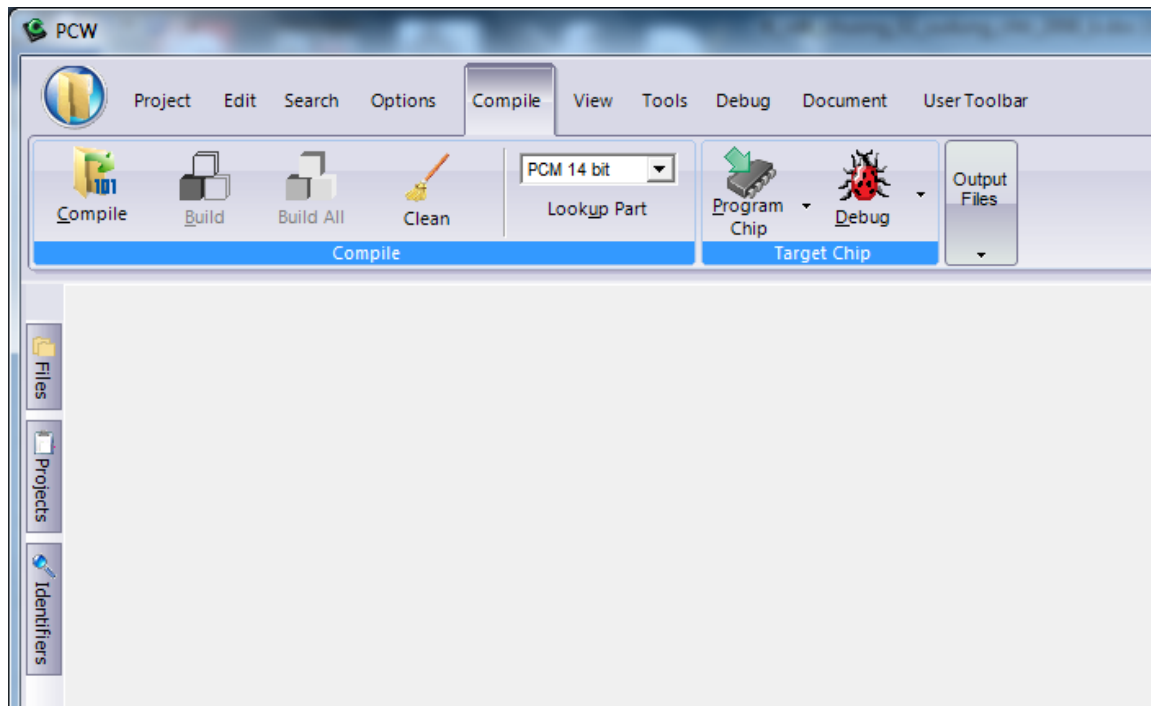
2.2.1 Biên soạn chương trình cơ bản

Bước 1: Nên tạo 1 thư mục để lưu các chương trình lập trình cho vi điều khiển.

Tài liệu này sử dụng thư mục “D:\TH_VDK_PIC_N1_SANG_THU_2”

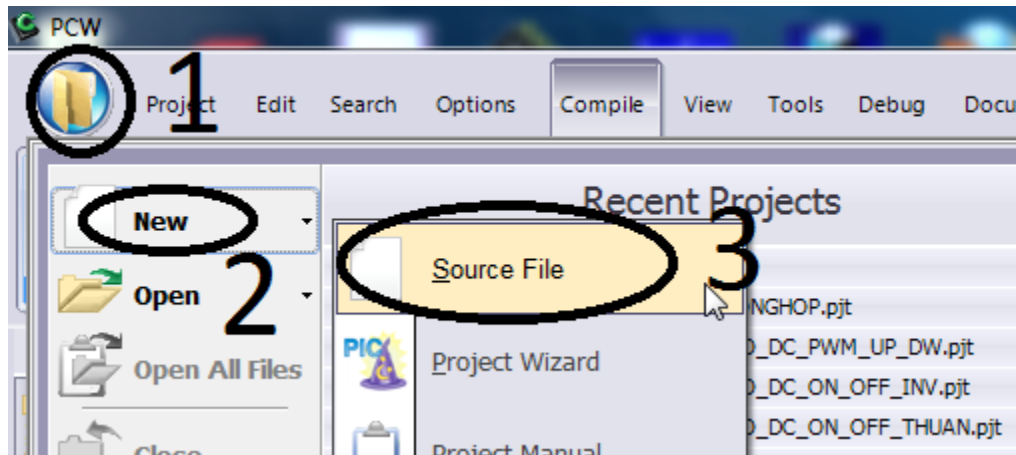
Bước 2: Khởi tạo chương trình CCS, giao diện xuất hiện như hình 2-2.

Thường thì phần mềm sẽ khởi tạo và mở một chương trình soạn thảo sau cùng hoặc mở một ứng dụng có trong phần mềm.



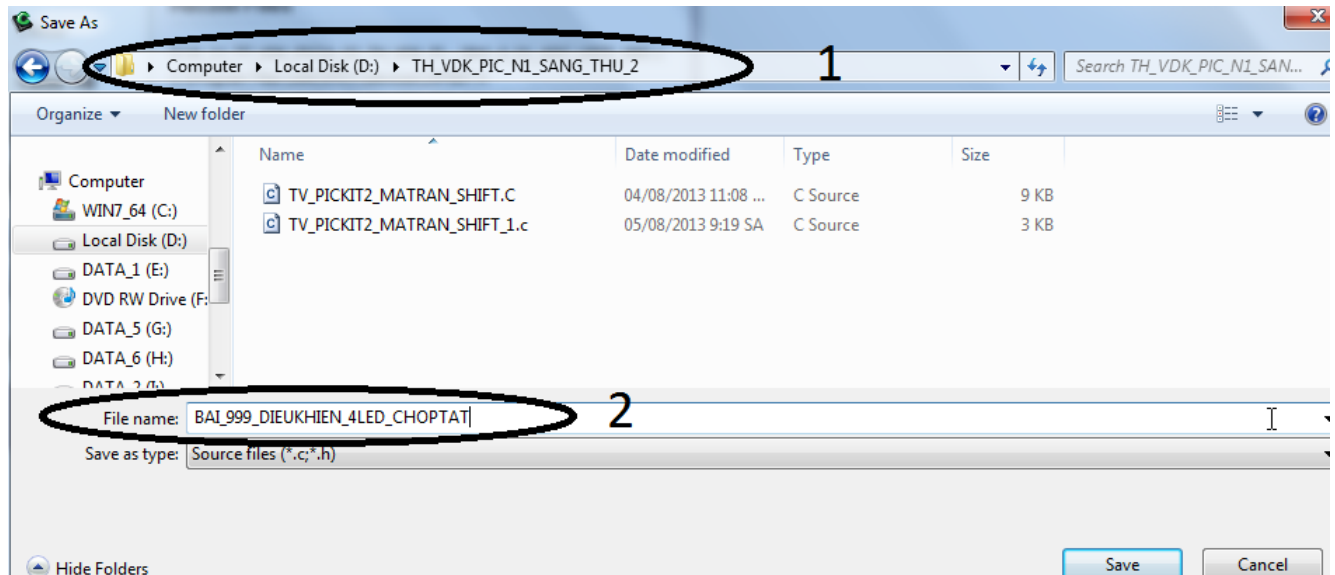
Hình 2-2. Giao diện phần mềm CCS.

Bước 3: Tiến hành chọn biểu tượng open, rồi chọn mục “New” và chọn “New source” theo thứ tự 1, 2, 3 như trong hình 2-3:



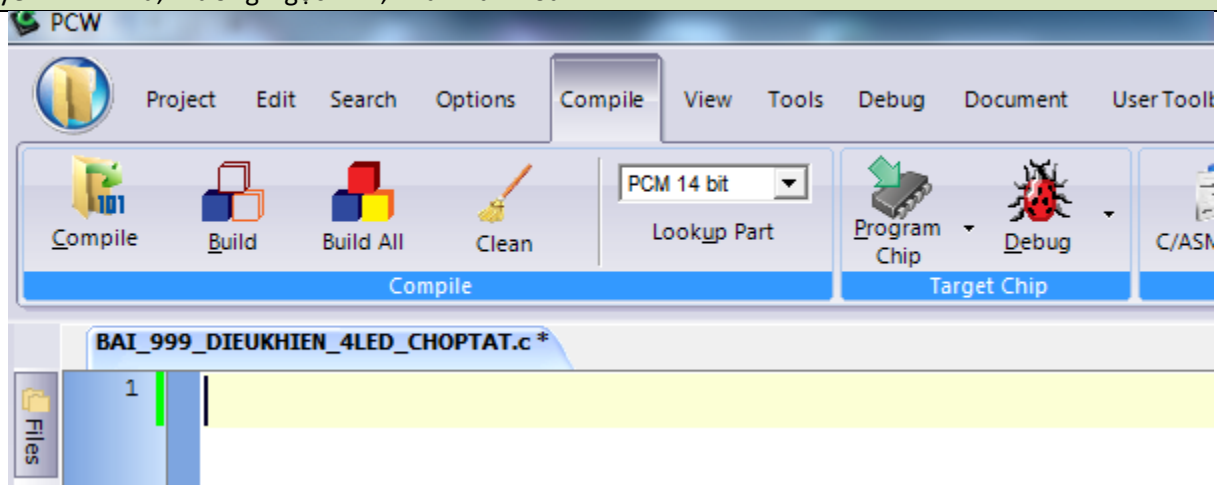
Hình 2-3. Tạo file mới.

Chọn new và chọn thư mục “D:\TH_VDK_PIC_N1_SANG_THU_2” để lưu file nguồn và đánh tên file theo thứ tự 1, 2 như trong hình 2-4:



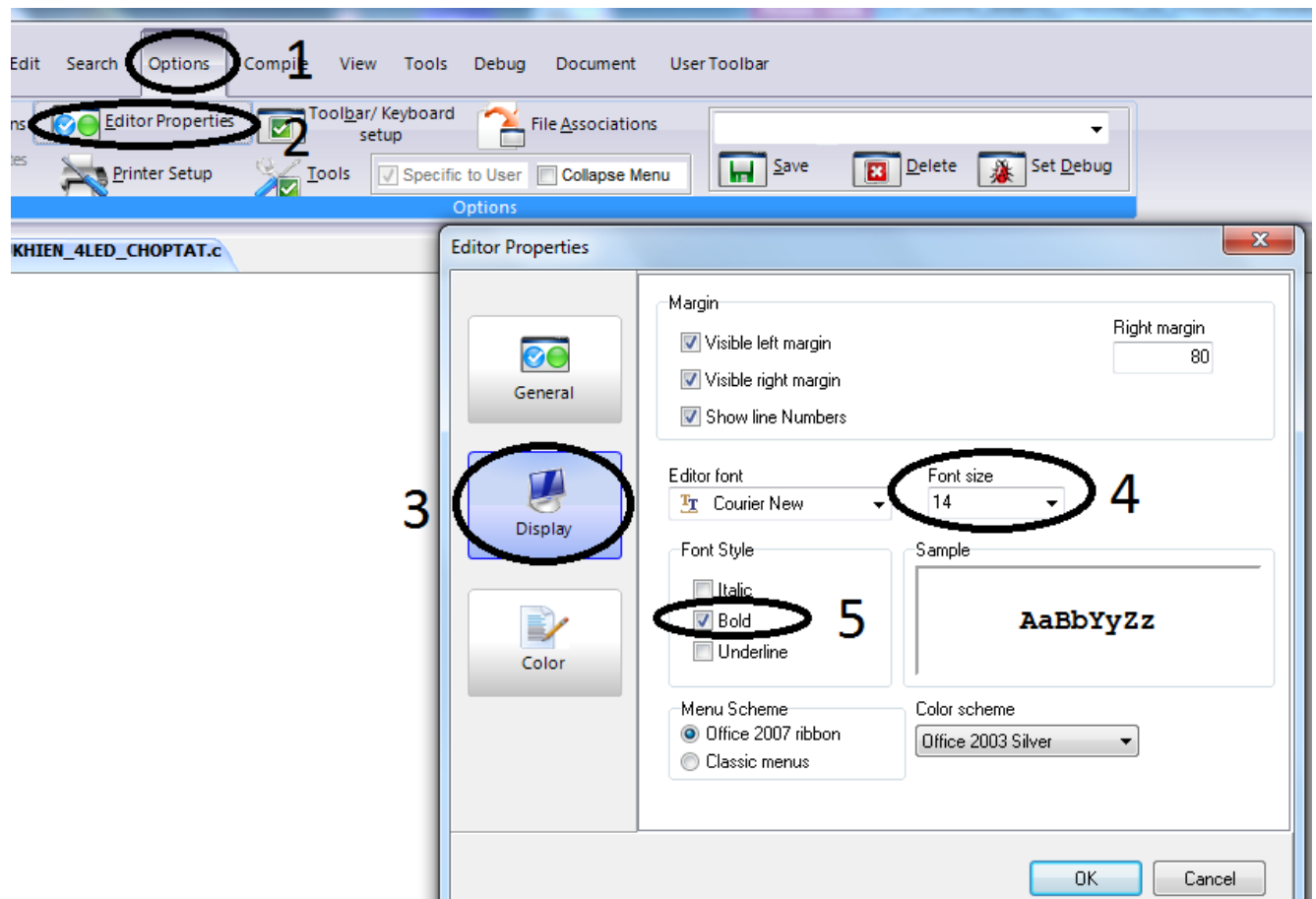
Hình 2-4. Đặt tên file mới và đường dẫn.

Giao diện sau xuất hiện như hình 2-5 và trong file chưa có gì:



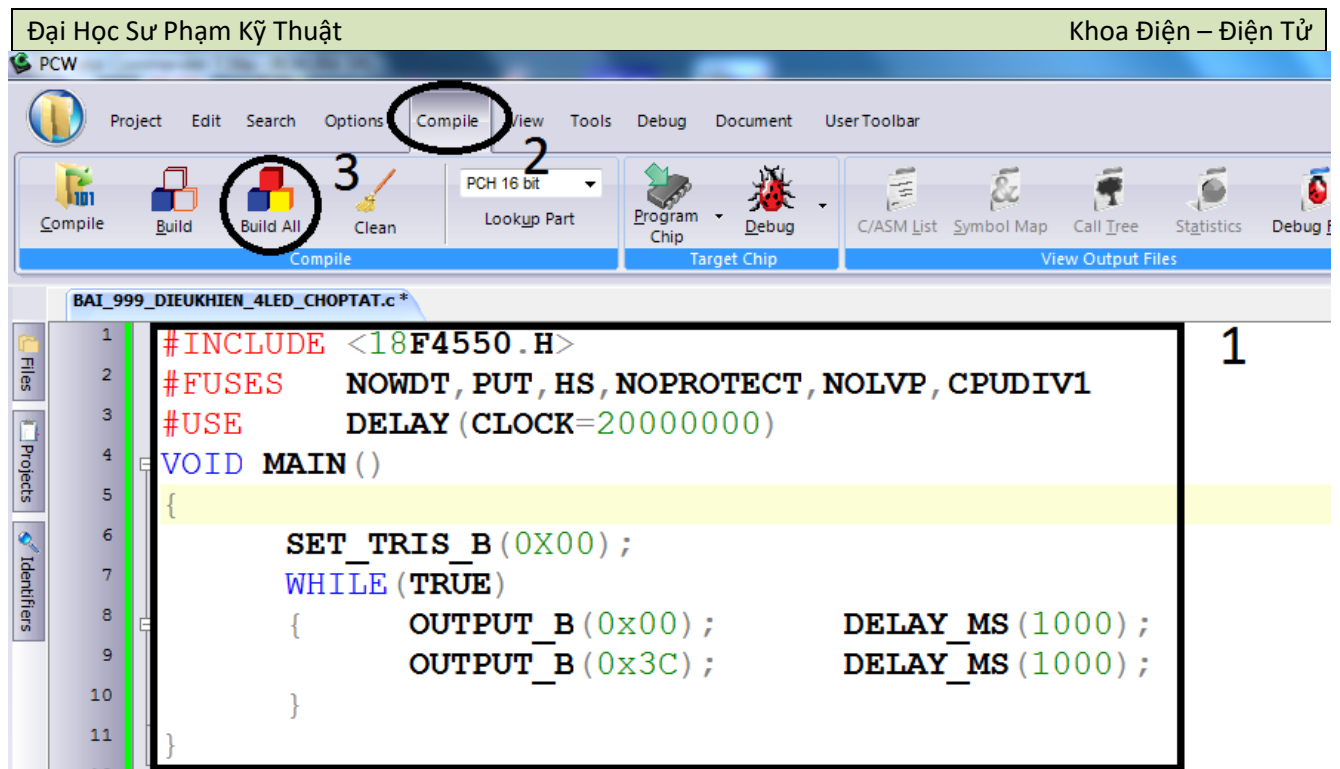
Hình 2-5. Màn hình soạn thảo.

Bước 4: điều chỉnh font chữ cho phù hợp: tiến hành chọn mục “Option” (số 1), chọn “editor Properties” (số 2), chọn tiếp “Display” (số 3), chọn tiếp “Font size” (số 4) cho phù hợp và nên chọn là 14 và chọn mục Bold (số 5), tất cả theo trình tự từ 1 đến 5.



Hình 2-6. Màn hình thay đổi font.

Bước 5: sau khi điều chỉnh xong thì tiến hành biên soạn chương trình chớp tắt đơn giản như hình như hình 2-7:



Hình 2-7. Màn hình của chương trình chớp tắt 4 led.

Tiến hành đánh các lệnh của chương trình trong hình vuông vào (số 1).

Sau khi đánh xong thì nhấn tổ hợp “ctrl” và “S” để lưu file.

Chương trình này có chức năng chớp tắt 4 led nối với 4 bit: bit thứ 2, 3, 4 và 5 của portB – xem hình 1-12 của chương 1. Kit vi điều khiển sử dụng portB giao tiếp với bàn phím ma trận và có kết nối 4 led để báo hiệu – khi không nhấn thì đèn tắt, khi nhấn thì đèn sáng, ta sử dụng 4 led này để thực hiện bài lập trình đầu tiên cho đơn giản.

Trong chương trình thì hành đầu tiên là thư viện của vi điều khiển PIC18F4550, file này chính là file gốc của phần mềm CCS.

Hàng lệnh thứ 2 là khai báo cấu hình: không sử dụng bộ định thời giám sát (No Watchdog Timer: NOWDT), sử dụng bộ định thời kéo dài thời gian reset vi điều khiển thêm 72ms để chờ nguồn điện ổn định sau khi vi điều khiển được mở nguồn (Power Up Timer: PUT), chọn bộ dao động tần số cao từ 4MHz đến 20MHz (High Speed: HS), không bảo vệ code (No protect), không chọn chế độ lập trình điện áp thấp (NOLVP: no low voltage program).

Hàng thứ 3 là khai báo tần số tự thành anh sử dụng là 20MHz.

Ba hàng trên thường là cố định cho tất cả các chương trình, khi viết chương trình khác thì nên copy từ chương trình đã có sang.

Hàng thứ 4 là khai báo chương trình chính.

Các lệnh tiếp theo là của chương trình chính đều nằm trong cặp dấu ngoặc nhọn.

Lệnh “SET_TRIS_B (0x00);” có chức năng định cấu hình cho portB là port xuất dữ liệu ra điều khiển led.

Lệnh WHILE(TRUE) là lệnh lặp có điều kiện và điều kiện cho ở đây luôn đúng và những gì trong vòng lặp while này sẽ thực hiện từ lệnh đầu tiên đến lệnh cuối cùng rồi lặp lại cho đến khi nào mất điện thì thôi – nên thường gọi là vòng lặp vô tận.

Trong vòng lặp while này có 4 lệnh: lệnh “OUTPUT_B(0x00);” là gán portB bằng 00H – kết quả lệnh này làm 8 ngõ ra của portB bằng 0, do led tích cực mức 0 và chỉ có 4 led nên 4 led sáng.

Lệnh tiếp theo là delay 1 giây để ta nhìn thấy led sáng.

Lệnh “OUTPUT_B(0x3C);” là gán portB bằng 3CH – kết quả lệnh này làm 4 ngõ ra của portB bằng 1 làm led tắt.

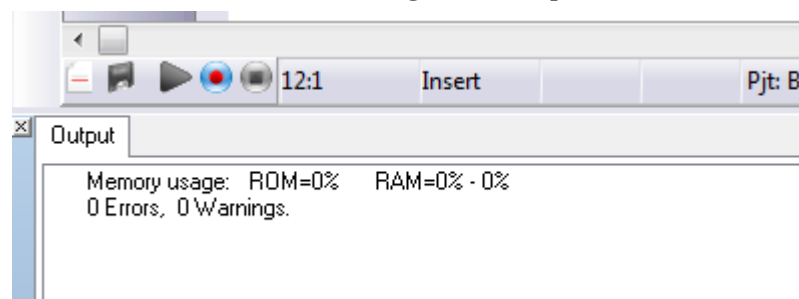
Lệnh tiếp theo là delay 1 giây để ta nhìn thấy led tắt.

Vòng lặp while đã thực hiện xong hết các lệnh và bây giờ sẽ quay lại thực hiện tiếp.

Ngôn ngữ lập trình CCS không phân biệt chữ hoa hay chữ thường.

Tiến hành biên dịch bằng cách chọn tab “Compile” (số 2) và chọn “Build All” (số 3) – xem hình 2-7 hoặc nhấn phím F9.

Sau khi biên dịch sẽ xuất hiện của sổ thông báo kết quả biên dịch như hình 2-8.



Hình 2-8. Màn hình thông báo kết quả biên dịch thành công.

Do lập trình đúng nên khi biên dịch thông báo “0 error ...” và cho biết phần trăm bộ nhớ đã sử dụng.

Nếu sai cú pháp thì biên dịch không thành, phải hiệu chỉnh cho hết lỗi.

Trong thông báo còn cho biết 2 thông tin dung lượng bộ nhớ ROM và RAM đã sử dụng.

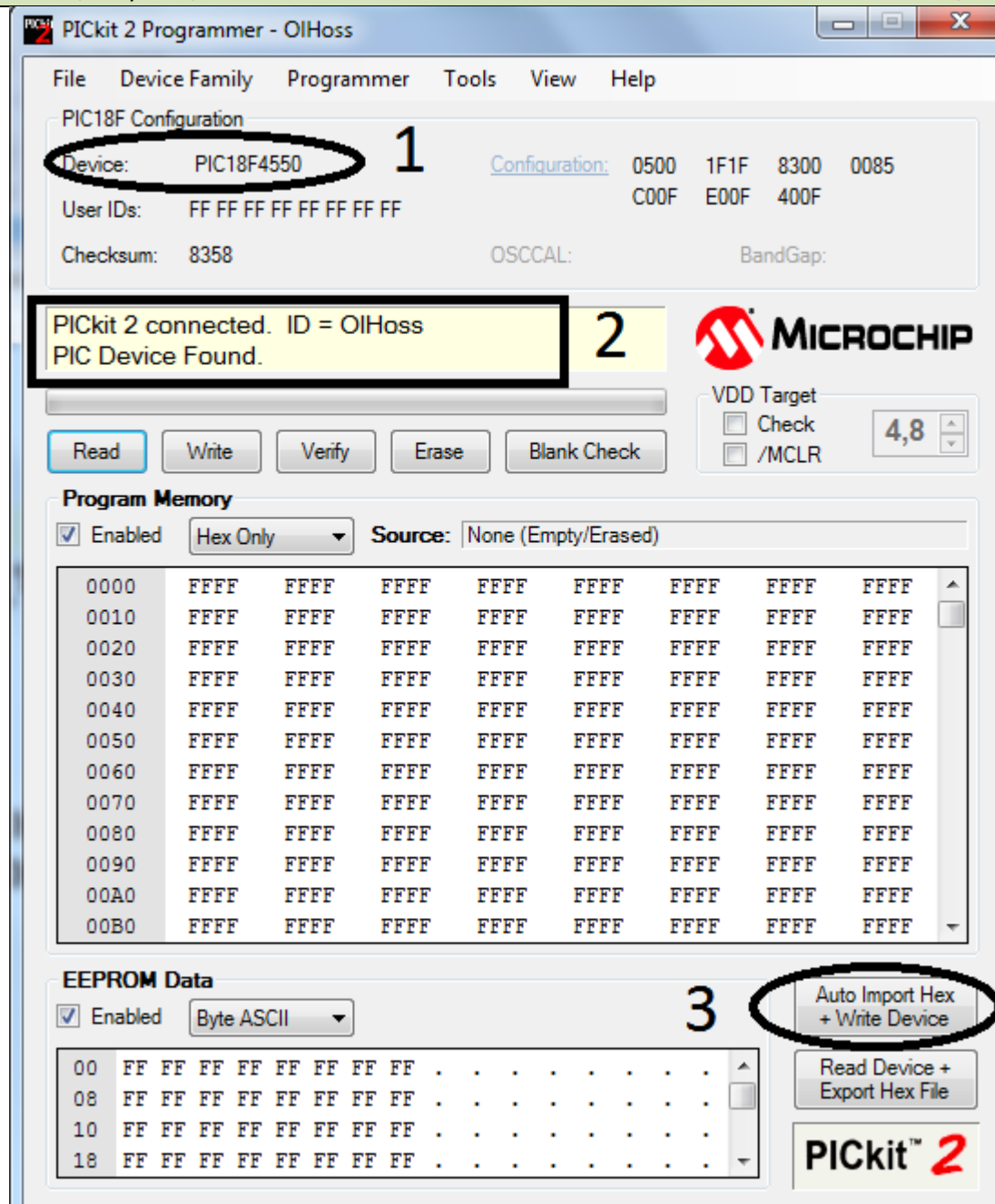
Có thể tắt màn hình thông báo bằng cách nhấn phím “ESC”.

Sau khi biên dịch xong thì sẽ được file chứa các số hex để nạp vào bộ nhớ của vi điều khiển.

Bước theo sau sẽ hướng dẫn nạp file hex vào bộ nhớ của vi điều khiển.

2.2.2 Nạp chương trình vào bộ nhớ vi điều khiển

Bước 1: Mở chương trình: Kit sử dụng mạch nạp của “PICKIT2”, tiến hành cài PICKIT2, kết nối cáp USB bộ thực hành vi điều khiển với máy tính và mở chương trình PICKIT2, sau khi kết nối thành công thì trên màn hình sẽ hiển thị loại CPU đang sử dụng và thông báo kết nối thành công theo trình tự 1, 2 như trong hình 2-9.



Hình 2-9. Màn hình phần mềm nạp PICKIT2.

Bước 2: Mở file hex để nạp: sau khi chọn xong thiết bị thì nhấp chuột vào nút nhấn ở vị trí số 3 trong hình 2-9.

Một màn hình mới xuất hiện và bạn hãy chọn đường dẫn chứa file hex của chương trình biên soạn ở trên rồi chọn file cần nạp và bấm open thì phần mềm sẽ tiến hành nạp.

Sau khi nạp xong sẽ thông báo và vi điều khiển sẽ chạy đúng chương trình điều khiển 4 led chớp tắt.

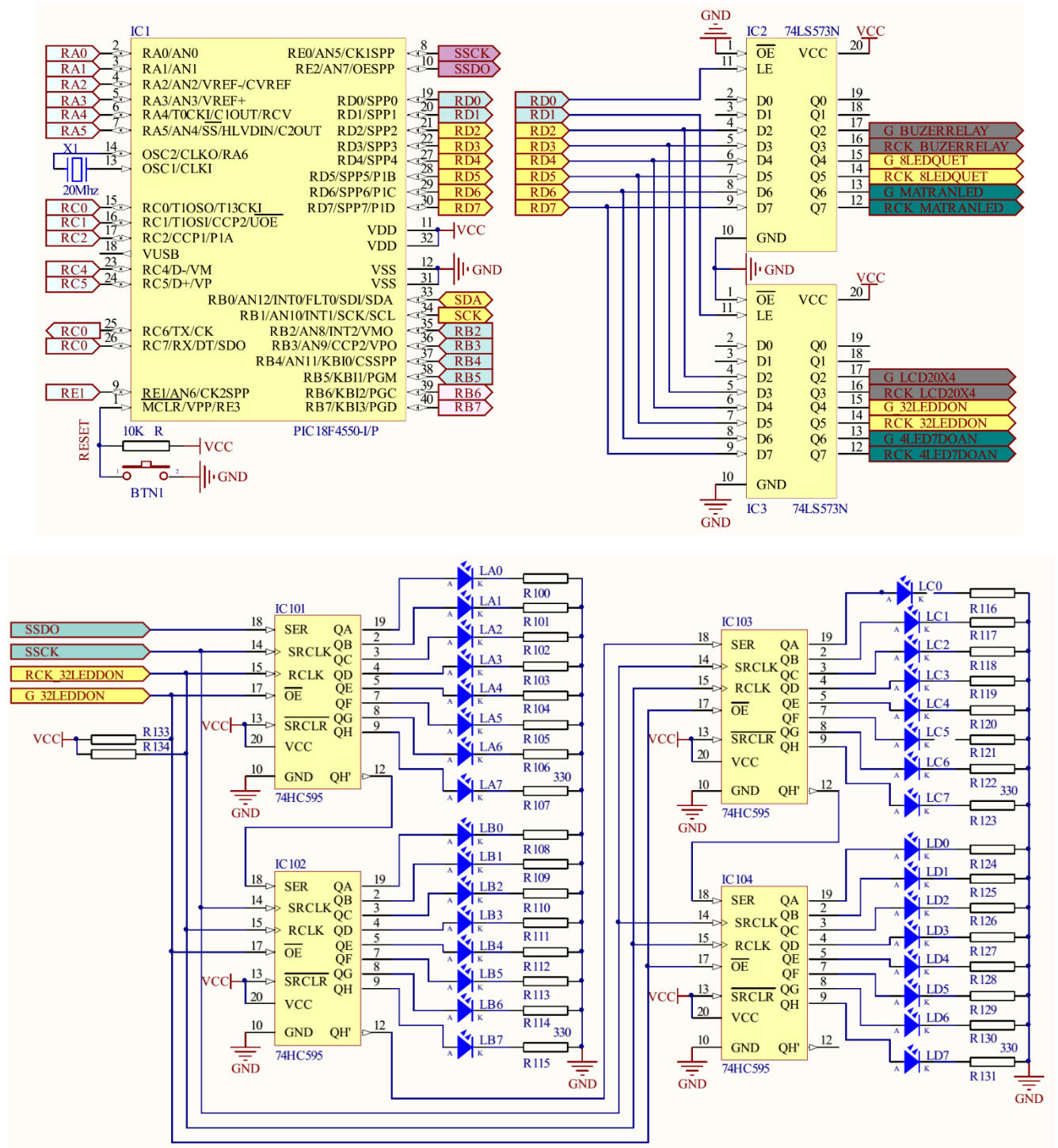
*Chương 3: THỰC HÀNH
MODULE 1– 32 LED ĐƠN
DÙNG THANH GHI DỊCH
74HC595, NÚT NHẤN, BÀN
PHÍM MA TRẬN*

3.1 CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN 32 LED ĐƠN

3.1.1 Mạch giao tiếp vi điều khiển với module 32 led đơn

Phần này thực hành các bài giao tiếp với 32 led đơn từ các bài đơn giản đến các bài phức tạp và tổng hợp nhiều bài lại với nhau.

Mạch điện giao tiếp vi điều khiển với module 32 led đơn như hình 3-1.



Hình 3-1. Sơ đồ nguyên lý giao tiếp vi điều khiển với module 32 led đơn.

Trong mạch sử dụng 4 IC thanh ghi 74HC595 để mở rộng port điều khiển 32 led đơn.

Phần vi điều khiển PIC giao tiếp với IC thanh ghi dịch

Có 4 tín hiệu điều khiển là:

- SSDO dùng để dịch dữ liệu nối tiếp.
- SSCK dùng để cấp xung clock.
- RCK_32LEDDON dùng điều khiển nạp dữ liệu song song từ bên trong IC thanh ghi dịch ra bên ngoài để điều khiển led đơn.
- G_32LEDDON dùng để cho phép xuất dữ liệu.

Phần IC thanh ghi dịch giao tiếp với 32 led

Mỗi IC thanh ghi dịch 74HC595 có 8 bit kết nối với 8 led, 4 IC điều khiển được 32 led.

Số lượng byte dữ liệu điều khiển module 32 led là 4 byte.

Khả năng mở rộng

Trong kit thực hành dùng 4 IC để điều khiển 32 led, sau này các ứng dụng thực tế cần nhiều led hơn thì bạn kết nối thêm IC thanh ghi nối tiếp vào một cách dễ dàng.

Chú ý nếu nhiều IC thanh ghi dịch thì phải tính toán fan-out cho phù hợp.

3.1.2 Các hàm điều khiển module 32 led đơn

Do sử dụng IC chốt để rộng thêm tín hiệu điều khiển các khối và các thanh ghi để mở rộng điều khiển các đối tượng led đơn, led 7 đoạn, LCD, nên chương trình điều khiển sẽ phức tạp hơn so với phương pháp dùng port điều khiển trực tiếp.

Để giúp các bạn hiểu được hệ thống thì ta cần khảo sát các hàm đã viết trong thư viện. Nên xem thêm các định nghĩa trong thư viện.

Hàm thứ 101: khởi tạo port và IC chốt, tắt hết các module:

```
result system_init()
{
    output_high(ALLMODULE_G);
    set_tris_a(0xff);    set_tris_b(0x3c);
    set_tris_c(0xf1);    set_tris_e(0x00);
    set_tris_d(0x00);    output_d(0x00);
    port_b_pullups(0xff);
    private_write_buzzer_relay();
    led32_display();
    d7seg_display();
    s7seg_display();
    write_lcd20x4(0xff,0xff);
    write_glcd128x64(0xff,0xff);
    write_ledmatrix(0,0,0,0);
    output_low(ALLMODULE_G);
    setup_timer_3(t3_internal|t3_div_by_8);
    return OK;
}
```

Khi bắt đầu lập trình thì tại chương trình chính bạn phải gọi hàm “system_init();” để khởi tạo các port điều khiển bao gồm định hướng vào ra cho các port, khởi tạo dữ liệu cho các IC chốt, gửi dữ liệu ra các IC chốt để cấm tất cả các module không được phép xuất dữ liệu. Tắt relay, triac, buzzer và các động cơ.

Trong chương trình chính ngay từ bắt đầu phải luôn gọi hàm này để định cấu hình.

Hàm thứ 102: là hàm xuất dữ liệu từng byte ra các thanh ghi dịch như sau:

```
void SSPI_WRITE(unsigned int8 x)
{
    if(x&0x80) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
    if(x&0x40) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
    if(x&0x20) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
    if(x&0x10) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
    if(x&0x08) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
    if(x&0x04) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
    if(x&0x02) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
    if(x&0x01) ssdo=1;
    else ssdo=0;
    ssck=0; ssck=1;
}
```

Hàm trên sẽ xuất từng bit của byte X được truyền vào ra thanh ghi dịch. Bắt đầu từ bit MSB và kết thúc là bit LSB. Hàm trên tác giả không sử dụng lệnh for nhằm tối ưu về thời gian thực thi giúp hệ thống hoạt động nhanh hơn.

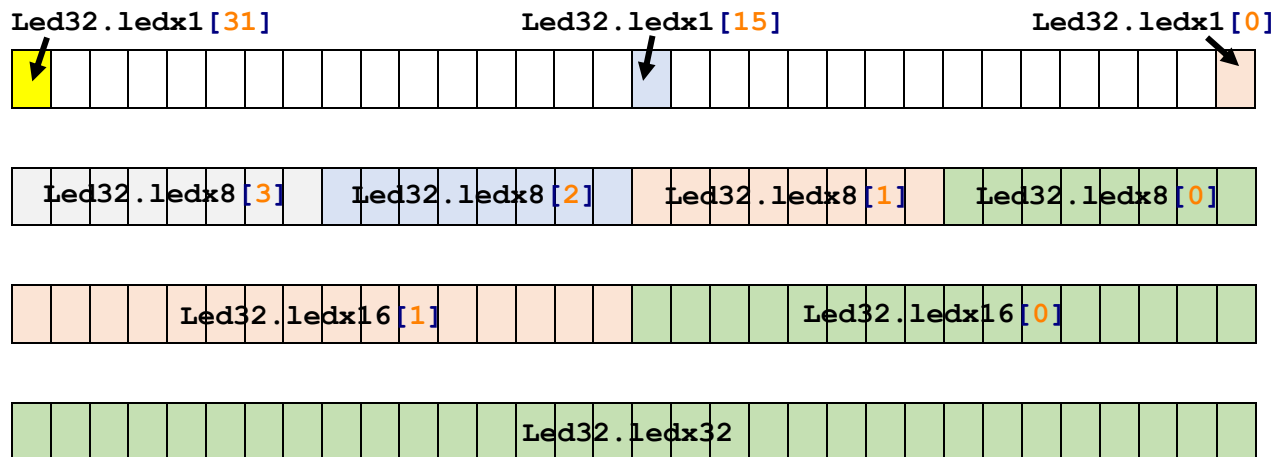
Hàm thứ 301: là hàm xuất dữ liệu ra module 32 led đơn:

```
result led32_display()
{
    SSPI_WRITE(led32.ledx8[3]); SSPI_WRITE(led32.ledx8[2]);
    SSPI_WRITE(led32.ledx8[1]); SSPI_WRITE(led32.ledx8[0]);
    output_high(LED32_RCK); output_low(LED32_RCK);
    return OK;
}
```

Hàm trên có chức năng xuất 4 byte dữ liệu ra module 32 LED đơn. Trong đó biến “led32” là biến thuộc kiểu union như sau:

```
typedef union
{
    int1        ledx1[32];
    unsigned int8 ledx8[4];
    unsigned int16 ledx16[2];
    unsigned int32 ledx32;
} LED32_UNION;
LED32_UNION led32;
```

Với khai báo trên ta có thể truy xuất module 32 LED theo 1, 8, 16 hoặc 32 LED đều được.



Hình 3-4. Các kiểu truy xuất module 32 LED đơn

3.1.3 Các chương trình điều khiển 32 led đơn dùng lệnh for

Phần này thực hành các bài điều khiển led dùng vòng lặp for, các chương trình đơn giản dễ hiểu.

Bài mẫu 301. Chương trình điều khiển 8 LED đơn tận cùng bên phải chớp tắt.
Lưu tên file là “BAI_301_CHOPTAT_8LED”.

- Mục đích: biết cách viết chương trình điều khiển led đơn chớp tắt.
- Chương trình:

```
#include<tv_boards.c>
void main()
{
    system_init();           // Cấu hình Port, tắt các module
    while(true)
    {
        led32.ledx8[0]=~led32.ledx8[0]; // Đảo 8 LED tận cùng bên phải
        led32_display();           // Xuất ra module 32 LED đơn
        delay_ms(200);             // Delay để quan sát
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.

Bài tập 302. Hãy hiệu chỉnh chương trình để điều khiển 16 LED tận cùng bên phải chớp tắt.

Lưu tên file là “BAI_302_CHOPTAT_16LED”.

Bài tập 303. Hãy hiệu chỉnh chương trình để điều khiển 32 LED chớp tắt.

Lưu tên file là “BAI_303_CHOPTAT_32LED”.

Bài mẫu 304. Chương trình điều khiển 8 LED sáng dần và tắt dần phải sang trái.
Lưu tên file là “BAI_304_STD_PST_8LED”.

- Mục đích: biết cách viết chương trình điều khiển 8 led sáng dần và tắt dần phải sang
- Chương trình:

```
#include<tv_boards.c>
unsigned int8 n;
void main()
{
    system_init();
    while(true)
    {
        for(n=0;n<=8;n++)    // Sáng dần từ phải qua trái
        {
            led32.ledx8[0]= ~( 0xff<<n) ;
            led32_display();
            delay_ms(100);
        }
        for(n=0;n<=8;n++)    // Tắt dần từ phải sang trái
        {
            led32.ledx8[0]= 0xff<<n;
            led32_display();
            delay_ms(100);
        }
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- Giải thích chương trình:
 - Chương trình tắt dần:
 - Ban đầu n=0, led32.ledx8[0]= 0xff<<0 = 0b11111111
 - Tiếp đó n=1, led32.ledx8[0]= 0xff<<1 = 0b11111110
 - Tiếp đó n=2, led32.ledx8[0]= 0xff<<2 = 0b11111100
 - Tiếp đó n=3, led32.ledx8[0]= 0xff<<3 = 0b11111000
 - Tiếp đó n=4, led32.ledx8[0]= 0xff<<4 = 0b11110000
 -
 - Cuối cùng n=8, led32.ledx8[0]= 0xff<<8 = 0b00000000
 - Chương trình sáng dần:
 - Nhận thấy trạng thái sáng dần từ phải sang trái là đảo của trạng thái tắt dần từ phải sang trái nên ta chỉ cần thêm lệnh đảo trước “(0xff<<n)” để thành “~(0xff<<n)”

Bài mẫu 305. Chương trình điều khiển 16 LED tận cùng bên phải sáng dần và tắt dần phải sang trái. 8 LED tận cùng bên trái chớp tắt.
Lưu tên file là “BAI_305_STD_PST_16LED_CHOPTAT_8LED”.

- Mục đích: biết cách mở rộng cho 16 led và kết hợp các dạng truy xuất.
- Chương trình:

```

#include<tv_boards.c>
unsigned int8 n ;
void main()
{
    system_init();
    while(true)
    {
        for(n=0;n<=16;n++)
        {
            led32.ledx16[0]= ~(0xffff<<n); // 16 LED sáng dần PST
            led32.ledx8[3]=~led32.ledx8[3]; // 8 LED chớp tắt
            led32_display();
            delay_ms(100);
        }
        for(n=0;n<=16;n++)
        {
            led32.ledx16[0]= 0xffff<<n; // 16 LED tắt dần PST
            led32.ledx8[3]=~led32.ledx8[3]; // 8 LED chớp tắt
            led32_display();
            delay_ms(100);
        }
    }
}

```

- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.

Bài tập 306. Hãy viết chương trình điều khiển 32 LED chia làm 2 nhóm 16 led sáng dần và tắt dần từ trái sang phải cùng lúc.

Lưu tên file là “BAI_306_STD_TSP_16LED_2NHOM”.

Bài mẫu 307. Chương trình điều khiển 32 LED sáng dần và tắt dần trái sang phải.

Lưu tên file là “BAI_307_STD_TSP_32LED”.

- a. Mục đích: biết các viết chương trình điều khiển 32 led sáng tắt dần.
- b. Chương trình:

```

#include<tv_boards.c>
unsigned int8 n;
void main()
{
    system_init();
    while(true)
    {
        for(n=0;n<=32;n++)
        {
            led32.ledx32= ~(0xffffffff >>n); // 32 LED sáng dần TSP
            led32_display();
            delay_ms(100);
        }
        for(n=0;n<=32;n++)
        {

```

```

        led32.ledx32= 0xffffffff >>n;    // 32 LED tắt dần TSP
        led32_display();
        delay_ms(100);
    }
}

```

- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.

Bài mẫu 308. Chương trình điều khiển 2 nhóm 16 LED chạy các hiệu ứng.

1. 16 LED bên trái sáng dần TSP, 16 LED bên phải sáng dịch PST.
2. 16 LED bên trái sáng dần PST, 16 LED bên phải sáng dịch TSP.

Lưu tên file là “BAI_308_SANGDAN_SANGDICH_2x16LED”.

- a. Mục đích: biết các viết chương trình điều khiển 32 led chia làm 2 nhóm.
- b. Chương trình:

```

#include<tv_boards.c>
unsigned int8 n;
void main()
{
    system_init();
    while(true)
    {
        for(n=0;n<=16;n++)
        {
            led32.ledx16[1] = ~(0xffff>>n); // 16 LED trái sáng dần TSP
            led32.ledx16[0] = (int16)1<<n; // 16 LED phải sáng dịch PST
            led32_display();
            delay_ms(100);
        }
        for(n=0;n<=16;n++)
        {
            led32.ledx16[1] = ~(0xffff<<n); // 16 LED trái sáng dần PST
            led32.ledx16[0] = 0x8000>>n; // 16 LED phải sáng dịch TSP
            led32_display();
            delay_ms(100);
        }
    }
}

```

- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.

Bài mẫu 309. Chương trình điều khiển 32 LED đơn sáng dần PST và TSP.

Lưu tên file là “BAI_309_SANGDON_PST_TSP_32LED”.

- a. Mục đích: biết các viết chương trình điều khiển 32 led chia làm 2 nhóm.
- b. Chương trình:

```

#include<tv_boards.c>
unsigned int8 n,m;
void main()

```

```

{
    system_init();
    while(true)
    {
        for(n=0;n<=32;n++)          // Sáng dần PTS
            for(m=0;m<32-n;m++)
            {
                // Sáng dần PST OR Sáng dịch TSP
                led32.ledx32= (~(0xffffffff<<n)) | (0x80000000>>m);
                led32_display();
                delay_ms(50);
            }
        for(n=0;n<=32;n++)          // Sáng dần TSP
            for(m=0;m<32-n;m++)
            {
                // Sáng dần TSP OR Sáng dịch PST
                led32.ledx32= (~(0xffffffff>>n)) | ((int32)1<<m);
                led32_display();
                delay_ms(50);
            }
    }
}

```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.

Bài tập 310. Chương trình điều khiển 32 LED tổng hợp nhiều chương trình gồm:

- Điều khiển 32 LED chớp tắt 5 lần.
- Điều khiển 32 LED sáng dần và tắt dần phải sang trái 2 lần.
- Điều khiển 32 LED chớp tắt 5 lần.
- Điều khiển 32 LED sáng dần và tắt dần trái sang phải 2 lần.
- Điều khiển 32 LED chớp tắt 5 lần.
- Điều khiển 32 LED sáng dần và tắt dần từ ngoài vào 2 lần.
- Điều khiển 32 LED chớp tắt 5 lần.
- Điều khiển 32 LED sáng dần và tắt dần từ trong ra 2 lần.

Lưu tên file là “BAI_310_TONGHOP_32LED”.

3.1.4 Các chương trình điều khiển 32 led đơn dùng lệnh if

Các chương trình dùng lệnh for thì không có vấn đề gì đối với từng chương trình hoặc nhiều chương trình điều khiển led đơn thực hiện một cách tuần tự, tuy nhiên khi hệ thống thực hiện nhiều chương trình với nhiều chức năng khác nhau với yêu cầu đáp ứng nhanh thì các chương trình điều khiển dùng vòng lặp for sẽ không đáp ứng được.

Ví dụ chương trình con điều khiển 32 led sáng tắt dần từ phải sang trái dùng vòng lặp for với thời gian delay là 1 giây thì để thực hiện xong 1 chu kỳ điều khiển của chương trình này phải mất hết 64 giây (32 giây cho sáng dần, 32 giây cho tắt dần). Trong lúc thực hiện chương trình này nếu ta có nhấn phím để ngừng chương trình này và chuyển sang chương trình khác thì cũng vô tác dụng vì chương trình chưa kết thúc. Nếu dùng nút nhấn để chuyển chương trình thì phải nhấn và giữ trong thời gian 64 giây thì mới có khả năng đáp ứng được.

Một ví dụ khác là vừa điều khiển 32 sáng tắt dần với thời gian trễ 1 giây vừa đếm sản phẩm hiển thị trên 4 led 7 đoạn: với cách viết dùng vòng lặp for thì phải đợi xong 1 chu kỳ điều khiển 32 led thì mới cập nhật kết quả đếm 1 lần – trong lúc đang điều khiển 32 led nếu có sản phẩm thì counter cũng đếm nhưng không hiển thị ra led được.

Vậy làm sao để thực hiện nhiều chương trình với đáp ứng nhanh và liên tục?

Đáp ứng nhanh và liên tục có nghĩa là giả sử đang điều khiển led thứ 3 sáng dần mà ta nhấn phím thì nó phát hiện và xử lý ngay hoặc khi phát hiện có xung thì nó cập nhật kết quả ngay và sau đó tiếp tục điều khiển led thứ 4 sáng dần ...

Để thực hiện được điều này thì có nhiều cách:

- Cách thứ 1 là vẫn dùng vòng lặp for nhưng can thiệp vào chương trình delay để kiểm tra phím nhấn, xung đếm của counter, ... Cách này không hay.
- Cách thứ 2 là viết không dùng vòng lặp. Các chương trình bây giờ viết theo cách khác và mỗi lần gọi thì phải kiểm tra đang thực hiện đến đâu và làm tiếp công việc và chỉ thực hiện 1 công việc đơn giản rồi thoát ngay.

Ví dụ chương trình điều khiển 32 led sáng dần và tắt dần viết theo cách này thì viết như thế nào?

Với chương trình này thì chu kỳ thực hiện ta biết là 64 lần: 32 lần cho led sáng dần và 32 lần cho led tắt dần, phải dùng 1 biến đếm để đếm số lần thực hiện, dữ liệu cho chương trình này là độc lập.

Mỗi khi chương trình này được gọi thì nó sẽ kiểm tra xem biến đếm nếu nhỏ hơn 32 thì đang ở phần làm led sáng dần thì ta tiến hành xuất dữ liệu ra module 32 led và dịch mức 1 vào rồi delay theo ý muốn, tăng biến đếm lên 1 và sau đó thoát.

Giả sử thời gian delay vẫn là 1 giây thì chương trình này khi được gọi thì sau 1 giây sẽ thoát chứ không phải chờ đến 64 giây như cách dùng vòng lặp for.

Cứ thế sau 32 lần gọi, 32 led đã sáng dần hết rồi thì bây giờ kiểm tra xem biến đếm nhỏ hơn 64 hay không, nếu đúng thì làm tắt dần 1 led, delay, tăng biến đếm rồi thoát.

Cứ thế cho đến khi biến đếm bằng 64 thì xong 1 chu kỳ thì ta cho biến đếm bằng 0 lại để bắt đầu chu kỳ tiếp theo.

Tóm lại chương trình có n công việc theo trình tự thời gian thì ta viết sao cho mỗi lần gọi thì chỉ thực hiện 1 công việc, có biến giám sát công việc đang thực hiện.

Với cách viết này thì mới có khả năng đáp ứng được một hệ thống điều khiển lớn được.

Bài mẫu 311. Hãy viết chương trình điều khiển 32 led sáng dần tắt dần từ phải sang trái viết không dùng vòng lặp, chỉ dùng lệnh if.

Lưu tên file là “BAI_311_STD_PST_32LED_NO_LOOP”.

- Mục đích: biết cách viết chương trình điều khiển 32 led sáng tắt dần theo cách không dùng vòng lặp.
- Chương trình:

```
#include<tv_boards.c>
unsigned int8 n=0,hu=0;
```

```

void sangdan_pst()
{
    if(n<=32) // chưa chạy xong thì tiếp tục chạy hiệu ứng hiện tại
    {
        led32.ledx32= ~(0xffffffff<<n);
        led32_display();
        delay_ms(100);
        n++;
    }
    else // Chạy xong rồi thì chuyển sang hiệu ứng kế tiếp
    {
        n=0; hu++;
    }
}
void tatdan_pst()
{
    if(n<=32) // chưa chạy xong thì tiếp tục chạy hiệu ứng hiện tại
    {
        led32.ledx32= 0xffffffff<<n;
        led32_display();
        delay_ms(100);
        n++;
    }
    else // Chạy xong rồi thì chuyển sang hiệu ứng kế tiếp
    {
        n=0; hu++;
    }
}
void main()
{
    system_init();
    while(true)
    {
        if (hu==0) sangdan_pst();
        else if(hu==1) tatdan_pst();
        else if(hu==2) hu=0;
    }
}

```

- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.

Bài tập 312. Hãy viết chương trình điều khiển 32 led sáng dần tắt dần từ phải sang trái, trái sang phải dùng lệnh if – không dùng lệnh for.

Lưu tên file là “BAI_312_STD_PST_TSP_32LED”.

Bài tập 313. Hãy viết lại bài 310 chỉ dùng lệnh if – không dùng lệnh for.

Lưu tên file là “BAI_313_TONGHOP_32LED”.

3.2 CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN 32 LED ĐƠN VÀ NÚT NHẤN

3.2.1 Giới thiệu

Nút nhấn, bàn phím dùng để giao tiếp giữa con người và mạch điện tử để điều khiển, ví dụ: bàn phím máy tính, bàn phím điện thoại, bàn phím máy bán xăng dầu dùng nhập số tiền cần bán, số lít cần bán ... máy giặt tự động có bàn phím để chỉnh chế độ giặt, chọn mực nước ...

Có 2 dạng giao tiếp vi điều khiển với bàn phím, nút nhấn:

- Hệ thống ít phím: ví dụ điều khiển động cơ bằng 3 phím: Start, Stop, Inv, đồng hồ có 3 đến 4 phím để chỉnh thời gian.
- Hệ thống nhiều phím: bàn phím máy tính, bàn phím điện thoại, ...

3.2.2 Mạch điện giao tiếp vi điều khiển với 4 nút nhấn đơn

Phần này thực hành các bài giao tiếp vi điều khiển với 4 nút nhấn đơn để thực hiện các yêu cầu điều khiển như điều khiển led tắt mở, điều khiển đảo chiều led và các ứng dụng sau này đều có liên quan đến nút nhấn.

Mạch điện giao tiếp vi điều khiển với các nút nhấn như hình 3-5.

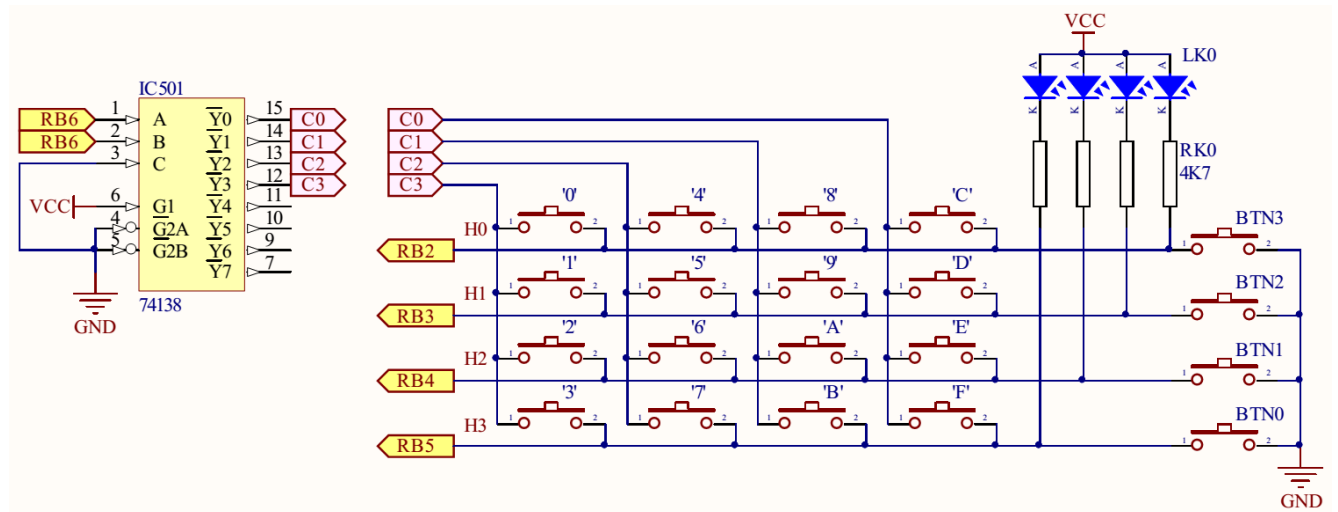
Vi điều khiển dùng 4 bit của portB giao tiếp với 4 nút nhấn đơn, các nút nhấn còn lại của bàn phím ma trận thì chưa cần quan tâm đến.

Các nút nhấn bình thường hở nên trạng thái tín hiệu ở mức logic '1', khi nhấn thì ngắn mạch nối mass làm mức logic về '0'. Khi nhả phím thì về lại bình thường mức logic '1'.

Để kiểm tra có nhấn hay không thì ta kiểm tra trạng thái nếu bằng '1' thì không nhấn phím, bằng '0' thì có nhấn.

Mạch có 4 LED : bình thường thì 4 LED tắt, khi nhấn 1 phím thì LED tương ứng sáng.

Khi đèn sáng thì chắc chắn phím nhấn còn tốt. Nhiều bạn lập trình sai thường đổ lỗi cho nút nhấn hỏng.



Hình 3-2. Sơ đồ nguyên lý giao tiếp vi điều khiển với 4 nút nhấn đơn.

3.2.3 Định nghĩa tên các nút nhấn trong thư viện

Kit thực hành có 4 nút nhấn đơn có tên là BT0 đến BT3 kết nối trực tiếp với 4 bit của portB.

Trong thư viện ta có định nghĩa sẵn nhiều tên khác nhau cho các nút nhấn để phù hợp với các bài thực hành, ví dụ ta nhấn ON thì đèn sáng, nhấn nút OFF thì đèn tắt hay hơn là nhấn BT0

đèn sáng và BT1 thì đèn tắt. Khi viết hàm ta đặt tên hàm là PHIM_ON và PHIM_OFF nó rõ ràng hơn là PHIM_BT0 và PHIM_BT1. Khi thấy hàm PHIM_ON nó gợi cho ta chức năng là mở ngay, còn khi thấy hàm PHIM_BT0 thì ta phải đọc nội dung mới biết nó làm gì khi mà ta viết nhiều phím và chương trình đã viết lâu rồi nay đọc lại sẽ mất nhiều thời gian.

Sau đây là các định nghĩa về 4 nút nhấn trong thư viện:

```
#DEFINE BT0 PIN_B5
#define BT1 PIN_B4
#define BT2 PIN_B3
#define BT3 PIN_B2
#define ON BT0
#define OFF BT1
#define INV BT2
#define UP BT0
#define MOD BT1
#define DW BT2
#define CLR BT3
#define STOP BT3
#define ON1 BT0
#define OFF1 BT1
#define ON2 BT2
#define OFF2 BT3
```

Việc định nghĩa này giúp ta thống nhất và khi viết chương trình không phải định nghĩa nữa.

3.2.4 Các chương trình điều khiển dùng các nút nhấn đơn

Bài mẫu 321. Chương trình điều khiển 8 LED bằng 2 nút ON và OFF. Khi có điện thì led tắt, khi nhấn ON thì 8 led sáng, khi nhấn OFF thì 8 led tắt.

Lưu tên file là “BAI_321_ON_OFF_8LED”

- Mục đích : Biết cách lập trình kiểm tra nút nhấn và điều khiển.
- Chương trình:

```
#include<tv_boards.c>
void main()
{
    system_init();
    while(true)
    {
        if(input(ON) ==0) led32.ledx8[0]=0xff;
        if(input(OFF)==0) led32.ledx8[0]=0 ;
        led32_display();
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: sau khi nạp xong chương trình thì nhấn nút ON sẽ làm 8 led sáng, nhấn nút OFF sẽ làm 8 led tắt, nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.

Bài mẫu 322. Chương trình điều khiển 8 LED bằng 3 nút ON, OFF, INV. Khi có điện thì 8 led tắt, khi nhấn ON thì 4 led sáng, khi nhấn OFF thì led tắt, khi nhấn INV thì 4 led sáng thành tắt, 4 led tắt thành sáng.

Lưu tên file là “BAI_322_ON_OFF_INV_8LED ”

- Mục đích: điều khiển để thấy hiện tượng dội gây ra khi nhấn nút trong điều khiển.
- Chương trình:

```
#include<tv_boards.c>
void main()
{
    system_init();
    while(true)
    {
        if(input(ON) ==0)    led32.ledx8[0]=0x0f;
        if(input(OFF)==0)    led32.ledx8[0]=0;
        if( (input(INV)==0) && (led32.ledx8[0]!=0) )
            led32.ledx8[0]=~ led32.ledx8[0];
        led32_display();
    }
}
```

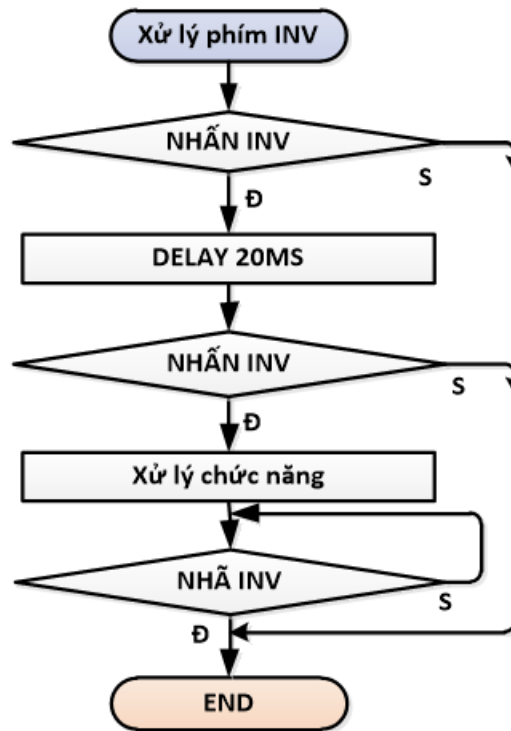
- Tiến hành biên dịch và nạp.
- Quan sát kết quả: Sau khi nạp xong chương trình thì nhấn nút ON sẽ làm 4 led sáng, nhấn nút OFF sẽ làm 4 led tắt, nhấn nút INV sẽ làm 4 led sáng thành tắt, 4 led tắt thành sáng, nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- Giải thích chương trình:

Trong chương trình này khi chạy thực tế thì do tốc độ của vi điều khiển quá nhanh, nên khi nhấn đảo chiều thì do thời gian nhấn phím dài nên vi điều khiển thực hiện đảo led liên tục, ta sẽ nhìn thấy 8 led sáng luôn cho đến khi ta buông phím, hoặc ta nhấn nhanh thì trạng thái đảo của led không thể xác định rõ ràng.

Để điều khiển chính xác thì phải chống dội phím nhấn, rồi xử lý chức năng và kiểm tra buông phím.

Trong bài điều khiển chỉ có phím INV gây ra ảnh hưởng nên lưu đồ và chương trình xử lý phần chống dội và chờ buông phím INV theo sau.

Để chống dội phím INV ta thực hiện theo lưu đồ sau:



Hình 3-6. Lưu đồ chống dội

Trong thư viện “**TV_LOWLEVEL**” đã viết sẵn chương trình chống dội dùng chung cho tất cả các nút nhấn theo lưu đồ trên như sau:

```

result inputcd(int16 pin)
{
    static unsigned int16 oldpin=0xffff;
    static unsigned int16 t;
    static unsigned int8 step=0;
    output_float(pin);
    switch (step)
    {
        case 0:
            // Nếu phát hiện có nhấn phím thì bắt đầu delay
            // và chuyển sang bước 1
            if(input(pin)==0)
            {
                oldpin    = pin;
                t          = delay_start();
                step       = 1;
            }
            break;
        case 1:
            // Nếu chưa đủ 30ms mà thả phím thì trở về bước 0
            if(input(oldpin)) step = 0;
            // Nếu giữ phím quá 30ms thì trả về OK - có nhấn phím
            // và bắt đầu delay, rồi chuyển sang bước 2
            else if((pin==oldpin)&&(delay_stop(t)>=30))
            {
                t          = delay_start();
            }
    }
}
  
```

```

        step = 2;
        return OK;
    }
    break;
case 2:
    // Nếu chưa đủ 500ms mà thả phím thì trở về bước 0
    if(input(oldpin)) step = 0;
    // Nếu giữ phím quá 500ms thì bắt đầu delay và chuyển
    // qua chế chờ nhấn giữa ở bước 3
    else if((pin==oldpin)&&(delay_stop(t)>=500))
    {
        t      = delay_start();
        step    = 3;
    }
    break;
case 3:
    // Nếu giữa không đủ 100ms thì trở về bước 0
    if(input(oldpin)) step = 0;
    // Nếu giữ quá 100ms thì trả về OK - có nhấn phím
    else if((pin==oldpin)&&(delay_stop(t)>=100))
    {
        t = delay_start();
        return OK;
    }
    break;
}
return BUSY;
}

```

Khi sử dụng ta chỉ việc thay hàm “input” bằng “inputcd” thì đã chống dội xong.

Bài mẫu 323. Chương trình điều khiển 8 LED bằng 3 nút ON, OFF, INV. Khi có điện thì 8 led tắt, khi nhấn ON thì 4 led sáng, khi nhấn OFF thì led tắt, khi nhấn INV thì 4 led sáng thành tắt, 4 led tắt thành sáng, có chống dội phím.

Lưu tên file là “BAI_323_ON_OFF_INV_CDOI_8LED”

- Mục đích: chống dội cho nút nhấn gây ra hiện tượng dội, biết cách lập trình chống dội cơ bản.
- Chương trình:

```

#include<tv_boards.c>
void main()
{
    system_init();
    while(true)
    {
        if(input(ON) ==0) led32.ledx8[0]=0x0f;
        if(input(OFF)==0) led32.ledx8[0]=0;
        if((inputcd(INV)==0)&&(led32.ledx8[0]!=0))
            led32.ledx8[0]=~ led32.ledx8[0];
        led32_display();
    }
}

```


- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: trong bài này thì chống dội và chờ nhả phím thì mới thực hiện.

Bài tập 324. Dùng vi điều khiển giao tiếp với 16 led đơn và 3 nút nhấn được đặt tên là UP, DW, CLR. Khi cấp điện thì 16 led tắt, khi nhấn UP thì led sáng dần lên từ phải sang trái – mỗi lần nhấn thì 1 led sáng, khi nhấn DW thì led tắt dần theo chiều ngược lại, khi nhấn CLR thì xóa hết.

Lưu tên file là “BAI_324_16LED_UP_DW_CLR_1”.

Bài tập 325. Giống bài 324 nhưng nút nhấn CLR có thêm chức năng đảo chiều sáng và tắt của led: ví dụ khi nhấn UP thì sáng dần từng led từ phải sang trái, thì sau khi nhấn CLR thì khi nhấn UP led sẽ sáng từng led theo chiều từ trái sang phải, tương tự cho nút nhấn DW.

Lưu tên file là “BAI_325_16LED_UP_DN_CLR_2”.

Bài tập 326. Hãy lập trình dùng vi điều khiển giao tiếp với 32 led đơn và 3 nút nhấn được đặt tên là UP, DN, CLR.

Hệ thống điều khiển 32 led với các chương trình như sau:

1. Chớp tắt 32 led.
2. Sáng tắt dần 32 led từ phải sang trái.
3. Sáng tắt dần 32 led từ trái sang phải.
4. Sáng tắt dần 32 led từ ngoài vào.
5. Sáng tắt dần 32 led từ trong ra.
6. Sáng dồn 32 led từ phải sang trái.
7. Sáng dồn 32 led từ trái sang phải.

Khi cấp điện mặc nhiên vi điều khiển chạy chương trình 1.

Khi nhấn UP thì chuyển sang chương trình kế theo chiều tăng - khi đến chương trình số 7 thì ngừng.

Khi nhấn DN thì chuyển sang chương trình kế theo chiều giảm - khi đến chương trình số 1 thì ngừng.

Khi nhấn CLR thì quay về chương trình số 1.

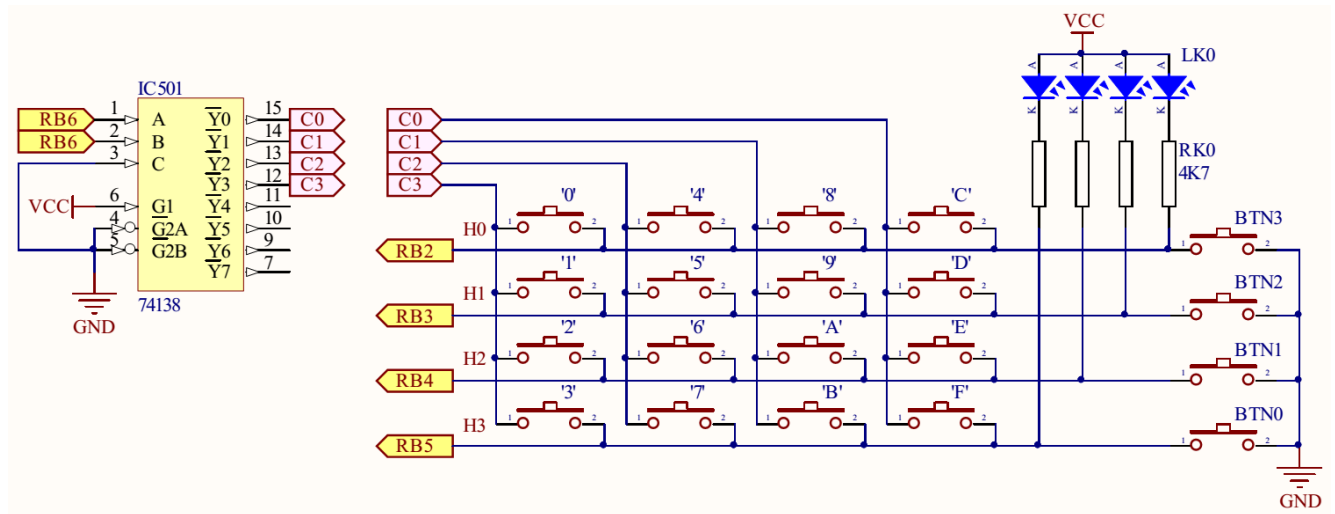
Chú ý là khi nhấn thì mạch phải chuyển ngay lập tức.

Lưu tên file là “BAI_326_TONGHOP_UP_DN_CLR” để lưu tất cả các file của project.

3.2.5 Mạch điện giao tiếp vi điều khiển với ma trận phím

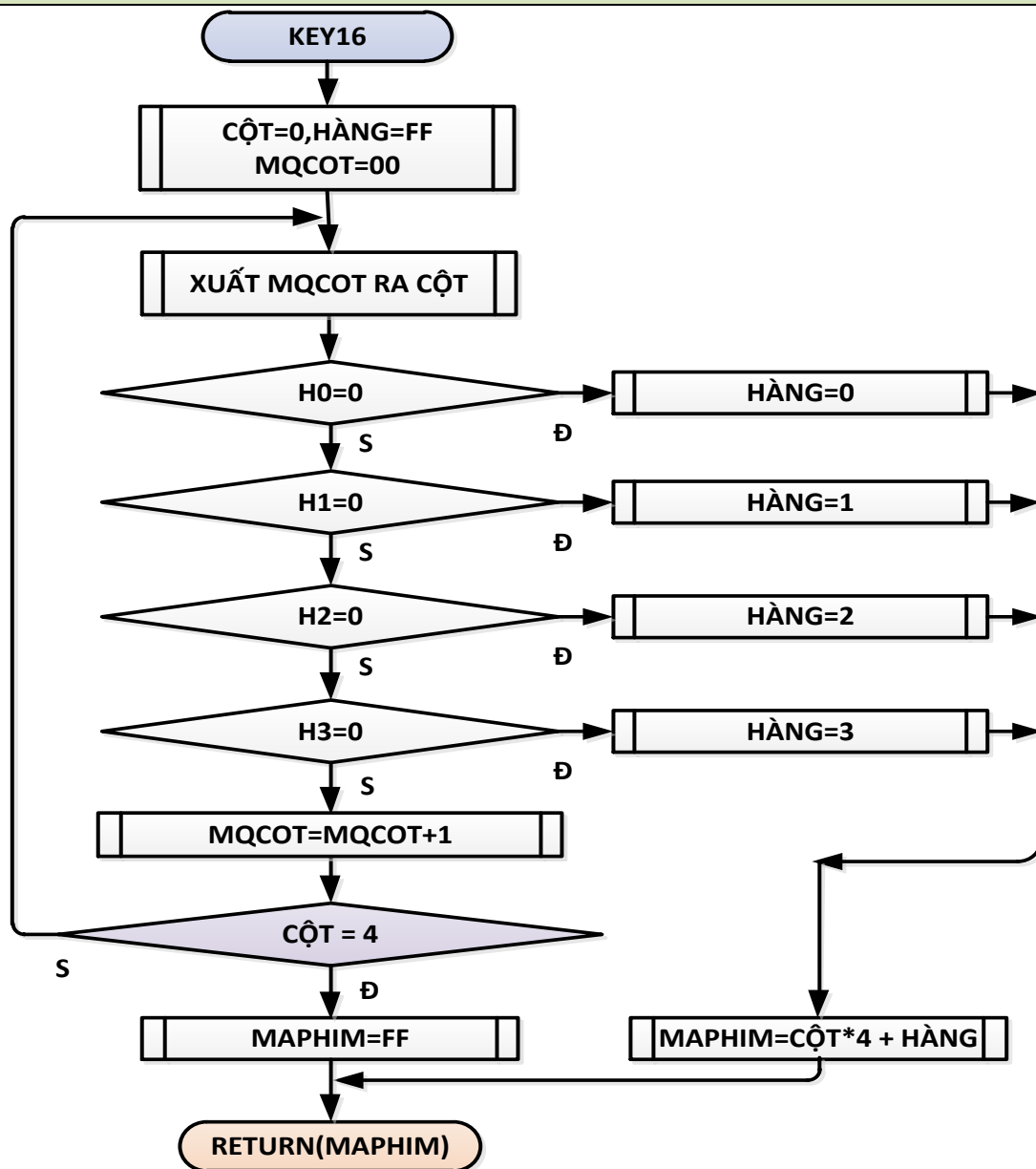
Phần này thực hành các bài giao tiếp vi điều khiển với ma trận bàn phím 4x4.

Mạch điện giao tiếp vi điều khiển với các nút nhấn như hình 3-9.



Hình 3-3. Sơ đồ nguyên lý giao tiếp vi điều khiển với ma trận 16 phím.

Vi điều khiển dùng 6 bit của portB giao tiếp với ma trận 4x4: 4 bit RB2 đến RB5 đóng vai trò là hàng, 2 bit RB6 và RB7 xuất ra số nhị phân có 4 trạng thái 00, 01, 10, 11 qua mạch giải mã 2 đường sang 4 đường ngõ ra tích cực mức 0 sẽ tạo ra 4 trạng thái tương ứng là 1110, 1101, 1011, 0111 dùng để quét 4 cột. Lưu đồ quét ma trận phím:



Hình 3-4. Lưu đồ quét ma trận 16 phím.

3.2.6 Các chương trình điều khiển dùng ma trận phím

Thư viện quét phím ma trận “TV_KEY4X4”.

(SV không đánh máy lại vì file thư viện này đã có sẵn)

```

typedef struct
{
    unsigned int8 key;
}KEY4x4_STRUCT;
KEY4x4_STRUCT key4x4;

unsigned int8 KEYREAD()
{
    unsigned int8 cot;
    for(cot=0; cot<4; cot++)
  
```

```

    {
        output_bit(pin_b7,cot&0x02);           // Xuất mã quét cột ra IC74138
        output_bit(pin_b6,cot&0x01);           // Xuất mã quét cột ra IC74138
        IF      (!INPUT(PIN_B2))      return (3 + cot*4);
        ELSE    IF  (!INPUT(PIN_B3))  return (2 + cot*4);
        ELSE    IF  (!INPUT(PIN_B4))  return (1 + cot*4);
        ELSE    IF  (!INPUT(PIN_B5))  return (0 + cot*4);
    }
    return 0xff;
}

result key4x4_read()
{
    static unsigned int8  step=0;
    static unsigned int16 t;
    key4x4.key = KEYREAD();
    switch (step)
    {
        // Các bước chống dội giống phím nhấn đơn
        // Các bạn xem lại hàm "inputcd"
        case 0:
            if( key4x4.key!=0xff)
            {
                t = delay_start();
                step=1;
            }
            break;
        case 1:
            if(key4x4.key==0xff) step =0;
            else if(delay_stop(t)>=30)
            {
                step = 2;
                t = delay_start();
                return OK;
            }
            break;
        case 2:
            if(key4x4.key==0xff) step =0;
            else if(delay_stop(t)>=500)
            {
                t = delay_start();
                step = 3;
            }
            break;
        case 3:
            if(key4x4.key==0xff) step =0;
            else if(delay_stop(t)>=100)
            {
                t = delay_start();
                return OK;
            }
            break;
    }
    return BUSY;
}

```

- Khi gọi hàm “**key4x4_read()**” nếu có phím nhấn thì mã của phím nhấn sẽ được lưu trong biến “**key4x4.key**”, đây là biến cấu trúc mặc dù chỉ có 1 phần tử nhưng tác giả vẫn khai báo dạng cấu trúc để cho đồng bộ với các module về sau.

Bài mẫu 332. Chương trình quét phím ma trận 4x4, khi nhấn phím nào thì mã của phím hiển thị ở các led đơn dạng số nhị phân.

Khi nhả phím thì mã mới xuất hiện trên led.

Lưu tên file là “BAI_332_MATRANPHIM_LEDDON”

- Mục đích: biết cách viết thư viện, biết cách lập trình giao tiếp bàn phím ma trận khi nhấn thì hiển thị mã phím nhị phân trên led.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_key4x4.c>
void main()
{
    system_init();
    while(true)
    {
        if(key4x4_read()==0K)
        {
            led32.ledx32= key4x4.key;
            led32_display();
        }
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: Bạn tiến hành nhấn các phím và sau đó nhả phím thì mã nhị phân của phím sẽ hiển thị trên led đơn và cả ở led 7 đoạn. Tương tự cho các phím còn lại.
- Giải thích chương trình: Hãy giải thích chương trình con quét bàn phím ma trận.

Bài mẫu 333. Chương trình điều khiển 8 led với yêu cầu: khi nhấn phím số ‘1’ thì 8 led sáng, khi nhấn phím số ‘0’ thì 8 led tắt. Phím số ‘0’ và ‘1’ là của bàn phím ma trận.

Lưu tên file là “BAI_333_MATRANPHIM_8LED_ON_OFF”

- Mục đích: biết ứng dụng bàn phím ma trận để điều khiển.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_key4x4.c>
void main()
{
    system_init();
    while(true)
    {
        if(key4x4_read()==0K)
        {
            if (key4x4.key==1) led32.ledx8[0]=0xff;
            else if (key4x4.key==0) led32.ledx8[0]=0;
        }
    }
}
```

```

        led32_display();
    }
}

```

- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: khi nhấn phím ‘1’ thì 8 led sáng, nhấn phím ‘0’ thì 8 led tắt, những led còn lại không có tác dụng.

Bài tập 334. Hãy viết chương trình điều khiển 8 led (có số thứ tự từ led0 đến led7).

Yêu cầu: 8 nút nhấn từ 0 đến 7 có chức năng ON: Khi nhấn phím số ‘0’ thì led0 sáng, tương tự cho đến số 7 thì led7 sáng.

8 nút nhấn từ A đến F có chức năng OFF: khi nhấn phím số ‘8’ thì led0 tắt, tương tự cho đến số F thì led7 tắt.

Lưu tên file là “BAI_334_MATRANPHIM_8LED_16_BTN_ON_OFF”.

Bài tập 335. Hãy làm lại bài 3-26 nhưng sử dụng 7 phím từ 0 đến 7 của bàn phím ma trận, không dùng UP, DW và CLR.

Lưu tên file là “BAI_335_7CT_7BTN”.

3.3 CÁC CÂU HỎI ÔN TẬP

Câu 3-1. Hãy vẽ sơ đồ mạch giao tiếp 4 nút nhấn với vi điều khiển PIC 18F4550 của kit.

Câu 3-2. Hãy vẽ sơ đồ mạch giao tiếp bàn phím ma trận với vi điều khiển PIC 18F4550 của kit.

Câu 3-3. Hãy vẽ lưu đồ quét bàn phím ma trận 4x4.

Câu 3-4. Hãy vẽ dạng sóng đội khi nhấn phím và lưu đồ chống dội nút nhấn.

Câu 3-5. Hãy vẽ mạch điện giao tiếp vi điều khiển với 32 led đơn.

Câu 3-6. Hãy giải thích nguyên lý vi điều khiển gửi dữ liệu ra 32 led đơn.

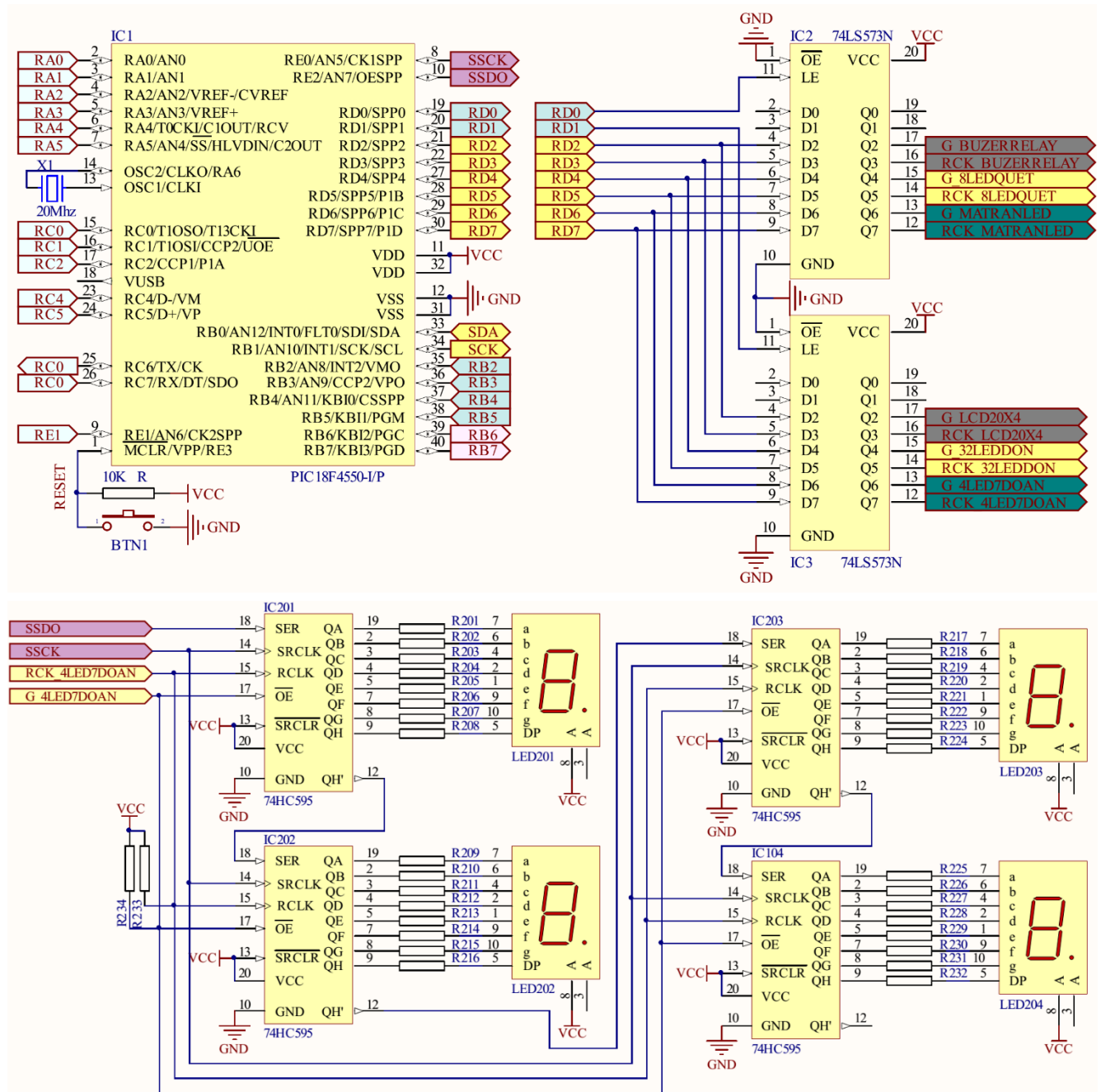
Chương 4: THỰC HÀNH
MODULE 2 – 4 LED 7 ĐOẠN
ANODE DÙNG THANH GHI
DỊCH 74HC595,
COUNTER/TIMER

4.1 CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN 4 LED 7 ĐOẠN

4.1.1 Mạch điện giao tiếp vi điều khiển với module 4 led 7 đoạn

Phần này thực hành các bài giao tiếp với 4 led 7 đoạn để thực hiện các bài đếm thời gian, đếm sản phẩm, hiển thị các thông tin là các số thập phân.

Mạch điện giao tiếp vi điều khiển với module 4 led 7 đoạn như hình 4-1.



Hình 4-1. Sơ đồ nguyên lý giao tiếp vi điều khiển với module 4 led 7 đoạn.

Trong mạch sử dụng 4 IC thanh ghi 74HC595 để mở rộng port điều khiển 4 led 7 đoạn.

Phần vi điều khiển PIC giao tiếp với IC thanh ghi dịch

Có 4 tín hiệu điều khiển là:

- SSDO dùng để dịch dữ liệu nối tiếp.
- SSCK dùng để cấp xung clock.
- RCK_4LED7DOAN dùng để điều khiển nạp dữ liệu song song từ bên trong IC thanh ghi dịch ra bên ngoài để điều khiển led 7 đoạn.
- G_4LED7DOAN dùng để cho phép xuất dữ liệu.

Phần IC thanh ghi dịch giao tiếp với 4 led 7 đoạn

Mỗi IC 74HC595 có 8 bit kết nối với 1 led 7 đoạn, 4 IC điều khiển được 4 led 7 đoạn.

Số lượng byte dữ liệu điều khiển module 4 led 7 đoạn là 4 byte.

Khả năng mở rộng

Trong kit thực hành dùng 4 IC để điều khiển 4 led, sau này các ứng dụng thực tế cần nhiều led hơn thì bạn cứ kết nối thêm IC thanh ghi nối tiếp vào một cách dễ dàng.

4.1.2 Hàm điều khiển module 4 led 7 đoạn

Hàm thứ 401: xuất dữ liệu ra module 4 LED 7 đoạn

```
result d7seg_display()
{
    SSPI_WRITE(d7seg.led[0]); SSPI_WRITE(d7seg.led[1]);
    SSPI_WRITE(d7seg.led[2]); SSPI_WRITE(d7seg.led[3]);
    output_high(D7SEG_RCK); output_low(D7SEG_RCK);
    return OK;
}
```

Trong đó “d7seg” là biến cấu trúc gồm 4 phần tử mặc định bằng 0xff (để tắt LED)

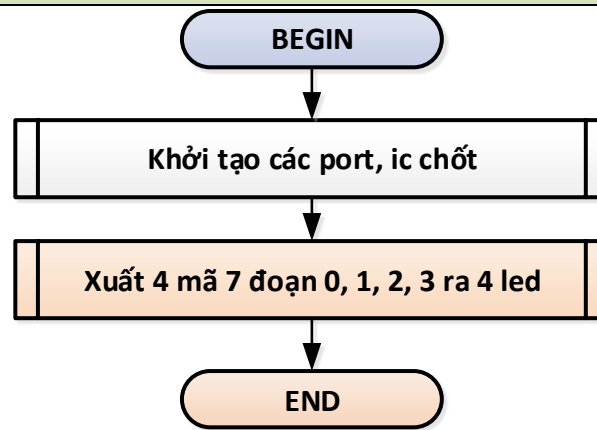
```
typedef struct
{
    unsigned int8    led[4];
} D7SEG_STRUCT;
D7SEG_STRUCT      d7seg = {0xff, 0xff, 0xff, 0xff};
```

4.1.3 Các chương trình điều khiển 4 led 7 đoạn

Bài mẫu 401. Chương trình điều khiển 4 LED 7 đoạn hiển thị 4 số từ 0 đến 3.

Lưu tên file là “BAI_401_HIEN THI_4SO”.

- Mục đích: biết điều khiển 4 led 7 đoạn hiển thị 4 số.
- Lưu đồ:



Hình 4-2. Lưu đồ điều khiển module 4 led 7 đoạn hiển thị 4 số 0, 1, 2, 3.

c. Chương trình:

```

#include<tv_boards.c>
void main ()
{
    system_init();
    d7seg.led[3] = m7d[3]; // m7d là mảng chứa mã LED 7 đoạn anode chung
    d7seg.led[2] = m7d[2];
    d7seg.led[1] = m7d[1];
    d7seg.led[0] = m7d[0];
    d7seg_display();
    while(true)
    {
    }
}
    
```

d. Tiến hành biên dịch và nạp.

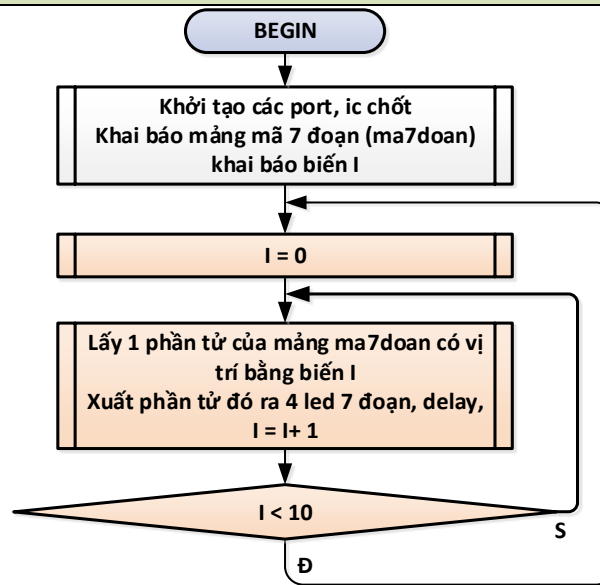
e. Quan sát kết quả: 4 led 7 đoạn sẽ hiển thị 4 số: 3, 2, 1, 0.

Bài mẫu 402. Chương trình điều khiển 1 LED 7 đoạn đếm từ 0 đến 9.

Lưu tên file là “BAI_402_DEM_0_9”.

a. Mục đích: Biết cách lập trình bài đếm thời gian 1 số hiển thị trên 1 led 7 đoạn.

b. Giải thuật:



Hình 4- 3. Lưu đồ điều khiển đếm từ 0 đến 9.

c. Chương trình:

```

#include<tv_boards.c>
unsigned int8 dem ;
void main()
{
    system_init();
    while(true)
    {
        for(dem=0;dem<10; dem++)
        {
            d7seg.led[0] = m7d[dem];
            d7seg_display();
            delay_ms(500);
        }
    }
}

```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: Sẽ có 1 led 7 đoạn đếm từ 0 đến 9.
- Giải thích chương trình: Bạn hãy giải thích chương trình.

Bài mẫu 403. Chương trình điều khiển 2 LED 7 đoạn đếm từ 00 đến 99.

Lưu tên file là “BAI_403_DEM_00_99”.

- Mục đích: Biết cách lập trình bài đếm thời gian 2 số hiển thị trên 2 led 7 đoạn.
- Lưu đồ: Sinh viên hãy tự viết.
- Chương trình:

```

#include<tv_boards.c>
unsigned int8 dem;
void main()

```

```
{
    system_init();
    while(true)
    {
        for(dem=0;dem<100;dem++)
        {
            d7seg.led[1] = m7d[dem/10 %10];
            d7seg.led[0] = m7d[dem%10];
            d7seg_display();
            delay_ms(500);
        }
    }
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: sẽ có 2 led 7 đoạn đếm từ 00 đến 99.
- f. Giải thích chương trình: bạn hãy giải thích chương trình.

Bài tập 404. Hãy viết chương trình điều khiển 2 led của 4 LED 7 đoạn đếm giây từ 00 đến 59, thời gian delay là 1000ms. Nếu muốn chạy nhanh thì giảm delay.

Lưu tên file là “BAI_404_DEM_GIAY_00_59”.

Bài tập 405. Hãy viết chương trình điều khiển 2 led của 4 LED 7 đoạn đếm từ 00 đến 59, rồi đếm về 00, rồi lặp lại, thời gian delay là 200ms.

Lưu tên file là “BAI_405_DEM_00_59_00”.

Bài tập 406. Hãy viết chương trình điều khiển 2 led của 4 LED 7 đoạn đếm từ 00 đến 59, với thời gian delay là 200ms, khi bằng 59 thì ngừng lại khoảng 3 giây rồi bắt đầu đếm xuống 00, với thời gian trễ là 200ms, khi bằng 00 thì ngừng 3 giây rồi lặp lại.

Sử dụng lệnh do while.

Lưu tên file là “BAI_406_DEM_00_59_3S_00_3S”.

Bài mẫu 407. Chương trình điều khiển 3 LED 7 đoạn đếm từ 000 đến 999.

Lưu tên file là “BAI_407_DEM_000_999”.

- a. Mục đích: Biết cách lập trình bài đếm thời gian 3 số hiển thị trên 3 led 7 đoạn.
- b. Lưu đồ: sinh viên hãy tự viết.
- c. Chương trình:

```
#include<tv_boards.c>
unsigned int16 dem=0;          // Chú ý: Biến "dem" phải khai báo 16bit
void main()
{
```

```

system_init();
while(true)
{
    for(dem=0;dem<1000;dem++)
    {
        d7seg.led[2] = m7d[dem/100%10];
        d7seg.led[1] = m7d[dem/10%10];
        d7seg.led[0] = m7d[dem%10];
        d7seg_display();
        delay_ms(500);
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: sẽ có 3 led 7 đoạn đếm từ 000 đến 999.
- f. Giải thích chương trình: bạn hãy giải thích chương trình.

Bài tập 408. Hãy viết chương trình điều khiển 4 LED 7 đoạn đếm từ 0000 đến 9999. Lưu tên file là “BAI_408_DEM_0000_9999”.

Gợi ý: mở rộng giới hạn cho vòng lặp for, hiệu chỉnh phần giải mã hiển thị.

Bài tập 409. Hãy viết chương trình điều khiển 4 LED 7 đoạn đếm phút giây. Lưu tên file là “BAI_409_DEM_PHUT_GIAY”.

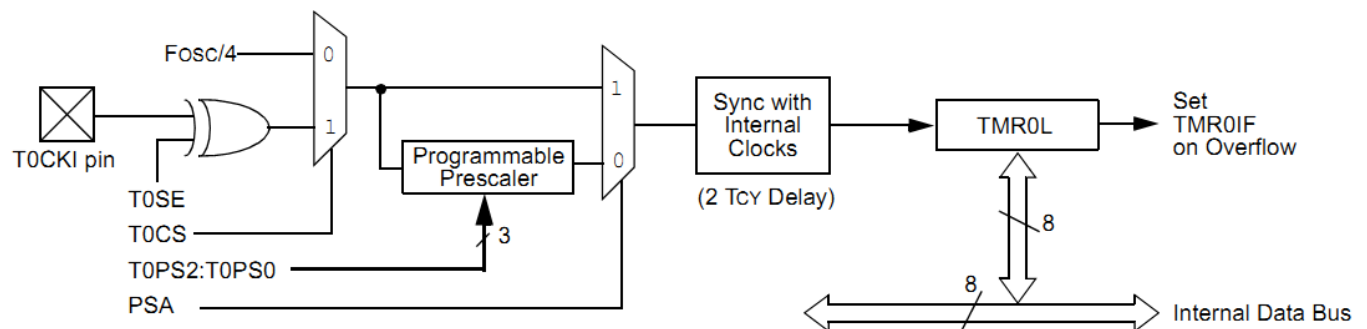
4.2 TIMER VÀ COUNTER CỦA VI ĐIỀU KHIỂN PIC

Vi điều khiển PIC 18F4550 có 4 timer T0, T1, T2 và T3.

T0, T1, T3 là timer/counter 16 bit, cả 3 đều có bộ chia trước. T2 là timer 8 bit có bộ chia trước và chia sau phục vụ cho các ứng dụng đặc biệt.

4.2.1 Tóm tắt timer/counter T0

Phần này thực hiện đếm xung ngoại dùng timer T0. Vì nguồn xung từ cảm biến trên bộ thí nghiệm đã cấp cho Timer T0 nên ta chỉ khảo sát timer T0.



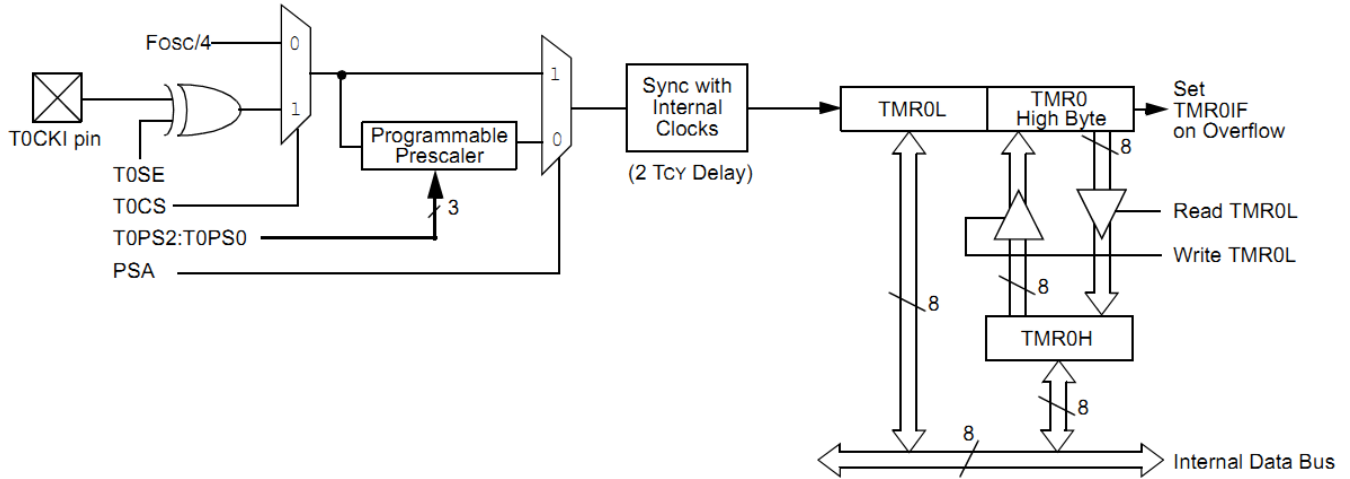
Note: Upon Reset, Timer0 is enabled in 8-bit mode with clock input from T0CKI maximum prescale.

Hình 4-4. Sơ đồ khối timer T0: chế độ 8 bit.

Vi điều khiển PIC 18F4550 có timer T0 có thể hoạt động ở 2 chế độ 8 bit hoặc 16 bit. Chế độ 8 bit thì giống họ vi điều khiển PIC16.

Về nguyên lý thì những gì của timer T0 ở họ PIC16 thì đều có thể thực hiện ở họ PIC18. Phần này không trình bày chi tiết mà chỉ tóm tắt.

Sơ đồ khối timer T0 hoạt động ở chế độ 8 bit như hình 4-4 và 16 bit ở hình 4-5.



Note: Upon Reset, Timer0 is enabled in 8-bit mode with clock input from T0CKI maximum prescale.

Hình 4-5. Sơ đồ khối timer T0: chế độ 16 bit.

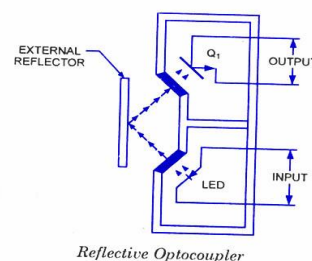
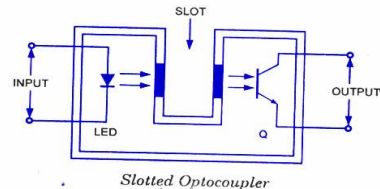
Bảng 4-1. Các thanh ghi có liên quan đến timer T0.

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset Values on page
TMR0L	Timer0 Register Low Byte								54
TMR0H	Timer0 Register High Byte								54
INTCON	GIE/GIEH	PEIE/GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF	53
INTCON2	RBPƯ	INTEDG0	INTEDG1	INTEDG2	—	TMR0IP	—	RBIP	53
T0CON	TMR0ON	T08BIT	T0CS	T0SE	PSA	T0PS2	T0PS1	T0PS0	54
TRISA	—	TRISA6 ⁽¹⁾	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	56

Các lệnh liên quan đến timer được trình bày ở tài liệu lý thuyết, các bit điều khiển liên quan đến timer thì có thể xem thêm trong thư viện 18F4550.h.

4.2.2 Cảm biến thu phát hồng ngoại HY860F

Để đếm xung ngoại thì nguồn xung đếm có thể tạo ra ở nhiều dạng khác nhau, nhưng trong bộ thí nghiệm thì sử dụng led thu phát hồng ngoại để nhận biết một sản phẩm đi qua để nó gần với thực tế ứng dụng bên ngoài.



Hình 4-6. Hình ảnh led thu phát HY860F và sơ đồ nguyên lý.

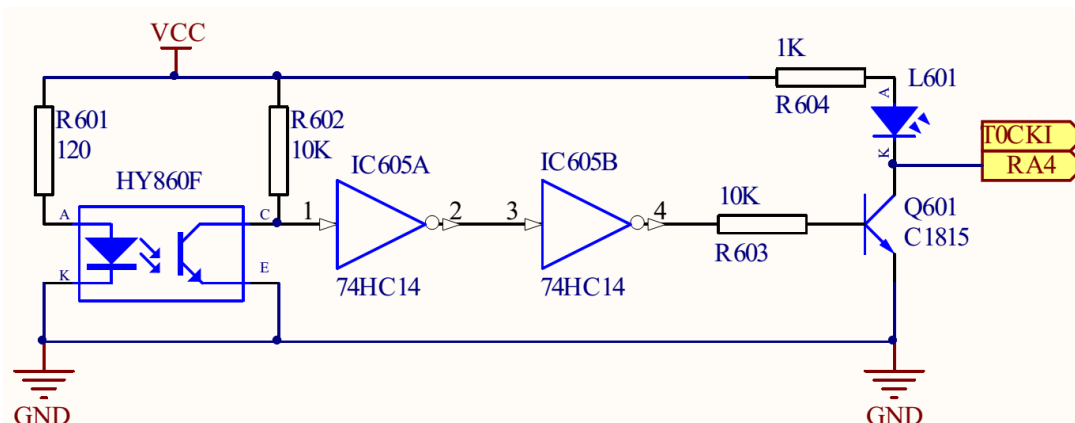
BTN sử dụng led thu phát HY860F có hình dạng và sơ đồ nguyên lý như hình 4-6.

Có 1 led phát ra ánh sáng hồng ngoại và 1 transistor thu.

Trong các robot có các động cơ và có gắn encoder thì có sử dụng dạng cảm biến này.

4.2.3 Mạch điện dùng T0 đếm xung ngoại

Khi sử dụng HY860F thì ta phải kết nối thêm điện trở và IC như hình sau:



Hình 4-7. Sơ đồ nguyên lý module thu phát hồng ngoại tạo xung.

Led phát sẽ nối với điện trở lên nguồn 5V để led luôn hoạt động phát ra ánh sáng hồng ngoại. Transistor có ngõ ra nối với điện trở và nối lên nguồn, đóng vai trò là điện trở tải để lấy tín hiệu.

Tín hiệu ngõ ra được nối với 2 cổng not của IC 74HC14 có chức năng đệm và schmitt trigger sửa dạng xung cho vuông và sau đó có thể đưa trực tiếp đến ngõ vào counter T0 hoặc đưa qua transistor để cho phép kết nối nhiều tín hiệu chung với nhau dạng cực thu để hở.

Khi không có sản phẩm nào che giữa led phát và led thu thì transistor được phân cực và ngõ ra transistor bằng 0, qua 2 cổng not thì cũng bằng 0, điều khiển transistor tắt, ngõ ra xung RA4/T0CKI bằng 1, led không có dòng nên tắt.

Khi bạn che ánh sáng giữa led phát và transistor thì transistor tắt, ngõ ra lên 1, qua 2 cổng not cũng bằng 1, điều khiển transistor dẫn. Ngõ ra xung RA4/T0CKI bằng 0, led có dòng nên led sáng.

Mỗi lần 1 sản phẩm đi qua thì tạo ra 1 xung, xung sẽ đưa đến counter T0 để đếm và hiển thị trên led.

4.2.4 Các ứng dụng đếm xung ngoại dùng counter T0

Phần này thực hành các bài đếm xung ngoại dùng counter T0 và hiển thị kết quả đếm trên module 4 led 7 đoạn.

Bài mẫu 411. Chương trình đếm sản phẩm hiển thị trên 4 led 7 đoạn dùng counter T0, giới hạn đếm từ 0 đến 100. Khi bằng 101 thì quay về 1.

Lưu tên file là “BAI_411_DEM_XUNG_T0”.

- Mục đích: Biết cách lập trình bài đếm xung ngoại dùng counter T0, hiển thị led 7 đoạn.
- Chương trình:

```
#include<tv_boards.c>
unsigned int8 T0;
void main()
{
    system_init();
    setup_timer_0 (t0_ext_1_to_h | t0_div_1);
    set_timer0(0); // Khi test SV thay đổi giá trị đặt cho nhanh
    while(true)
    {
        T0= get_timer0();
        if(T0>100) set_timer0(1);
        d7seg.led[2] = m7d[T0/100%10];
        d7seg.led[1] = m7d[T0/10%10];
        d7seg.led[0] = m7d[T0%10];
        d7seg_display();
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: dùng giấy che giữa led phát và led thu thì mạch cảm biến hồng ngoại sẽ tạo ra xung, có đèn báo hiệu sáng lên rồi tắt tương ứng với xung, kết quả giá trị trên led sẽ tăng lên 1.
- Giải thích chương trình: gồm có khởi tạo IC chốt, khởi tạo T0 chế độ đếm xung ngoại, hệ số chia là 1.

Vòng lặp while thực hiện đọc giá trị đếm được của T0 đem giải mã hiển thị và so sánh giới hạn để kết lặp lại. Bài này đơn giản giúp các bạn dễ hiểu chính vì thế cần phải thực hiện tiếp để cải tiến cho tốt hơn qua các bài tiếp theo.

Bài mẫu 412. Chương trình đếm sản phẩm hiển thị trên 4 led 7 đoạn dùng counter T0, giới hạn đếm từ 0 đến 100. Khi bằng 101 thì quay về 1 và có xóa số 0 vô nghĩa.

Lưu tên file là “BAI_412_DEM_XUNG_T0_XOA_SO0_VN”.

- Mục đích: Biết cách xóa số 0 vô nghĩa.
- Chương trình:

```
#include<tv_boards.c>
```

```

unsigned int8 T0;
void d7seg_xoa0VN(unsigned int16 d)
{
    unsigned int8 n=0;
    do
    {
        d7seg.led[n]= m7d[d%10];
        d/=10;n++;
    }while(d!=0);
    for(;n<4;n++) d7seg.led[n]= 0xff;
    d7seg_display();
}
void main()
{
    system_init();
    setup_timer_0 (T0_EXT_H_TO_L | T0_DIV_1);
    set_timer0(0);
    while(true)
    {
        T0= get_timer0();
        if(T0>100) set_timer0(1);
        d7seg_xoa0VN(T0);
    }
}

```

- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: giống bài trước nhưng thực hiện cải tiến là xóa số 0 vô nghĩa.
- e. Giải thích chương trình:

Bài mẫu 413. Giống bài 412 nhưng chỉ giải mã hiển thị khi kết quả đếm của counter thay đổi.

Lưu tên file là “BAI_413_DEM_XUNG_T0_XOA_SO0_VN_CT”.

- a. Mục đích: Biết cách lập trình tối ưu để tiết kiệm thời gian, tránh làm những việc không cần thiết.
- b. Chương trình:

```

#include<tv_boards.c>
unsigned int8 T0,T0_TAM=0xff;
void d7seg_xoa0VN(unsigned int16 d)
{
    unsigned int8 n=0;
    do
    {
        d7seg.led[n]= m7d[d%10];
        d/=10;n++;
    }while(d!=0);
    for(; n<4;n++) d7seg.led[n]= 0xff;
    d7seg_display();
}
void main()

```

```

{
    system_init();
    setup_timer_0 (T0_EXT_H_TO_L | T0_DIV_1);
    set_timer0(0);
    while(true)
    {
        T0= get_timer0();
        if(T0>100) set_timer0(1);
        if(T0!=T0_TAM)
        {
            T0_TAM=T0;
            d7seg_xoa0VN(T0);
        }
    }
}

```

- c. Tiến hành biên dịch và nạp.
- d. Quan sát kết quả: giống bài trước nhưng thực hiện cải tiến là xóa số 0 vô nghĩa.

Bài tập 414. Hãy viết chương trình đếm sản phẩm hiển thị trên 4 led 7 đoạn dùng counter T0, giới hạn đếm từ 240 đến 255. Khi bằng 255 thì quay về 240.

Đúng ra là đếm từ 0 nhưng để đếm 255 thì bạn phải tạo tới 255 xung sẽ mất nhiều thời gian nên để nhanh chóng ta bắt đầu đếm từ 240 để thêm 15 xung nữa là đạt 255 và 1 xung nữa thì về lại 240. Mục đích là cho thấy mạch đếm ở chế độ 8 bit.

Lưu tên file là “BAI_414_DEM_XUNG_T0_240_255”.

Bài tập 415. Chương trình đếm sản phẩm hiển thị trên 4 led 7 đoạn dùng counter T0, giới hạn đếm từ 250 đến 300. Khi bằng 301 thì quay về 250. Giống bài 414 nhưng chế độ đếm 16 bit. Xóa bỏ bit **T0_8_BIT** trong lệnh khởi tạo T0.

Lưu tên file là “BAI_415_DEM_XUNG_T0”.

Chú ý: Để kiểm tra giới hạn đếm tạo xung đếm thì nên bắt đầu giá trị đếm từ 250.

Bài tập 416. Chương trình đếm sản phẩm hiển thị trên 4 led 7 đoạn dùng counter T0, giới hạn đếm từ 0 đến 9999. Khi bằng 10000 thì quay về 1 và có xóa số 0 vô nghĩa.

Lưu tên file là “BAI_416_DEM_XUNG_T0”.

Bài tập 417. Giống bài 413 nhưng có thêm 2 nút nhấn ON, OFF có chức năng cho phép đếm và ngừng đếm. Khi đang ngừng đếm thì 32 led đơn tắt, có xung cũng không đếm. Nhấn ON thì cho phép đếm và 32 led sáng.

Lưu tên file là “BAI_417_DEM_XUNG_T0_START_STOP_32LED”.

4.3 KẾT HỢP ĐIỀU KHIỂN 32 LED VÀ ĐẾM XUNG DÙNG COUNTER T0

Phần này thực hành kết hợp 2 module gồm module 32 led đơn và 4 led 7 đoạn.

Bài mẫu 421. Vừa đếm sản phẩm vừa điều khiển 32 led đơn sáng nhiều chương trình thay đổi bằng 2 nút UP và DW.

Lưu tên file là “BAI_421_DEM_XUNG_T0_QUANG_CAO_32LED”.

- Mục đích: Biết cách lập trình kết hợp 2 module: 32 led đơn với 4 led 7 đoạn.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
unsigned int8 T0,T0_TAM=0xff,n=0,hu=0;
void d7seg_xoa0VN(unsigned int16 d)
{
    unsigned int8 m=0;
    do
    {
        d7seg.led[m]= m7d[d%10];
        d/=10;m++;
    }while(d!=0);
    for(;m<4;m++) d7seg.led[m]= 0xff;
    d7seg_display();
}
void sangdan_pst()
{
    if(n<=32)
    {
        led32.ledx32= ~(0xffffffff<<n);
        led32_display();
        delay_ms(50);
        n++;
    }
    else n=0;
}
void sangdan_tsp()
{
    if(n<=32)
    {
        led32.ledx32= ~(0xffffffff>>n);
        led32_display();
        delay_ms(50);
        n++;
    }
    else n= 0;
}
void tatdan_pst()
{
    if(n<=32)
    {
        led32.ledx32= 0xffffffff<<n;
        led32_display();
        delay_ms(50);
        n++;
    }
    else n=0;
}
```

```

}
void tatdan_tsp()
{
    if(n<=32)
    {
        led32.ledx32= 0xffffffff>>n;
        led32_display();
        delay_ms(50);
        n++;
    }
    else n=0;
}
void main()
{
    system_init();
    setup_timer_0 (T0_EXT_H_TO_L | T0_DIV_1);
    set_timer0(0);
    while(true)
    {
        // Xử lý đếm đếm sản phẩm
        T0= get_timer0();
        if(T0>100) set_timer0(1);
        if(T0!=T0_TAM)
        {
            T0_TAM=T0;
            d7seg_xoa0VN(T0);
        }
        // Xử lý quảng cáo
        if((inputcd(UP)==0)&&(hu<3)){hu++;n=0;}
        if((inputcd(DW)==0)&&(hu>0)){hu--;n=0;}
        if(hu==0)sangdan_pst();
        else if(hu==1)sangdan_tsp();
        else if(hu==2)tatdan_pst();
        else tatdan_tsp();
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: vừa đếm sản phẩm vừa điều khiển 32 led chạy nhiều chương trình.
- f. Giải thích chương trình: tổng hợp 2 chương trình đã viết.

Chú ý: có sử dụng hàm xuất 4 led đoạn có luôn giải mã hiển thị và xóa số 0.

Bài tập 422. Kết hợp bài 413 và bài 326 nhưng chương trình chạy tự động.

Lưu tên file là “BAI_422_DEM_XUNG_T0_QUANG_CAO_32LED”.

Bài tập 423. Hãy viết lưu đồ và chương trình kết hợp bàn phím ma trận với 4 led 7 đoạn có chức năng khi bắt đầu thì hiển thị 4 dấu chấm, nếu nhấn phím số nào thì mã 7 đoạn của phím đó dịch vào bên phải các dữ liệu trước đó dịch sang trái.

Lưu tên file là “BAI_423_KEY_4X4_4LED_DICH”.

Bài tập 424. Hãy viết lưu đồ và chương trình giống bài 423 như chỉ cho dịch các số từ 0 đến 9. Phím C có chức năng xóa về lại từ đầu. Phím B có chức năng undo tối đa 4 cấp. Lưu tên file là “BAI_424_KEY_4X4_4LED_DICH_UNDO4”.

4.4 CÁC CÂU HỎI ÔN TẬP

- Câu 4-1.** Hãy vẽ sơ đồ mạch giao tiếp 4 led quét với các IC 74HC595.
- Câu 4-2.** Hãy vẽ sơ đồ mạch giao tiếp IC 74HC595 của module 4 led với vi điều khiển và IC chốt.
- Câu 4-3.** Ở bài mẫu 401 bạn hãy cho biết khi thực hiện hàm gọi dữ liệu thì chúng sẽ xuất hiện ở đâu trong sơ đồ mạch phần cứng và có chức năng gì? Dòng điện chạy từ đâu đến đâu?
- Câu 4-4.** Hãy cho biết ưu điểm của các hàm 401, 402, 403 và 404.
- Câu 4-5.** Hãy cho biết các thông số của timer T0.
- Câu 4-6.** Hãy vẽ sơ đồ khối của timer T0 và giải thích chức năng các kí hiệu và các bit có trong sơ đồ.
- Câu 4-7.** Hãy cho biết các thanh ghi liên quan đến timer T0.
- Câu 4-8.** Hãy cho biết tên và chức năng từng bit của thanh ghi T0CON.
- Câu 4-9.** Hãy mở file thư viện 18f4550.h và xem các định nghĩa có liên quan đến các thông số của timer T0. Tại sao người ta định nghĩa hằng số T0_DIV_1 có giá trị là 8? Tương tự cho các hằng số khác.
- Câu 4-10.** Hãy viết lại hàm thư viện 405 và giải thích.
- Câu 4-11.** Hãy vẽ sơ đồ mạch tạo xung và giải thích nguyên lý tạo xung.
- Câu 4-12.** Hãy cho biết ưu điểm của bài 413.

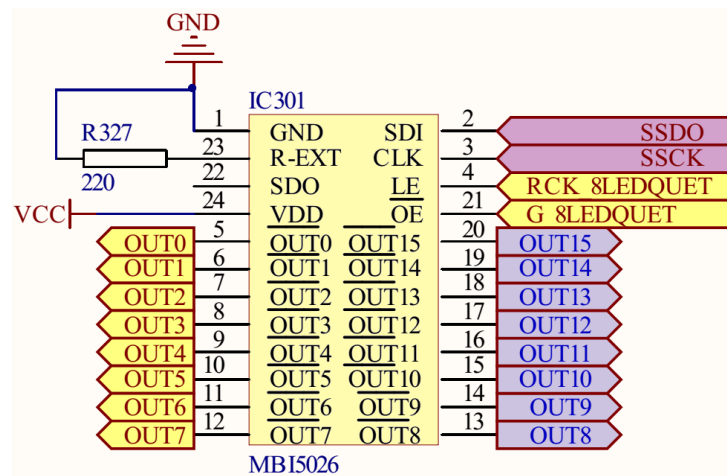
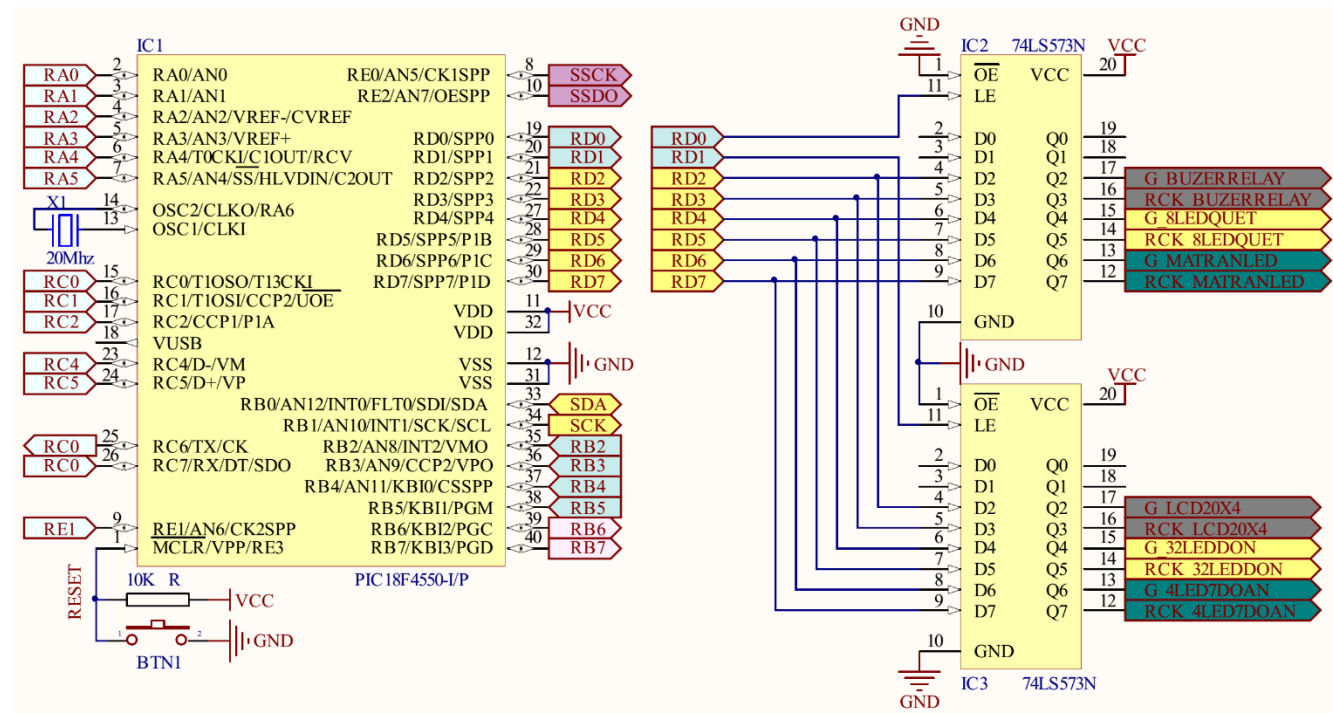
Chương 5: THỰC HÀNH
MODULE 3 – 8 LED 7 ĐOẠN
ANODE CHUNG KẾT NỐI
THEO PP QUÉT DÙNG THANH
GHI DỊCH MBI5026

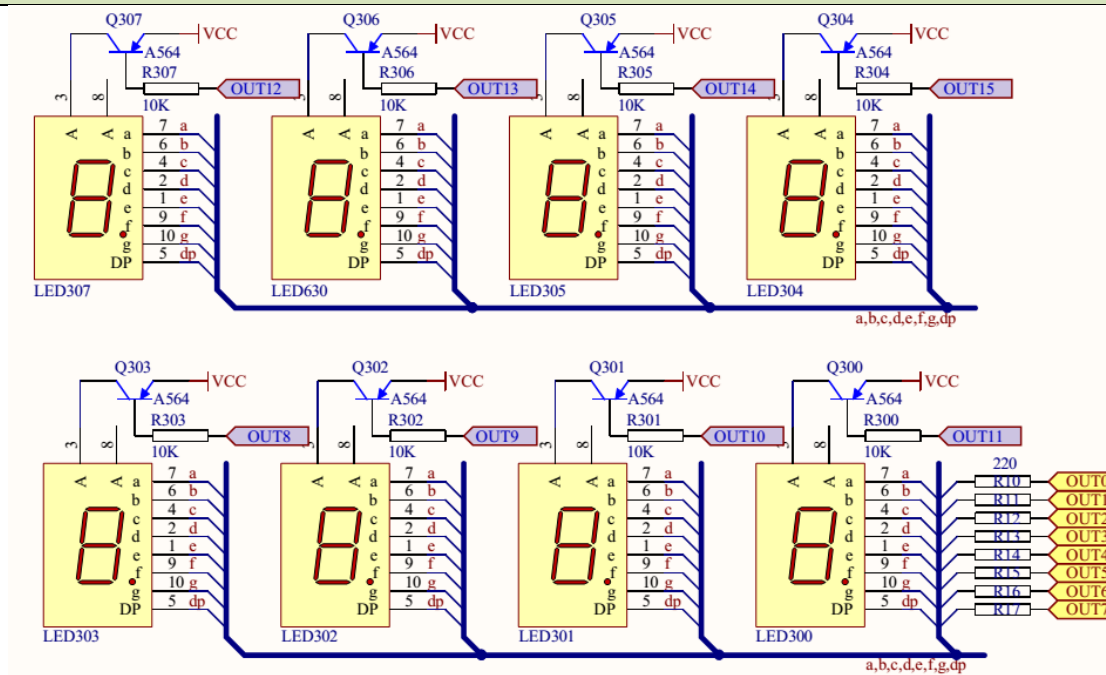
5.1 MẠCH ĐIỆN GIAO TIẾP VI ĐIỀU KHIỂN VỚI 8 LED 7 ĐOẠN QUÉT

5.1.1 Mạch điện giao tiếp vi điều khiển với module 8 led 7 đoạn

Phần này thực hành các bài giao tiếp với 8 led 7 đoạn theo phương pháp quét dùng thanh ghi dịch 16 bit MBI5026.

Mạch điện giao tiếp vi điều khiển với module 8 led 7 đoạn như hình 5-1.





Hình 5-1. Sơ đồ nguyên lý giao tiếp vi điều khiển với module 8 led 7 đoạn.

Trong mạch sử dụng 1 IC thanh ghi MBI5026 để mở rộng port điều khiển 8 led 7 đoạn.

Phần vi điều khiển PIC giao tiếp với IC thanh ghi dịch

Có 4 tín hiệu điều khiển là:

- SSDO dùng để dịch dữ liệu nối tiếp.
- SSCK dùng để cấp xung clock.
- RCK_8LEDQUET dùng điều khiển nạp dữ liệu song song từ bên trong IC thanh ghi dịch ra bên ngoài để điều khiển led.
- G_8LEDQUET dùng để cho phép xuất dữ liệu.

Phần IC thanh ghi dịch giao tiếp với 8 led 7 đoạn quét

IC thanh ghi dịch MBI5026 có 16 bit ngõ ra kết nối với 8 đoạn của 8 led (các đoạn nối chung với nhau) và điều khiển 8 transistor để điều khiển led sáng hay tắt.

Số lượng byte dữ liệu điều khiển module 8 led 7 đoạn theo phương pháp quét là 2 byte: 1 byte cho mã đoạn và 1 byte mã quét điều khiển 8 transistor.

Byte mã quét sẽ có 1 bit bằng 0 và 7 bit còn lại bằng 1 để cho phép 1 led sáng và sau khi dịch ra thì xuất hiện ở 8 ngõ ra từ out8 đến out15 để điều khiển 1 transistor dẫn, 7 transistor còn lại tắt.

Byte mã 7 đoạn cần hiển thị sau khi dịch ra thì xuất hiện ở 8 ngõ ra từ out0 đến out7 để điều khiển 8 đoạn gồm a, b, c, d, e, f, g, dp tích cực mức thấp.

Kết hợp lại thì sẽ có 1 led sáng số thập phân.

Giải thích về dòng điện thì dòng điện sẽ chạy từ nguồn Vcc vào cực E của 1 transistor dẫn rồi tách làm 2 dòng: dòng thứ nhất chạy ra khỏi cực B chạy về IC MBI 5026 – dòng này gọi là dòng phân cực, dòng thứ 2 chạy ra khỏi cực C chạy vào đầu anode chung của 1 led 7 đoạn và chạy ra các đoạn có mức tích cực là mức 0 và chạy về IC MBI5026.

Sau khi cho 1 led sáng trong 1 thời gian ngắn ta sẽ điều khiển led kế sáng và tiếp tục cho đến led thứ 8 rồi lặp lại led thứ 1.

Chu kỳ quét phải lớn hơn đáp ứng của mắt người để ta quan sát không thấy led nhấp nháy.

Bảng 5-1. Bảng dữ liệu điều khiển quét lần lượt các transistor.

TT led sáng	Mã quét 8 led								Mã số hex
	Chỉ cho 1 transistor dẫn để 1 led sáng								
	7	6	5	4	3	2	1	0	
7	1	1	1	1	1	1	1	0	FE
6	1	1	1	1	1	1	0	1	FD
5	1	1	1	1	1	0	1	1	FB
4	1	1	1	1	0	1	1	1	F7
3	1	1	1	0	1	1	1	1	EF
2	1	1	0	1	1	1	1	1	DF
1	1	0	1	1	1	1	1	1	BF
0	0	1	1	1	1	1	1	1	7F

Khả năng mở rộng

Trong kit thực hành dùng 1 IC để điều khiển 8 led, sau này các ứng dụng thực tế cần nhiều led hơn thì bạn cứ kết nối thêm IC thanh ghi nối tiếp vào một cách dễ dàng. Nhưng do phương pháp quét nếu mở rộng nhiều và nếu chu kỳ quét lớn thì led sẽ mờ hay nhấp nháy.

5.1.2 Các hàm điều khiển module 8 led 7 đoạn quét

Module 8 led 7 đoạn quét gồm các hàm xuất dữ liệu ra như sau:

Hàm thứ 501: hàm quét 8 LED 7 đoạn:

```

result s7seg_display()
{
    for(int8 i=0x80,n=0;n<8;n++,i>=>1)
    {
        SSPI_WRITE(i);
        SSPI_WRITE(~s7seg.led[n]);
        output_high(S7SEG_RCK); output_low(S7SEG_RCK);
        delay_ms(1);
        SSPI_WRITE(~0xff);
        output_high(S7SEG_RCK); output_low(S7SEG_RCK);
    }
    return OK;
}

```

Trong đó “s7seg” là biến cấu trúc gồm 8 phần tử mặc định bằng 0xff (để tắt LED)

```
typedef struct
{
    unsigned int8    led[8];
} S7SEG_STRUCT;
S7SEG_STRUCT s7seg={0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff};
```

Do IC MBI5026 là thanh ghi dịch đảo nên khi xuất thì cả 2 byte đều phải đảo mức logic. Byte mã quét led xuất trước, byte hiển thị xuất sau. Các lệnh còn lại giống các hàm đã nêu.

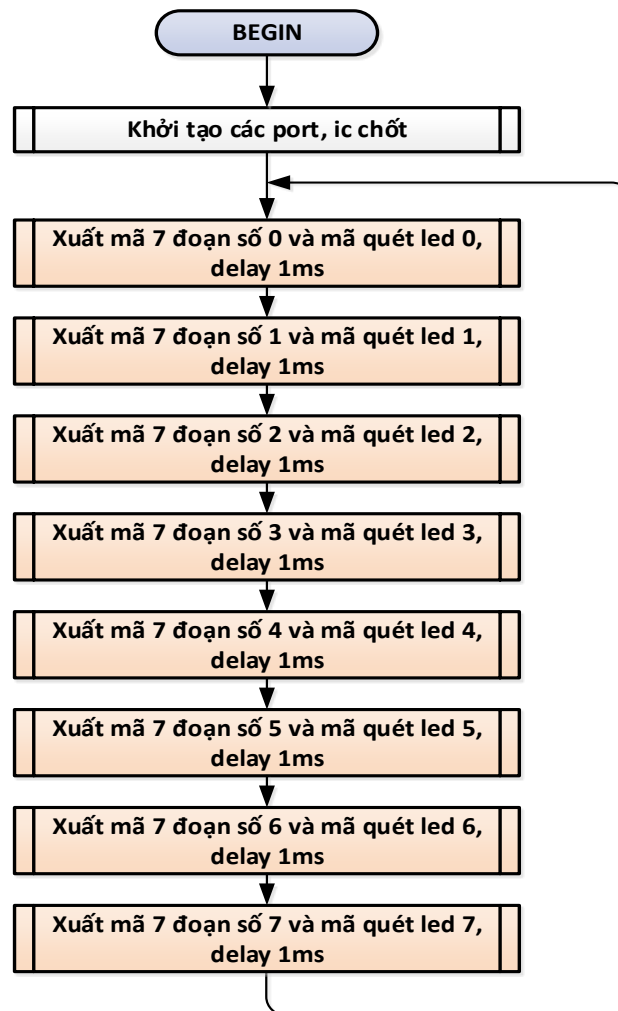
5.1.3 Các chương trình hiển thị 8 led 7 đoạn quét

Phần này thực hành các bài điều khiển 8 led 7 đoạn kết nối theo phương pháp quét để giúp bạn hiểu nguyên lý quét led 7 đoạn. Thấy được hiện tượng quét nhanh và chậm, biết cách làm cho vấn đề phức tạp trở nên đơn giản hơn.

Bài mẫu 501. Chương trình điều khiển 8 LED 7 đoạn hiển thị 8 số từ 0 đến 7.

Lưu tên file là “BAI_501_HIEN THI_8SO”.

- Mục đích: biết cách viết chương trình điều khiển led 7 đoạn theo phương pháp quét.
- Lưu đồ:



Hình 5-2. Lưu đồ điều khiển 8 led sáng từ 0 đến 7 trên module 8 led 7 đoạn.

c. Chương trình:

```
#include<tv_boards.c>
void main()
{
    system_init();
    s7seg.led[7] = m7d[7];
    s7seg.led[6] = m7d[6];
    s7seg.led[5] = m7d[5];
    s7seg.led[4] = m7d[4];
    s7seg.led[3] = m7d[3];
    s7seg.led[2] = m7d[2];
    s7seg.led[1] = m7d[1];
    s7seg.led[0] = m7d[0];
    while(true)
    {
        s7seg_display();
    }
}
```

- d. Tiến hành biên dịch và nạp.
- e. Ở bài trên ta tiến hành giải mã trước while(true) do dữ liệu hiển thị là cố định. Nếu dữ liệu hiển thị có sự thay đổi thì ta phải thực hiện giải mã trong while(true)
- f. Câu hỏi: Dựa vào hàm “s7seg_display();” SV hãy cho biết thời gian sáng và thời gian tắt của mỗi led là bao nhiêu của mỗi lần quét ?

Bài tập 502. Chương trình hiển thị MSSV trên 8 LED 7 đoạn quét .

Lưu tên bài là “BAI_502_HIEN THI_MSSV”.

Bài tập 503. Chương trình khi nhấn nút ON thì hiển thị MSSV lên 8 LED quét. Khi nhấn nút OFF thì tắt MSSV.

Lưu tên file là “BAI_503_BAT_TAT_MSSV”.

Bài tập 504. Chương trình khi nhấn nút ON thì MSSV hiển thị dần từ trái qua phải nếu hiển thị đủ 8 chữ số của MSSV rồi thì sẽ nhấp nháy. Khi nhấn nút OFF thì tắt MSSV.

Lưu tên file là “BAI_504_SANGDAN_NHAPNHAY_MSSV”.

Bài tập 505. Chạy chuỗi “HELLO. CHO 1 LY CAFE” từ phải sang trái và tuần hoàn liên tục (giống LED quảng cáo)

Lưu tên file là “BAI_505_CHAYCHUOI_QUANGCAO”.

5.2 CÁC CHƯƠNG TRÌNH ĐẾM THỜI GIAN HIỂN THỊ 8 LED

Phần này thực hành các bài điều đếm thời gian nhưng hiển thị trên module 8 led 7 đoạn quét.

Sử dụng timer T1 định thời báo ngắt để đếm chính xác về thời gian.

5.2.1 Đếm giây dùng delay

Bài mẫu 511. Chương trình đếm giây từ 00 đến 59 hiển thị trên 2 led bên phải, không cần chính xác về thời gian.

Lưu tên file là “BAI_511_DEM_GIAY”.

- Mục đích: biết cách viết chương trình điều khiển led 7 đoạn quét đếm giây.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
unsigned int8 giay,i;
void main()
{
    system_init();
    while(true)
    {
        for(giay=0;giay<60;giay++)
        {
            s7seg.led[1] = m7d[giay/10%10];
            s7seg.led[0] = m7d[giay%10];
            for(i=0;i<50;i++) s7seg_display();    // delay - hiển thị
        }
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: đếm từ 00 đến 59. Điều chỉnh giá trị của biến I trong chương trình hiển thị delay sẽ thay đổi thời gian đếm, giá trị nhỏ đếm nhanh, giá trị lớn đếm chậm.
- Giải thích chương trình: Khi điều khiển LED quét ta không được phép sử dụng delay lâu vì sẽ gây nhấp nháy LED nên ở chương trình trên ta thay việc delay bằng quét LED 50 lần.

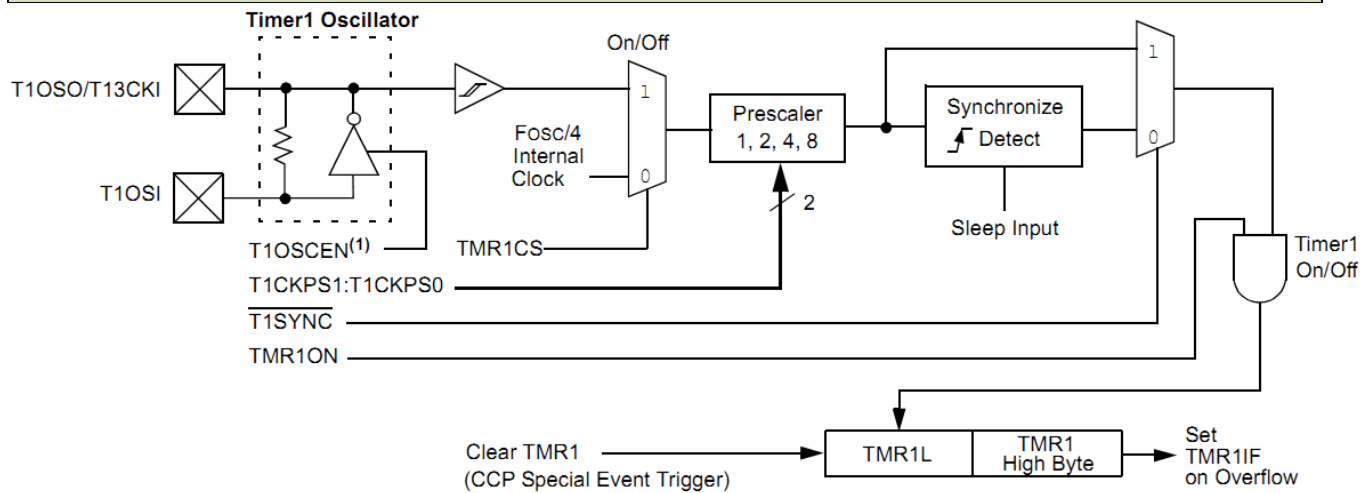
5.2.2 Đếm giây dùng timer T1 định thời

Ở bài đếm giây trên, nếu muốn đếm chính xác thì rất khó để tính toán:

- Thứ nhất ta không biết được thời gian thực hiện 1 lệnh trong ngôn ngữ C là bao nhiêu.
- Thứ hai là khi mở rộng đếm nhiều thông số như giờ phút giây thì các chu kỳ thực hiện không đồng đều nên dẫn đến sai số.

Để đếm chính xác thì ta giao công việc đó cho timer đếm xung nội với các chu xung nội là hằng số và ổn định, chính xác. Khi đó nhiệm vụ của vi điều khiển bây giờ lo quét led và kiểm tra xem đủ thời gian 1 giây chưa, nếu chưa thì tiếp tục quét led, nếu đủ thì tăng giây và kiểm tra giới hạn.

Do có sử dụng timer T1 nên phần này có trình bày lại sơ đồ của timer T1 như hình 5-3:



Hình 5- 3. Sơ đồ khối của timer T1.

Bài mẫu 512. Chương trình đếm giây chính xác dùng timer T1 báo ngắt.
Lưu tên file là “BAI_512_DEM_GIAY_CX”.

- Mục đích: biết sử dụng timer T1 định thời và ngắt để đếm giây chính xác hiển thị 8 led quét.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
unsigned int8 giay=0,bdn=0;
void main()
{
    system_init();
    setup_timer_1(T1_INTERNAL| T1_DIV_BY_8);
    set_timer1(3036);
    enable_interrupts(INT_TIMER1);
    enable_interrupts(GLOBAL);
    while(true)
    {
        if(bdn>=10)        // Tràn đủ 10 lần là được 1 giây
        {
            bdn=bdn-10;
            giay++;
            giay%=60;      // Không chế giây trong phạm vi từ 0 đến 59
        }
        s7seg.led[4] = m7d[giay/10%10]; // Giải mã hàng chục của giây
        s7seg.led[3] = m7d[giay%10];    // Giải mã hàng đơn vị của giây
        s7seg_display();                // Quét LED
    }
}

#INT_TIMER1                // Chương trình con xử lý ngắt T1
void ngat_timer1()         // Không viết gì vào giữa 2 hàng này
{
    set_timer1(3036);
    bdn++;
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: đếm giây chính xác từ 00 đến 59.
- f. Giải thích chương trình:

Khởi tạo timer1 đếm xung nội có tần số 20MHz qua bộ chia 4 còn 5MHz. Sử dụng bộ chia trước có hệ số chia là 8 nên xung vào bộ đếm với tần số còn lại là $5\text{MHz}/8 = 0.625\text{MHz}$ hay bằng 625,000Hz.

Tần số 625,000Hz có nghĩa là trong thời gian 1 giây sẽ có 625,000 xung hay đếm 625,000 xung sẽ tương đương 1 giây.

Sử dụng timer 1 để đếm nhưng timer T1 chỉ có 16 bit và giới hạn đếm lớn nhất là 65536 xung thì tràn, vậy ta thiết lập timer đếm 62500 thì tràn và cho đếm 10 lần.

Mỗi lần tràn thì tăng biến đếm ngắt và nếu biến đếm ngắt bằng 10 thì được 625,000 xung sẽ tương đương giây.

Thiết lập giá trị đếm bắt đầu cho timer1 là 3036 nên timer T1 đếm 62500 xung sẽ tràn và báo ngắt.

Khi ngắt xảy ra thì giá trị của timer T1 được khởi tạo lại và đếm tiếp rồi báo lần tràn tiếp theo.

Muốn thay đổi thời gian đếm cho nhanh để kiểm tra kết quả chương trình thì có thể thay đổi giá trị “T1_DIV_BY_8” bằng “T1_DIV_BY_1” hoặc thay đổi thêm giá trị biến đếm ngắt BDN bằng 2 và hiệu chỉnh lệnh BDN = BDN - 2.

- g. Câu hỏi: Tại sao các chu kỳ thực hiện không đồng đều? Hãy giải thích tại sao không so sánh BDN bằng 10 và khi bằng thì gán bằng 0 mà lại làm phức tạp như trong chương trình.

Bài tập 513. Hãy viết chương trình đếm phút giây chính xác.

Lưu tên file là “BAI_513_DEM_PHUT_GIAY”.

Bài tập 514. Hãy viết chương trình đếm giờ phút giây chính xác.

Lưu tên file là “BAI_514_DEM_GIO_PHUT_GIAY”.

Bài tập 515. Hãy viết chương trình đếm giờ phút giây chính xác có thêm 3 nút UP, DW và MOD để chỉnh thời gian. Nút MOD chọn chế độ chỉnh giây, nhấn nửa chỉnh phút, nhấn nữa thì chỉnh giờ, nhấn nữa thì không cho chỉnh. Nhấn UP thì chỉnh tăng, nhấn DW thì chỉnh giảm.

Lưu tên file là “BAI_515_GIO_PHUT_GIAY_KEY”.

Bài mẫu 516. Cải tiến bài 515 sao cho giá trị nào được chỉnh thì có dấu ‘.’ xuất hiện kế bên.

Lưu tên file là “BAI_516_GIO_PHUT_GIAY_KEY_CAITIEN_DAUCHAM”.

Bài mẫu 517. Cải tiến bài 515 sao cho giá trị nào được chỉnh thì nhấp nháy.

Lưu tên file là “BAI_517_GIO_PHUT_GIAY_KEY_CAITIEN_NHAPNHAY”.

Bài tập 518. Hãy viết chương trình đếm xung ngoại dùng counter T0 hiển thị kết quả đếm trên 8 led quét, giới hạn đếm từ 00 đến 99. Có nút clear khi nhấn thì xóa về 00, có nút start_stop để cho phép đếm hoặc dừng đếm sản phẩm.

Lưu tên file là “BAI_518_DEM_XUNG_T0_SSC”.

5.3 KẾT HỢP 3 MODULE

Phần này thực hành kết hợp 3 module gồm module 32 led đơn, 4 led 7 đoạn và 8 led 7 đoạn quét.

Bài mẫu 521. Kết hợp bài 517 và bài 422 để thực hiện chức năng: vừa đếm thời gian giờ phút giây hiển thị 8 led 7 đoạn quét, vừa đếm sản phẩm hiển thị ở module 4 led và điều khiển 32 led đơn sáng 5 chương trình tùy chọn.

Lưu tên file là “BAI_521_3MODULE_123”.

5.4 CÁC CÂU HỎI ÔN TẬP

Câu 5-1. Hãy vẽ sơ đồ mạch giao tiếp 8 led quét với IC MBI5026.

Câu 5-2. Hãy vẽ sơ đồ mạch giao tiếp IC MBI5026 của module 8 led với vi điều khiển và IC chốt.

Câu 5-3. Hãy giải thích 2 dữ liệu trong bài 501 là 0xFE và 0xC0 khi thực hiện hàm gọi dữ liệu thì chúng sẽ xuất hiện ở đâu trong sơ đồ mạch phần cứng và có chức năng gì. Dòng điện chạy từ đâu đến đâu?

Câu 5-4. Hãy cho biết ưu điểm của bài 503, bài 504.

Câu 5-5. Hãy so sánh bài đếm giây 511 và 404.

Câu 5-6. Hãy vẽ lưu đồ bài đếm giây 511.

Câu 5-7. Hãy vẽ lưu đồ đếm giờ phút giây dựa vào bài 511. Và cho biết tại sao cách này không chính xác.

Câu 5-8. Hãy vẽ lưu đồ bài đếm giây 512. Và cho biết tại sao cách này chính xác.

Câu 5-9. Hãy trình bày cách tính toán thời gian delay 1 giây dùng timer T1.

Câu 5-10. Hãy vẽ lưu đồ bài đồng hồ số 515.

Câu 5-11. Hãy vẽ lưu đồ bài đồng hồ số 516.

Câu 5-12. Hãy vẽ lưu đồ bài đồng hồ số 517.

Chương 6: THỰC HÀNH *MODULE 4 – LCD 20×4, GLCD* *128×64*

6.1 LÝ THUYẾT LCD

6.1.1 Giới thiệu LCD

Ở các phần giao tiếp với led 7 đoạn có hạn chế vì chỉ hiển thị được các số từ 0 đến 9 hoặc số hex từ 0 đến F – không thể nào hiển thị được các thông tin kí tự khác, nhưng chúng sẽ được hiển thị đầy đủ trên LCD.

LCD có rất nhiều dạng phân biệt theo kích thước từ vài kí tự đến hàng chục kí tự, từ 1 hàng đến vài chục hàng. Ví dụ LCD 16×2 có nghĩa là có 2 hàng, mỗi hàng có 16 kí tự. LCD 20×4 có nghĩa là có 4 hàng, mỗi hàng có 20 kí tự.

LCD 40×4 như hình 6-1:



Hình 6-1. Hình ảnh mặt trước của LCD.

6.1.2 Sơ đồ chân của LCD

LCD có nhiều loại và số chân của chúng cũng khác nhau nhưng có 2 loại phổ biến là loại 14 chân và loại 16 chân. Sự khác nhau là các chân nguồn cung cấp cho đèn nền, còn các chân điều khiển thì không thay đổi, khi sử dụng loại LCD nào thì phải tra datasheet của chúng để biết rõ các chân.

Sơ đồ chân của LCD như bảng 6-1.

Bảng 6-1. Các chân của LCD.

Chân số	Tên chân	Input/output	Chức năng tín hiệu
1	VSS	Power	GND
2	VDD	Power	+5V
3	VO	Analog	Contrast Control
4	RS	Input	Register Select. H: data signal, L: instruction signal
5	R/W	Input	Read/Write. H: read mode, L: write mode
6	E	Input	Enable (strobe)
7	D0	I/O	Data (LSB)
8	D1	I/O	Data

9	D2	I/O	Data
10	D3	I/O	Data
11	D4	I/O	Data
12	D5	I/O	Data
13	D6	I/O	Data
14	D7	I/O	Data (MSB)
15	LED_A	input	Backlight Anode
16	LED_K	input	Backlight Cathode

Trong 16 chân của LCD được chia ra làm 4 dạng tín hiệu như sau:

Các chân cấp nguồn: Chân số 1 là chân nối mass (0V), chân thứ 2 là Vdd nối với nguồn +5V. Chân thứ 3 dùng để chỉnh contrast thường nối với biến trở, chỉnh cho đến khi thấy được kí tự thì ngừng, trong bộ thực hành thì đã chỉnh rồi.

Các chân điều khiển: Chân số 4 là chân RS dùng để điều khiển lựa chọn thanh ghi. Chân R/W dùng để điều khiển quá trình đọc và ghi. Chân E là chân cho phép dạng xung chốt.

Các chân dữ liệu D7÷D0: Chân số 7 đến chân số 14 là 8 chân dùng để trao đổi dữ liệu giữa thiết bị điều khiển và LCD.

Các chân LED_A và LED_K: Chân số 15, 16 là 2 chân dùng để cấp nguồn cho đèn nền để có thể nhìn thấy vào ban đêm.

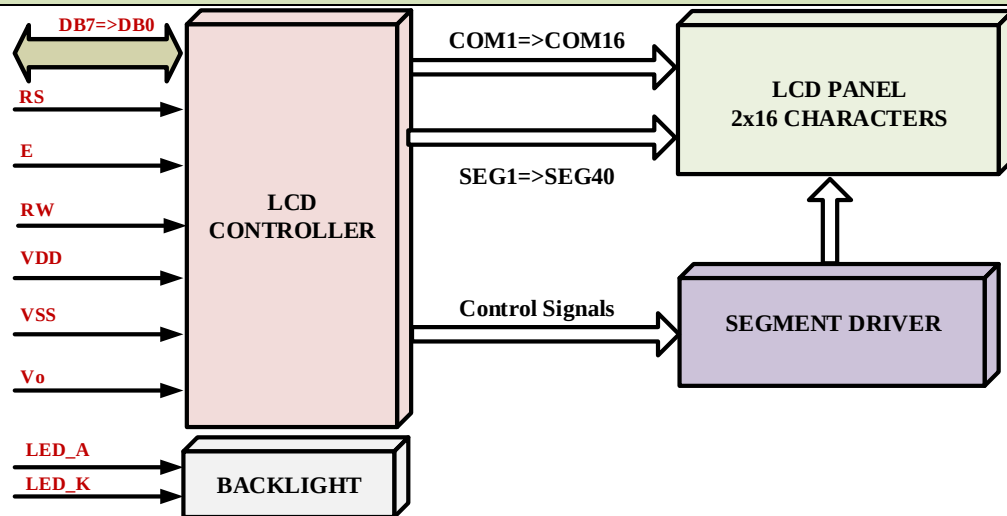
6.1.3 Bộ điều khiển LCD và các vùng nhớ

Để điều khiển LCD thì có các IC chuyên dùng được tích hợp bên dưới LCD có mã số 447801 đến các IC 447809 như trong hình 6-2.



Hình 6-2. Hình ảnh mặt sau của LCD.

Sơ đồ khối của bộ điều khiển LCD như hình sau:



Hình 6-3. Sơ đồ khối của bộ điều khiển LCD

Sơ đồ khối gồm có 4 phần: bộ điều khiển LCD, bảng kí tự LCD, bộ thúc tín hiệu các đoạn, đèn nền.

Bộ điều khiển LCD có ba vùng nhớ nội, mỗi vùng có chức năng riêng. Bộ điều khiển phải khởi động trước khi truy cập bất kỳ vùng nhớ nào.

a. Bộ nhớ DDRAM

Bộ nhớ chứa dữ liệu để hiển thị (Display Data RAM: DDRAM) lưu trữ những mã kí tự để hiển thị lên màn hình. Mã kí tự lưu trữ trong vùng DDRAM sẽ tham chiếu với từng bitmap kí tự được lưu trữ trong CGROM đã được định nghĩa trước hoặc đặt trong vùng do người sử dụng định nghĩa.

b. Bộ phát kí tự ROM - CGROM

Bộ phát kí tự ROM (Character Generator ROM: CGROM) chứa các kiểu bitmap cho mỗi kí tự được định nghĩa trước mà LCD có thể hiển thị, như được trình bày bằng mã ASCII.

Mã kí tự lưu trong DDRAM cho mỗi vùng kí tự sẽ được tham chiếu đến một vị trí trong CGROM.

Ví dụ, mã kí tự số hex 0x53 lưu trong DDRAM được chuyển sang dạng nhị phân 4 bit cao là DB[7:4] = "0101" và 4 bit thấp là DB[3:0] = "0011" chính là kí tự chữ 'S' sẽ hiển thị trên màn hình LCD.

c. Bộ phát kí tự RAM – CGRAM

Bộ phát kí tự RAM (Character Generator RAM: CG RAM) cung cấp vùng nhớ để tạo ra 8 kí tự tùy ý. Mỗi kí tự gồm 5 cột và 8 hàng.

6.1.4 Các lệnh điều khiển LCD

Tập lệnh của LCD

Bảng 6-2. Các lệnh điều khiển LCD.

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Mô tả	clock
(1) NOP	0	0	0	0	0	0	0	0	0	0	No operation	0

(2) Clear display	0	0	0	0	0	0	0	0	0	1	Write "20H" to DDRAM and set DDRAM address counter to "00H".	1.52ms
(3) Cursor home	0	0	0	0	0	0	0	0	1	0	Sets address counter to zero, returns shifted display to original position. DDRAM contents remain unchanged.	39μs
(4) Entry mode set	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction, and specifies automatic shift.	39μs
(5) Display control	0	0	0	0	0	0	1	D	C	B	Turns display (D), cursor on/off (C) or cursor blinking (B).	39μs
(6) Cursor display shift	0	0	0	0	0	1	S/C	R/L	0	0	Set cursor moving and display shift control bit, and the direction, without changing DDRAM data.	39μs
(7) Function set	0	0	0	0	1	DL	N	F	0	0	Set interface data length (DL : 4-bit/8-bit), numbers of display line (N : 1-line/2-line), display font type(F : 5 X 8 dots/ 5 X 11 dots)	39μs
(8) Set CGRAM addr	0	0	0	1	AC5	AC4	AC3	AC2	AC1	AC0	Set CGRAM address in address counter	39μs
(9) Set DDRAM addr	0	0	1	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Set DDRAM address in address counter	39μs

Đại Học Sư Phạm Kỹ Thuật											Khoa Điện – Điện Tử	
(10) Read Busy Flag and Address	0	1	BF	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Whether during internal operation or not can be known by reading BF. The contents of address counter can also be read.	0 μ s
(11) Write data to RAM	1	0	D7	D6	D5	D4	D3	D2	D1	D0	Write data to CGRAM or DDRAM	43 μ s
(12) Read data from RAM	1	1	D7	D6	D5	D4	D3	D2	D1	D0	Read data from CGRAM or DDRAM	43 μ s

Lệnh thiết chức năng “Function set”: lệnh này dùng để thiết lập chức năng giao tiếp.

Bảng 6-3. Mã lệnh Function set.

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Số hex
(7) Function set	0	0	0	0	1	DL	N	F	0	0	
			0	0	1	1	1	0	0	0	0x38

Bit DL (data length) = 1 thì cho phép giao tiếp 8 đường data D7 ÷ D0, nếu bằng 0 thì cho phép giao tiếp 4 đường D7 ÷ D4.

Bit N (number of line) = 1 thì cho phép hiển thị 2 hàng, nếu bằng 0 thì cho phép hiển thị 1 hàng.

Bit F (font) = 1 thì cho phép hiển thị với ma trận 5×8, nếu bằng 0 thì cho phép hiển thị với ma trận 5×11.

Các bit cao còn lại là hằng số không đổi.

Lệnh điều khiển hiển thị “Display Control”:

Bảng 6- 4. Mã lệnh Display control.

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Số hex
(5) Display control	0	0	0	0	0	0	1	D	C	B	
			0	0	0	0	1	1	0	0	0x0C

Bit D: cho phép LCD hiển thị thì D = 1, không cho hiển thị thì bit D = 0.

Bit C: cho phép con trỏ hiển thị thì C = 1, không cho hiển thị con trỏ thì bit C = 0.

Bit B: cho phép con trỏ nhấp nháy thì B = 1, không cho con trỏ nhấp nháy thì bit B = 0.

Với các bit như trên thì để hiển thị phải cho D = 1, 2 bit còn lại thì tùy chọn, trong thư viện thì cho 2 bit đều bằng 0, không cho phép mở con trỏ và nhấp nháy, nếu bạn không thích thì hiệu chỉnh lại.

Lệnh xoá màn hình “Clear Display”:

Bảng 6-5. Mã lệnh Clear Display.

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Số hex
(2) Clear display	0	0	0	0	0	0	0	0	0	1	0x01

Khi thực hiện lệnh này, vi điều khiển của LCD sẽ ghi kí tự khoảng trắng “ ” vào tất cả các vùng nhớ hiển thị và bộ đếm địa chỉ được xoá về 0. Vì phải ghi 128 kí tự vào 128 byte ô nhớ DDRAM nên mất 1,52ms. Khá nhiều so với các lệnh khác.

Lệnh thiết lập lỗi vào “Entry mode set”:

Bảng 6-6. Mã lệnh entry mode.

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Số hex
(4)Entry mode set	0	0	0	0	0	0	0	1	I/D	S	
			0	0	0	0	0	1	1	0	0x06

Lệnh này dùng để thiết lập lỗi vào cho các kí tự hiển thị.

Bit I/D = 1 thì con trỏ tự động tăng lên 1 mỗi khi có 1 byte dữ liệu ghi vào bộ hiển thị, khi I/D = 0 thì con trỏ sẽ tự động giảm đi 1 mỗi khi có 1 byte dữ liệu ghi vào bộ hiển thị.

Bit S = 1 thì cho phép dịch chuyển dữ liệu mỗi khi nhận 1 byte hiển thị.

Trong thư viện có định nghĩa lệnh này như sau:

Lệnh thiết lập lỗi vào “Cursor/Display shift”:

Bảng 6-7. Mã lệnh entry mode.

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Số hex
(6) Cursor display shift	0	0	0	0	0	1	S/C	R/L	0	0	
Cho phép dịch trái			0	0	0	1	1	0	0	0	0x18
Cho phép dịch phải			0	0	0	1	1	1	0	0	0x1C

Lệnh này dùng để điều khiển dịch màn hình và di chuyển con trỏ:

Bit SC: SC = 1 cho phép dịch chuyển, SC = 0 thì không cho phép.

Bit RL xác định hướng dịch chuyển: RL = 1 thì dịch phải, RL = 0 thì dịch trái. Nội dung bộ nhớ DDRAM vẫn không đổi.

Vậy khi cho phép dịch thì có 2 tùy chọn: dịch trái và dịch phải.

Trong thư viện thì có định nghĩa lệnh này như sau:

#DEFINE LCD_SHIFT_LEFT 0X18

#DEFINE LCD_SHIFT_RIGHT 0X1C

Lệnh di chuyển con trỏ về đầu màn hình “Cursor Home”: khi thực hiện lệnh này thì bộ đếm địa chỉ được xoá về 0, phần hiển thị trở về vị trí gốc đã bị dịch trước đó. Nội dung bộ nhớ RAM hiển thị DDRAM không bị thay đổi.

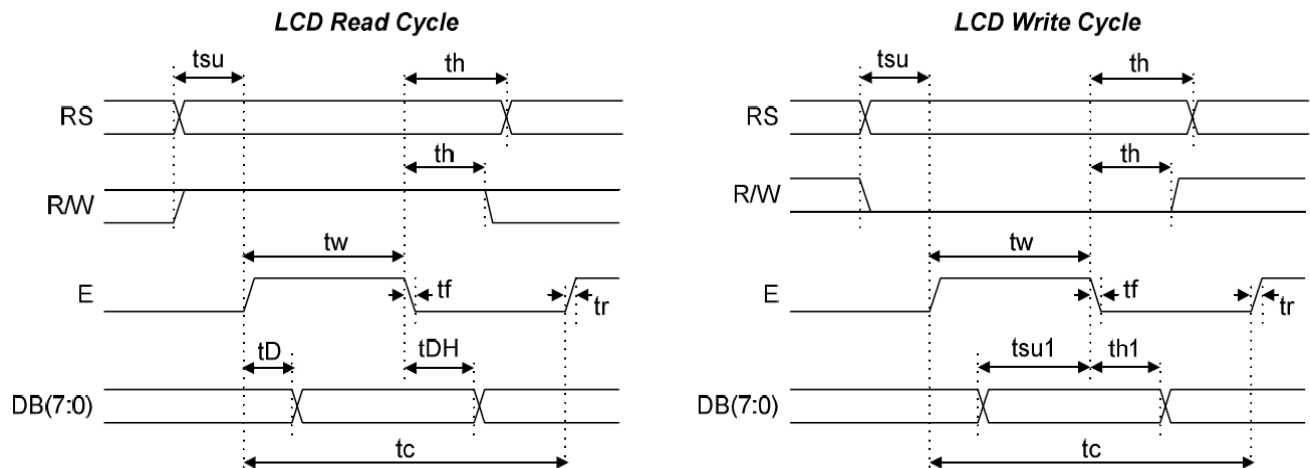
Lệnh thiết lập địa chỉ cho bộ nhớ RAM phát kí tự “Set CGRAM Addr”: lệnh này dùng để thiết lập địa chỉ cho bộ nhớ RAM phát kí tự.

Lệnh thiết lập địa chỉ cho bộ nhớ RAM hiển thị “Set DDRAM Addr”: lệnh này dùng để thiết lập địa chỉ cho bộ nhớ RAM lưu trữ các dữ liệu hiển thị.

Hai lệnh cuối cùng là lệnh đọc và lệnh ghi dữ liệu LCD.

6.1.5 Các hoạt động đọc ghi LCD

Có 2 chế độ hoạt động là đọc và ghi LCD, hai dạng sóng tương ứng như hình 6-4.



Hình 6-4. Dạng sóng điều khiển của LCD.

Hoạt động đọc:

- Điều khiển tín hiệu RS.
- Điều khiển tín hiệu R/W lên mức 1.
- Điều khiển tín hiệu E lên mức cao để cho phép.
- Đọc dữ liệu từ bus dữ liệu DB7÷DB0 (data bus).
- Điều khiển tín hiệu E về mức thấp.

Hoạt động ghi:

- Điều khiển tín hiệu RS.
- Điều khiển tín hiệu R/W xuống mức 0.
- Điều khiển tín hiệu E lên mức cao để cho phép.
- Xuất dữ liệu ra bus dữ liệu DB7÷DB0 (data bus).
- Điều khiển tín hiệu E về mức thấp.

Các thông số thời gian cho ở bảng 6-8.

Bảng 6-8. Cho biết các thông số thời gian của LCD.

Parameter	Symbol	Min	Max	Unit	Test Pin
Enable cycle time	tc	500		ns	E
Enable High pulse width	tw	220		ns	E
Enable rise/fall time	tr, tf		25	ns	E
RS, R/W setup time	tsu	40		ns	RS, R/W
RS, R/W hold time	th	10		ns	RS, R/W
Read data output delay	tD	60	120	ns	DB0-DB7
Read data hold time	tDH	20		ns	DB0-DB7
Write data setup time	tsu1	40		ns	DB0-DB7
Write data hold time	th1	10		ns	DB0-DB7

6.1.1 Mã ASCII

LCD sử dụng mã ASCII để hiển thị các kí tự, các số, các kí hiệu, ... có tổng cộng là 256 kí tự. Mã ASCII như ở bảng 6-9.

Trong các bài thực hành hiển thị các kí tự và các bài đếm thì ta hiển thị các con số từ 0 đến 9 và tra trong bảng thì ta thấy các con số sẽ có mã từ 0x30 đến 0x39.

Để hiển thị trên Led 7 đoạn các giá trị đếm từ giá trị số nhị phân thì ta phải chuyển sang số BCD hàng đơn vị, hàng chục, hàng trăm, ... rồi lấy mã 7 đoạn tương ứng.

Để hiển thị trên LCD từ giá trị số nhị phân thì ta phải chuyển sang số BCD rồi sau đó cộng thêm với 0x30 mới hiển thị được trên LCD.

Bảng 6-9. Bảng mã ASCII.

Lower 4 Bits	Upper 4 Bits	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
xxxx0000	CG RAM (1)			0	a	P	`	P					-	3	3	α	p
xxxx0001	(2)			!	1	A	Q	a	4				6	7	7	ä	q
xxxx0010	(3)			"	2	B	R	b	r				「	イ	ツ	β	θ
xxxx0011	(4)			#	3	C	S	c	s				」	ウ	チ	ε	ω
xxxx0100	(5)			\$	4	D	T	d	t				、	エ	ト	μ	Ω
xxxx0101	(6)			%	5	E	U	e	u				・	オ	ナ	℃	Û
xxxx0110	(7)			&	6	F	V	f	v				ヲ	カ	ニ	ρ	Σ
xxxx0111	(8)			'	7	G	W	g	w				ア	キ	ヌ	g	π
xxxx1000	(1)			(	8	H	X	h	x				ィ	ク	ネ	「	又
xxxx1001	(2)			)	9	I	Y	i	y				ッ	ク	ル	「	y
xxxx1010	(3)			*	:	J	Z	j	z				エ	コ	レ	j	千
xxxx1011	(4)			+	;	K	L	k	l				オ	サ	ヒ	※	万
xxxx1100	(5)			,	<	L	¥	l	l				ヤ	シ	フ	¢	円
xxxx1101	(6)			-	=	M	I	m	}				ユ	ズ	ン	£	÷
xxxx1110	(7)			.	>	N	^	n	÷				ヨ	セ	ホ	ん	
xxxx1111	(8)			/	?	O	_	o	←				ッ	ソ	マ	〇	■

6.1.2 Vùng nhớ hiển thị DDRAM

Trong chip điều khiển LCD, đã tích hợp đầy đủ bảng mã ASCII, muốn hiển thị kí tự nào thì ta tiến hành gởi mã của kí tự đó ra vùng nhớ DDRAM, khi đó vi điều khiển của LCD sẽ lấy mã ma trận của kí tự đó đem ra hiển thị trên màn hình.

Trong tập lệnh LCD thì có lệnh thiết lập vùng nhớ để hiển thị là lệnh thứ 9 Set DDRAM addr. Vùng nhớ này chứa mã để hiển thị, trong lệnh có 7 bit địa chỉ AC6 ÷ AC0

Bảng 6-10. Địa chỉ vùng nhớ hiển thị DDRAM.

	Set DDRAM address in address counter								Không tính D7 = 1	Tính D7 = 1
	D7	D6	D5	D4	D3	D2	D1	D0		
(9) Set DDRAM addr	1	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Số hex	Số hex
Địa chỉ đầu	1	0	0	0	0	0	0	0	0x00	0x80
	1	0	0	0	0	0	0	1	0x01	0x81

	1	...							
Địa chỉ cuối cùng	1	1	1	1	1	1	1	0x7F	0xFF

Nếu tính luôn bit lệnh $D7 = 1$ thì địa chỉ vùng nhớ này từ 0×80 đến $0 \times FF$ có tổng cộng 128 byte có thể chứa được 128 mã kí tự để hiển thị.

Vùng nhớ này được chia ra làm 2:

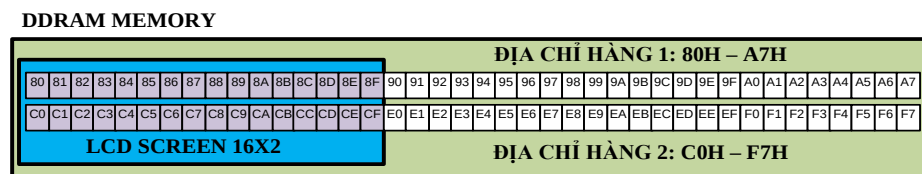
- Vùng 1: các ô nhớ có địa chỉ từ 0×80 đến $0 \times BF$ sẽ lưu các kí tự hiển thị hàng 1 của LCD.
- Vùng 2: các ô nhớ có địa chỉ từ $0 \times C0$ đến $0 \times FF$ sẽ lưu các kí tự hiển thị hàng 2 của LCD.

Các nhà chế tạo LCD sản xuất ra nhiều loại LCD khác nhau, ví dụ LCD 16×2 có 2 hàng và mỗi hàng hiển thị được 16 kí tự, khi đó:

Hàng 1 chỉ hiển thị được 16 kí tự có địa chỉ từ 0×80 đến $0 \times 8F$, từ 0×90 đến $0 \times BF$ nằm ẩn bên trong, muốn hiển thị thì phải dùng lệnh dịch.

Hàng 2 chỉ hiển thị được 16 kí tự có địa chỉ từ $0 \times C0$ đến $0 \times CF$, từ $0 \times B0$ đến $0 \times FF$ nằm ẩn bên trong, muốn hiển thị thì phải dùng lệnh dịch.

Xem hình sau:



Hình 6-5. Hiển thị 32 kí tự ở LCD 16×2

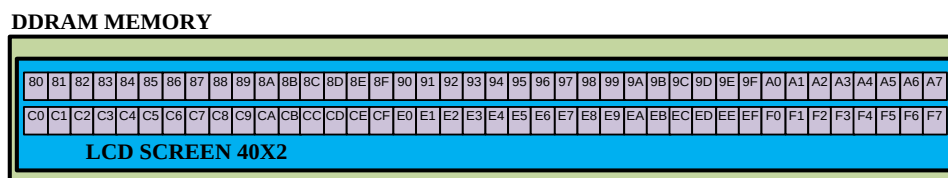
Bộ thí nghiệm vi điều khiển dùng LCD 20×4 , có 4 hàng và mỗi hàng 20 kí tự. Nhưng nếu theo bố trí bộ nhớ thì thông số 20×4 sẽ trở thành 40×2 có nghĩa là có 2 hàng và mỗi hàng có 40 kí tự.

Hàng 1 chỉ hiển thị được 40 kí tự có địa chỉ từ 0×80 đến $0 \times A7$, từ $0 \times A8$ đến $0 \times BF$ nằm ẩn bên trong, muốn hiển thị thì phải dùng lệnh dịch.

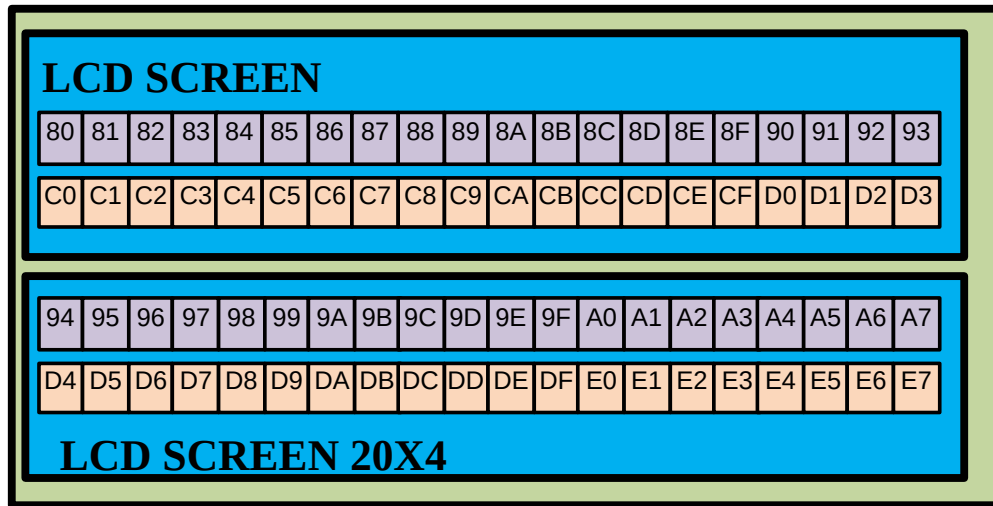
Hàng 2 chỉ hiển thị được 16 kí tự có địa chỉ từ $0 \times C0$ đến $0 \times E7$, từ $0 \times E8$ đến $0 \times FF$ nằm ẩn bên trong, muốn hiển thị thì phải dùng lệnh dịch.

Khi chế tạo 2 hàng và mỗi hàng 40 kí tự thì LCD sẽ dài và mỏng nên nhà chế tạo bố trí lại thành 20×4 , khi đó bạn sẽ thấy hàng 1 và hàng 3, hàng 2 và hàng 4 liên tục nhau.

Xem hình sau:



DDRAM MEMORY



Hình 6-6. Hiển thị 80 kí tự ở LCD 20×4.

Địa chỉ chi tiết từng kí tự trên từng hàng cho ở bảng theo sau.

Bảng 6-11. Địa chỉ từng kí tự của LCD 20×4.

ĐC	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	90	91	92	93
Hàng 1																				
ĐC	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF	D0	D1	D2	D3
Hàng 2																				
ĐC	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F	A0	A1	A2	A3	A4	A5	A6	A7
Hàng 3																				
ĐC	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF	E0	E1	E2	E3	E4	E5	E6	E7
Hàng 4																				

Trong LCD còn vùng nhớ CGRAM được trình bày ở phần theo sau.

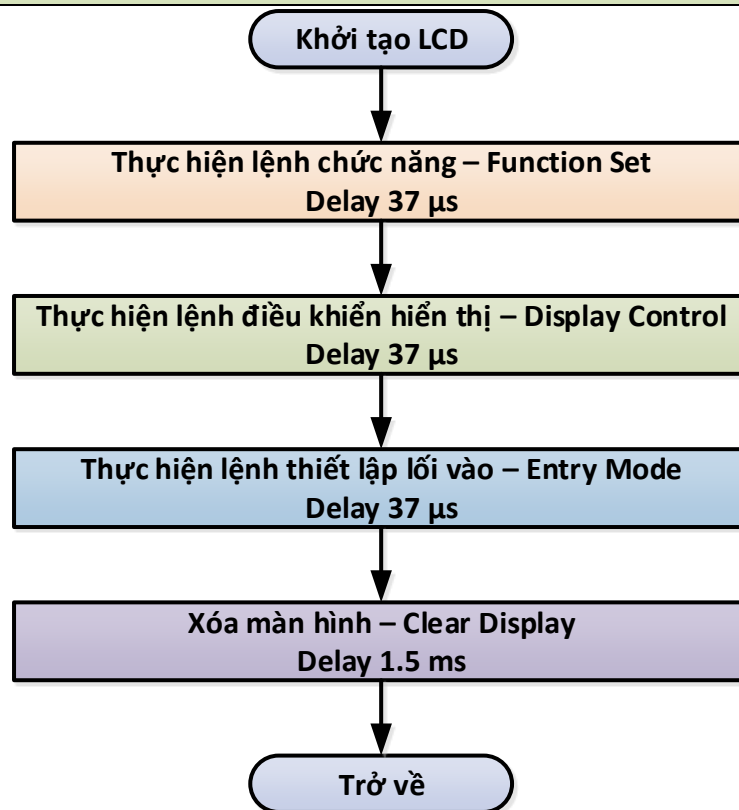
6.1.3 Lưu đồ khởi tạo LCD

Khi sử dụng LCD để hiển thị các kí tự thì ta phải khởi tạo LCD. Trình tự khởi tạo LCD được thể hiện qua hình 6-7.

Sau khi cấp điện thì phải đợi 20ms sau đó thì tiến hành thực hiện lệnh khởi tạo Function Set – là lệnh định cấu hình giao tiếp bus dữ liệu bao nhiêu bit nên phải thực hiện đầu tiên.

Tương tự cho các lệnh còn lại và chú ý thời gian, đây là trình tự do nhà sản xuất cung cấp.

Sau khi khởi tạo xong thì mới gửi dữ liệu ra LCD để hiển thị.

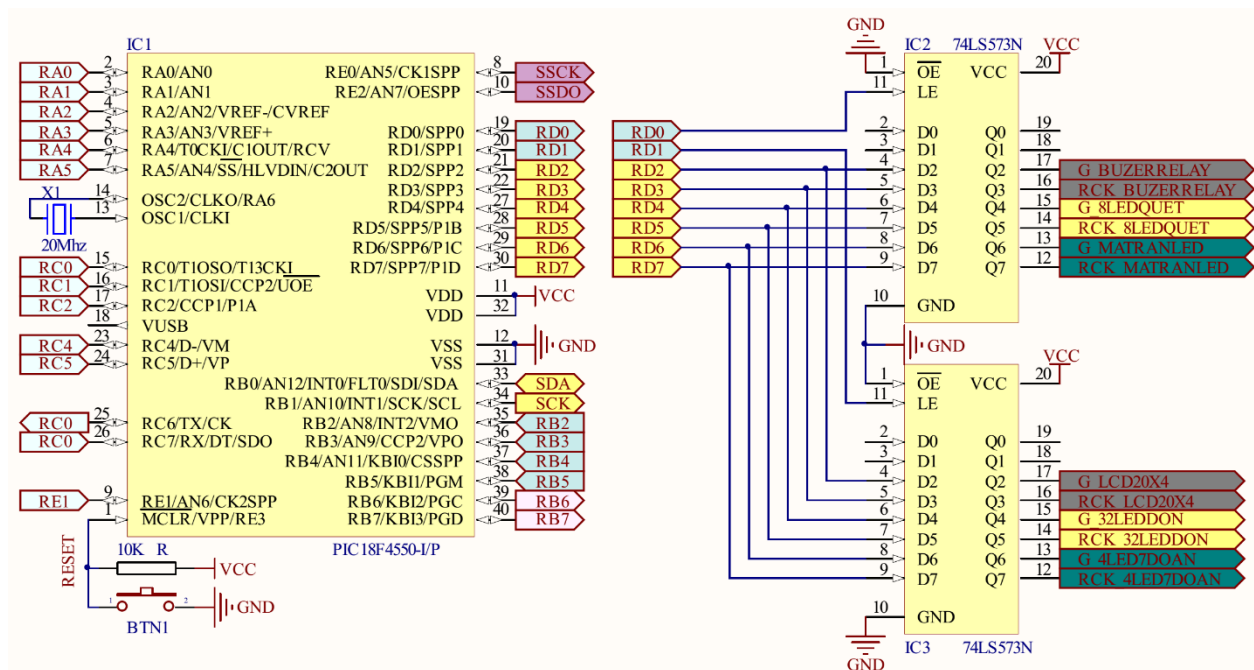


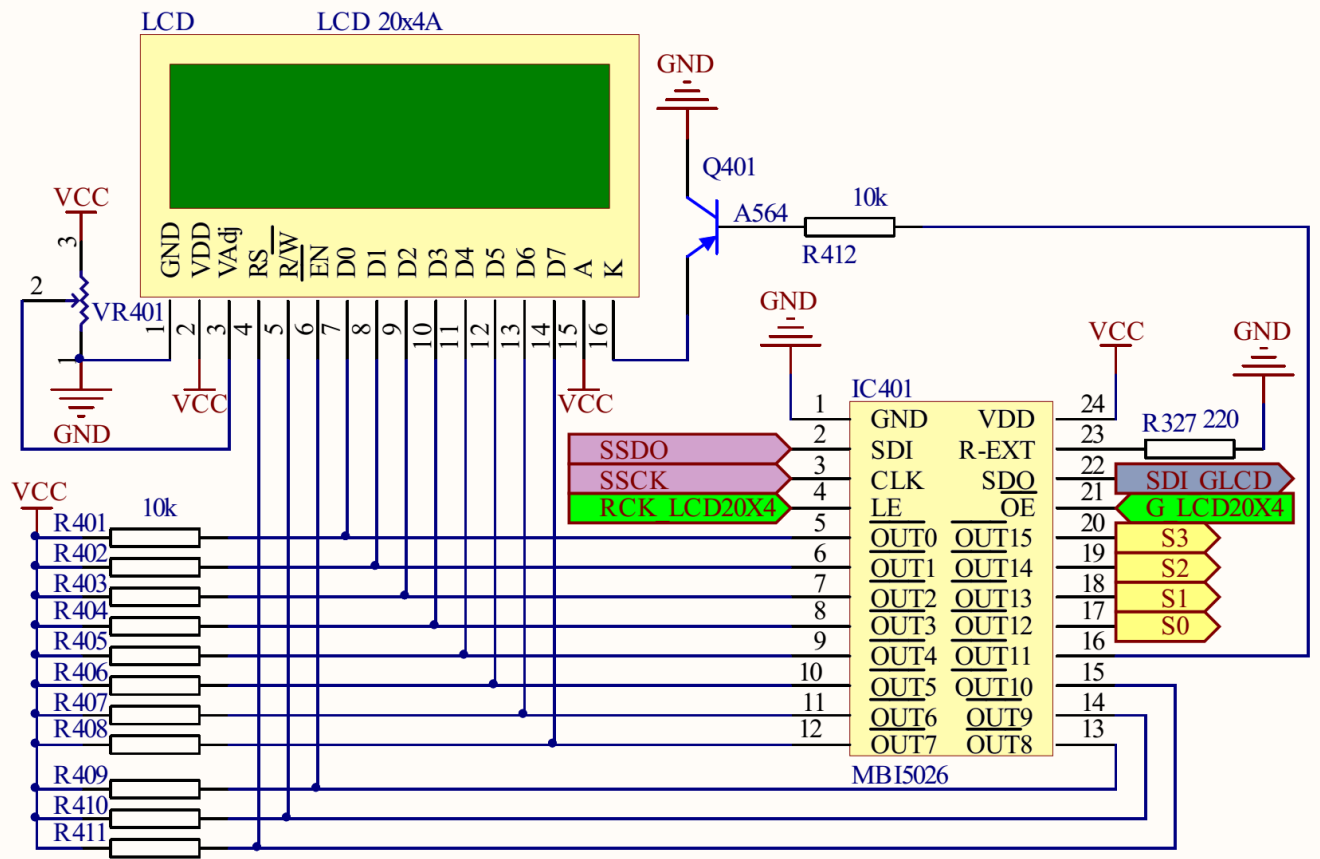
Hình 6-7. Trình tự khởi tạo LCD.

6.2 LCD TRONG KIT THỰC HÀNH

6.2.1 Mạch giao tiếp vi điều khiển với LCD

Trong phần này sẽ trình bày phân giao tiếp vi điều khiển PIC18F4550 với LCD như hình 6-8.





Hình 6-8. Giao tiếp vi điều khiển PIC với LCD qua thanh ghi dịch MBI5026.

Như đã trình bày, trong bộ thực hành sử dụng IC thanh ghi MBI5026 để mở rộng port điều khiển LCD.

Phần vi điều khiển PIC giao tiếp với IC thanh ghi dịch MBI5026:

Có 4 tín hiệu điều khiển là:

- SSDO dùng để dịch dữ liệu nối tiếp.
- SSCK dùng để cấp xung clock.
- RCK_LCD20X4 dùng để điều khiển nạp dữ liệu song song từ bên trong IC thanh ghi dịch MBI5026 ra bên ngoài để điều khiển LCD.
- G_LCD20X4 dùng để cho phép xuất dữ liệu.

Phần IC thanh ghi dịch MBI5026 giao tiếp với LCD:

Có 16 tín hiệu ngõ ra với các chức năng như sau:

- Từ out0 đến out7: giao tiếp với bus 8 dữ liệu của LCD theo chế độ 8 bit (vẫn có thể thực hành được chế độ 4 bit).
- Ba tín hiệu out8, out9, out10 làm các tín hiệu điều khiển LCD tương ứng với E, RW và RS.
- Tín hiệu out11 làm tín hiệu điều khiển đóng ngắt transistor cho phép tắt mở đèn nền để tiết kiệm nguồn năng lượng (P) và tăng tuổi thọ của đèn nền khi không sử dụng. Mức logic 0 làm đèn nền sáng, mức 1 hoặc khi ngõ ra ở trạng thái tổng trở cao thì đèn nền tắt.

- Bốn tín hiệu S0 đến S3 dùng để giao tiếp IC cảm biến ánh sáng.
- Ngõ ra SDO nối sang một IC MBI 5026 khác để điều khiển GLCD và sẽ trình bày ở phần GLCD.

6.2.2 Dữ liệu và hàm điều khiển LCD và GLCD

Theo kết nối phần cứng đã trình bày thì dữ liệu điều khiển LCD gồm có 2 byte:

- Byte thứ 1 (LCDDATA) dùng để tải dữ liệu.
- Byte thứ 2 (LCDCONTROL) dùng để tải 4 tín hiệu điều khiển E, RW, RS, P (không tính đến 4 tín hiệu S0 đến S3).

Trong kit thực hành còn có module GLCD chung các tín hiệu điều khiển với LCD và cũng dùng IC MBI 5026, phần cứng giao tiếp, dữ liệu điều khiển cũng tương tự như LCD. Có 2 byte để điều khiển GLCD.

- Byte thứ 1 (GLCDDATA) dùng để tải dữ liệu.
- Byte thứ 2 (GLCDCONTROL) dùng để tải 6 tín hiệu điều khiển sẽ được trình bày chi tiết ở GLCD.

Vậy có tổng cộng 4 byte dữ liệu khi giao tiếp với module LCD, GLCD.

Trình tự gọi dữ liệu:

- GLCDCONTROL
- GLCDDATA
- LCDCONTROL
- LCDDATA

Vi điều khiển giao tiếp với cả 2 module LCD và GLCD nên phát sinh các trường hợp thực hành như sau:

- Chỉ thực hành các bài điều khiển LCD.
- Chỉ thực hành các bài điều khiển GLCD.
- Kết hợp cả 2 module LCD và GLCD.

Các hàm gọi dữ liệu ra LCD, GLCD và các biến để lưu các dữ liệu và các tín hiệu điều khiển như sau:

Hàm thứ 601: xuất 4 byte ra module LCD và GLCD:

```
typedef struct
{
    RBDC_UNION          rbdc;           // Biến quản lý Relay, Buzzer, Động cơ
    unsigned int8        lcddata;       // SV sẽ được học ở các bài sau
    unsigned int8        lcdcontrol;
    unsigned int8        glcddata;
    unsigned int8        glcdcontrol;
}SYSTEM_VAR_STRUCT;
SYSTEM_VAR_STRUCT system_var={0,0,0,0,0};

result private_write_glcd_lcd()
{
    SSPI_WRITE(system_var.glcdcontrol);
}
```

```

SSPI_WRITE(system_var.glcddata);
SSPI_WRITE(system_var.lcdcontrol);
SSPI_WRITE(system_var.lcddata);
output_high(LCD_RCK);
output_low(LCD_RCK);
return OK;
}

```

Hàm này gọi lần lượt 4 byte ra 2 IC thành ghi dịch.

Hàm thứ 602: xuất dữ liệu ra module LCD còn dữ liệu GLCD thì không đổi:

```

result write_lcd20x4(unsigned int8 lcd_signal, lcd_ins_hthi)
{
    system_var.lcdcontrol = ~lcd_signal;
    system_var.lcddata     = ~lcd_ins_hthi;
    private_write_glcd_lcd();
    return OK;
}

```

Nếu bạn chỉ điều khiển 1 module LCD thì nên dùng hàm này.

Hàm thứ 603: xuất dữ liệu ra GLCD còn dữ liệu LCD thì không đổi:

```

result write_glcd128x64(unsigned int8 glcd_signal, glcd_ins_hthi)
{
    system_var.glcdcontrol = ~glcd_signal;
    system_var.glcddata     = ~glcd_ins_hthi;
    private_write_glcd_lcd();
    return OK;
}

```

Nếu bạn chỉ điều khiển 1 module GLCD thì nên dùng hàm này.

6.3 CÁC CHƯƠNG TRÌNH SỬ DỤNG VÙNG NHỚ DDRAM CỦA LCD

Trong LCD có 2 vùng nhớ thì phần này sử dụng vùng nhớ DDRAM, phần sau sẽ dùng vùng nhớ CGRAM còn lại.

6.3.1 Các ứng dụng hiển thị kí tự, dịch chuỗi trên LCD

Thư viện LCD. Chương trình thư viện điều khiển LCD 20x4 đã cho trong thư mục của bạn (SV không đánh máy lại) có tên là “TV_LCD.h” và “TV_LCD.c”.

Bạn có thể sử dụng cho các bài thực hành và nên đọc hiểu.

a. File thư viện “TV_LCD.h”:

```

#ifndef __tv_lcd_h__
#define __tv_lcd_h__
typedef enum                                // Khai báo các lệnh thường dùng
{
    lcd_clear_display    = 0x01,
    lcd_shift_left       = 0x18,
    lcd_shift_right      = 0x1c,
}LCD_COMMANDS;
result lcd_command(int8 command); // Hàm xuất mã lệnh ra LCD
result lcd_data(int8 data);       // Hàm xuất dữ liệu hiển thị

```

```
#ifndef TV_LCD_C
```

```

#define __TV_LCD_C__
#include<TV_LCD.h>
result lcd_command(int8 command)    // Hàm xuất mã lệnh
{
    write_lcd20x4(0x01,command);    // P=0,RS=0,RW=0,E=1
    write_lcd20x4(0x00,command);    // P=0,RS=0,RW=0,E=0
    delay_us(200);
    return OK;
}
result lcd_data(int8 data)          // Hàm xuất dữ liệu hiển thị
{
    write_lcd20x4(0x05,data);        // P=0,RS=1,RW=0,E=1
    write_lcd20x4(0x04,data);        // P=0,RS=1,RW=0,E=0
    delay_us(200);
    return OK;
}
result lcd_setup()                  // Hàm khởi tạo LCD
{
    int8 n;
    lcd_command(0x3c); delay_ms(5);
    lcd_command(0x3c); delay_ms(5);
    lcd_command(0x0c);
    lcd_command(0x06);
    lcd_command(0x40);                // di chuyển đến CGRAM
    for(n=0;n<64;n++) lcd_data(LCD_MA_8DOAN[n]); // ghi ma Font vào CGRAM
    lcd_command(lcd_clear_display); delay_ms(2);
    return OK;
}
/* Hàm di chuyển con trỏ
   colum_0_19 : cột cần di chuyển - phạm vi từ 0 đến 19
   row_0_3     : hàng cần di chuyển - phạm vi từ 0 đến 3
*/
result lcd_goto( int8 colum_0_19, int8 row_0_3)
{
    const unsigned int8 dc[]={0x80,0xc0,0x94,0xd4};
    lcd_command(dc[row_0_3]+colum_0_19);
    return OK;
}
/* Hàm hiển thị 1 số FONT 2x3 - 2 hàng và 3 cột
   num_0_9     : Số cần hiển thị - phạm vi từ 0 đến 9
   colum_0_17: Vị trí cột bắt đầu hiển thị - phạm vi từ 0 đến 17
   row_0_2     : Vị trí hàng bắt đầu hiển thị - phạm vi từ 0 đến 2
*/
result lcd_write_2x3_num(signed int8 num_0_9, int8 colum_0_17,int8 row_0_2)
{
    lcd_goto(colum_0_17,row_0_2);
    for (int8 i=0;i<6;i++)
    {
        if (i==3) lcd_goto(colum_0_17,row_0_2+1);
        lcd_data(lcd_so_x[num_0_9][i]);
    }
    return OK;
}
/* Hàm hiển thị 1 số FONT 4x3 - 4 hàng và 3 cột
   num_0_9     : Số cần hiển thị - phạm vi từ 0 đến 9

```

```

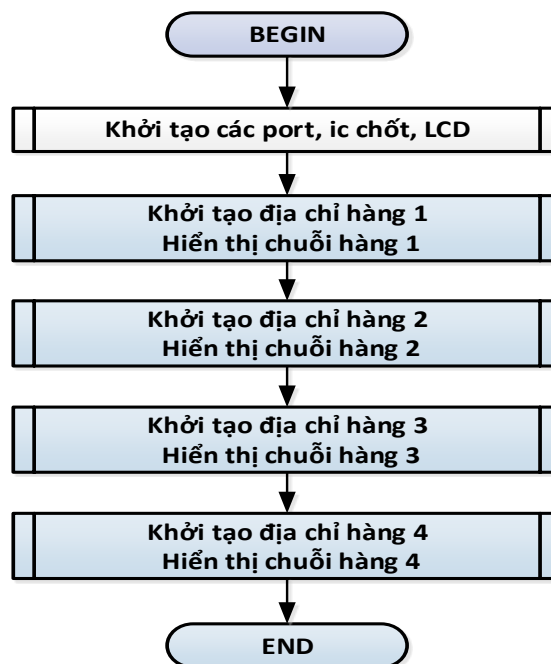
column_0_17: Vị trí cột bắt đầu hiển thị - phạm vi từ 0 đến 17
*/
result lcd_write_4x3_num(signed int8 num_0_9, int8 column_0_17)
{
    for(int8 n=0;n<12;n++)
    {
        if(n%3==0) LCD_GOTO(column_0_17,n/3);
        LCD_DATA(LCD_SO_X1[num_0_9][n]);
    }
    return OK;
}
#endif

```

Bài mẫu 601. Chương trình điều khiển LCD 20×4 hiển thị 4 hàng kí tự. Bạn có thể thay đổi nội dung hiển thị ở 4 hàng tùy thích nhưng chỉ trong phạm vi 20 kí tự cho mỗi hàng.

Lưu tên file là “BAI_601_LCD_HIEN THI_4HANG”

- Mục đích: Biết khởi tạo LCD, biết hiển thị chuỗi kí tự trên các hàng của LCD.
- Lưu đồ:



Hình 6-9. Lưu đồ điều khiển LCD hiển thị 4 chuỗi kí tự.

- Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
void main()
{
    system_init();
    lcd_setup();
    lcd_goto(0,0);           // Di chuyển con trỏ đến cột 0 hàng 0
    lcd_data("DAI HOC SPKT TPHCM");
    lcd_goto(0,1);          // Di chuyển con trỏ đến cột 0 hàng 1
}

```

```

    lcd_data("KHOA DIEN - DIEN TU");
    lcd_goto(0,2);           // Di chuyển con trỏ đến cột 0 hàng 2
    lcd_data("BO MON DTCN-YS");
    lcd_goto(0,3);           // Di chuyển con trỏ đến cột 0 hàng 3
    lcd_data("TTVXL MOUDLE LCD");
    while(true)
    {
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: hiển thị nội dung 4 hàng đã khai báo trong chương trình.
- f. Giải thích chương trình:

Bài mẫu 602. Chương trình đếm giây không cần chính xác hiển thị hàng cuối LCD. Hàng đầu LCD hiển thị chuỗi “HIEN THI BIEN GIAY”

Lưu tên file là “BAI_602_LCD_HT_BIEN”

- a. Mục đích: Biết cách hiển thị giá trị của biến.
- b. Lưu đồ: sinh viên hãy tự viết.
- c. Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
unsigned int8 giay=0;
void main()
{
    system_init();
    lcd_setup();
    lcd_goto(0,0);           // Nội dung cố định thì để ngoài while(true)
    lcd_data("HIEN THI BIEN GIAY");
    while(true)
    {
        lcd_goto(9,3);
        lcd_data(giay/10%10 +0x30); // Hiển thị hàng chục của giây
        lcd_data(giay%10 +0x30);    // Hiển thị hàng đơn vị của giây
        giay++;                      // Tăng giây
        giay%=60;                    // Không chế giây từ 0 đến 59
        delay_ms(1000);
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả.
- f. Giải thích chương trình.

Bài mẫu 603. Chương trình đếm giây giống bài 602 nhưng dùng lệnh printf.

Lưu tên file là “BAI_603_LCD_HT_BIEN_PRINTF”

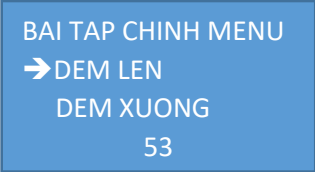
- a. Mục đích: Biết cách hiển thị giá trị của biến bằng lệnh printf.
- b. Lưu đồ: sinh viên hãy tự viết.
- c. Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
unsigned int8 giay=0;
void main()
{
    system_init();
    lcd_setup();
    lcd_goto(0,0);
    lcd_data("HIEN THI BIEN GIAY");
    while(true)
    {
        lcd_goto(9,3);
        printf(lcd_data,"%02u ",giay);
        // khi biến là unsigned int8 ta dùng %u
        // khi biến là signed int8 ta dùng %d
        // khi biến là unsigned int16 ta dùng %lu
        // khi biến là signed int16 ta dùng %ld
        // khi biến là unsigned int32 ta dùng %lu
        // khi biến là signed int32 ta dùng %ld
        // khi biến là float ta dùng %f
        // trong ví dụ trên %02u có nghĩa biến cần hiển thị thuộc kiểu
        // unsigned int8 và muốn lấy 2 số - không xóa số 0 vô nghĩa
        giay++;
        giay%=60;
        delay_ms(1000);
    }
}

```

Bài mẫu 604. Hiện thị menu như hình dưới. Dùng nút INV để di chuyển mũi tên đi lên hoặc đi xuống để chọn chế độ đếm lên hoặc đếm xuống của giây. Và hiện thị giá trị đếm của giây ở hàng cuối LCD - có xóa số 0 vô nghĩa.



```

BAI TAP CHINH MENU
→DEM LEN
DEM XUONG
53

```

Lưu tên file là “BAI_604_LCD_CHINH_MENU”

- Mục đích: Biết cách hiển thị ký tự không đánh máy được và biết cách chỉnh menu
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
signed int8 giay=0,n; // Chú ý biến "giay" và "chieu" phải là biến signed
signed int8 chieu=1; // Mặc định ban đầu chọn chế độ đếm lên
void hienthi_muiten ()
{
    if(chieu==1)
    {
        lcd_goto(0,1); // Nếu đếm lên thì hiển thị mũi tên lên ở đầu hàng 1
    }
}

```



```

        lcd_data(0x7e); // Tra bảng mã ASCII để có mã dấu mũi tên
        lcd_goto(0,2); // Xóa mũi tên cũ ở đầu hàng 2
        lcd_data(" ");
    }
    else
    {
        lcd_goto(0,1); // Nếu đếm xuống thì xóa mũi tên cũ ở đầu hàng 1
        lcd_data(" ");
        lcd_goto(0,2); // Hiển thị mũi tên ở đầu hàng 2
        lcd_data(0x7e);
    }
}
void kiểmtra_inv()
{
    if(inputcd(INV)==0) // Đếm lên thì "chieu =1", đếm xuống "chieu=-1"
    {
        chieu=-chieu;
        hienthi_muiten(); // Cập nhật lại vị trí mũi tên
    }
}
void main()
{
    system_init();
    lcd_setup();
    lcd_goto(0,0);
    lcd_data("BAI TAP CHINH MENU");
    lcd_goto(1,1);
    lcd_data("DEM LEN");
    lcd_goto(1,2);
    lcd_data("DEM XUONG");
    hienthi_muiten(); // Hiển thị dấu mũi tên
    while(true)
    {
        lcd_goto(9,3);
        printf(lcd_data,"%d ",giay);
        giay = giay + chieu;
        // "chieu=1" thì đếm lên,"chieu=-1" thì đếm xuống
        if(giay>=60)    giay=0; // Không chế giới hạn khi đếm lên
        else if(giay<0)    giay=59; // Không chế giới hạn khi đếm xuống
        // Kiểm tra nút nhiều lần thay cho delay để nút đáp ứng tốt
        for(n=0;n<100;n++)
        {
            kiểmtra_inv();
            delay_ms(10);
        }
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả.
- f. Giải thích chương trình.

Bài tập 605. Cải tiến bài 604 bằng cách thay nội dung ở hàng đầu LCD bằng “TRUONG DAI HOC SU PHAM KY THUAT TP.HCM” chạy từ phải sang trái giống bảng quảng cáo.

Lưu tên file là “BAI_605_LCD_CHAYCHUOI”

Bài tập 606. Hiệu chỉnh bài 605 để chuỗi “TRUONG DAI HOC SU PHAM KY THUAT TP.HCM” chạy từ phải sang trái đến khi xuất hiện chữ “TP.HCM” ở cuối hàng thì bắt đầu chạy ngược lại từ trái sang phải cho đến khi xuất hiện chữ “TRUONG” ở đầu hàng thì lại đảo chiều. Và cứ thế chạy tuần hoàn liên tục.

Lưu tên file là “BAI_606_LCD_CHAYCHUOI_QUALAI”

6.3.2 Đếm thời gian và đếm xung ngoại hiển thị trên LCD

Bài mẫu 611. Chương trình đếm giây chính xác dùng TIMER hiển thị trên LCD 20×4
Lưu tên file là “BAI_611_LCD_DEM_GIAY”

- Mục đích: Biết kết hợp module LCD và TIMER.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_lcd.c>
unsigned int8 bdn=0,giay=0;
void main()
{
    system_init();
    lcd_setup();
    setup_timer_1(T1_INTERNAL | T1_DIV_BY_8);
    set_timer1(3036);
    enable_interrupts(INT_TIMER1);
    enable_interrupts(GLOBAL);
    while(true)
    {
        if(bdn>=10)
        {
            bdn=bdn-10;
            giay++;
            giay%=60;
        }
        lcd_goto(9,0);
        printf lcd_data,"%02u",giay);
    }
}
#INT_TIMER1
void ngattimer1()
{
    set_timer1(3036);
    bdn++;
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: giây sẽ đếm và hiển thị ở hàng 0
- f. Giải thích chương trình:

Bài tập 612. Chương trình đếm phút giây hiển thị trên LCD 20×4

Lưu tên file là “BAI_612_LCD_DEM_PHUT_GIAY”

Bài tập 613. Hãy viết chương trình đếm giờ phút giây hiển thị trên LCD 20×4.

Lưu tên file là “BAI_613_LCD_DEM_GIO_PHUT_GIAY”

Bài tập 614. Hãy viết chương trình đếm giờ phút giây hiển thị trên LCD 20×4, có thêm 3 nút nhấn để chỉnh được thời gian – chỉnh cái gì thì cái đó nhấp nháy.

Lưu tên file là “BAI_614_LCD_CHINH_GIO_PHUT_GIAY”

Bài tập 615. Hãy hiệu chỉnh bài 614 vừa hiển thị trên LCD 20×4 và hiển thị trên module 8 led quét.

Lưu tên file là “BAI_615_LCD_8LED_GIO_PHUT_GIAY”

Bài tập 616. Chương trình đếm sản phẩm dùng counter T0 hiển thị trên LCD 20×4 ở hàng 0, hiển thị trên module 4 led 7 đoạn, 8 led quét, giới hạn đếm từ 000 đến 100.

Lưu tên file là “BAI_616_LCD_4LED_8LED_DEM_SP_T0”

Bài tập 617. Hãy hiệu chỉnh bài 616 thêm phần cài giới hạn đếm bằng 3 nút UP, DW và CLR. Giới hạn cài từ 01 đến 99. Khi nhấn CLR thì giá trị cài về 01, giá trị đang đếm về 00.

Hàng 1 hiển thị giá trị cài, hàng 0 hiển thị giá trị đếm.

4 led 7 đoạn thì 2 led trái hiển thị giá trị cài, 2 led phải hiển thị giá trị đếm.

8 led 7 đoạn quét thì 2 led tận cùng bên trái hiển thị giá trị cài, 2 led tận cùng bên phải hiển thị giá trị đếm.

Lưu tên file là “BAI_617_LCD_4LED_8LED_DEM_SP_T0_UD”

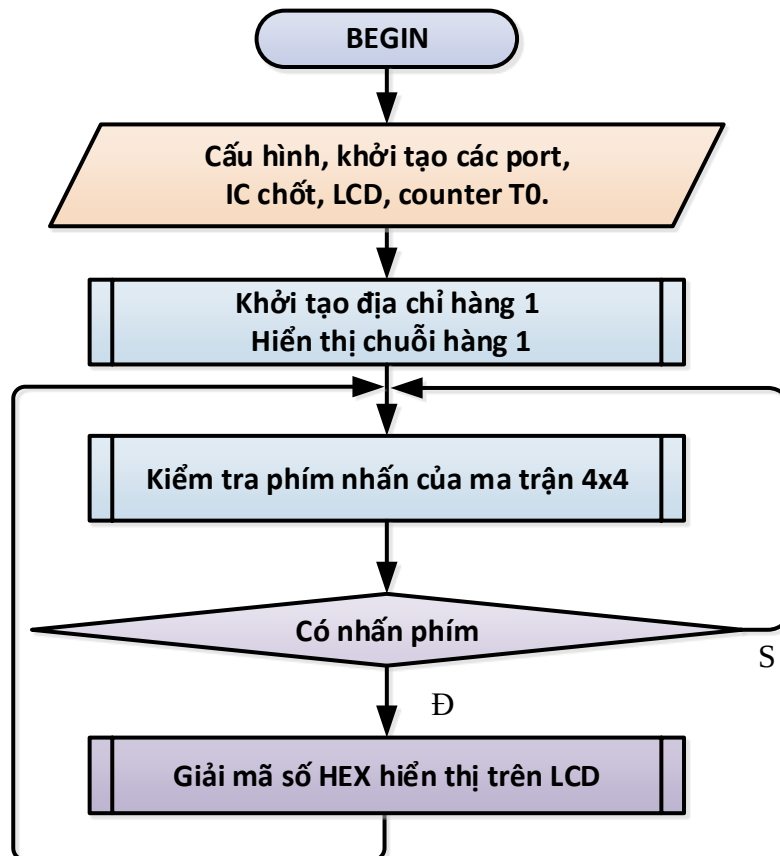
6.3.3 Các ứng dụng kết hợp bàn phím ma trận và LCD

Bài mẫu 621. Chương trình có chức năng khi nhấn phím bất kỳ của bàn phím ma trận 4x4 thì mã phím đó xuất hiện tại vị trí đầu tiên bên phải của hàng 1, ví dụ khi nhấn phím số 0 thì số 0 hiển thị, khi nhấn phím số 5 thì số 5 hiển thị, số trước đó sẽ mất đi.

Hàng 1 hiển thị thông tin: “HAY NHAN PHIM:”

Lưu tên file là “BAI_621_LCD_KEX4X4”

- Mục đích: Biết kết hợp bàn phím ma trận với LCD.
- Lưu đồ:



Hình 6-10. Lưu đồ quét phím ma trận hiển thị trên LCD.

- Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
#include<tv_key4x4.c>
void main()
{
    system_init();
    lcd_setup();
    lcd_goto(3,0);
    lcd_data("HAY NHAN PHIM");
    while(true)
    {
        if(key4x4_read()==OK)
    
```

```

    {
        lcd_goto(19,1);
        if(key4x4.key<10) lcd_data( key4x4.key +0x30);
        else                lcd_data( key4x4.key +0x37);
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- f. Giải thích chương trình: tại sao cộng 0x30 và 0x37?

Bài tập 622. Hãy viết chương trình có chức năng khi nhấn phím bất kỳ của bàn phím ma trận 4×4 thì mã phím đó xuất hiện tại tận cùng bên phải của hàng 2, các ký tự đã nhấn sẽ dịch sang trái. Các ký tự hàng 1 không thay đổi.

Hàng 1 hiển thị thông tin : “HAY NHAN PHIM:”

Lưu tên file là “BAI_622_LCD_KEX4X4_DICH”

Bài tập 623. Hãy viết chương trình có chức năng khi mới cấp điện thì 4 hàng hiển thị ký tự trắng, khi nhấn phím bất kỳ của bàn phím ma trận 4×4 thì mã phím đó xuất hiện tại tận cùng bên phải của hàng 1, nhấn nữa thì dịch sang trái, cho đến khi được 20 ký tự mà nhấn nữa thì xuống hàng 2, khi hàng 2 đủ 20 ký tự thì dịch xuống hàng 3 và khi hàng 3 đủ 20 thì xuống hàng 4.

Lưu tên file là “BAI_623_LCD_KEX4X4_DICH_4HANG_1234”

Bài tập 624. Hãy viết chương trình có chức năng hiển thị 2 hàng thông tin và điều khiển buzzer bằng cách khi nhấn phím A của bàn phím ma trận 4×4 thì buzzer sẽ kêu ứng với thời gian Y nằm trong giới hạn từ 0 đến 9 được thay đổi bằng 10 phím số từ 0 đến 9 của bàn phím ma trận.

Khi buzzer kêu thì XXX sẽ hiển thị chữ “ ON”, khi chuông hết kêu thì hiển thị chữ “OFF”. Thời gian Y mặc nhiên khi bắt đầu là 1s, nếu thời gian bằng 0 thì chuông không kêu nếu nhấn A.

Hàng 1 hiển thị thông tin: “ DIEU KHIEN BUZZER ”

Hàng 2 hiển thị thông tin: “BUZZER: XXX, TG:Y”

Lưu tên file là “BAI_624_LCD_KEX4X4_DICH”

Bài tập 625. Hãy viết lại bài 617 nhưng dùng bàn phím ma trận để cài giới hạn.

Ví dụ, muốn cài 45 thì nhấn 4 hiển thị ở hàng đơn vị, sau đó nhấn 5 thì giá trị 4 sẽ dịch sang hàng chục và 5 thành hàng đơn vị, kết quả là 45.

Nhấn phím C thì xóa về 01.

Lưu tên file là “ BAI_625_LCD_4LED_DEM_SP_T0_MATRIX_KEY ”

6.4 CÁC ỨNG DỤNG VÙNG NHỚ CGRAM CỦA LCD

6.4.1 Vùng nhớ CGRAM

Ở mục trước ta đã sử dụng LCD có chức năng hiển thị các kí tự hoặc các chuỗi kí tự thì các kí tự đó thuộc mã ASCII và đã tạo sẵn trong chip điều khiển LCD, bạn chỉ cần gởi đúng mã ASCII thì kí tự đó sẽ được hiển thị.

Phần này chúng ta sẽ sử dụng vùng nhớ còn lại là CGRAM mà nhà chế tạo LCD dành cho người dùng tự thiết lập các kí tự mong muốn.

Trong bảng tập lệnh thì có lệnh thiết lập địa chỉ CGRAM có dạng bảng như sau:

Bảng 6- 12. Thiết lập địa chỉ CGRAM.

D7	D6	D5	D4	D3	D2	D1	D0	HEX	
0	1	A5	A4	A3	A2	A1	A0		
0	1	0	0	0	0	0	0	0×40	Địa chỉ đầu
0	1	...	...	...	...	...	...		
0	1	1	1	1	1	1	1	0×7F	Địa chỉ cuối

Vùng nhớ CGRAM này có địa chỉ từ 0×40 đến 0×7F cho phép lưu 64 byte dữ liệu.

Mỗi kí tự hiển thị trên LCD là một ma trận gồm 8 byte nên vùng nhớ CGRAM cho phép lưu được 8 kí tự.

Bảng 6-13. Vùng nhớ CGRAM.

Vùng thứ	Địa chỉ cuối	B7	B6	B5	B4	B3	B2	B1	B0	Địa chỉ đầu	Địa chỉ kí tự
0	0×47	Vùng nhớ 8 byte để lưu kí tự tự tạo								0×40	0
1	0×4F									0×48	1
2	0×57									0×50	2
3	0×5F									0×58	3
4	0×67									0×60	4
5	0×6F									0×68	5
6	0×77									0×70	6

7	0×7F		0×78	7
---	------	--	------	---

6.4.2 Cách tìm mã kí tự mong muốn

Mỗi kí tự là một ma trận điểm 5×8 như bảng sau, muốn điểm nào sáng thì bit trong ô đó bằng 1, với hình trái tim thì byte thứ 0 là 0×0A – tính theo hàng ngang, tương tự cho các hàng còn lại.

Bảng 6-14. Mã của trái tim.

	BIT					
BYTE	4	3	2	1	0	HEX
0		X		X		0×0A
1	X		X		X	0×15
2	X				X	0×11
3	X				X	0×11
4		X		X		0×0A
5			X			0×04
6						0×00
7						0×00

Cách để hiển thị trái tim:

- Khai báo mảng lưu 8 byte mã của trái tim.
- Khởi tạo địa chỉ bắt đầu của 1 trong 8 vùng ví dụ chọn 0x40 để lưu dữ liệu vào vùng nhớ CGRAM từ 0×40 đến 0×47.
- Copy 8 byte dữ liệu của trái tim vào vùng nhớ CGRAM từ 0×40 đến 0×47.
- Gọi lệnh hiển thị kí tự thứ 0.

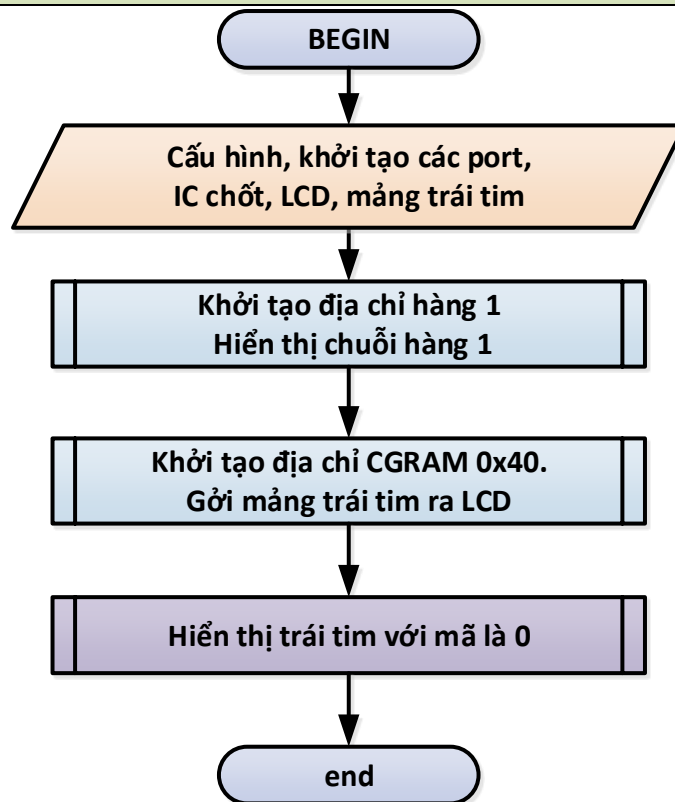
6.4.3 Các chương trình ứng dụng các kí tự - tự tạo trên LCD

Bài mẫu 631. Chương trình hiển thị kí tự hay biểu tượng tự tạo.

Hàng 1 hiển thị thông tin: “ HIEN THI TRAI TIM ”

Lưu tên file là “BAI_631_LCD_TRAITIM”

- Mục đích: biết lập trình hiển thị các kí tự mong muốn.
- Lưu đồ:



Hình 6-11. Lưu đồ hiển thị mã tự tạo trái tim trên LCD.

c. Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
const unsigned int8 traitim[]={0x0a,0x15,0x11,0x11,0x0a,0x04,0,0};
unsigned int8 n;
void main()
{
    system_init();
    lcd_setup();
    lcd_command(0x40); // Di chuyển đến vùng nhớ CGRAM
    for(n=0;n<8;n++) lcd_data(traitim[n]); // Ghi mã trái tim vào CGRAM
    lcd_goto(0,0);
    lcd_data( 0);      //Hiển thị trái tim
    while(true)
    {
    }
}
    
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- f. Giải thích chương trình:

Bài tập 632. Hãy hiệu chỉnh bài 631 sao cho hàng 2 hiển thị 20 trái tim.

Lưu tên file là “BAI_632_LCD_20_TRAITIM”

Bài mẫu 633. Chương trình hiển thị biểu tượng cục pin đang sạc.

Lưu tên file là “BAI_633_LCD_SACPIN”

- Mục đích: biết lập trình hiển thị các kí tự mong muốn.
- Lưu đồ:
- Chương trình

```
#include<tv_boards.c>
#include<tv_lcd.c>           // Tạo 6 cục pin với 6 mức khác nhau
const unsigned int8 cucpin[]={0x04,0x1f,0x11,0x11,0x11,0x11,0x11,0x1f,
                                0x04,0x1f,0x11,0x11,0x11,0x11,0x1f,0x1f,
                                0x04,0x1f,0x11,0x11,0x11,0x1f,0x1f,0x1f,
                                0x04,0x1f,0x11,0x11,0x1f,0x1f,0x1f,0x1f,
                                0x04,0x1f,0x11,0x1f,0x1f,0x1f,0x1f,0x1f,
                                0x04,0x1f,0x1f,0x1f,0x1f,0x1f,0x1f,0x1f};

unsigned int8 n;
void main()
{
    system_init();
    lcd_setup();
    lcd_command(0x40);
    for(n=0;n<48;n++) lcd_data(cucpin[n]);
    while(true)
    {
        for(n=0;n<6;n++) // Lần lượt hiển thị 6 hình cục pin khác nhau
        {                 // đè lên nhau
            lcd_goto(19,0);
            lcd_data(n);
            delay_ms(200);
        }
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả:
- Giải thích chương trình:

Bài tập 634. Dựa vào bài mẫu 633 hãy tạo cột sóng điện thoại với 8 mức.

Lưu tên file là “BAI_634_LCD_COTSONG_DT”

Bài tập 635. Hãy lập trình hiển thị trái táo (Apple) chiếm 4 ma trận (mỗi 1 ma trận hiển thị 1 kí tự trên LCD).

Hàng 1 hiển thị thông tin: “ BIEU TUONG APPLE ”

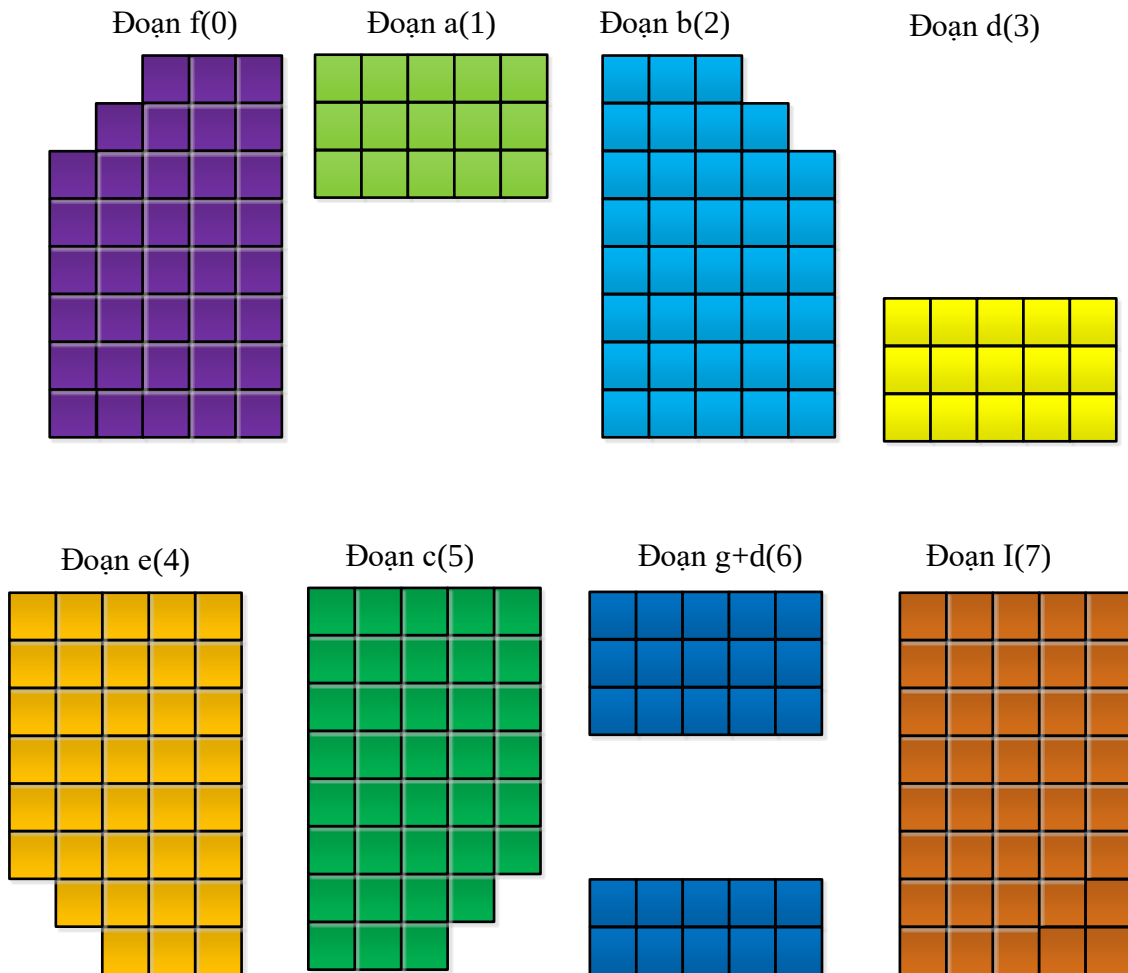
Lưu tên file là “BAI_635_LCD_APPLE”

Bài tập 636. Hãy thêm vào bài 635 sao cho trái táo di chuyển từ phải sang trái rồi từ trái sang phải.

6.4.4 Tạo mã 7 đoạn kích thước lớn trên LCD

Bước 1: xây dựng mã cho các đoạn

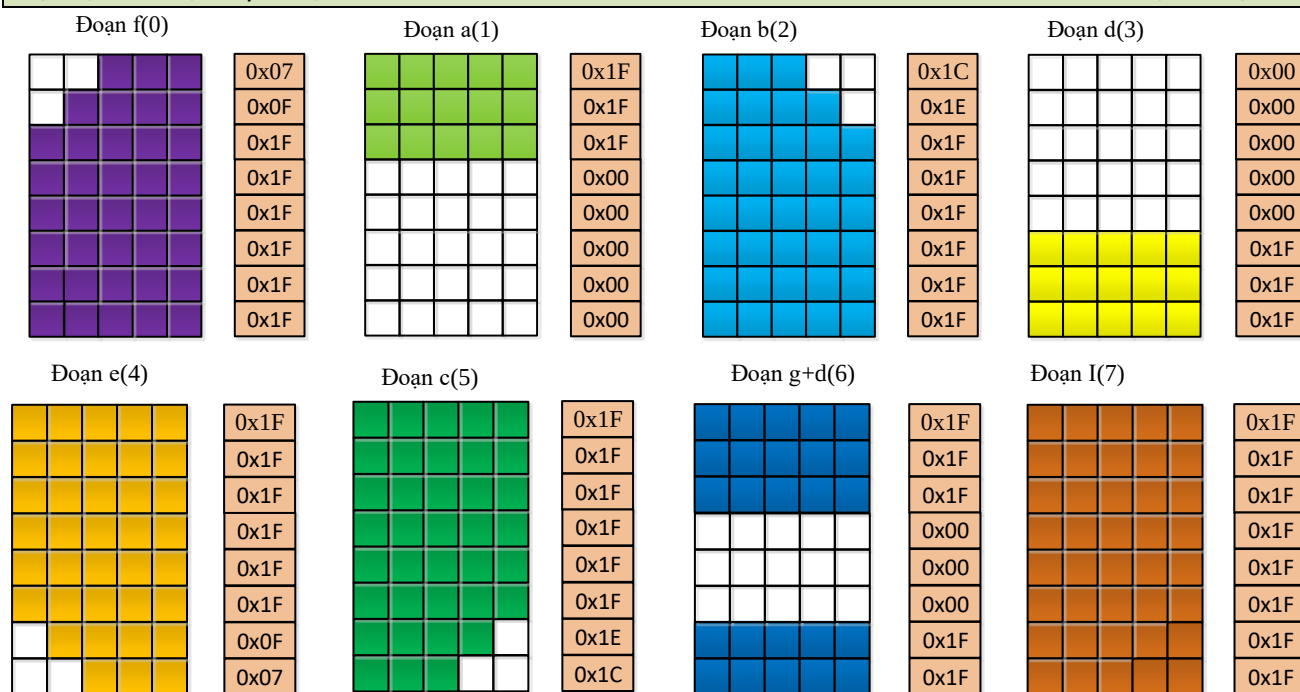
Để tạo các số từ 0 đến 9 để hiển thị lớn hơn kí tự bình thường cho dễ nhìn thì cách tạo ra gần giống mã của led 7 đoạn. Các đoạn được đánh tên tương ứng là a, b, c, d, e, f, g, i



Hình 6-12. Hình ảnh các đoạn cho số lớn trên LCD.

Các con số thứ tự đi kèm là trình tự khai báo trong mảng.

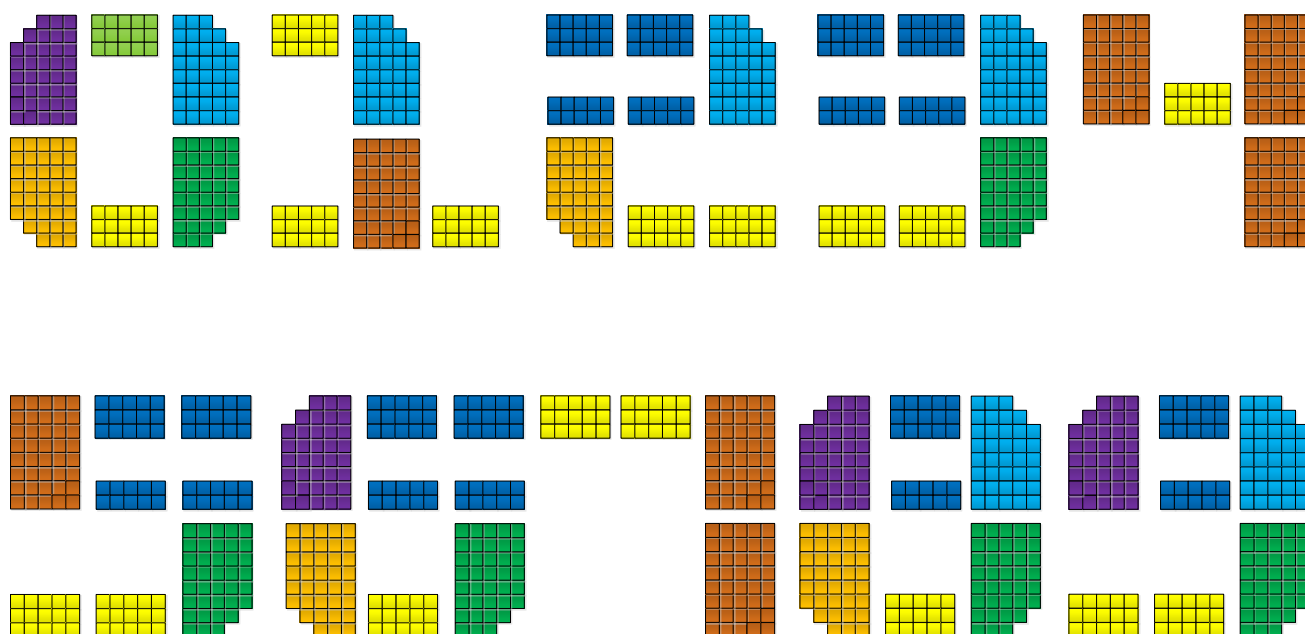
Mã hex của các đoạn tương ứng:



Hình 6-13. Tìm mã của các đoạn cho số lớn trên LCD.

Bước 2: Kết hợp các đoạn để xây dựng các con số từ 0 đến 9

Để sáng con số lớn thì ta kết hợp các đoạn để được các con số từ 0 đến 9, mỗi con số sẽ chiếm 6 vị trí như sau:



Hình 6- 14. Hình ảnh các số lớn từ 0 đến 9 trên LCD.

Để thuận tiện cho tạo dữ liệu và lập trình, các đoạn được tính theo thứ tự: f, a, b, c, d, e, f, hoặc g+d.

Mỗi kí tự số lớn sẽ sử dụng 6 kí tự bình thường, khi tính thì tính 3 kí tự hàng thứ nhất trước rồi đến 3 kí tự hàng thứ 2 và khi lập trình cũng vậy.

Muốn sáng số 0 thì các đoạn: f, a, b, e, d, c sáng – tương ứng với số thứ tự là: 0, 1, 2, 4, 3, 5.

Muốn sáng số 1 thì các đoạn: a, b, ' ', d, i, d, sáng – tương ứng với số thứ tự là: 1, 2, 0×20, 3, 7, 3.

Muốn sáng số 2 thì các đoạn: g+d, g+d, b, e, d, d sáng – tương ứng với số thứ tự là: 6, 6, 2, 4, 3, 3.

Muốn sáng số 3 thì các đoạn: g+d, g+d, b, d, d, c sáng – tương ứng với số thứ tự là: 6, 6, 2, 3, 3, 5.

Muốn sáng số 4 thì các đoạn: i, d, i, 0, 0, i sáng – tương ứng với số thứ tự là: 7, 3, 7, 0×20, 0×20, 7.

Muốn sáng số 5 thì các đoạn: i, g+d, g+d, d, d, c sáng – tương ứng với số thứ tự là: 7, 6, 6, 3, 3, 5.

Muốn sáng số 6 thì các đoạn: f, g+d, g+d, e, d, c sáng – tương ứng với số thứ tự là: 0, 6, 6, 4, 3, 5.

Muốn sáng số 7 thì các đoạn: a, a, i, 0, 0, i sáng – tương ứng với số thứ tự là: 1, 1, 7, 0×20, 0×20, 7.

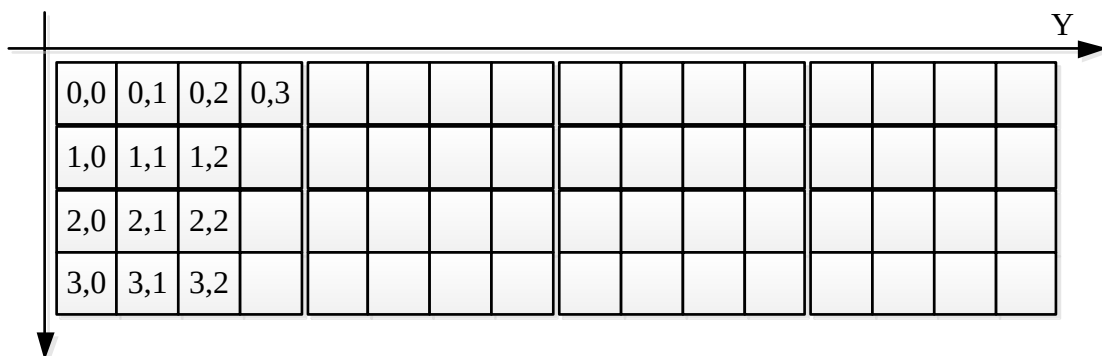
Muốn sáng số 8 thì các đoạn: f, g+d, b, e, d, c sáng – tương ứng với số thứ tự là: 0, 6, 2, 4, 3, 5.

Muốn sáng số 9 thì các đoạn: f, g+d, b, d, d, c sáng – tương ứng với số thứ tự là: 0, 6, 2, 3, 3, 5.

Có 1 số led hiển thị khoảng trắng có mã là 0×20 hay 32.

Bước 3: Thiết lập lại vị trí các kí tự trên màn hình LCD 20×4:

LCD 20×4 có 4 hàng và 20 cột có tổng cộng là 80 kí tự theo tọa độ 2 trục X và Y như hình 6-17.



Hình 6-15. LCD tính theo địa chỉ hàng và cột.

Ô có địa chỉ 0×80 bây giờ gọi là ô có tọa độ $(x, y) = (0,0)$.

Ô có địa chỉ 0×81 bây giờ gọi là ô có tọa độ $(x, y) = (0,1)$.

Ô có địa chỉ $0 \times C0$ bây giờ gọi là ô có tọa độ $(x, y) = (1, 0)$.

Ô có địa chỉ $0 \times C1$ bây giờ gọi là ô có tọa độ $(x, y) = (1, 1)$.

Tương tự cho các ô còn lại.

Để sáng 1 con số sẽ dùng 6 kí tự, mỗi hàng chiếm 3 kí tự, ví dụ muốn sáng số 0 bắt đầu tại tọa độ $(x, y) = (0, 0)$ thì 6 ô sau sẽ được sử dụng là:

3 ô hàng thứ nhất: ô $(0, 0)$, ô $(0, 1)$, ô $(0, 2)$

3 ô hàng thứ hai: ô $(1, 0)$, ô $(1, 1)$, ô $(1, 2)$

Mã số 0 có các đoạn là f, a, b, e, d, c sẽ hiển thị lần lượt vào 6 ô nhớ trên.

Tại tọa độ $(x, y) = (0, 0)$ ta cho hiển thị đoạn thứ 1 là f. Tăng giá trị y lên 1: $(x, y) = (0, 1)$ để hiển thị đoạn thứ 2 là a. Tăng giá trị y lên 1: $(x, y) = (0, 2)$ để hiển thị đoạn thứ 3 là b.

Tăng giá trị x lên 1: $(x, y) = (1, 0)$ ta cho hiển thị đoạn thứ 4 là e. Tăng giá trị y lên 1: $(x, y) = (1, 1)$ để hiển thị đoạn thứ 5 là d. Tăng giá trị y lên 1: $(x, y) = (1, 2)$ để hiển thị đoạn thứ 6 là c.

Các giá trị tọa độ hiển thị được tính theo trục x và y nhưng địa chỉ thật được tính từ giá trị xuất phát là 0×80 , 0×81 , 0×82 và $0 \times C0$, $0 \times C1$, $0 \times C2$.

Cách tính:

- Nếu x bằng 0 thì bắt đầu là 0×80 .
- Nếu x bằng 1 thì bắt đầu là $0 \times C0$.
- Nếu x bằng 2 thì bắt đầu là 0×94 .
- Nếu x bằng 3 thì bắt đầu là $0 \times D4$.
- Sau đó ta cộng địa chỉ trên với giá trị của y.

Bước 4: Viết chương trình:

Các mã của các đoạn đã viết trong chương trình thư viện của LCD bao gồm mảng 2 chiều cho 10 kí tự từ 0 đến 9, mỗi kí tự có 8 byte khi hiển thị sẽ tạo thành con số.

Chương trình con LCD_GOTO() có chức năng di chuyển con trỏ đến tọa độ (x, y) .

Các dữ liệu này đã viết trong hàm thư viện của LCD.

Hàm 641. Hàm hiển thị số Font 2x3 – hàm này đã có sẵn trong thư viện TV_LCD.C

```
/* Hàm hiển thị 1 số FONT 2x3 - 2 hàng và 3 cột
   num_0_9      : Số cần hiển thị - phạm vi từ 0 đến 9
   colum_0_17: Vị trí cột bắt đầu hiển thị - phạm vi từ 0 đến 17
   row_0_2      : Vị trí hàng bắt đầu hiển thị - phạm vi từ 0 đến 2
*/
result lcd_write_2x3_num(signed int8 num_0_9, int8 colum_0_17, int8
row_0_2)
{
    lcd_goto(colum_0_17, row_0_2);
    for (int8 i=0; i<6; i++)
    {
        if (i==3)    lcd_goto(colum_0_17, row_0_2+1);
        lcd_data lcd_so_x[num_0_9][i];
    }
}
```

```

    }
    return OK;
}

```

Trong đó “`lcd_so_x[11][6]`” là mảng 2 chiều liệt kê 6 ký tự cấu hình nên các số từ 0 đến 9 của font 2x3. Ký tự thứ 11 bao gồm 6 khoảng trắng dùng để xóa 1 số dạng font 2x3

```

const unsigned int8 LCD_SO_X[11][6] ={           // Font 3x2
    0,1,2,5,3,4,           // SO 0
    1,2,32,3,7,3,          // SO 1
    6,6,2,5,3,3,           // SO 2
    6,6,2,3,3,4,           // SO 3
    5,3,7,32,32,7,         // SO 4
    7,6,6,3,3,4,           // SO 5
    0,6,6,5,3,4,           // SO 6
    1,1,7,32,32,7,         // SO 7
    0,6,2,5,3,4,           // SO 8
    0,6,2,3,3,4,           // SO 9
    32,32,32,32,32,32      // XOÁ
};

```

Hàm 642. Hàm hiển thị số Font 4x3 – hàm này đã có sẵn trong thư viện TV_LCD.C

```

/* Hàm hiển thị 1 số FONT 4x3 - 4 hàng và 3 cột
   num_0_9   : Số cần hiển thị - phạm vi từ 0 đến 9
   colum_0_17: Vị trí cột bắt đầu hiển thị - phạm vi từ 0 đến 17
*/
result lcd_write_4x3_num(signed int8 num_0_9, int8 colum_0_17)
{
    for(int8 n=0;n<12;n++)
    {
        if(n%3==0) LCD_GOTO(colum_0_17,n/3);
        LCD_DATA(LCD_SO_X1[num_0_9][n]);
    }
    return OK;
}

```

Trong đó “`lcd_so_x1[11][12]`” là mảng 2 chiều liệt kê 12 ký tự cấu hình nên các số từ 0 đến 9 của font 4x3. Ký tự thứ 11 bao gồm 12 khoảng trắng dùng để xóa 1 số dạng font 4x3

```

const unsigned int8 LCD_SO_X1[11][12] ={           //Font 3x4
    0,1,2,7,32,7,7,32,7,5,3,4,           //0
    1,2,32,32,7,32,32,7,32,3,7,3,         //1
    1,1,2,3,3,7,7,32,32,5,3,3,           //2
    0,1,2,32,3,4,32,32,2,5,3,4,           //3
    7,32,7,5,3,7,32,32,7,32,32,7,         //4
    7,1,1,7,3,3,32,32,7,5,3,4,           //5
    0,1,2,7,3,3,7,32,7,5,3,4,           //6
    1,1,7,32,32,4,32,0,32,32,7,32,        //7
    0,1,2,5,3,4,0,32,2,5,3,4,           //8
    0,1,2,5,3,7,32,32,7,5,3,4,           //9
    32,32,32,32,32,32,32,32,32,32,32,32; //XOÁ

```

Bài mẫu 641. Chương trình hiển thị số 01234 theo font 2x3 ở 2 hàng giữa LCD20×4.
Lưu tên file là “BAI_641_LCD_SO_01234_FONT2x3”

- Mục đích: biết lập trình hiển thị con số có kích thước lớn trên màn hình LCD.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_lcd.c>
void main()
{
    system_init();
    lcd_setup();
    lcd_write_2x3_num(0,0,1); // Hiển thị số 0 ở hàng 1 cột 0
    lcd_write_2x3_num(1,4,1); // Hiển thị số 1 ở hàng 1 cột 4
    lcd_write_2x3_num(2,8,1); // Hiển thị số 2 ở hàng 1 cột 8
    lcd_write_2x3_num(3,12,1); // Hiển thị số 3 ở hàng 1 cột 12
    lcd_write_2x3_num(4,16,1); // Hiển thị số 4 ở hàng 1 cột 16
    while(true)
    {
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- Giải thích chương trình:

Bài tập 642. Hãy viết chương trình hiển thị kí tự số 01234 theo font 4x3 trên LCD20×4.
Lưu tên file là “BAI_642_LCD_SO_01234_FONT4x3”

Bài mẫu 643. Chương trình đếm từ 00 đến 99 hiển thị theo font 1x1 ở đầu hàng, font 2x3 ở giữa hàng và 4x3 ở cuối hàng của LCD.

Lưu tên file là “BAI_643_LCD_DEM_00_99_HT_3KIEUFONT”

- Mục đích: biết lập trình đếm giải mã hiển thị con số có kích thước lớn trên màn hình LCD.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_lcd.c>
unsigned int8 dem=0;
void main()
{
    system_init();
    lcd_setup();
```

```

while(true)
{
    for(dem=0;dem<100;dem++)
    {
        lcd_goto(0,2);
        printf lcd_data,"%02u",dem); // Font 1x1
        lcd_write_2x3_num(dem/10%10,6,1); // Hàng chục Font 2x3
        lcd_write_2x3_num(dem%10,9,1); // Hàng đơn vị Font 2x3
        lcd_write_4x3_num(dem/10%10,14); // Hàng chục font 4x3
        lcd_write_4x3_num(dem%10,17); // Hàng đơn vị font 4x3
        delay_ms(300);
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- f. Giải thích chương trình:

Bài tập 644. Hãy hiệu chỉnh bài 616 đếm sản phẩm hiển thị trên led 7 đoạn và LCD nhưng sử dụng font 2x3.

Hàng 1 hiển thị thông tin: “**DEM SAN PHAM ** ”

Lưu tên file là “BAI_644_LCD_DEM_SP_KTL”

Bài tập 645. Thêm vào bài 614 nút INV có chức năng chọn font 1x1 - 2x3 - 4x3 cho giá trị giờ, phút, giây trên LCD. Nhấn UP, DW, MOD để chỉnh thời gian – cái nào được chỉnh thì cái đó nhấp nháy.

Lưu tên file là “BAI_645_LCD_TIMER_3FONT”

6.5 LÝ THUYẾT GLCD

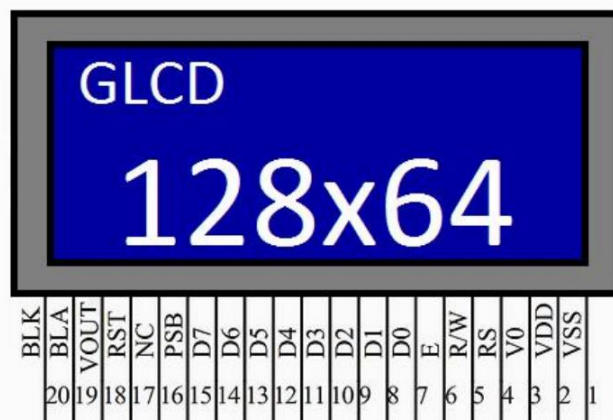
6.5.1 Giới thiệu GLCD

Ở phần trên ta đã sử dụng hết các khả năng của LCD để hiển thị kí tự ASCII và kí tự tự tạo.

Phần này ta sẽ sử dụng GLCD có chức năng giống LCD và thêm khả năng hiển thị hình ảnh.

GLCD có rất nhiều dạng phân biệt theo kích thước điểm ảnh và GLCD có trong kit thực hành là GLCD 128x64 có nghĩa là có 128 điểm ảnh theo hàng và 64 điểm ảnh theo cột.

GLCD 128x64 như hình 6-18:



Hình 6-16. Hình ảnh của GLCD.

6.5.2 Sơ đồ chân của GLCD

Sơ đồ chân của GLCD như bảng sau.

Bảng 6-15. Các chân của GLCD.

Chân số	Tên chân	Input/output	Chức năng tín hiệu
1	VSS	Power	GND
2	VDD	Power	+5V
3	VO	Analog	Contrast Control
4	RS (CS_SPI)	Input	Register Select. H: data signal, L: instruction signal
5	R/W (SDI_SPI)	Input	Read/Write. H: read mode, L: write mode
6	E (SCK_SPI)	Input	Enable (strobe)
6-14	DB0 ~ DB7	H/L	Data bit 0~7
15	PSB (CS1)	Input	Interface selection (Chip select signal for IC1) 0: serial mode 1: 8/4 bits parallel bus mode
16	NC (CS2)	Input	No connection (Chip select signal for IC2)
17	$\overline{RESET}$	Input	Reset signal
18	V_{out} (V_{EE})	O	Output voltage for GLCD driving
15	BLA	input	Backlight Anode
16	BLK	input	Backlight Cathode

20 chân của GLCD được chia ra thành 4 dạng tín hiệu như sau:

Các chân cấp nguồn: Chân số 1 là chân V_{ss} sẽ nối mass (0V), chân thứ 2 là V_{dd} sẽ nối với nguồn +5V. Chân thứ 3 dùng để chỉnh contrast thường nối với biến trở, biến trở có 1 đầu nối với 0V và điện áp ra ở chân số 18 là V_{out} , chỉnh biến trở cho đến khi thấy được ký tự thì ngừng, trong bộ thực hành thì đã chỉnh rồi.

Các chân BLA và BLK: là 2 chân dùng để cấp nguồn cho đèn nền để có thể nhìn thấy vào ban đêm. Giống LCD cũng được điều khiển bằng transistor để tắt mở đèn nền.

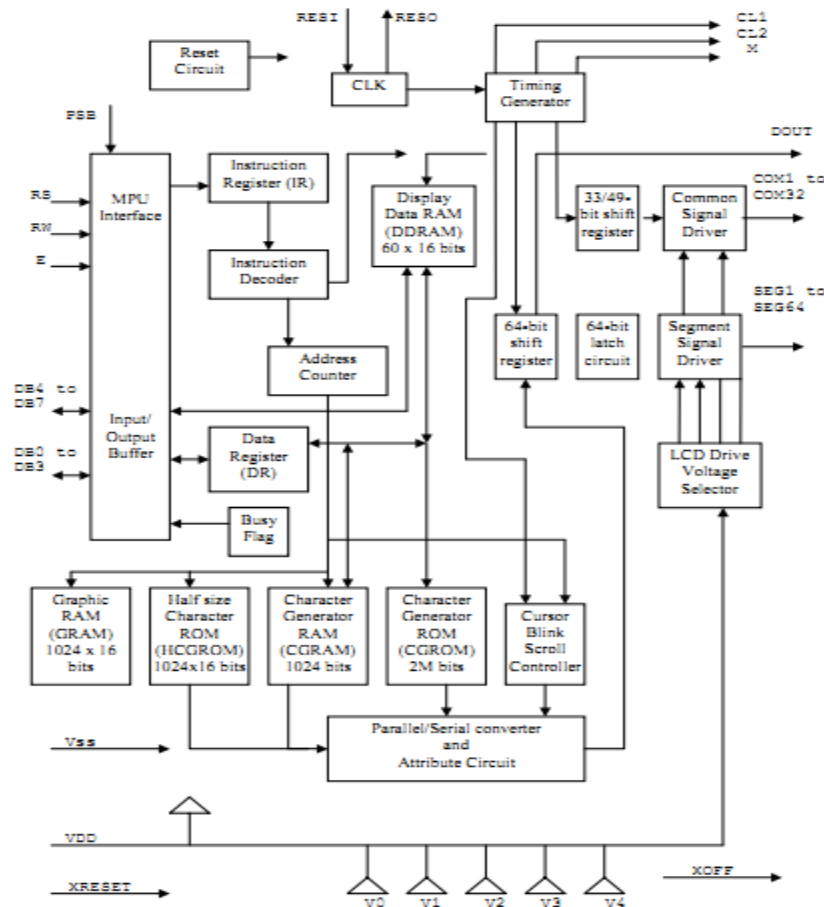
Các chân điều khiển: bao gồm tín hiệu RS dùng để điều khiển lựa chọn thanh ghi. Tín hiệu R/W dùng để điều khiển quá trình đọc và ghi. Tín hiệu E là chân cho phép dạng xung chốt. Ba tín hiệu này còn có chức năng giao tiếp chuẩn SPI ở một số GLCD. Hai tín hiệu PSB và tín hiệu reset tích cực mức thấp.

Các chân dữ liệu D7÷D0: Chân số 7 đến chân số 14 là 8 chân dùng để trao đổi dữ liệu giữa thiết bị điều khiển và GLCD.

6.5.3 IC điều khiển GLCD ST7920

Để điều khiển GLCD, có các IC chuyên dùng được tích hợp bên dưới GLCD và có nhiều IC điều khiển, GLCD của kit thực hành thì dùng IC có mã số là ST7920.

Sơ đồ khối của IC ST7920 như hình 6-19.



Hình 6- 17. Cấu trúc IC ST7920.

Các khối bao gồm:

Khối giao tiếp với vi điều khiển có tên là MPU interface gồm có 5 tín hiệu RS, RW, E, PSB, RESET và 8 đường dữ liệu.

Có 2 thanh ghi gồm thanh ghi lệnh (Instruction Register) và thanh ghi dữ liệu (Data Register).

Vùng nhớ DDRAM chứa dữ liệu để hiển thị (Display Data RAM) giống LCD gồm 60 ô nhớ và mỗi ô nhớ là 16 bit (60×16 bit).

Vùng nhớ CGROM là vùng nhớ ROM phát kí tự (Character Generator ROM) có dung lượng 2M bit, vùng nhớ này giống LCD chứa sẵn các kí tự phổ biến.

Vùng nhớ CGRAM là vùng nhớ RAM phát kí tự tự tạo (Character Generator RAM) có dung lượng 1024 bit, vùng nhớ này giống LCD dành cho người dùng thiết lập các kí tự tự tạo.

Vùng nhớ HCGROM là vùng nhớ ROM phát kí tự nhưng kích thước mỗi kí tự giảm một nửa (Half Character Generator ROM) có dung lượng 1024 ô nhớ, mỗi ô 16 bit (1024×16 bit).

Vùng nhớ GRAM chứa dữ liệu hình ảnh để hiển thị (Graphic RAM) có dung lượng 1024 ô nhớ và mỗi ô nhớ chứa 16 bit (1024×16 bit).

6.5.4 Mô tả chức năng ic điều khiển GLCD ST7920

Giao tiếp với MPU

IC điều khiển ST7920 hỗ trợ 3 chế độ giao tiếp với MPU: chế độ giao tiếp song-song 8 bit, chế độ giao tiếp song – song 4 bit và chế độ giao tiếp đồng bộ nối tiếp SPI.

Tín hiệu PSB khi bằng 1 thì chọn chế độ giao tiếp song – song.

Tín hiệu PSB khi bằng 0 thì chọn chế độ giao tiếp nối tiếp SPI.

Còn chế độ giao tiếp song – song 8 bit hay 4 bit thì tùy thuộc vào bit DL trong lệnh thiết lập chức năng (Function set) – giống LCD.

Hai thanh ghi 8 bit DR và IR được dùng cho các hoạt động ghi và đọc của IC ST7920.

Thanh ghi dữ liệu DR dùng để truy xuất dữ liệu các vùng nhớ DDRAM/CGRAM/GRAM và IRAM với con trỏ địa chỉ lưu trong bộ đếm AC (Address Counter).

Thanh ghi lệnh IR dùng để lưu trữ lệnh từ MPU gửi đến ST7920.

Các vùng nhớ

Vùng nhớ phát kí tự CGROM 16×16 và HCGROM 8×16

IC ST7920 cung cấp CGROM lên đến 8192 font kí tự 16×16 và 126 kí tự 8×16 . Vùng nhớ này hỗ trợ cho ngôn ngữ Trung Quốc và tiếng Anh. Hai byte liên tiếp được dùng để chỉ định một kí tự 16×16 hoặc 2 kí tự 8×16 .

Các mã kí tự được ghi vào DDRAM và font tương ứng được ánh xạ từ vùng nhớ CGROM hoặc HCGROM để điều khiển hiển thị trên LCD.

Vùng nhớ chứa kí tự tự tạo CGRAM

IC ST7920 cung cấp cho người dùng khả năng tự tạo kí tự theo font mong muốn.

Vùng nhớ này có địa chỉ 6 bit nên có thể lưu được tối đa 4 mã kí tự tự tạo có kích thước 16×16 . 4 bit thấp từ 3 đến 0 dùng để định địa chỉ 16 từ dữ liệu của kí tự tự tạo, 2 bit 5 và 4 dùng để truy xuất 1 trong 4 kí tự.

4 kí tự tự tạo sẽ có mã hiển thị là 0000H, 0002H, 0004H và 0006H.

Khi tạo mã kí tự thì hàng cuối cùng thường để trống dùng để nhập nháy con trỏ.

Vùng nhớ DDRAM

DDRAM có 64×2 byte (64×16 bit) nên có thể lưu trữ 16 kí tự có kích thước 16×16 cho 1 hàng, hoặc 32 kí tự 8×16 cho 1 hàng. Các mã kí tự lưu trong vùng nhớ DDRAM trỏ đến vùng nhớ với font lưu trong CGROM, HCGROM và CGRAM.

ST7920 hiển thị font với độ cao phân nửa của vùng nhớ HCGROM, font do người dùng định nghĩa ở vùng nhớ CGRAM và font đầy đủ 16×16 ở vùng nhớ CGROM.

Các mã dữ liệu 02H đến 7FH được dùng cho font kí tự với độ cao giảm phân nửa.

Bảng 6-16. Các font chữ 16×16 của các vùng nhớ của GLCD.

DDRAM data (char. code)					CGRAM Addr.					CGRAM data (higher byte)								CGRAM data (lower byte)													
B15~ B4	B3	B2	B1	B0	B5	B4	B3	B2	B1	B0	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0					
0	X	00	X	00							0	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0					
											0	0	0	1	1	1	1	1	1	0	0	1	0	0	0	0					
											0	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	1	0	0	
											0	0	1	1	0	0	0	1	0	0	0	0	0	1	1	1	1	1	1	0	
											0	1	0	0	0	0	1	0	0	1	0	0	1	0	0	0	0	1	0	0	
											0	1	0	1	0	0	1	1	1	1	0	0	1	0	0	0	0	1	0	0	
											0	1	1	0	0	1	1	0	0	1	0	1	0	1	0	0	1	0	0	0	
											0	1	1	1	1	0	1	0	0	1	1	0	0	1	0	0	1	0	0	0	
											1	0	0	0	0	0	1	0	0	1	0	0	1	0	0	1	0	0	0	0	
											1	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	0	1	0	0	
											1	0	1	0	0	0	1	0	0	1	0	0	0	1	0	0	0	0	0	0	
											1	0	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0	1	0	0	0
											1	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	1	0	0	0
											1	1	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0
											1	1	1	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0
											1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	X	01	X	01							0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1					
											0	0	0	1	0	0	0	1	1	0	1	0	0	0	0	0	1	0	0		
											0	0	1	0	0	0	1	0	0	0	0	1	0	0	1	1	0	1	0	0	
											0	0	1	1	0	1	0	1	1	1	0	1	1	0	1	0	0	1	0	0	
											0	1	0	0	1	0	0	0	0	0	0	0	0	1	0	1	0	0	1	0	
											0	1	0	1	0	1	1	1	1	1	1	1	0	0	1	0	0	1	0	0	
											0	1	1	0	0	1	0	0	0	0	0	1	0	0	1	0	0	1	0	0	
											0	1	1	1	0	1	1	1	1	1	1	1	0	0	1	0	0	1	0	0	
											1	0	0	0	0	1	0	0	0	0	0	1	0	0	1	0	0	1	0	0	
											1	0	0	1	0	1	1	1	1	1	1	1	0	0	1	0	0	1	0	0	
											1	0	1	0	0	1	0	0	0	0	0	0	0	0	1	0	0	1	0	0	
											1	0	1	1	0	0	1	1	1	1	1	1	1	0	0	1	0	0	1	0	
											1	1	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	0	1	0	
											1	1	0	1	0	1	0	1	1	1	1	1	1	0	0	0	1	1	1	0	0
											1	1	1	0	0	1	0	1	0	0	0	0	0	1	0	0	0	1	0	0	0
											1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Các mã dữ liệu A140H đến D75FH được dùng cho code BIG5 và A1A0H đến F7FFH cho code GB.

- Để hiển thị font HCGROM thì ghi 2 byte vào DDRAM để hiển thị 2 font 8×16. Một byte tượng trưng cho 1 font kí tự. Dữ liệu của mỗi byte từ 02H đến 7FH.
- Để hiển thị font CGRAM thì ghi 2 byte vào DDRAM để hiển thị 1 font 16×16. Chỉ có các từ sau là cho phép 0000H, 0002H, 0004H, 0006H.
- Để hiển thị font CGROM thì ghi 2 byte vào DDRAM để hiển thị 1 font 16×16. Các từ sau A140H, D75FH là cho code BIG5 và A1A0H, F7FFH là cho code GB.

Byte cao D15 đến D7 được gọi đến GLCD trước và sau đó là byte thấp từ D7 đến D0. Hãy tham khảo thêm bảng 6-16.

Bảng 6-17. Các font chữ 8x16 của GLCD.

H/L	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☹	♥	♦	♣	♠	•	◐	◑	♂	♀	♂	♀	♂	♀
1	▶	◀	↕	!!	¶	§	—	↑	↓	→	←	↶	↷	▲	▼	
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	Δ

Các ký tự có kích thước 8×16 nằm trong bảng theo sau:

Các font của CGRAM và CGROM chỉ có thể được hiển thị ở vị trí bắt đầu của mỗi địa chỉ như bảng 6-18.

Bảng 6-18. Địa chỉ gọi mã của font 16x16.

80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
H	L	H	L	H	L	H	L	H	L	H	L	H	L	H	L
S	i	t	r	o	n	i	x								
矽	創	電	子	.	.	中	文	編	碼	(	正	確	)		
矽	創	電	子	.	.	.	中	文	編	碼					

Table 4

Incorrect position

Trong hình trên thì để hiển thị ký tự kích thước 16×16 thì phải gọi 2 byte liên tiếp vào cùng 1 địa chỉ, ví dụ bạn phải gọi 2 byte cùng 1 địa chỉ 0x85 chứ không được gọi 1 byte ở địa chỉ 0x85 và 1 byte ở địa chỉ 0x86.

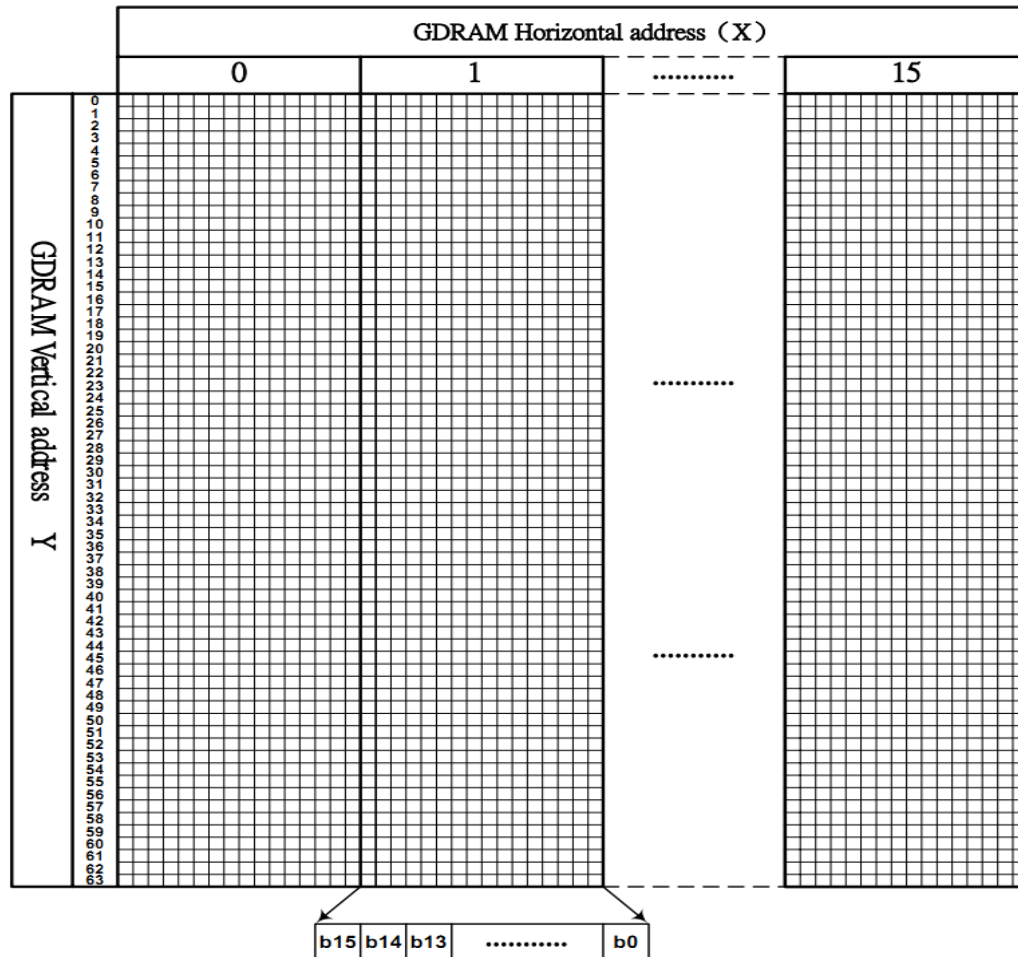
Vùng nhớ GDRAM

Vùng nhớ RAM hiển thị hình ảnh hỗ trợ không gian ánh xạ có kích thước 64×256 bit như hình 6-20.

Địa chỉ vùng nhớ GDRAM được thiết lập bằng cách ghi 2 byte liên tiếp cho địa chỉ chiều ngang và chiều dọc. Hai byte dữ liệu ghi vào GDRAM cho một địa chỉ. Bộ đếm địa chỉ tự động tăng lên 1 để lưu 2 byte tiếp theo.

Trình tự thực hiện như sau:

- Thiết lập địa chỉ dọc (Y) cho GDRAM
- Thiết lập địa chỉ ngang (X) cho GDRAM
- Ghi 8 bit D15 đến D8 vào GDRAM – byte thứ nhất
- Ghi 8 bit D7 đến D0 vào GDRAM – byte thứ hai

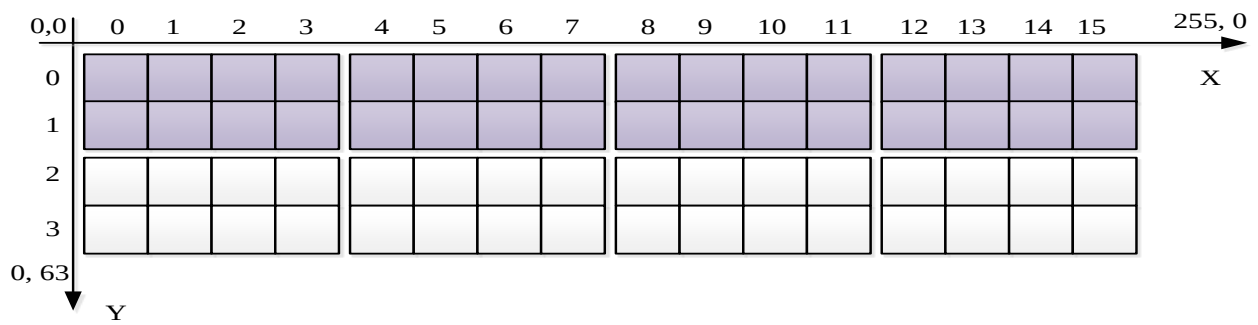


Hình 6-18. Tung độ hiển thị GDRAM và địa chỉ tương ứng của GLCD.

Hình 6-18 cho thấy địa chỉ theo chiều ngang x (horizontal address) là 16 (từ 0 đến 15) tính theo word 16 bit, tính theo bit thì bằng $16 \times 16 = 256$ bit.

Địa chỉ theo chiều dọc y (vertical address) là 64 bit (từ 0 đến 63).

Để giúp các bạn dễ hiểu thì hình 6-18 được vẽ lại như hình 6-21.



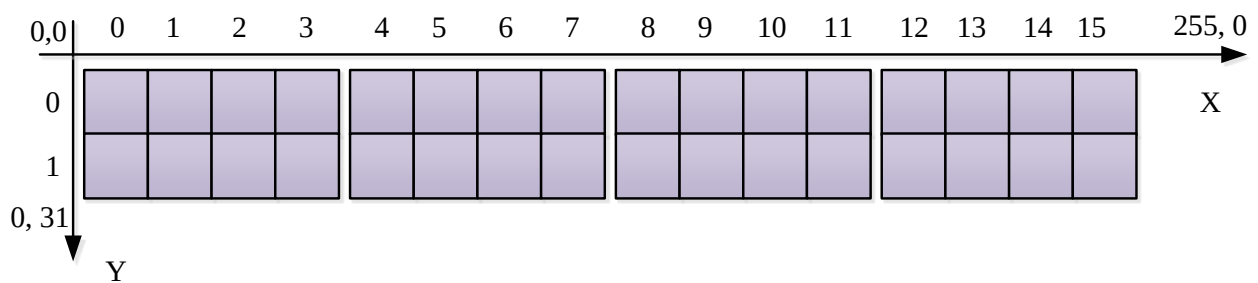
Hình 6- 19. Vẽ lại hình GDRAM theo khối bộ nhớ.

Theo cột thì vẫn là 16 cột, mỗi cột có dữ liệu là 16 bit tương ứng 16 pixel.

Theo hàng thì cũng chia ra thành 4 hàng, mỗi ô của 1 hàng có 16 bit tương ứng 16 pixel.

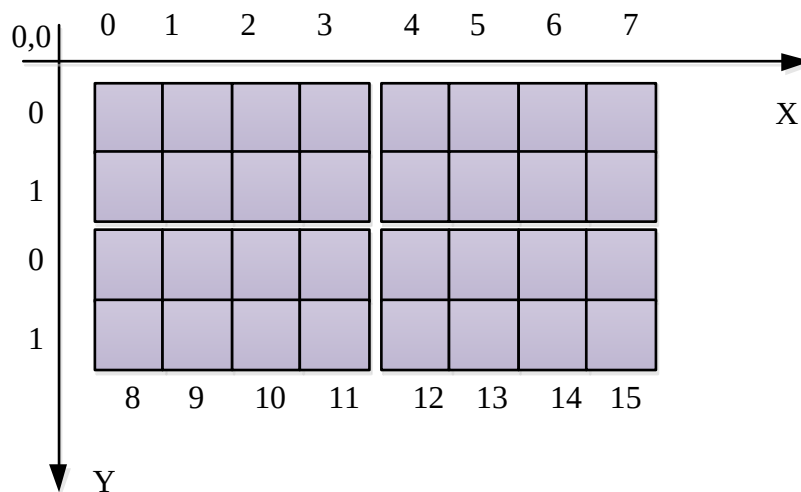
Số lượng pixel mà IC ST7920 có thể điều khiển là 64×256 .

Màn hình GLCD gắn ở kit thực hành thì có kích thước là 64×128 . Thực tế thì kích thước này đúng là 32×256 , điều đó có nghĩa là người ta cắt bỏ phần bên dưới và vùng nhớ hiển thị còn lại như hình 6-22.



Hình 6-20. Bộ nhớ GDRAM của GLCD có kích thước 32×256 .

Khi chế tạo, GLCD có kích thước dài và mỏng nên thì người ta bố trí lại như hình 6-23.

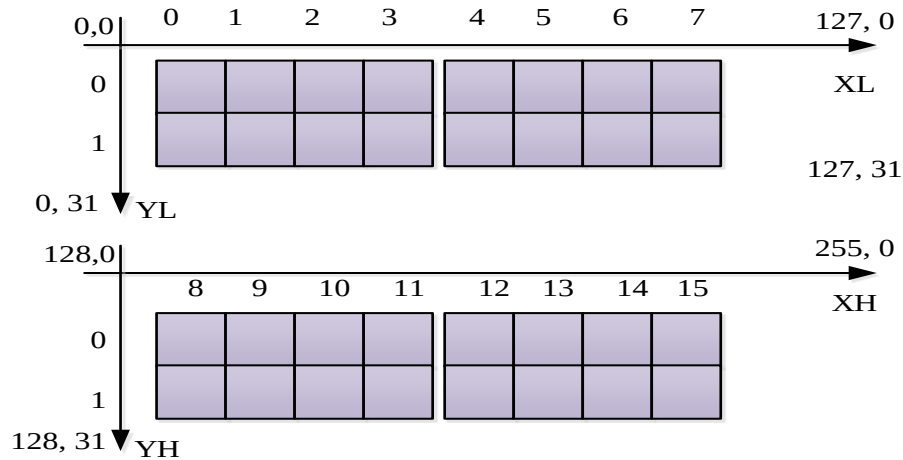


Hình 6-21. Bộ nhớ GDRAM của GLCD 32×256 bố trí theo kích thước 64×128 .

Người ta cắt 8 vùng nhớ ở bên phải đem xuống đặt bên dưới, khi đó tính theo cách bố trí này thì GLCD có kích thước 64×128 .

Khi lập trình điều khiển 1 pixel hiển thị thì ta không thể tính theo kích thước 64×128 của hình 6-21 được mà phải tính theo kích thước 32×256 của hình 6-22.

Muốn lập trình cho tương thích với kích thước 64×128 , ta vẽ lại hình 6-21 như hình 6-24.



Hình 6-22. Bộ nhớ GDRAM của GLCD 32×256 chi tiết.

Giới hạn các tọa độ pixel: Khi lập trình điều khiển 1 pixel hiển thị theo kích thước ($y \times x = 64 \times 128$) thì y phải nằm trong giới hạn từ 0 đến 63 và x từ 0 đến 127.

Khi muốn điều khiển hiển thị 1 pixel, ta tiến hành kiểm tra y : nếu y nằm trong phạm vi nhỏ hơn 32 thì điểm pixel đó nằm ở 2 hàng trên (XL, YL), nếu lớn 32 thì ta phải trừ đi cho 32 và tăng kích thước của x lên 128 và khi đó chúng sẽ nằm ở 2 hàng dưới (XH, YH).

Công việc ta vừa trình bày là quy đổi kích thước 64×128 về kích thước thực là 32×256 .

Các tính toán trên theo trục ngang là tính toán theo pixel, còn tính toán theo từng khối thì có 16 khối từ 0 đến 15. Trong hình thì các con số 0, 1, 2 ... là theo khối.

Chỉ có trục ngang là theo khối còn dọc thì không.

Tại sao lại quan tâm đến khối vì trong lệnh khởi tạo địa chỉ cho trục ngang theo khối chỉ có 4 bit địa chỉ nên các tọa độ pixel theo kích thước sau khi chuyển đổi về 64×128 thì ta phải tiếp tục chuyển giá trị 128 về địa chỉ theo khối bằng cách lấy địa chỉ theo trục ngang chia cho 16.

6.5.5 Các lệnh điều khiển GLCD

IC ST7920 có 2 tập lệnh cơ bản và tập lệnh mở rộng như sau:

Tập lệnh cơ bản khi bit RE bằng 0.

Bảng 6-19. Tập lệnh điều khiển cơ bản của GLCD.

LỆNH	R S	R W	D 7	D6	D5	D4	D3	D2	D1	D0	Mô tả	cloc k
(1) Clear display	0	0	0	0	0	0	0	0	0	1	Write "20H" to DDRAM and set DDRAM address counter to "00H".	1.6m s

Đại Học Sư Phạm Kỹ Thuật											Khoa Điện – Điện Tử	
(2) Cursor home	0	0	0	0	0	0	0	0	1	0	Sets DDRAM address counter to zero, returns shifted display to original position. DDRAM contents remain unchanged.	72μs
(3) Entry mode set	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction, and specifies automatic shift.	72μs
(4) Display control	0	0	0	0	0	0	1	D	C	B	Turns display (D), cursor on/off (C) or cursor blinking (B).	72μs
(5) Cursor display shift	0	0	0	0	0	1	S/C	R/L	0	0	Set cursor moving and display shift control bit, and the direction, without changing DDRAM data.	72μs
(6) Function set	0	0	0	0	1	DL	x	0 (RE)	0	0	Set interface data length (DL: 4-bit/8-bit). RE = 1 extended instruction. RE = 0 basic instruction.	72μs
(7) Set CGRAM address	0	0	0	1	AC5	AC4	AC3	AC2	AC1	AC0	Set CGRAM address in address counter. Make sure that in extended instruction SR=0 (Scroll or RAM address select)	72μs
(8) Set DDRAM address	0	0	1	0 AC6	AC5	AC4	AC3	AC2	AC1	AC0	Set DDRAM address in address counter. AC6 is fixed to 0.	72μs
(9) Read Busy Flag and Address	0	1	B F	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Read busy flag (BF) for completion of internal operation, also read out the value of address counter (AC).	0μs
(10) Write data to RAM	1	0	D 7	D6	D5	D4	D3	D2	D1	D0	Write data to CGRAM/DDRAM/GDRAM	72μs

(11) Read data from RAM	1	1	D 7	D6	D5	D4	D3	D2	D1	D0	Read data from internal CGRAM/DDRAM/GDR AM	72μs
-------------------------------------	---	---	--------	----	----	----	----	----	----	----	--	------

Tập lệnh cơ bản khi bit RE bằng 1.

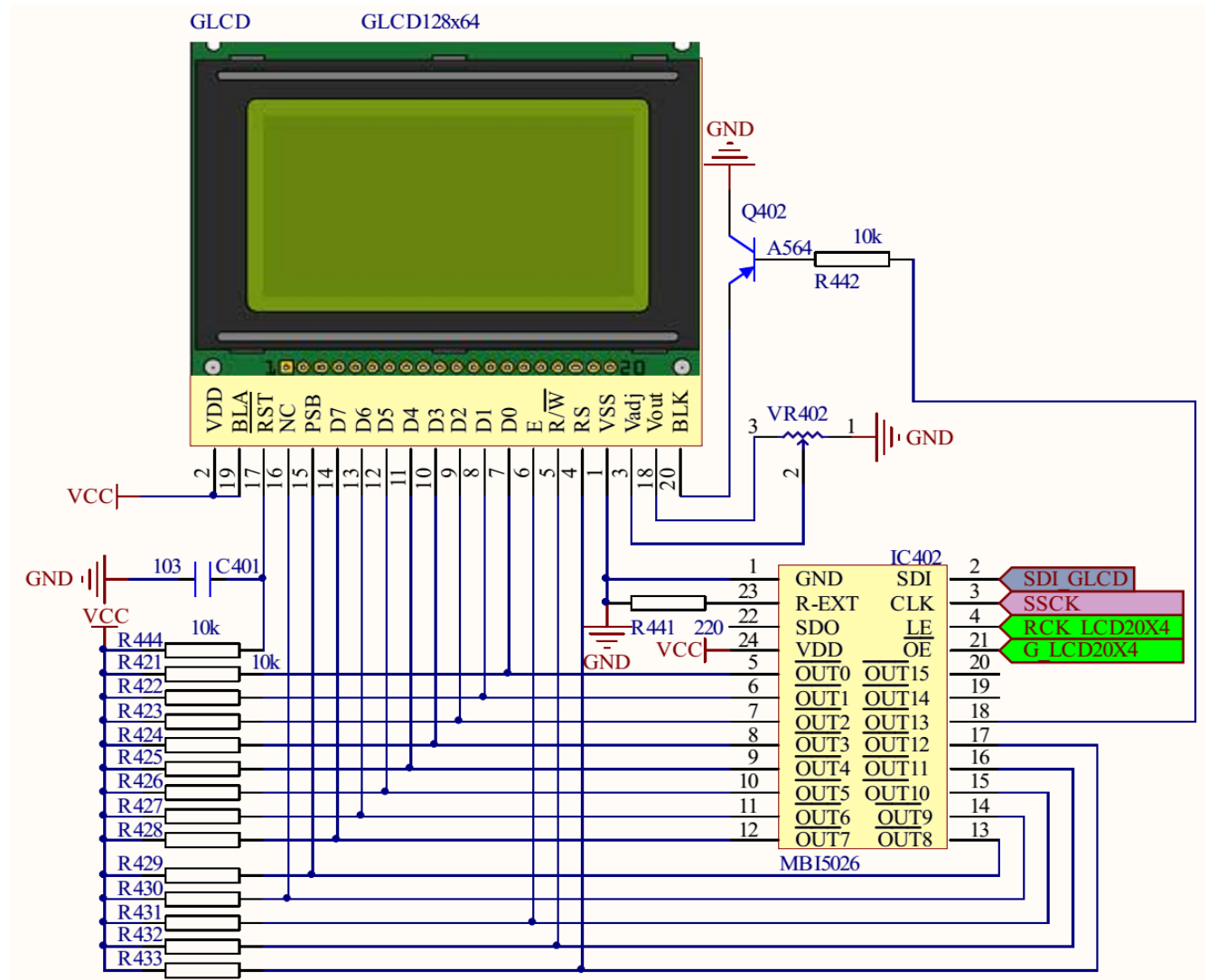
Bảng 6-20. Tập lệnh điều khiển mở rộng của GLCD.

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Mô tả	clock
(1) Stand by	0	0	0	0	0	0	0	0	0	1	Enter standby mode, any others instruction can terminate (Com1 .. 32 halted)	1.6ms
(2) Scroll or RAM addr select	0	0	0	0	0	0	0	0	1	SR	SR = 1: enable vertical scroll position SR = 0: enable CGRAM address (basic instruction)	72μs
(3) reverse	0	0	0	0	0	0	0	1	R1	R0	Select 1 out of 4 line (in DDRAM) and decide whether to reverse the display by toggling this instruction R1, R0 initial value is 00	72μs
(4) Extended Function Set	0	0	0	0	1	DL	x	1 (RE)	G	0	Set interface data length (DL: 4-bit/8-bit). RE = 1 extended instruction. RE = 0 basic instruction. G = 1 graphic display on. G = 0 graphic display off.	72μs
(5) Set IRAM or scroll addr	0	0	0	1	AC5	AC4	AC3	AC2	AC1	AC0	SR = 1: AC5 – AC0 the address of vertical scroll.	72μs
(6) Set GRAPHIC RAM addr	0 0	0 0	1 1	0 0	AC5 0	AC4 0	AC3 AC3	AC2 AC2	AC1 AC1	AC0 AC0	Set GDRAM address to address counter (AC). First set vertical address and horizontal address by consecutive writing. Vertical address range AC5 ... AC0. Horizontal address range AC3 ... AC0.	72μs

6.6 CÁC BÀI THỰC HÀNH DÙNG GLCD

6.6.1 Sơ đồ mạch của GLCD

Kit thực hành có kết nối 1 GLCD 128×64 qua IC mở rộng port là MBI 5026 có sơ đồ như hình sau:



Hình 6-23. Sơ đồ nguyên lý module GLCD 128×64.

Phần điều khiển GLCD đã trình bày ở phần LCD.

6.6.2 Các bài thực hành hiển thị kí tự trên GLCD

GLCD có khả năng hiển thị kí tự và hình ảnh, phần này thực hành các bài hiển thị kí tự trên GLCD giống như trên LCD.

Thư viện GLCD. Chương trình thư viện dùng cho GLCD đã có trong thư mục của bạn, bạn có thể sử dụng.

Tên file là “TV_GLCD.C” và “TV_GLCD.H”

a. File thư viện “TV GLCD.H”

```

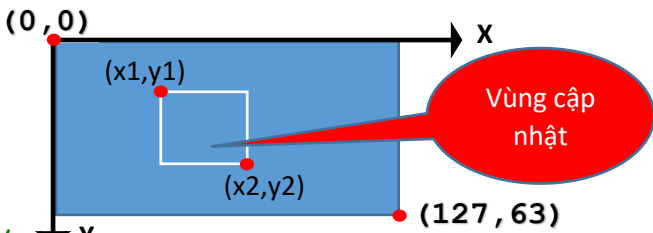
#ifndef __TV_GLCD_H__
#define __TV_GLCD_H__
#define GLCD_NGANG 16 // Định nghĩa chiều ngang thực tế GLCD (16x16)
#define GLCD_DOC 32 // Định nghĩa chiều dọc thực tế của GLCD
#define GLCD_WIDTH 128 // Định nghĩa chiều ngang hiển thị
typedef union
{
    unsigned int16 word;
    unsigned int8 nbyte[2];
} GLCD_DOTS;

typedef struct
{
    int8 x;
    int8 y;
    int8 size;
    int1 mau ;
    int1 tt_text;
    int1 tt_update;
    int1 overlay;
    unsigned int8 x1;
    unsigned int8 y1;
    unsigned int8 x2;
    unsigned int8 y2;
    GLCD_DOTS PIXEL[GLCD_DOC][GLCD_NGANG];
} GLCD_VAR_STRUCT;
GLCD_VAR_STRUCT GLCD;
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx Các hàm điều khiển xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
result glcd_command(int8 data);
result glcd_data(int8 data);
result glcd_setup(void);
result glcd_clear(int1 color);
result glcd_text_setup(unsigned int16 x, unsigned int16 y,int8 size, int1
color,int1 overlay);
result glcd_text(int8 t);
result glcd_pixel(unsigned int8 x, unsigned int8 y, int1 color);
result glcd_line(unsigned int8 x1, unsigned int8 y1, unsigned int8 x2,
unsigned int8 y2, int1 color);
result glcd_bar(unsigned int8 x1, unsigned int8 y1, unsigned int8 x2,
unsigned int8 y2, unsigned int8 width, int1 color);
result glcd_rect(unsigned int8 x1, unsigned int8 y1, unsigned int8 x2,
unsigned int8 y2, int1 fill, int1 color);
result glcd_circle(unsigned int8 x, unsigned int8 y, unsigned int8 radius,
int1 fill, int1 color);
result glcd_triangle(signed int16 x1,signed int16 y1, signed int16 x2,
signed int16 y2, signed int16 x3, signed int16 y3,int1 fill,int1 color);
result glcd_spkt_logo_raw(int x) ;
result glcd_update(void);
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx FONT xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
const unsigned int8 GLCD_FONT[51][5] = {...}; // Bảng Font Graphic,đã xóa cho
const unsigned int8 GLCD_FONT2[44][5]= {...}; // gọn,xem đầy đủ trong thư viện
const unsigned int8 GLCD_Logo_DHSPKTP [] = {...}; // Hình Logo SPKT
#endif

```

b. File thư viện “TV_GLCD.C”

```

#ifndef __TV_GLCD_C__
#define __TV_GLCD_C__
#include<tv_glcd.h>
/* Hàm cập nhật một vùng dữ liệu từ RAM VĐK ra GLCD- Không nên dùng
- x1,y1,x2,y2 là tọa độ vùng cần cập nhật
*/


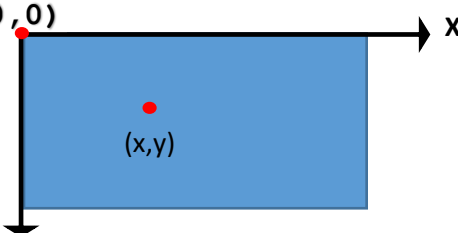
```

result GLCD_UPDATE_COORDINATE(unsigned int8 x1,unsigned int8 y1,
 unsigned int8 x2, unsigned int8 y2)
{
 if(GLCD.tt_update==0)
 {
 GLCD.x1 = x1;
 GLCD.x2 = x2;
 GLCD.y1 = y1;
 GLCD.y2 = y2;
 }
 else
 {
 if(x1<GLCD.x1) GLCD.x1=x1;
 if(x2>GLCD.x2) GLCD.x2=x2;
 if(y1<GLCD.y1) GLCD.y1=y1;
 if(y2>GLCD.y2) GLCD.y2=y2;
 }
 GLCD.tt_update=1;
 return OK;
}
/* Hàm xóa toàn bộ dữ liệu đồ họa trong RAM VĐK
- color : màu cần xóa (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì màn hình mới bị xóa */
result glcd_clear(int1 color)
{
 int16 d,doc,ngang;
 if(color) d=0xffff; else d=0;
 for (DOC=0; DOC < GLCD_DOC; DOC++)
 {
 for (NGANG=0; NGANG < GLCD_NGANG; NGANG++)
 {
 GLCD.PIXEL[DOC][NGANG].word = d;
 }
 }
 GLCD.x1 = 0;
 GLCD.x2 = 127;
 GLCD.y1 = 0;
 GLCD.y2 = 63;
 GLCD.tt_update=1;
}

```


```

```

    return OK;
}
/* Hàm ghi 1 điểm ảnh (pixel) ra màn hình GLCD
(0,0)

- color : màu cần ghi (0:xanh- 1:trắng)
*/
result glcd_pixel(int8 x, int8 y, int1 color)
{
    int8 b,doc,ngang;
    if(y>31){x += 128; y-= 32;};
    DOC = y;
    NGANG = x/16;
    b = 15 - (x % 16);

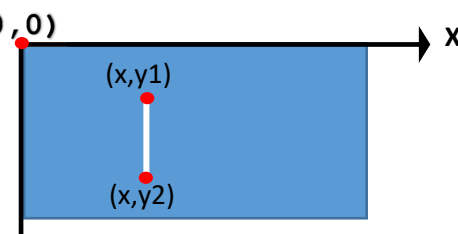
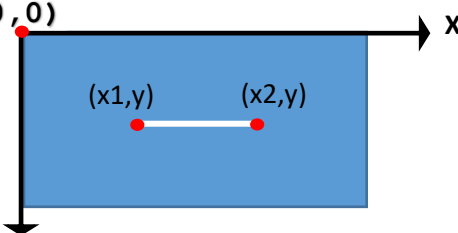
    if (color == 1) bit_set (GLCD.PIXEL[DOC][NGANG].word, b);
    else bit_clear (GLCD.PIXEL[DOC][NGANG].word, b);
    GLCD_UPDATE_COORDINATE(x,y, x, y);
    return OK;
}

/* Hàm ghi 1 điểm ảnh (pixel) vào RAM VĐK
- x, y : tọa độ điểm ảnh cần ghi
- color : màu cần ghi (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị
*/
result GLCD_PIXEL_TEMP(int8 x, int8 y, int1 color)
{
    int8 b,doc,ngang;
    if(y>31){x += 128; y-= 32;};
    DOC = y;
    NGANG = x/16;
    b = 15 - (x % 16);

    if (color == 1) bit_set (GLCD.PIXEL[DOC][NGANG].word, b);
    else bit_clear (GLCD.PIXEL[DOC][NGANG].word, b);
    return OK;
}
/* Hàm cập nhật nội dung hiển thị từ RAM VĐK ra màn hình GLCD
*/
result glcd_update ()
{
    int8 hesox,hesox2,hesoy,doc,ngang;
    if(GLCD.x1>GLCD.x2) {hesox=GLCD.x1; GLCD.x1=GLCD.x2, GLCD.x2= hesox;}
    if(GLCD.y1>GLCD.y2) {hesoy=GLCD.y1; GLCD.y1=GLCD.y2, GLCD.y2= hesoy;}
    if (GLCD.tt_update)
    {
        GLCD.x1 = GLCD.x1/16; GLCD.x2=GLCD.x2/16;
        for (DOC = GLCD.y1; DOC<=GLCD.y2; DOC++)

```

```

    {
        if(DOC>31) {hesox = 8+GLCD.x1;hesox2= GLCD.x2+8; hesoy = DOC-32;}
        else {hesox = GLCD.x1 ;hesox2 = GLCD.x2 ; hesoy = DOC; }
        GLCD_COMMAND( 0x80 |hesoy);
        GLCD_COMMAND( 0x80 |hesox);
        for (NGANG=hesox; NGANG <=hesox2; NGANG++)
        {
            GLCD_DATA( GLCD.PIXEL[hesoy][NGANG].nbyte[1]);
            GLCD_DATA( GLCD.PIXEL[hesoy][NGANG].nbyte[0]);
        }
    }
    GLCD.tt_update = FALSE;
}
return OK;
}
/* Hàm vẽ một đoạn thẳng đứng
(0,0)

- color : màu cần ghi (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị
*/
result GLCD_VERTICAL_LINE(unsigned int8 x, unsigned int8 y1,
                          unsigned int8 y2, int1 color)
{
    unsigned int8 i;
    for(i=y1;i<=y2;i++) glcd_pixel_temp(x, i, color) ;
    return OK;
}
/* Hàm vẽ một đoạn thẳng nằm ngang
(0,0)

- color : màu cần ghi (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị
*/
result GLCD_HORIZONE_LINE(unsigned int8 x1, unsigned int8 x2,
                          unsigned int8 y, int1 color)
{
    int8 doc;
    unsigned int8 i,right,left;
    if(y>31){x1 += 128;x2 += 128; y-= 32;};
    left=x1/16; right = x2/16;DOC = y;

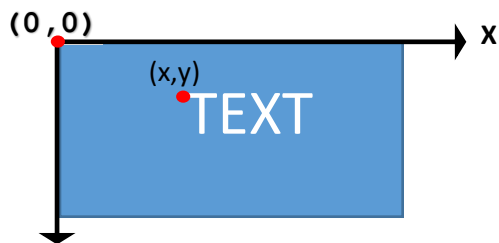
```

```

if (left==right)
{
    if(color)
        GLCD.PIXEL[DOC][left].word= GLCD.PIXEL[DOC][left].word
            | (0xffff<<(15-(x2%16))) & (0xffff>>(x1%16));
    else
        GLCD.PIXEL[DOC][left].word= GLCD.PIXEL[DOC][left].word
            & (~(0xffff<<(15-(x2%16))) & (0xffff>>(x1%16)));
}
else
{
    if(color)
    {
        GLCD.PIXEL[DOC][right].word = GLCD.PIXEL[DOC][right].word
            | (0xffff<<(15-(x2%16)));
        GLCD.PIXEL[DOC][left].word =
            GLCD.PIXEL[DOC][left].word | (0xffff>>(x1%16));
        for(i = left + 1; i < right; i++)
            GLCD.PIXEL[DOC][i].word=0xffff;
    }
    else
    {
        GLCD.PIXEL[DOC][right].word = GLCD.PIXEL[DOC][right].word &
            (~(0xffff<<(15-(x2%16))));
        GLCD.PIXEL[DOC][left].word = GLCD.PIXEL[DOC][left].word &
            (~(0xffff>>(x1%16)));
        for(i = left + 1; i < right; i++) GLCD.PIXEL[DOC][i].word=0;
    }
}
return OK;
}

```

/* Hàm cài đặt các thông số trước khi muốn hiển thị chuỗi ký tự



- x,y : Tọa độ muốn hiển thị chuỗi ký tự như hình trên
- size : kích thước chữ 1 là 5x7 (n là nx(5x7)- vd n=2 là 10x14)
- color : màu cần ghi (0:xanh- 1:trắng)
- overlay: Có ghi đè không (1: có ghi đè - 0: không ghi đè)

*/

```

result glcd_text_setup(unsigned int16 x, unsigned int16 y,int8 size, int8
color,int1 overlay)
{
    GLCD.mau = color;
    GLCD.size = size;
    GLCD.x    = x;
    GLCD.y    =y;
    GLCD.tt_text=0;
    GLCD.overlay=overlay;
    return OK;
}

```



```

/* Hàm hiển thị chuỗi ký tự (hàm này dùng giống "LCD_DATA()" của LCD)
- t : chuỗi ký tự cần hiển thị
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị
*/
result glcd_text( char t)
{
    static unsigned int8 x1,x2,y1,y2;
    unsigned int8 j, k, l, m;
    unsigned int8 pixelData[5];
    if(GLCD.overlay)
    {
glcd_rect(GLCD.x,GLCD.y,GLCD.x+6*GLCD.size,GLCD.y+7*GLCD.size,1,!GLCD.mau);
    }
    if(GLCD.tt_text==0)
    {
        x1=GLCD.x;
        y1=GLCD.y;
        GLCD.tt_text=1;
    }
    if(t < 'S')
        memcpy(pixelData, GLCD_FONT[t - ' '], 5);
    else if(t <= '~')
        memcpy(pixelData, GLCD_FONT2[t - 'S'], 5);
    else
        memcpy(pixelData, GLCD_FONT[0], 5);
    if(GLCD.x+5*GLCD.size >= GLCD_WIDTH)
    {
        GLCD.x = 0;
        GLCD.y += 7*GLCD.size + 1;
    }
    for(j=0; j<5; ++j, GLCD.x+=GLCD.size)
    {
        for(k=0; k < 7; ++k)
        {
            if(bit_test(pixelData[j], k))
            {
                for(l=0; l < GLCD.size; ++l)
                    GLCD_horizone_line(GLCD.x, GLCD.x+GLCD.size-
                                        1,GLCD.y+k*GLCD.size+1,GLCD.mau);
            }
        }
    }
    x2 = GLCD.x;
    GLCD.x+=GLCD.size;
    y2= GLCD.y + 7*GLCD.size;
    GLCD_UPDATE_COORDINATE(x1,y1, x2,y2);
    return OK;
}
/* Hàm vẽ logo SPKT kích thước 64x64
- x : cột bắt đầu vẽ
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị
*/

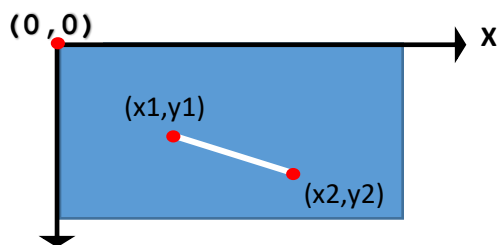
```

```

result glcd_spkt_logo_raw(int x)
{
    unsigned int8 i=0, j=0, k=0, y=0;
    unsigned int16 count=0;
    GLCD_UPDATE_COORDINATE(x, 0, x+64, 63);
    for(j=0; j<64; j++)
    {
        for(; i<64;)
        {
            for(k=8; k>0; k--)
            {
                glcd_pixel_temp(i+x, j+y, bit_test(GLCD_Logo_DHSPKTTP[count], (k-1)));
                i++;
            }
            count++;
        }
        i=0;
    }
    return OK;
}

```

/* Hàm vẽ một đoạn thẳng đi qua 2 điểm



- color : màu cần ghi (0: xanh- 1: trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị

*/

```

result glcd_line(unsigned int8 x1, unsigned int8 y1, unsigned int8 x2,
unsigned int8 y2, int1 color)
{
    unsigned int16    dy, dx;
    signed int8  addx=1, addy=1;
    signed int16 P, diff;
    unsigned int8 i=0, xt1, xt2, yt1, yt2;
    dx = abs((signed int8) (x2 - x1));
    dy = abs((signed int8) (y2 - y1));
    xt1=x1; xt2=x2;
    yt1=y1, yt2=y2;
    if(x1 > x2)
    {
        addx = -1;
        xt2=x1;
        xt1=x2;
    }
    if(y1 > y2)
    {
        addy = -1;
        yt2=y1;
        yt1=y2;
    }
}

```

```
GLCD_UPDATE_COORDINATE(xt1,yt1, xt2,yt2);
if(dx >= dy)
{
    dy *= 2;
    P = dy - dx;
    diff = P - dx;

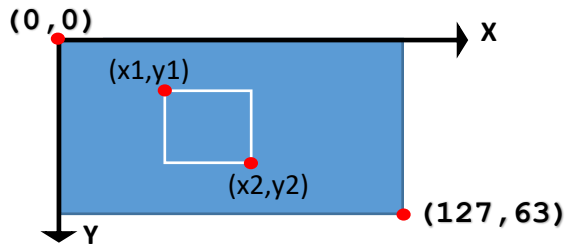
    for(; i<=dx; ++i)
    {
        glcd_pixel_temp(x1, y1, color);

        if(P < 0)
        {
            P += dy;
            x1 += addx;
        }
        else
        {
            P += diff;
            x1 += addx;
            y1 += addy;
        }
    }
}
else
{
    dx *= 2;
    P = dx - dy;
    diff = P - dy;

    for(; i<=dy; ++i)
    {
        glcd_pixel_temp(x1, y1, color);

        if(P < 0)
        {
            P += dx;
            y1 += addy;
        }
        else
        {
            P += diff;
            x1 += addx;
            y1 += addy;
        }
    }
}
return OK;
}
```

/* Hàm vẽ hình chữ nhật



- fill : có tô kín bên trong hay không (1: tô kín - 0: không tô kín)
- color : màu cần ghi (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị

*/

```
result glcd_rect(unsigned int8 x1, unsigned int8 y1, unsigned int8 x2,
unsigned int8 y2, int1 fill, int1 color)
```

```
{
```

```
    unsigned int8  xmin, xmax, ymin, ymax,i;
```

```
    xmin = x1;
```

```
    xmax = x2;
```

```
    ymin = y1;
```

```
    ymax = y2;
```

```
    if(x1 > x2)
```

```
    {
```

```
        xmin = x2;
```

```
        xmax = x1;
```

```
    }
```

```
    if(y1 > y2)
```

```
    {
```

```
        ymin = y2;
```

```
        ymax = y1;
```

```
    }
```

```
    if(fill)
```

```
    {
```

```
        for(i=ymin; i <= ymax; i++)
```

```
            glcd_horizone_line(xmin, xmax,i, color);
```

```
    }
```

```
    else
```

```
    {
```

```
        glcd_vertical_line(xmin, ymin,ymax, color);
```

```
        glcd_vertical_line(xmax, ymin,ymax, color);
```

```
        glcd_horizone_line(x1, x2, y1, color);
```

```
        glcd_horizone_line(x1, x2, y2, color);
```

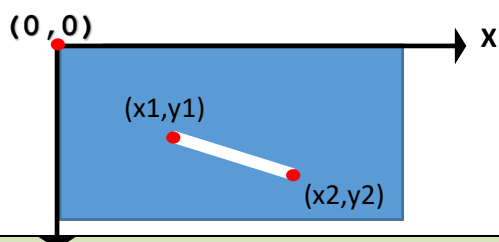
```
    }
```

```
    GLCD_UPDATE_COORDINATE(xmin,ymin, xmax,ymax);
```

```
    return OK;
```

```
}
```

/* Hàm vẽ một đoạn thẳng có bề dày đi qua 2 điểm



```

- width: bề dày của đoạn thẳng (nếu =1 thì thành hàm "glcd_line")
- color : màu cần ghi (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị
*/

result glcd_bar(unsigned int8 x1, unsigned int8 y1, unsigned int8 x2,
unsigned int8 y2, unsigned int8 width, int1 color)
{
    unsigned int8      half_width;
    signed int16 dy, dx;
    signed int8  addx=1, addy=1, j;
    signed int16 P, diff, c1, c2;
    unsigned int8 i=0,xt1,yt1,xt2,yt2;
    dx = abs((signed int8)(x2 - x1));
    dy = abs((signed int8)(y2 - y1));
    half_width = width/2;
    c1 = -(dx*x1 + dy*y1);
    c2 = -(dx*x2 + dy*y2);
    xt1=x1-half_width;
    xt2=x2+half_width;
    yt1=y1-half_width;
    yt2=y2+half_width;
    if(x1 > x2)
    {
        signed int16 temp;
        temp = c1;
        c1 = c2;
        c2 = temp;
        addx = -1;
        xt1=x2-half_width;
        xt2=x1+half_width;
    }
    if(y1 > y2)
    {
        signed int16 temp;
        temp = c1;
        c1 = c2;
        c2 = temp;
        addy = -1;
        yt1=y2-half_width;
        yt2=y1+half_width;
    }
    if(xt1>127)    xt1=0;
    if(yt1>127)    yt1=0;
    if(xt2>127)    xt2=127;
    if(yt2>63)     yt2=63;
    GLCD_UPDATE_COORDINATE(xt1,yt1, xt2,yt2);
    if(dx >= dy)
    {
        P = 2*dy - dx;
        diff = P - dx;

        for(i=0; i<=dx; ++i)
        {

```

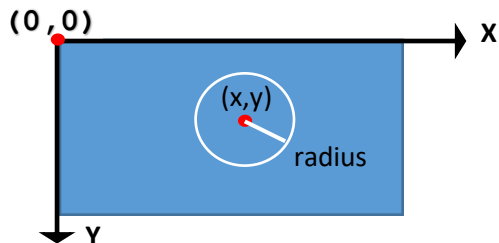
```

        for(j=-half_width; j<half_width+width%2; ++j)
        {
            glcd_pixel_temp(x1, y1+j, color);
        }
        if(P < 0)
        {
            P += 2*dy;
            x1 += addx;
        }
        else
        {
            P += diff;
            x1 += addx;
            y1 += addy;
        }
    }
}
else
{
    P = 2*dx - dy;
    diff = P - dy;

    for(i=0; i<=dy; ++i)
    {
        if(P < 0)
        {
            P += 2*dx;
            y1 += addy;
        }
        else
        {
            P += diff;
            x1 += addx;
            y1 += addy;
        }
        for(j=-half_width; j<half_width+width%2; ++j)
        {
            glcd_pixel_temp(x1+j, y1, color);
        }
    }
}
return OK;
}

```

/* Hàm vẽ hình tròn



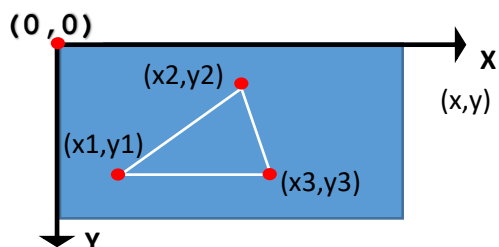
- fill : có tô kín bên trong hay không (1: tô kín - 0: không tô kín)
- color : màu cần ghi (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị

```

*/
result glcd_circle(unsigned int8 x, unsigned int8 y, unsigned int8 radius,
int1 fill, int1 color)
{
    unsigned int8 xt1,xt2, yt1, yt2;
    signed int8 a, b, P;
    a = 0;
    b = radius;
    P = 1 - radius;
    do
    {
        if(fill)
        {
            glcd_horizone_line(x-a, x+a, y+b, color);
            glcd_horizone_line(x-a, x+a, y-b, color);
            glcd_horizone_line(x-b, x+b, y+a, color);
            glcd_horizone_line(x-b, x+b, y-a, color);
        }
        else
        {
            glcd_pixel_temp(a+x, b+y, color);
            glcd_pixel_temp(b+x, a+y, color);
            glcd_pixel_temp(x-a, b+y, color);
            glcd_pixel_temp(x-b, a+y, color);
            glcd_pixel_temp(b+x, y-a, color);
            glcd_pixel_temp(a+x, y-b, color);
            glcd_pixel_temp(x-a, y-b, color);
            glcd_pixel_temp(x-b, y-a, color);
        }

        if(P < 0)
            P += 3 + 2 * a++;
        else
            P += 5 + 2 * (a++ - b--);
    } while(a <= b);
    xt1 = x-radius;
    xt2 = x+radius;
    yt1 = y-radius;
    yt2 = y+radius;
    if(xt1>127) xt1=0;
    if(yt1>127) yt1=0;
    if(xt2>127) xt2=127;
    if(yt2>63) yt2=63;
    GLCD_UPDATE_COORDINATE(xt1,yt1, xt2,yt2);
    return OK;
}
/* Hàm vẽ hình tam giác

```



```

- fill : có tô kín bên trong hay không (1: tô kín - 0: không tô kín)
- color : màu cần ghi (0:xanh- 1:trắng)
- Sau hàm này cần phải gọi hàm "glcd_update ()" thì mới hiển thị
*/

result glcd_triangle(signed int16 x1,signed int16 y1, signed int16 x2,
signed int16 y2, signed int16 x3, signed int16 y3,int1 fill,int1 color)
{
    unsigned int8 xt1, xt2, yt11,yt22;
    signed int16 x[3],y[3],n,yt1,yt2,yt,dx1,dx2,dx3,dy1,dy2,dy3;
    signed int16 sh1,sh2,sh3;
    x[0]=x1; x[1]=x2; x[2]=x3;y[0]=y1; y[1]=y2; y[2]=y3;
    if(x[0]>x[1]){yt=x[0];x[0]=x[1];x[1]=yt;yt=y[0];y[0]=y[1];y[1]=yt;}
    if(x[0]>x[2]){yt=x[0];x[0]=x[2];x[2]=yt;yt=y[0];y[0]=y[2];y[2]=yt;}
    if(x[1]>x[2]){yt=x[1];x[1]=x[2];x[2]=yt;yt=y[1];y[1]=y[2];y[2]=yt;}
    dx1= x[2]-x[0]; dx2= x[1]-x[0];dx3=x[2]-x[1];
    dy1= y[2]-y[0]; dy2= y[1]-y[0];dy3=y[2]-y[1];
    sh1=x[2]*y[0] - x[0]*y[2]; sh2=x[1]*y[0] - x[0]*y[1];
    sh3=x[2]*y[1] - x[1]*y[2];
    if(fill)
    {
        for(n=x[0] ; n<x[1]; n++)
        {
            yt1= (dy1*n + sh1)/dx1;
            yt2= (dy2*n + sh2)/dx2;
            if(yt2<yt1){yt= yt1; yt1=yt2; yt2=yt;}
            glcd_vertical_line(n,yt1,yt2,color);
        }
        for(n=x[1] ; n<x[2]; n++)
        {
            yt1= (dy1*n + sh1)/dx1;
            yt2= (dy3*n + sh3)/dx3;
            if(yt2<yt1){yt= yt1; yt1=yt2; yt2=yt;}
            glcd_vertical_line(n,yt1,yt2,color);
        }
    }
    else
    {
        glcd_line(x1,y1,x2,y2,color);
        glcd_line(x1,y1,x3,y3,color);
        glcd_line(x2,y2,x3,y3,color);
    }
    xt1 = x[0];
    xt2 = x[2];
    if(y[0]>y[1])yt22= y[0]; else yt22 = y[1];
    if(yt22<y[2]) yt22 = y[2];
    if(y[0]<y[1]) yt11= y[0]; else yt11 = y[1];
    if(yt11>y[2])yt11 = y[2];
    GLCD_UPDATE_COORDINATE(xt1,yt11, xt2,yt22);
    return OK;
}
/* Hàm xuất dữ liệu hiển thị ra GLCD
result glcd_data(int8 data)
{
    write_glcd128x64(0x17,data);
}

```

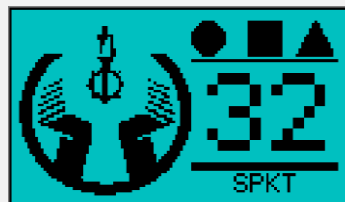


```

    write_glcd128x64(0x13,data);
    delay_us(9);
    return OK;
}
/* Hàm xuất mã lệnh ra GLCD
result glcd_command(int8 data)
{
    write_glcd128x64(0x07,data);
    write_glcd128x64(0x03,data);
    delay_us(2);
    return OK;
}
/* Hàm khởi tạo GLCD- Phải gọi hàm này thì GLCD mới hoạt động
result glcd_setup()
{
    GLCD_COMMAND(0x30);    DELAY_MS(1);
    GLCD_COMMAND(0x01);    DELAY_MS(10);
    GLCD_COMMAND(0x36);
    glcd_clear(0);
    glcd_update();
    GLCD.tt_update=0;
    GLCD.tt_text=0;
    return OK;
}
#endif

```

Bài mẫu 652. Chương trình hiển thị lên GLCD nội dung như hình dưới. Trong đó giá trị số đếm từ 00-99.



Lưu tên file là “BAI_652_GLCD_HIEN THI_COBAN”

- Mục đích: giúp SV làm quen với các hàm đồ họa cơ bản.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```

#include<tv_boards.c>
#include<tv_glcd.c>
unsigned int8 dem=0;
void main()
{
    system_init();
    // Cấu hình GLCD - Thiếu hàm này GLCD không chạy
    glcd_setup();
    // 1. Vẽ logo STKT (64x64) tại tọa độ X =0
    glcd_spkt_logo_raw( 0);
    // 1. Cập nhật logo SPKT ra màn hình
    glcd_update();
}

```

```
// 2. Vẽ hình tròn có tâm tại (70,7) , bán kính 6, tô kín, màu trắng
glcd_circle(70, 7, 6, 1, 1);
// 2. Cập nhật hình tròn ra màn hình
glcd_update();
// 3. Vẽ hình chữ nhật tại (81,1),(97,13), tô kín , màu trắng
glcd_rect(85,1, 97, 13, 1,1);
// 3. Cập nhật hình chữ nhật ra màn hình
glcd_update();
// 4. Vẽ hình tam giác tô kín, màu trắng
glcd_triangle(103,13,110,1,117,13,1,1);
// 4. Cập nhật hình tam giác ra màn hình
glcd_update();
// 5. Vẽ đoạn thẳng
glcd_line(64, 15, 118, 15, 1);
// 5. Cập nhật đoạn thẳng ra màn hình
glcd_update();
// 6. Vẽ thanh có độ dày là 3
glcd_bar(64, 53, 118, 53,3,1);
// 6. Cập nhật thanh ra màn hình
glcd_update();
// 7. Cấu hình chuỗi tại(80,57),font1 (5x7), màu trắng, không ghi đè
glcd_text_setup(80,56,1,1,0);
// 7. Hiển thị chuỗi SPKT
glcd_text("SPKT");
// 7. Cập nhật chuỗi SPKT ra màn hình
glcd_update();
while(true)
{
    // 8. Chuỗi tại(70,20),font 4x(5x7)=20x28, màu trắng, có ghi đè
    glcd_text_setup(70,20,4,1,1);
    // 8. Hiển thị giá trị của biến
    printf(glcd_text,"%02d",dem);
    glcd_update();
    dem++; dem%=100;
    delay_ms(200);
}
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả.
- f. Giải thích chương trình: Nếu hàm nào SV không rõ thì hãy đọc lại phần giải thích cách sử dụng các hàm đồ họa ở phần thư viện bên trên hoặc hỏi GV phụ trách.

Bài tập 653. Chương trình đếm sản phẩm từ 00 đến 24 dùng counter T0 và hiển thị trên GLCD theo font 3 (15x21)

Lưu tên file là “BAI_653_GLCD_DEM_SP_T0”

Bài tập 654. Hãy viết chương trình đếm sản phẩm dùng counter T0, hiển thị kết quả đếm trên 4 led 7 đoạn, LCD theo font 1x1, 2x3, 4x3 và GLCD.

Mục đích: biết lập trình đếm sản phẩm và giải mã hiển thị trên nhiều module.

Lưu tên file là “BAI_654_GLCD_LCD_4LED_DEM_SP_T0”

6.6.3 Các bài thực hành hiển thị hình ảnh trên GLCD

Phần này thực hành các bài hiển thị hình ảnh trên GLCD.

Để hiển thị hình ảnh số thì ta phải biết hình ảnh có được từ nhiều nguồn thiết bị khác nhau như máy chụp ảnh hay ảnh chụp trên màn hình máy tính hay do phần mềm vẽ tạo ra lưu trên máy tính với nhiều tên mở rộng khác nhau, ... và muốn hiển thị ảnh trên GLCD thì ta phải dùng phần mềm chuyển đổi ảnh cần hiển thị sang file nhị phân (viết gọn ở dạng số hex) – tạm gọi là file ảnh, sau đó ta viết một chương trình gọi lần lượt các mã nhị phân của file ảnh này ra GLCD.

GLCD của kit thực hành là loại đơn sắc chỉ có 2 màu xanh và trắng và ta có thể lập trình chọn 1 trong 2 chế độ màu này.

Để hiển thị 1 ảnh thì về cơ bản là ta vẽ lại từng điểm ảnh (pixel) trên màn hình GLCD và diện tích ảnh trên GLCD đúng với ảnh ta cần hiển thị.

Bài tập 661. Hiệu chỉnh bài 654 sao cho hiển thị logo đại học SPKT bên trái GLCD. Bên phải GLCD hiển thị kết quả đếm sản phẩm trong hình tròn.

Lưu tên file là “BAI_661_GLCD_CAITIEN_BAI654_GRAPHIC ”

Bài tập 662. Hiệu chỉnh bài 661 sao cho khi nhấn nút INV thì chuyển qua hoặc chuyển lại giữa hình thoi (giống 661) và hình tròn bao quanh giá trị đếm sản phẩm.

Lưu tên file là “BAI_662_GLCD_CAITIEN_BAI661_NUTNHAN ”

Bài mẫu 663. Viết chương trình hiển thị menu như hình dưới, nhấn nút UP (lên), DW(xuống) để di chuyển hình chữ nhật lên – xuống. Khi chọn COUNTER thì hiển thị kết quả đếm sản phẩm từ 0 đến 24 trong hình tròn bên phải. Khi chọn TIMER thì đếm giây chính xác dùng timer và hiển thị trong hình tròn bên phải. Khi chọn LOGO thì hiển thị logo SPKT ở bên phải.



Lưu tên file là “BAI_663_GLCD_CHINH_MENU”

- Mục đích: Biết cách tạo menu và chỉnh menu trên GLCD.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_glcd.c>
unsigned int8 mode=2 ,bdn=0,giay=0,sp;
```

```

void ve_giaodien()
{
    // Xóa toàn bộ màn hình
    glcd_clear(0) ;
    // Vẽ thanh chọn lựa
    glcd_rect(0,mode*20, 50, mode*20+20, 1,1);
    // Vẽ nội dung của menu
    glcd_text_setup(5,7,1,mode!=0,0);
    glcd_text("COUNTER");
    glcd_text_setup(5,27,1,mode!=1,0);
    glcd_text("TIMER");
    glcd_text_setup(5,47,1,mode!=2,0);
    glcd_text("LOGO");
    // Nếu mode bằng 2 thì hiển thị logo
    if(mode==2) glcd_spkt_logo_raw(64);
    // Nếu mode bằng 0 hoặc 1 thì hiển thị hình tròn
    else
    {
        // Vẽ đường tròn dày 3 pixel
        glcd_circle(95,31,20,1,1);
        glcd_circle(95,31,17,1,0);
    }
    // Hiển thị ra màn hình
    glcd_update();
}

void main()
{
    system_init();
    glcd_setup();
    ve_giaodien();
    setup_timer_0(T0_EXT_H_TO_L| T0_DIV_1);
    set_timer0(0);
    setup_timer_1(T1_INTERNAL| T1_DIV_BY_8);
    set_timer1(3036);
    enable_interrupts(INT_TIMER1);
    enable_interrupts(GLOBAL);
    while(true)
    {
        // Chú ý mỗi lần nhấn nút mới vẽ lại giao diện 1 lần
        // để tránh trì hoãn các tác vụ khác
        if((inputcd(DW)==0)&&(mode<2)) {mode++; ve_giaodien();}
        if((inputcd(UP)==0)&&(mode>0)) {mode--; ve_giaodien();}
        sp = get_timer0();
        if(sp>24) set_timer0(1);
        if(bdn>=10)
        {
            bdn=bdn-10; giay++;
            giay%=60;
        }
        glcd_text_setup(85,24,2,1,1);
        if(mode==0) printf(glcd_text,"%02u",sp);
        else if(mode==1) printf(glcd_text,"%02u",giay);
        glcd_update();
    }
}

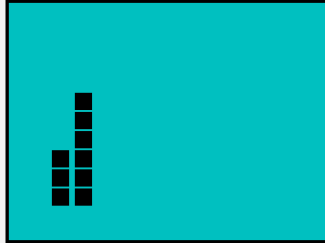
#INT_TIMER1
void ngatT1()

```

```
{
    set_timer1(3036);
    bdn++;
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả.
- f. Giải thích chương trình.

Bài tập 664. Hãy viết chương trình đếm sản phẩm từ 00-999 dùng counter T0, hiển thị kết quả đếm trên GLCD dạng hình ảnh theo cột. Ví dụ số sản phẩm là 36 thì hiển thị :



Lưu tên file là “BAI_664_GLCD_DEM_SP_T0_HT_COT”

Bài tập 665. Hãy viết chương trình đếm giây hiển thị trên màn hình giống đồng hồ kim. Vẽ vòng tròn và vẽ kim giây, sau 1 giây thì xóa kim giây hiện tại, thay đổi tọa độ cuối vẽ kim giây mới, sử dụng timer T1 định thời để đếm chính xác.

Lưu tên file là “BAI_665_GLCD_DEM_GIAY”

6.7 KẾT HỢP NHIỀU MODULE

Ở trên ta đã kết hợp được 3 module gồm GLCD, LCD và 4 led 7 đoạn để thực hiện bài đếm sản phẩm, phần này kết hợp thêm nhiều module.

Bài mẫu 671. Kết hợp nhiều module.

Module 32 led đơn thì chạy đèn quảng cáo 3 hiệu ứng.

Module 8 led quét thì đếm giờ phút giây.

Module LCD, GLCD, 4 led thì hiển thị kết quả đếm sản phẩm từ counter T0

Lưu tên file là “BAI_671_TONGHOP_5_MODULE”.

Bài tập 672. Giống bài 671 nhưng GLCD hiển thị logo spkt và đếm sản phẩm.

Lưu tên file là “BAI_672_TONGHOP_5_MODULE”.

Chương 7: THỰC HÀNH CHUYỂN ĐỔI TƯƠNG TỰ SANG SỐ - ADC, CÁC CẢM BIẾN

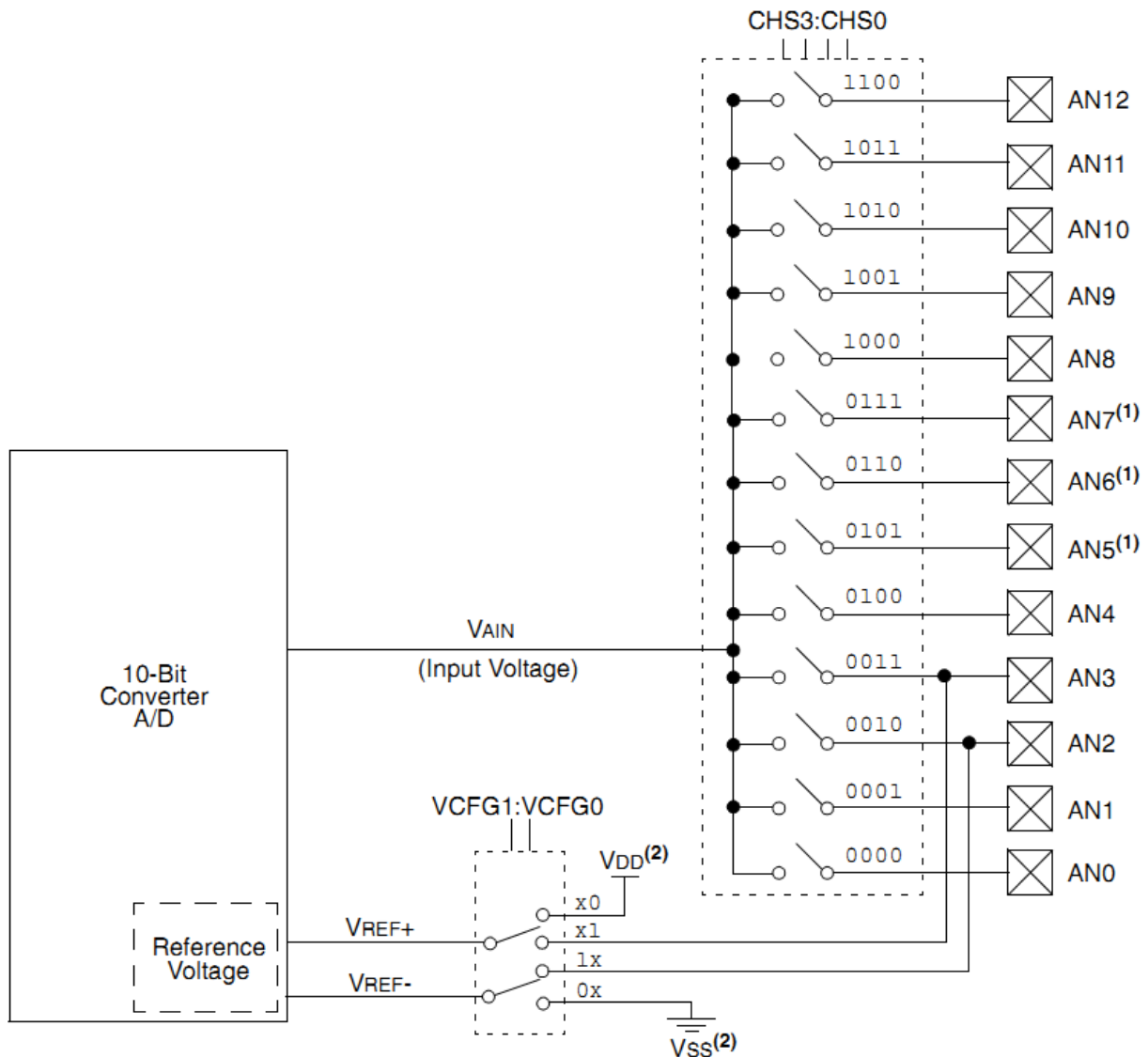
7.1 CHUYỂN ĐỔI TƯƠNG TỰ SANG SỐ CỦA PIC

Vi điều khiển PIC có bộ chuyển đổi tín hiệu tương tự sang tín hiệu số ADC 10 bit đa hợp nhiều kênh tùy thuộc vào từng loại vi điều khiển cụ thể, PIC16F877A có 8 kênh, PIC16F887 có 14 kênh và PIC18F4550 có 13 kênh. Mạch ADC dùng cho các ứng dụng giao tiếp với tín hiệu tương tự có thể nhận từ các cảm biến như cảm biến nhiệt độ LM35, cảm biến áp suất, cảm biến độ ẩm, cảm biến khoảng cách, ...

Phần này sẽ khảo sát chi tiết khối ADC của PIC, các thanh ghi của khối ADC, trình tự thực hiện chuyển đổi, tập lệnh lập trình C cho ADC và ứng dụng ADC để đo nhiệt độ.

7.1.1 Khảo sát ADC của PIC 18F4550

ADC của IC18F4550 có sơ đồ khối như hình 7-1:



Hình 7- 1. Sơ đồ khối của ADC PIC18F4550.

Chức năng các thành phần:

- AN0 đến AN12 (analog) là 13 ngõ vào của 13 kênh tương tự được đưa đến mạch đa hợp.
- CHS<3:0> là các ngõ vào chọn kênh của bộ đa hợp tương tự.
- Tín hiệu kênh tương tự đã chọn sẽ được đưa đến bộ chuyển đổi ADC.
- Điện áp tham chiếu dương Vref+ có thể lập trình nối với nguồn cung cấp dương V_{DD} hoặc điện áp tham chiếu bên ngoài nối với ngõ vào Vref+ của chân AN3. Điện áp tham chiếu âm Vref- có thể lập trình nối với nguồn cung cấp V_{SS} hoặc điện áp tham chiếu bên ngoài nối với ngõ vào Vref- của chân AN22 bit lựa chọn là VCFG1:VCFG0.
- Hai ngõ vào Vref+ và Vref- có chức năng thiết lập độ phân giải cho ADC.
- Bit ADON có chức năng cho phép ADC hoạt động hoặc tắt bộ ADC khi không hoạt động để giảm công suất tiêu tán, ADON bằng 1 thì cho phép, bằng 0 thì tắt.
- Kết quả chuyển đổi là số nhị phân 10 bit sẽ lưu vào cặp thanh ghi 16 bit có tên là ADRESH và ADRESL, 10 bit kết quả lưu vào thanh ghi 16 bit nên có dạng lưu là canh lề trái và canh lề phải tùy thuộc vào bit lựa chọn có tên ADFM.

ADC có 13 kênh nhưng mỗi thời điểm chỉ chuyển đổi 1 kênh và chuyển đổi kênh nào thì phụ thuộc vào 4 bit chọn kênh CHS4:CHS0. Hai ngõ vào điện áp tham chiếu dương và âm có thể lập trình nối với nguồn VDD và VSS hoặc nhận điện áp tham chiếu từ bên ngoài qua 2 chân RA3 và RA2.

Khối ADC độc lập với CPU nên có thể hoạt động khi CPU đang ở chế độ ngủ do xung cung cấp cho ADC lấy từ dao động RC bên trong của khối ADC.

7.1.2 Tập lệnh C cho khối ADC của PIC

Các lệnh của ngôn ngữ lập trình C liên quan đến ADC bao gồm:

Lệnh SETUP_ADC(MODE)

Lệnh SETUP_ADC_PORT(VALUE)

Lệnh SET_ADC_CHANNEL(CHAN)

Lệnh VALUE=READ_ADC(MODE)

a. LỆNH SETUP_ADC(MODE) - LỆNH ĐỊNH CẤU HÌNH CHO ADC

Cú pháp: setup_adc (*mode*);

Thông số: *mode*- Analog to digital mode. Xem trong file device.

Chức năng: định cấu hình cho ADC

b. LỆNH SETUP_ADC_PORT(VALUE) - LỆNH ĐỊNH CẤU HÌNH CHO PORT

Cú pháp: setup_adc_ports (*value*)

Hằng số: *value* - là hằng số định nghĩa trong file devices .h

Chức năng: Thiết lập chân của ADC là tương tự, số hoặc tổ hợp cả 2.

c. LỆNH SET_ADC_CHANNEL(chan) - LỆNH CHỌN KÊNH ADC

Cú pháp: Set_adc_channel (**chan**) - chọn kênh cần chuyển đổi khi đo nhiều hơn 1 kênh, nếu chỉ đo 1 kênh thì không cần.

Thông số: **Chan** là thứ tự kênh cần chuyển đổi. Kênh bắt đầu từ số 0.

Chức Chọn kênh cần chuyển đổi ADC

năng: Phải chờ một khoảng thời gian ngắn khi chuyển kênh, thường thì thời gian chờ khoảng $10\mu s$.

d. $LỆNH\ value = READ_ADC(mode) - LỆNH\ ĐỌC\ KẾT\ QUẢ\ CHUYỂN\ ĐỔI\ ADC$

Cú pháp: `value = read_adc ([mode])` - đọc kết quả sau khi chuyển đổi xong

Hằng số: mode là 1 trong các hằng số sau:

- `ADC_START_AND_READ` (thực hiện đọc liên tục, mặc nhiên là mode này)
- `ADC_START_ONLY` (bắt đầu chuyển đổi)
- `ADC_READ_ONLY` (đọc kết quả của lần chuyển đổi sau cùng)

Trả về: Kết quả 8 bit hay 16 bit tùy thuộc và khai báo #DEVICE ADC= directive.

Chức Lệnh này sẽ đọc giá trị số sau khi chuyển đổi xong

năng: Khai báo #DEVICE ADC= directive như sau:

#DEVICE	8 bit	10 bit	11 bit	16 bit
---------	-------	--------	--------	--------

7.2 ĐO NHIỆT ĐỘ DÙNG CẢM BIẾN LM35

7.2.1 Cảm biến LM35

Cảm biến LM35 hoạt động với điện áp từ +35V đến 4V.

Điện áp ra thay đổi 10mV cho 1 độ C.

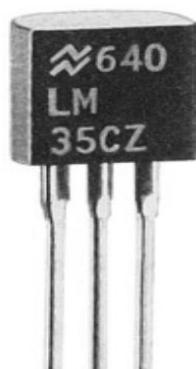
Điện áp ngõ ra từ $+6V$ đến $-1V$.

Dòng điện ngõ ra 10mA.

LM35CZ có tầm đo từ -40°C đến $+110^{\circ}\text{C}$, LM35DZ có tầm đo từ 0°C đến $+100^{\circ}\text{C}$.

Hình dạng bên ngoài và tên các tín hiệu

- Hình dạng bên ngoài và tên các tín hiệu như hình

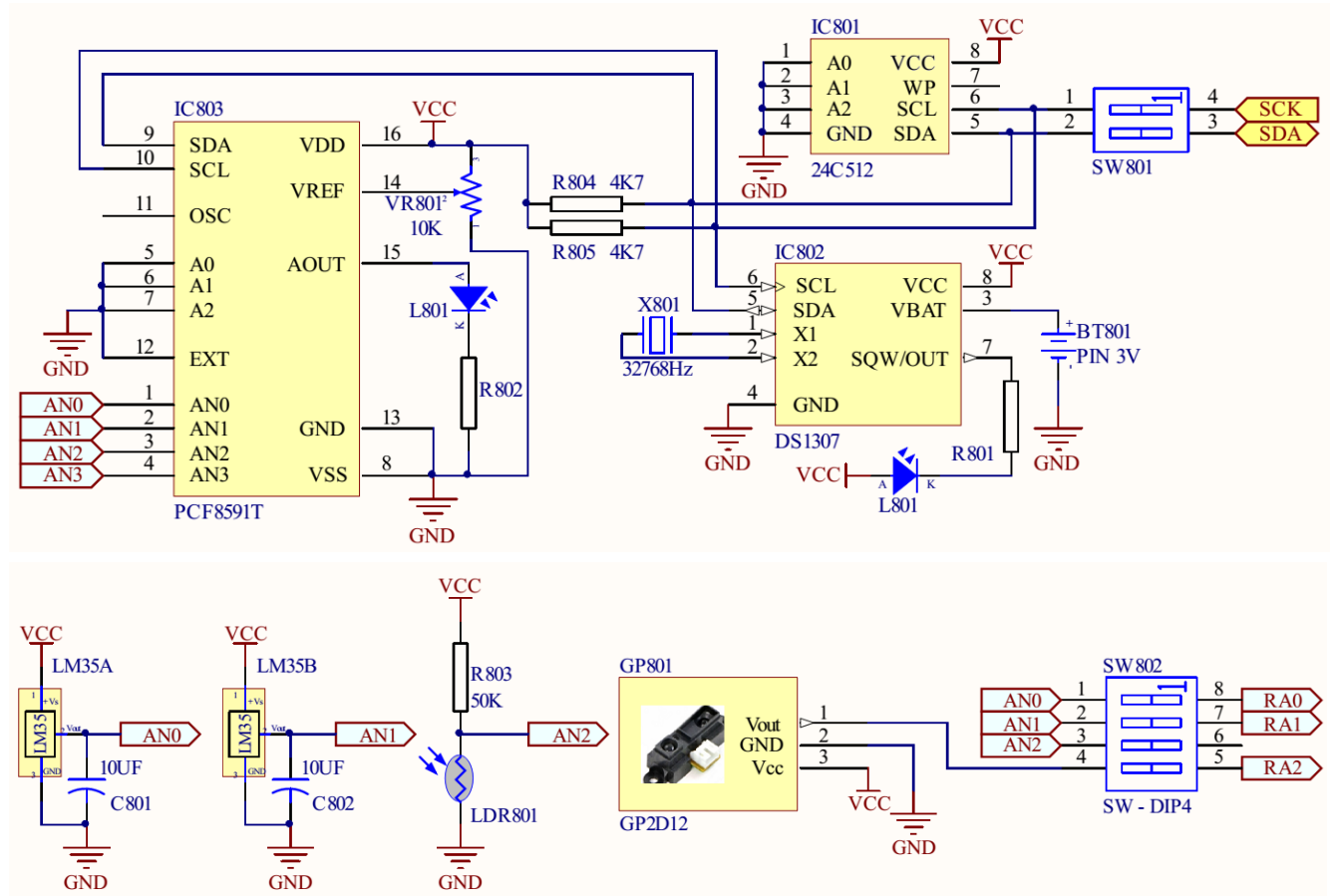


Hình 7-2. Hình ảnh cảm biến LM35CZ.

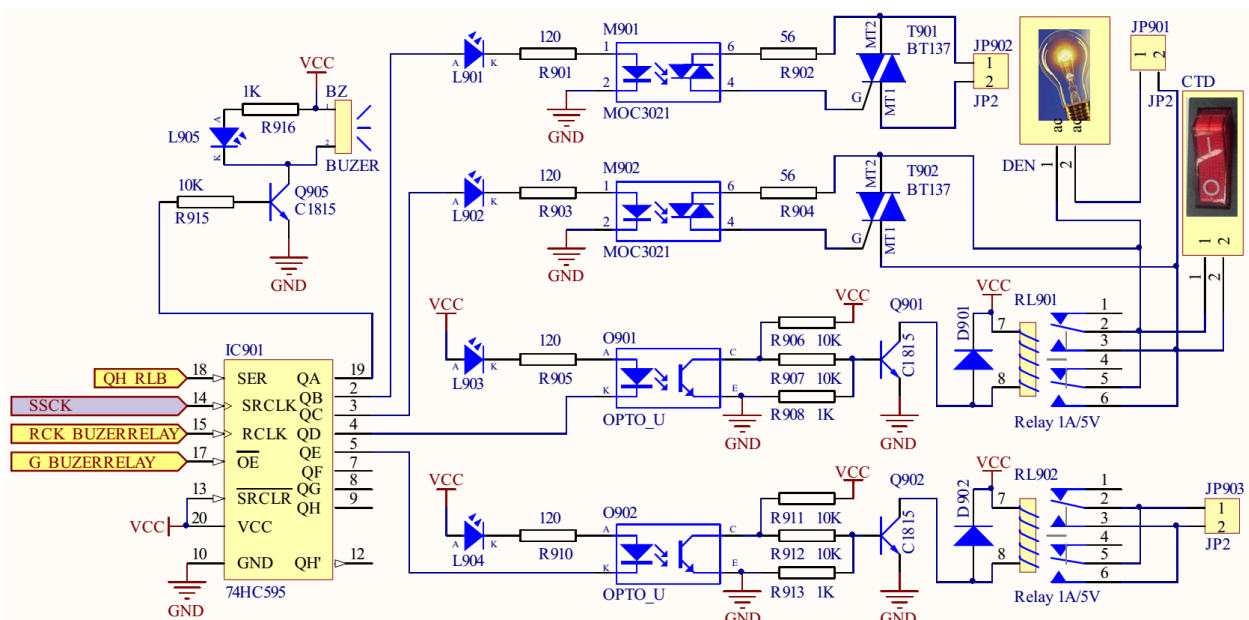
7.2.2 Mạch điện giao tiếp vi điều khiển PIC với cảm biến LM35

Phần này thực hành các bài chuyển đổi ADC với các tín hiệu vào là các cảm biến.

Mạch điện giao tiếp vi điều khiển với 2 cảm biến LM35A, LM35B, 1 cảm biến quang và 1 cảm biến khoảng cách có sơ đồ mạch như hình 7-3.



Hình 7-3. Sơ đồ nguyên lý module ADC và các cảm biến tương tự.



Hình 7-4. Sơ đồ nguyên lý module điều khiển Relay, Triac, tải 220V AC.

Các cảm biến LM35A và LM35B nối với 2 ngõ vào AN0 và AN1 của vi mạch ADC bên ngoài là PCF8591, đồng thời nối với 2 ngõ vào tương tự RA0/AN0, RA1/AN1 thông qua Switch SW802.

Cảm biến quang nối với ngõ vào AN2 của vi mạch ADC bên ngoài là PCF8591.

Cảm biến khoảng cách nối với ngõ vào tương tự AN2/RA2 thông qua Switch SW802.

Các cảm biến LM35A và LM35B chuyển đổi nhiệt độ thành tín hiệu điện. Trong bộ thí nghiệm thì các cảm biến nhiệt được đặt bên trong để gần bóng đèn tròn để có thể điều khiển thay đổi nhiệt độ.

Khi đèn sáng thì làm gia tăng nhiệt độ giúp bạn quan sát kết quả đo, khi đèn tắt thì nhiệt độ giảm dần cho đến khi bằng nhiệt độ môi trường.

Để mở đèn thì có 3 contact mắc song song như hình 7-4.

- Điều khiển đèn bằng tay: ta dùng contact thứ 2 nằm trên contact nguồn của bộ thí nghiệm.
- Điều khiển tự động: có 2 contact còn lại là dùng Triac hoặc dùng relay. Ở chế độ tự động thì ta phải chuyển contact bằng tay về chế độ OFF, nếu bạn để ON thì chế độ tự động tắt mở đèn trở nên vô nghĩa.

7.2.3 Đo nhiệt dùng cảm biến LM35 hiển thị trên 4 led 7 đoạn

Phần này thực hành ADC và cảm biến nhiệt LM35.

ADC được dùng để chuyển đổi điện áp tương tự từ cảm biến LM35 nên ta phải biết hệ số chuyển đổi của cảm biến.

Cảm biến LM35 có nhiều loại: LM35C và LM35D: loại LM35C thì tầm đo từ -40°C đến 110 °C và LM35D thì tầm đo từ 0°C đến 100 °C. Dòng tiêu thụ của cảm biến với nguồn cung cấp 5VDC là 50μA.

Hệ số chuyển đổi tuyến tính khi nhiệt độ thay đổi 1°C thì điện áp ra thay đổi 10mV.

- Tính toán độ phân giải của ADC:

Công thức tính độ phân giải của ADC (step size) là $SS_{ADC} = \frac{V_{REF+} - V_{REF-}}{2^{10} - 1} = \frac{V_{REF+} - V_{REF-}}{1023}$

Thông số 2^{10} là do ADC 10 bit.

Độ phân giải của ADC phải bằng độ phân giải của cảm biến và bằng 10mV:

$$SS_{LM35} = 10mV.$$

Nên ta có: $SS_{ADC} = \frac{V_{REF+} - V_{REF-}}{1023} = SS_{LM35} = 10mV$

Bây giờ ta tính điện áp tham chiếu cho ADC:

$$V_{REF+} - V_{REF-} = SS_{LM35} \times 1023 = 10mV \times 1023 = 10230mV$$

Một phương trình có 2 tham số nên ta phải chọn 1 tham số và tính tham số còn lại:

Chọn điện áp tham chiếu là $V_{REF-} = 0V$

Khi đó $V_{REF+} = 10,023V \cong 10V$

Điện áp này lớn hơn lớn hơn điện áp cho phép của ADC là 5V nên không phù hợp, trong khi đó ta có nguồn điện áp $V_{DD} = 5V$ nên ta chọn $V_{REF+} = V_{DD} = 5V$

Khi đó độ phân giải ADC sẽ là: $SS = \frac{V_{REF+} - V_{REF-}}{2^{10} - 1} = \frac{5000mV}{1023} = 4,887mV \cong 5mV$

Độ phân giải ADC là 5mV không tương thích với độ phân giải của cảm biến LM35 bằng 10mV, tỉ lệ chênh lệch là:

$$\frac{10mV}{5mV} = 2$$

Để có kết quả hiển thị đúng thì lấy kết quả chuyển đổi chia cho tỉ lệ chênh lệch.

Nếu tính chính xác hơn theo 4,887mV thì hệ số là 2,046.

Bài mẫu 701. Chương trình đo nhiệt độ LM35A hiển thị trên 4 led 7 đoạn.

Lưu tên file là “BAI_701_LM35A”.

- Mục đích: biết sử dụng cảm biến nhiệt LM35, lập trình ADC để đo và hiển thị nhiệt độ trên module 4 led 7 đoạn.
- Lưu đồ: SV tự vẽ
- Chương trình:

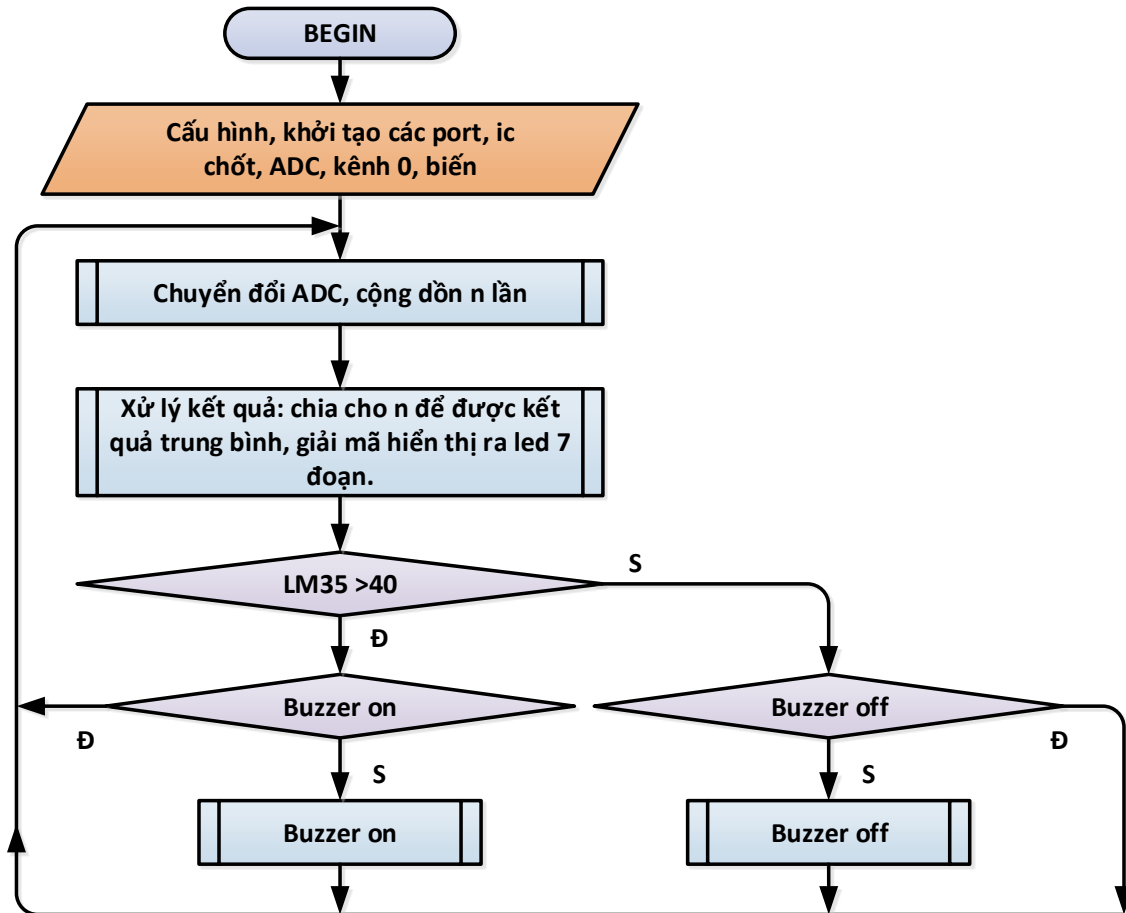
```
#include<tv_boards.c>
#include<tv_lcd.c>
unsigned int16 lm35a;
void main()
{
    system_init();
    setup_adc(adc_clock_div_32);
    setup_adc_ports(analog_pins|vss_vdd);
    // Chọn kênh tương tự nối với cảm biến LM35A
    set_adc_channel(lm35a_channel); delay_us(20);
    while(true)
    {
        // Đọc kết quả chuyển đổi và quy đổi ra nhiệt độ
        lm35a = read_adc() / 2.046;
        d7seg.led[1]=m7d[lm35a/10%10];
        d7seg.led[0]=m7d[lm35a%10];
        d7seg_display();
    }
}
```

- Quan sát kết quả: Mở contact cho đèn sáng để gia tăng nhiệt độ cho thấy sự thay đổi của nhiệt độ, sau đó tắt đèn để xem nhiệt độ giảm. Nhận thấy là kết quả nhiệt độ nhảy liên tục khó quan sát
- Giải thích chương trình:
- Hãy trả lời câu hỏi: Tại sao giá trị nhiệt độ khó quan sát?

Bài mẫu 702. Chương trình đo nhiệt độ LM35A hiển thị trên 4 led 7 đoạn. Đo trung bình 100 lần, khi nhiệt độ lớn hơn 40 độ thì báo động bằng buzzer, khi nhỏ hơn thì tắt báo động.

Lưu tên file là “BAI_702_LM35A_TRUNGBINH_BUZZ”.

- Mục đích: Biết cách đo trung bình và so sánh nhiệt độ để báo động, biết tắt mở buzzer.
- Lưu đồ:



Hình 7-5. Lưu đồ chuyển đổi ADC kênh thứ 0 để đọc nhiệt độ và so sánh điều khiển buzzer.

- Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
#define solan 100
unsigned int16 tong=0;
unsigned int8 n=0,lm35a;
void main()
{
    system_init();
    setup_adc(adc_clock_div_32);
    setup_adc_ports(analog_pins|vss_vdd);
    set_adc_channel(lm35a_channel); delay_us(20);
    while(true)
  
```

```

{
    tong += read_adc();
    n++;
    if(n == solan)
    {
        lm35a= tong/2.046/solan;
        tong=0; n= 0;
    }
    if(lm35a>40) buzzer_on(); // nd > 40 thì bật buzzer
    else buzzer_off(); // ngược lại thì tắt buzzer
    d7seg.led[1]=m7d[lm35a/10%10];
    d7seg.led[0]=m7d[lm35a%10];
    d7seg_display();
}
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: Ta thấy kết quả ít dao động hơn so với bài 701 do ta đo trung bình
- f. Giải thích chương trình:
- g. Hãy giải thích hàm làm buzzer on và off. Hãy vẽ mạch điều khiển buzzer và giải thích mạch.

Bài tập 703. Chương trình đo nhiệt độ LM35A hiển thị trên 4 led 7 đoạn, đo trung bình 100 lần. Tự động điều khiển mở bóng đèn sáng để tăng nhiệt cho đến khi lớn hơn 40 độ thì tắt đèn và báo động, khi nhiệt độ giảm dưới 40 độ thì tắt báo động, nhiệt độ nhỏ hơn 35 độ thì mở lại bóng đèn.

Chú ý:

- SV cần tắt công tắc đèn – bên trên công tắc nguồn
- Dùng hàm “triac_2_on();” để mở đèn và “triac_2_off();” để tắt đèn

Lưu tên file là “BAI_703_LM35A_BUZZ_AUTO”.

Bài tập 704. Làm lại bài 703 vừa hiển thị trên 2 led 7 đoạn vừa hiển thị trên LCD font2x3.

Lưu tên file là “BAI_704_LM35A_BUZZ_AUTO_FONT2x3”.

Bài mẫu 705. Chương trình đo nhiệt độ LM35A và LM35B hiển thị trên 4 led 7 đoạn. Đo trung bình 100 lần, mỗi kênh hiển thị trên 2 led 7 đoạn. Giữa 2 giá trị nhiệt độ cách nhau bởi dấu chấm.

Lưu tên file là “BAI_705_ADC_2LM35 ”.

- a. Mục đích: biết đo nhiệt độ nhiều kênh.
- b. Lưu đồ: sinh viên hãy tự viết.
- c. Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
#define solan 100
unsigned int16 tonga=0,tongb=0;
unsigned int8 n=0,lm35a,lm35b;
void main()
{
    system_init();
    setup_adc(adc_clock_div_32);
    setup_adc_ports(analog_pins|vss_vdd);
    while(true)
    {
        // Đo LM35A
        set_adc_channel(lm35a_channel); delay_us(20);
        tonga += read_adc();
        // Đo LM35B
        set_adc_channel(lm35b_channel); delay_us(20);
        tongb += read_adc();
        n++;
        // Đủ số lần rồi thì tính toán ra nhiệt độ
        if(n==solan)
        {
            lm35a= tonga/2.046/solan;
            lm35b= tongb/2.046/solan;
            tonga=0; tongb=0; n=0;
        }
        d7seg.led[1]=m7d[lm35a/10%10];
        d7seg.led[0]=m7d[lm35a%10];
        d7seg.led[3]=m7d[lm35b/10%10];
        d7seg.led[2]=m7d[lm35b%10]-128; // -128 để xuất hiện dấu chấm
        d7seg_display();
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- f. Giải thích chương trình:

Bài tập 706. Thêm vào yêu cầu bài 705 phần tự động điều khiển mở bóng đèn sáng để tăng nhiệt cho đến khi nhiệt độ của 1 trong 2 kênh lớn hơn 40 độ thì tắt đèn và báo động, khi nhiệt độ cả 2 kênh giảm dưới 40 độ thì tắt báo động, nhiệt độ cả 2 kênh nhỏ hơn 35 độ thì mở lại bóng đèn.

Lưu tên file là “BAI_706_2LM35_BUZZ”.

Bài tập 707. Làm lại bài 706 nhưng hiển thị thêm trên LCD fon2x3. Hàng 0 và 1 hiển thị LM35A, hàng 2 và 3 hiển thị LM35B. Có chú thích thông tin: ‘LM35A : XX’ và ‘LM35B : XX’. XX là nhiệt độ đo font chữ lớn, còn lại là kích thước nhỏ.

Lưu tên file là “BAI_707_2LM35_LCD”.

Bài tập 708. Hệ thống vừa đo nhiệt độ vừa đếm sản phẩm trên led 7 đoạn với yêu cầu như sau:

Đo nhiệt độ LM35A hiển thị trên 2 led 7 đoạn bên phải.

Đếm sản phẩm dùng counter T0 hiển thị trên 2 led 7 đoạn bên trái, giới hạn đếm từ 00 đến 99.

Lưu tên file là “BAI_708_LM35A_COUNTER_T0”.

Bài tập 709. Chương trình đo nhiệt độ LM35A hiển thị trên 2 led 7 đoạn và đo trung bình 100 lần.

2 led 7 đoạn còn lại hiển thị giá trị cài đặt trước (GTCĐT) để báo động, giá trị mặc nhiên thấp nhất là 35 độ, lớn nhất là 60 độ, có thể điều chỉnh bằng 2 nút nhấn UP và DW.

Khi giá trị đo lớn hơn GTCĐT thì tắt đèn và báo động, khi nhiệt độ giảm nhỏ hơn GTCĐT thì tắt báo động và khi nhỏ hơn GTCĐT-5 thì mở lại đèn gia nhiệt.

Lưu tên file là “BAI_709_LM35_UP_DW”.

Bài tập 710. Hệ thống đo nhiệt độ 2 kênh dùng LM35A và LM35B hiển thị trên LCD.

Hàng 0: “NHIET DO LM35A: XX”

Hàng 1 : “NHIET DO LM35B: XX”

Hàng 2 : “DEM SAN PHAM: XX”

Có tự động điều khiển đèn tắt khi nhiệt độ 2 kênh lớn hơn 40 và mở đèn khi cả hai nhỏ hơn 35.

Lưu tên file là “BAI_710_LM35_COUNTER_LCD”.

7.3 ĐO KHOẢNG CÁCH DÙNG CẢM BIẾN GP2D12

7.3.1 Cảm biến GP2D12

Phần này thực hành ADC và cảm biến khoảng cách GP2D12 của Sharp.

Tính chất của cảm biến GP2D12

- Tín hiệu ra là điện áp tương tự.
- Chu kỳ mỗi xung cho Led là 32ms.
- Thời gian đáp ứng trung bình là 39ms.
- Thời gian khởi động là 44ms
- Dòng tiêu thụ là 33mA.
- Phát hiện khoảng cách khu vực trong phạm vi 70cm là 6cm.

Hình dạng bên ngoài và tên các tín hiệu

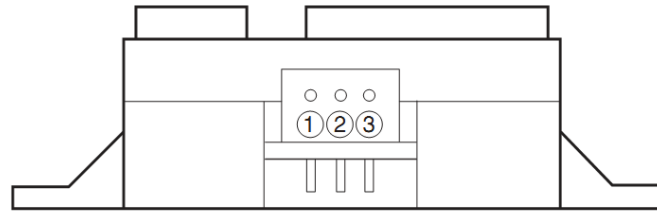
- Hình dạng bên ngoài và tên các tín hiệu như hình 7-7.

Điện áp hoạt động của cảm biến GP2D12:

- Điện áp nguồn cung cấp V_{CC} từ 4,5V đến 5,5V
- Điện áp tín hiệu ra V_O từ -0,3V đến ($V_{CC} + 0,3V$) phụ thuộc vào khoảng cách.
- Nhiệt độ làm việc từ -10 đến 60°C

Dạng sóng hoạt động của cảm biến GP2D12:

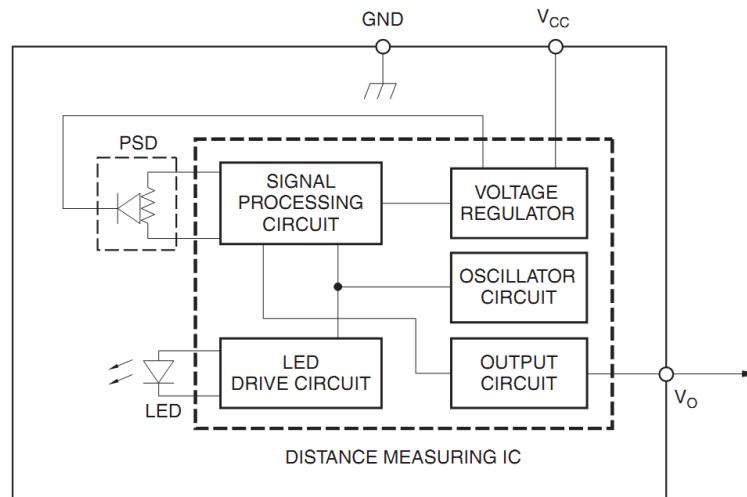
- Hình 7-9 trình bày dạng sóng của các lần đo của cảm biến:



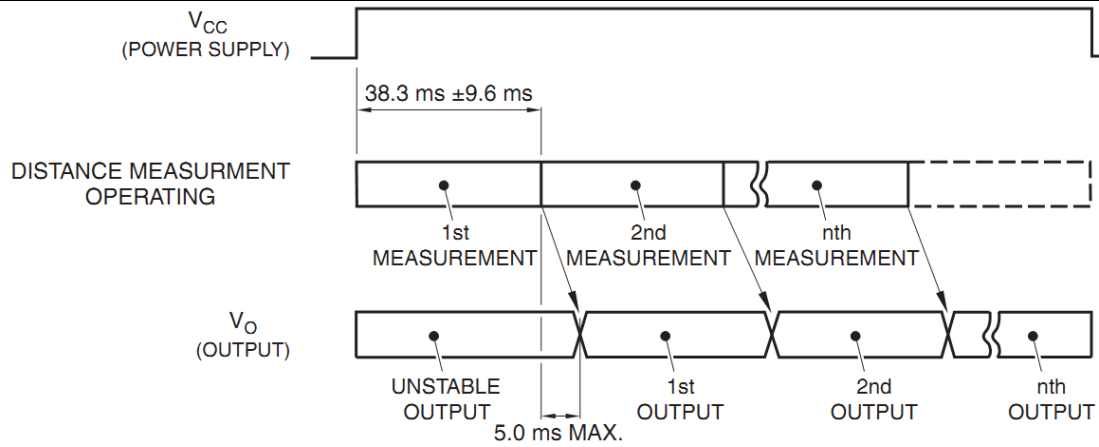
PIN	SIGNAL NAME
①	V_O
②	GND
③	V_{CC}

Hình 7-6. Hình cảm biến khoảng cách GP2D12.

Sơ đồ khối bên trong như hình 7-8:



Hình 7-7. Sơ đồ khối bên trong cảm biến khoảng cách GP2D12.



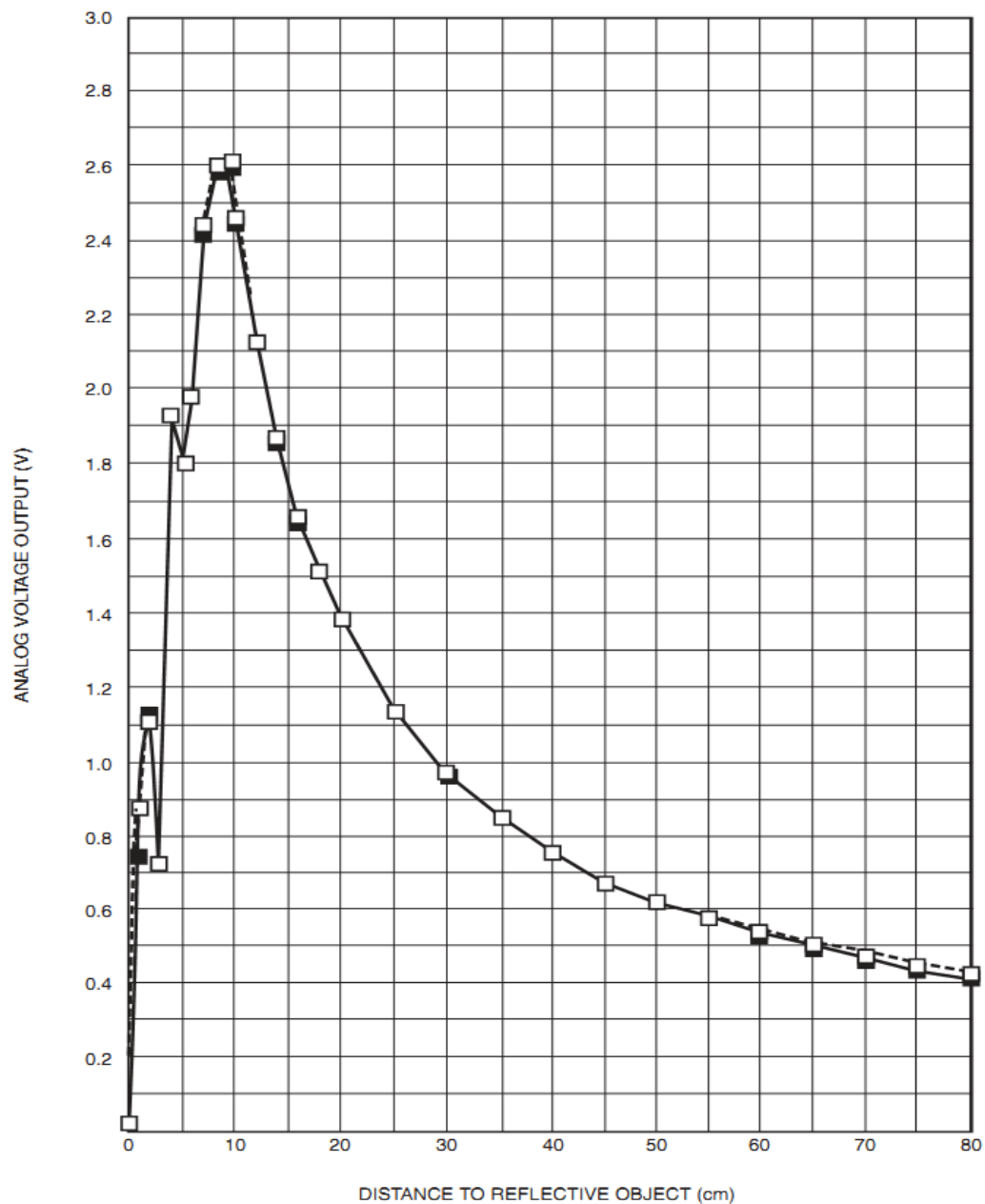
Hình 7-8. Dạng sóng của cảm biến khoảng cách GP2D12.

Quan sát dạng sóng ta thấy mỗi chu kỳ chuyển đổi tối đa là 38,3ms.

Chu kỳ đầu tiên khi cấp điện thì ngõ ra chưa ổn định nên bỏ.

Đồ thị chuyển đổi của cảm biến GP2D12:

- Hình 7-10 trình bày đồ thị chuyển đổi của cảm biến:



Hình 7-9. Đồ thị chuyển đổi của cảm biến khoảng cách GP2D12.

Quan sát đồ thị ta thấy trục hoành là khoảng cách với đơn vị là cm, trục tung là điện áp ra của cảm biến.

Bảng 7-1. Giá trị tại các tọa độ chính.

Toạ độ	Khoảng cách (cm)	Điện áp ra (mV)	DL số	Chênh lệch số	Chia cho 9 cấp
1	80	400	70		
2	70	435	87	7	0.77
3	60	530	106	19	2.11

4	50	620	124	18	2
5	40	750	150	26	2.88
6	30	970	196	46	5.11
7	20	1370	276	70	8.88
8	10	2450	490	214	23.77

7.3.2 Đo khoảng cách hiển thị trên 4 led 7 đoạn

Bài mẫu 711. Chương trình đo khoảng cách dùng led thu phát GP2D12 hiển thị trên 2 led 7 đoạn.

Lưu tên file là “BAI_711_GP2D12_D7SEG”.

- Mục đích: biết lập trình đo khoảng cách từ cảm biến GP2D12.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
unsigned int16 distance;
void main()
{
    system_init();
    setup_adc(adc_clock_div_32);
    setup_adc_ports(analog_pins|vss_vdd);
    while(true)
    {
        // Chọn kênh tương tự nối với cảm biến GP2D12
        set_adc_channel(gp2d12_channel); delay_us(20);
        distance = read_adc();
        distance = pow(4277 / distance, 1.115); // Tính khoảng cách (Cm)
        if (distance > 80) distance = 80; // Không chế giới hạn 10-80 cm
        else if (distance < 10) distance = 10;
        d7seg.led[1] = m7d[distance/10%10];
        d7seg.led[0] = m7d[distance%10];
        d7seg_display();
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: Bạn để tay che cảm biến GP2D12 cách khoảng 12cm và dịch chuyển tay ra xa và đồng thời quan sát kết quả trên led đơn tương ứng với khoảng cách đo được.
- Giải thích chương trình:

Bài tập 712. Đo khoảng cách hiển thị trên 4 led 7 đoạn, LCD theo font 2x3, GLCD theo font 4.

Lưu tên file là “BAI_712_GP2D12_HT_NHIEUMODULE”.

Bài tập 713. Đo khoảng cách hiển thị LCD theo yêu cầu sau :

- Khi khoảng cách bé hơn 30.cm thì hiển thị font 1x1 chính giữa LCD.
- Khi khoảng cách từ 30 đến 50 cm thì hiển thị font 2x3 chính giữa LCD.
- Khi khoảng cách lớn hơn 50 cm thì hiển thị font 4x3 chính giữa LCD.

Lưu tên file là “BAI_713_GP2D12_LCD_DOI_FONT”.

Bài tập 714. Hãy viết chương trình đo hiển thị trên LCD:

Đo nhiệt độ dùng LM35A và LM35B hiển thị ở hàng thứ 1.

Đếm sản phẩm dùng counter T0 hiển thị ở hàng thứ 2.

Đo khoảng cách dùng cảm biến GP2D12 và hiển thị hàng thứ 3.

Lưu tên file là “BAI_714_LCD_2LM35_T0_GP2D12”

7.4 ĐO KHOẢNG CÁCH DÙNG CẢM BIẾN SIÊU ÂM HC-SR 04

7.4.1 Cảm biến HC-SR 04

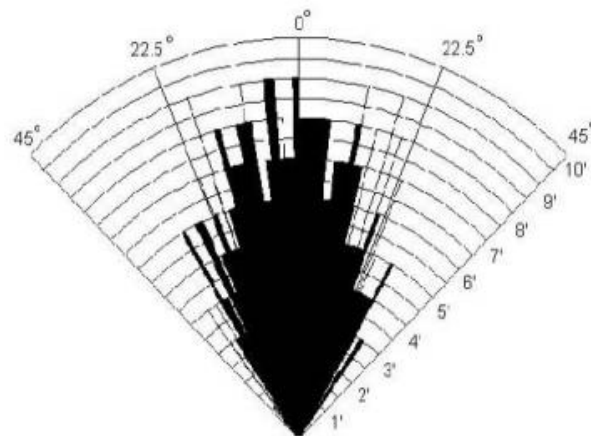
Phần này thực hành cảm biến siêu âm HC-SR 04.

Tính chất của cảm biến HC-SR 04

- Điện áp nguồn cung cấp 5V DC
- Dòng tính < 2mA
- Góc có hiệu lực < 15°
- Khoảng cách đo từ 2cm đến 500cm
- Độ phân giải 0.3 cm

Hình dạng bên ngoài và tên các tín hiệu

- Hình dạng bên ngoài và góc phát sóng như hình 7-11:

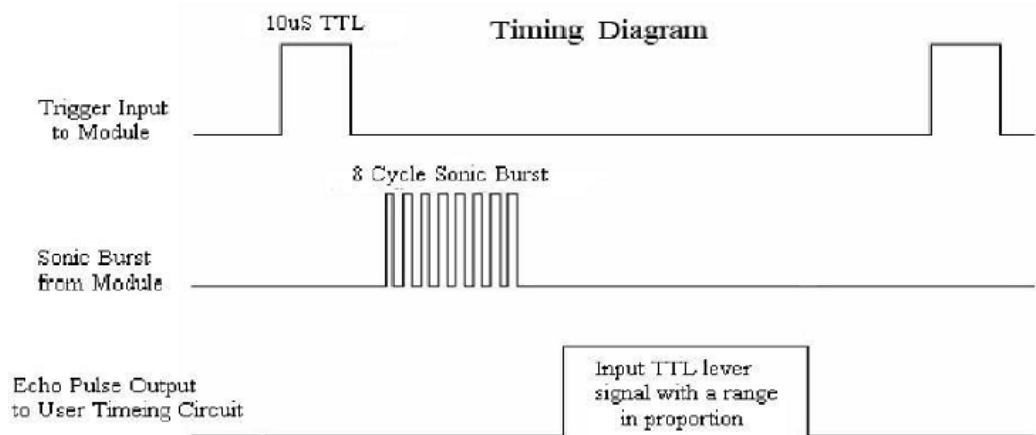


Hình 7-10. Hình cảm biến siêu âm và góc phát thu sóng.

Cảm biến HC-SR 04 có 4 chân: chân cấp nguồn Vcc, chân kích (trigger), chân nhận tín hiệu dội (echo) và chân GND.

Sau này có loại HC-SR 05 có thêm 1 tín hiệu out.

Giải đồ thời gian các tín hiệu:



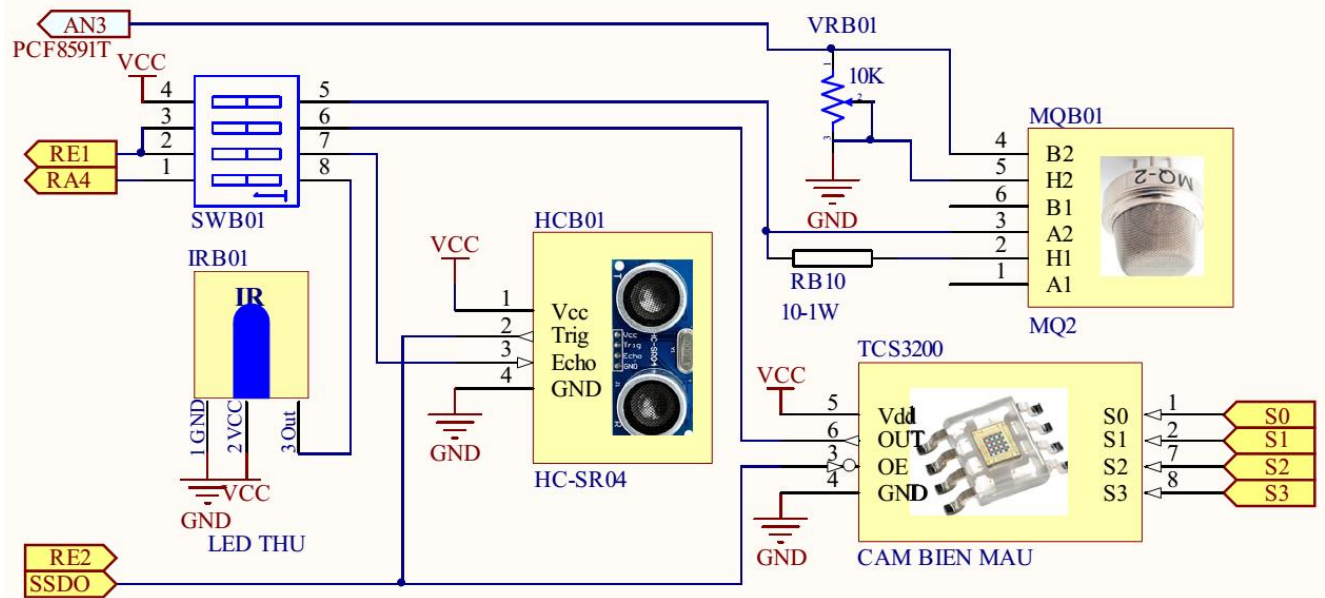
Hình 7-11. Giải đồ thời gian của cảm biến HC-SR04.

Để đo khoảng cách dùng cảm biến siêu âm thì hệ thống điều khiển phải kích tín hiệu chân trigger tạo ra 1 xung có mức logic 1 tối thiểu là $10\mu s$. Tín hiệu tương thích chuẩn TTL. Sau đó thì mạch cảm biến sẽ tạo ra một chuỗi 8 xung phát ra ở biên tử phát, sau khi gặp vật cản nằm trong giới hạn đo thì sóng sẽ phản xạ về và tín hiệu echo lên mức 1 với thời gian ở mức 1 tỷ lệ thuận với khoảng cách đo.

Hệ thống phải đo thời gian tín hiệu echo ở mức logic 1 theo đơn vị thời gian là μs , lấy thời gian đo được chia cho 58 sẽ được khoảng cách cần đo theo đơn vị cm, chia cho 148 sẽ được khoảng cách theo đơn vị inch – các hệ số này do nhà cung cấp cho chúng ta – dựa vào tốc độ sóng phát đi và phản xạ về.

7.4.2 Cảm biến HC-SR 04 trong kit thực hành

Trong kit thực hành có gắn 1 cảm biến HC-SR 04 có sơ đồ kết nối như hình 7-13:



Hình 7-12. Sơ đồ nguyên lý cảm biến khoảng cách siêu âm.

Cảm biến siêu âm đã cấp nguồn, chân tín hiệu Trigger nối trực tiếp đến chân RE2/SSDO và chân Echo nối đến SWB01 và nối với chân RE1 cùng chân với tín hiệu OUT của cảm biến màu.

Khi sử dụng thì switch của cảm biến màu chuyển sang OFF, switch của cảm biến siêu âm chuyển sang ON, thường thì switch đã chuyển sẵn, các bạn không cần chuyển.

Thư viện SRF04. Chương trình thư viện của cảm biến siêu âm đã cho trong thư mục của bạn có tên là “TV_SRF04.c” và “TV_SRF04.h”.

Bạn có thể sử dụng cho các bài thực hành và nên đọc hiểu.

File “TV_SRF04.h”

```
#ifndef __TV_SRF04_H__
#define __TV_SRF04_H__
typedef struct
{
    int8 step;
    unsigned int32 t;
} SRF04_VAR_STRUCT;
SRF04_VAR_STRUCT SRF04_VAR={0,0}; // Các biến sử dụng tạm trong quá trình đo
typedef struct
{
    unsigned int16 distance;
} SRF04_STRUCT; // Biến chứa kết quả đo khoảng cách
SRF04_STRUCT srf04;
result srf04_read(void);
#endif
```

File “TV_SRF04.c”

```
#ifndef __TV_SRF04_C__
#define __TV_SRF04_C__
#include<TV_SRF04.h>
result srf04_read()
{
```

```

unsigned int8 tam;
switch (SRF04_VAR.step)
{
    case 0: //Khởi tạo ngắt CCP2
        enable_interrupts(int_CCP2);
        enable_interrupts(global);
        SRF04_VAR.step=1;
        break;

    case 1: //Chân Triger mức cao
        output_high(SRF04_TRIGER);
        SRF04_VAR.t = delay_start();
        SRF04_VAR.step=2;
        break;

    case 2: //Đủ 1 ms
        if(delay_stop(SRF04_VAR.t)>=1)
        {
            output_low(SRF04_TRIGER); //Chân Triger mức thấp
            setup_ccp2(CCP_CAPTURE_RE); //CCP2 tích cực cạnh lên
            SRF04_VAR.t = delay_start();
            SRF04_VAR.step=3;
        }
        break;

    case 3: //Sau 100ms mà chưa thấy Echo lên 1 thì báo lỗi cảm biến
        if(delay_stop(SRF04_VAR.t)>=100)
        {
            SRF04_VAR.step=1;
            return ERROR;
        }
        break;

    case 5: //Tính khoảng cách khi đo xong
        srf04.distance=SRF04_VAR.t/36-1;
        SRF04_VAR.t = delay_start();
        SRF04_VAR.step=6;
        return OK;
        break;

    case 6:
        if(delay_stop(SRF04_VAR.t)>=100) // Delay 100ms rồi đo tiếp
            SRF04_VAR.step=1;
        break;
}
return BUSY;
}

#int_ccp2
void interrupt_ccp2()
{
    if(SRF04_VAR.step==3) //Có cạnh lên chân Echo thì bắt đầu định thời
    {
        setup_ccp2(CCP_CAPTURE_FE); //CCP2 tích cực cạnh xuống
        delay.timeline += get_timer3(); set_timer3(0);
        if(TIM3IF) delay.timeline +=65536; TIM3IF=0;
        SRF04_VAR.step=4;
    }
    else if(SRF04_VAR.step==4) //Có cạnh xuống chân Echo là đo xong
    {

```



```

        SRF04_VAR.t = get_timer3();
        SRF04_VAR.step=5;
    }
}
#endif

```

7.4.3 Đo khoảng cách dùng HCSR-04 hiển thị trên 4 led 7 đoạn

Bài mẫu 721. Chương trình đo khoảng cách dùng cảm biến khoảng cách HC-SR 04 hiển thị trên 4 led 7 đoạn.

Lưu tên file là “BAI_721_HCSR04_DO_KCSA”.

- Mục đích: biết đo khoảng cách dùng cảm biến siêu âm HCSR04 và xử lý kết quả đo hiển thị trên led.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```

#include<tv_boards.c>
#include<tv_srf04.c>
void main()
{
    system_init();
    while(true)
    {
        if(srf04_read()==OK)
        {
            d7seg.led[2]=m7d[srf04.distance/100%10];
            d7seg.led[1]=m7d[srf04.distance/10%10];
            d7seg.led[0]=m7d[srf04.distance%10];
            //Nếu led quét thì để ngoài lệnh if này vì các lệnh trong
            //đây chỉ thực thi 1 lần mỗi khi đo xong (độ trễ hơn 100ms)
            d7seg_display();
        }
    }
}

```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: Bạn để tay hoặc sách học che cảm biến siêu âm và dịch chuyển tay ra xa, đồng thời quan sát kết quả trên led đơn tương ứng với khoảng cách đo được, phạm vi đo khoảng 400 cm. Chú ý các nhóm đối diện nhau có thể ảnh hưởng sóng cho nhau.
- Giải thích chương trình

Bài tập 722. Hãy viết chương trình đo khoảng cách dùng cảm biến HCSR04 và hiển thị kết quả đo trên LCD theo font 2x3.

Lưu tên file là “BAI_722_LCD_HCSR04”.

Bài tập 722. Hãy viết chương trình đo khoảng cách dùng cảm biến HCSR04 hiển thị trên LCD.

Lần đo thứ 1 hiển thị ở hàng 0 cho đến khi nhấn BT0 thì giữ nguyên kết quả ở hàng 0.
 Lần đo thứ 2 hiển thị ở hàng 1 cho đến khi nhấn BT0 thì giữ nguyên kết quả ở hàng 1.
 Sau đó hàng 2 hiển thị tích của 2 lần đo ở hàng 0 và hàng 1.
 Nhấn BT1 thì xóa hết và tiến hành đo lại.
 Lưu tên file là “BAI_722_LCD_HCSR04_DO_DIEN TICH”.

7.5 CẢM BIẾN NHIỆT 1 DÂY

7.5.1 Cảm biến nhiệt DS18B20

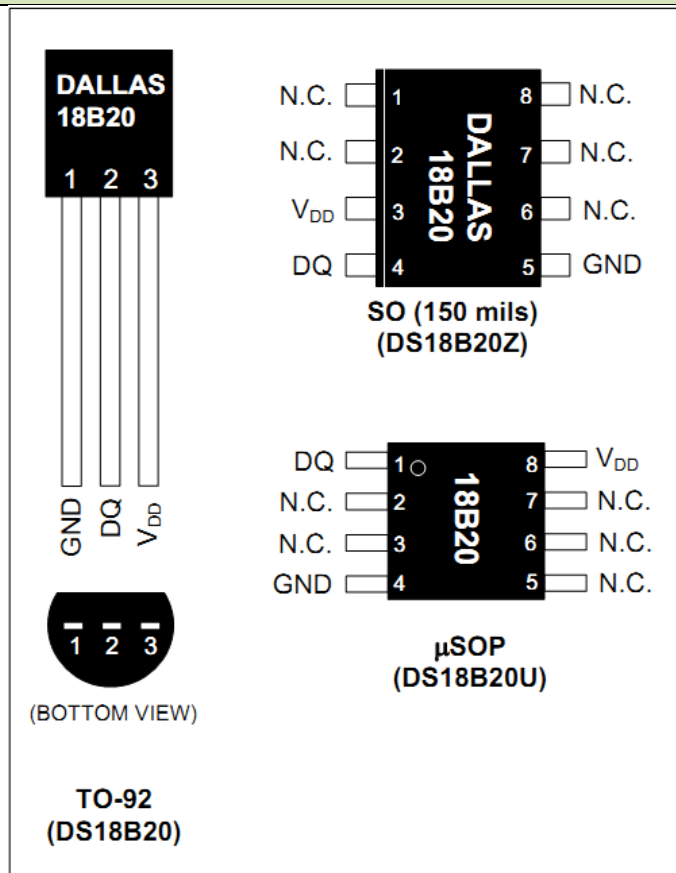
Ở phần trước chúng ta đã sử dụng cảm biến nhiệt là LM35 và bộ chuyển đổi ADC để chuyển đổi điện áp tương tự thành tín hiệu số, ở phần này chúng ta sẽ nghiên cứu và sử dụng cảm biến nhiệt có luôn bộ chuyển đổi ADC bên trong và chỉ có 1 tín hiệu giao tiếp vào ra còn gọi là 1 dây hay one wire.

Các thông tin của cảm biến như sau:

- Chỉ sử dụng một dây giao tiếp.
- Mỗi cảm biến đều có mã 64bit lưu trong bộ nhớ ROM.
- Là cảm biến đa năng thay cho hệ thống cảm biến nhiệt phức tạp.
- Phần cứng đơn giản vì chỉ dùng 1 dây không cần dùng ADC nên rất thích hợp cho vi điều khiển không tích hợp ADC.
- Có thể cấp nguồn từ đường tín hiệu, điện áp từ 3V-5.5V.
- Tầm đo từ -55°C đến +125°C (-67°F đến +257°F).
- Sai số 0.5°C cho tầm đo từ -10°C đến +85°C.
- Có thể chọn độ phân giải bằng phần mềm từ 9 đến 12 bit.
- Thời gian chuyển đổi lớn nhất cho 12bit là 750ms.
- Có thể cài đặt nhiệt độ cảnh báo lưu vào bộ nhớ ROM.
- Bộ cảnh báo luôn thăm dò để gửi lệnh đến thiết bị khi nhiệt độ vượt qua giới hạn cho phép.
- Thiết bị này có nhiều dạng chân: loại 3 chân, 8 chân, hay loại dán.
- Thiết bị được ứng dụng rộng rãi trong kiểm soát nhiệt độ: hệ thống công nghiệp, sản phẩm hàng tiêu dùng, nhiệt kế hay bất kỳ một hệ thống cảm biến nhiệt nào.

7.5.2 Hình ảnh và tên các chân của cảm biến

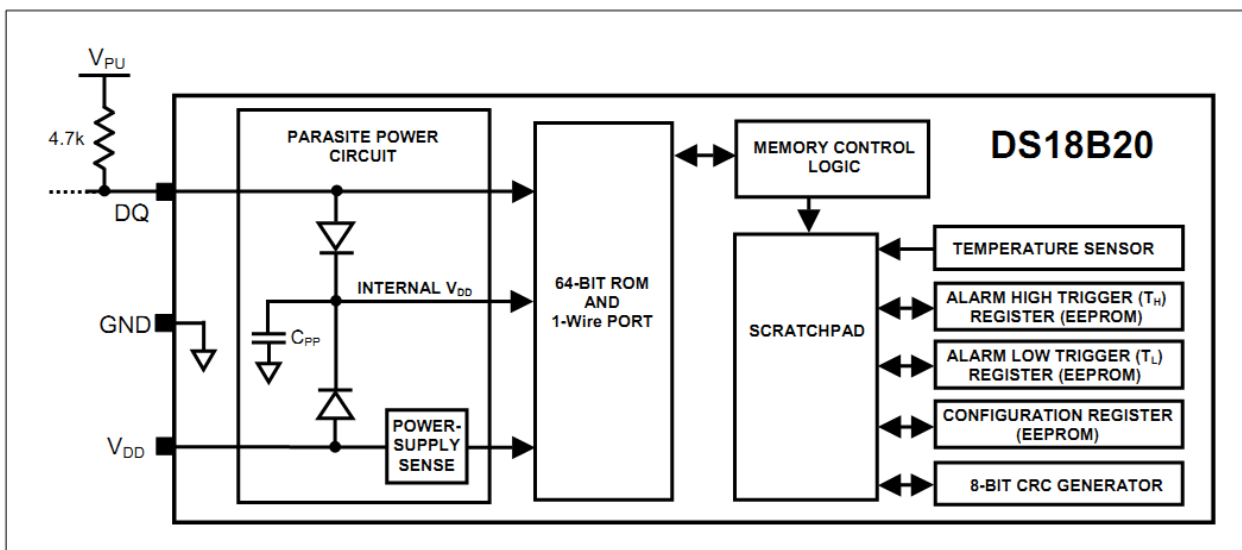
Các hình dạng khác nhau của cảm biến như hình 7-14.



Hình 7-13. Các dạng cảm biến DS18B20.

7.5.3 Sơ đồ khối của cảm biến DS18B20

Cảm biến DS18B20 có sơ đồ khối như hình 7-15.



Hình 7-14. Sơ đồ khối của cảm biến DS18B20.

Các khối bao gồm:

- Khối mạch nguồn cấp cho các khối bên trong hoạt động lấy từ nguồn bên ngoài.
- Khối bộ nhớ ROM 64 bit và port 1 dây.

- Khối điều khiển bộ nhớ.
- Khối scratchpad liên kết với các khối cảm biến nhiệt độ, khối lưu nhiệt độ byte cao T_H và byte thấp T_L để báo động quá nhiệt, khối thanh ghi định cấu hình và bộ kiểm tra mã dùng chuẩn CRC.

Bộ nhớ ROM lưu trữ mã 64 bit duy nhất của cảm biến.

Vùng nhớ scratchpad có 2 byte thanh ghi dùng để lưu trữ dữ liệu số là nhiệt độ từ cảm biến. Ngoài ra còn có 2 thanh ghi lưu nhiệt độ ngưỡng dưới và ngưỡng trên để so sánh với nhiệt độ đo và cảnh báo khi vượt quá giới hạn cài đặt, tên 2 thanh ghi là T_L và T_H .

Thanh ghi định cấu hình cho phép người dùng thiết lập độ phân giải chuyển đổi từ nhiệt độ sang dữ liệu số là 9, 10, 11 hoặc 12 bit. Giá trị thanh ghi định cấu hình và T_H , T_L còn được lưu vào bộ nhớ không bay hơi (EEPROM) nên dữ liệu vẫn còn khi mất điện.

7.5.4 Hoạt động của cảm biến DS18B20

Chức năng chính của DS18B20 là cảm biến nhiệt độ chuyển đổi trực tiếp sang dữ liệu số. Độ phân giải của cảm biến có thể định cấu hình là 9, 10, 11 hoặc 12 bit tương ứng với hệ số chuyển đổi theo thứ tự là 0,5°C, 0,25°C, 0,125°C và 0,0625°C. Độ phân giải mặc nhiên khi cấp điện là 12bit.

Khi cấp điện thì cảm biến DS18B20 ở trạng thái nghỉ để giảm công suất tiêu thụ. Để bắt đầu quá trình chuyển đổi tương tự sang số của nhiệt độ đo thì thiết bị chủ phải cấp phát lệnh chuyển đổi (Convert_T có mã là 44H). Sau khi chuyển đổi xong thì dữ liệu nhiệt độ lưu vào 2 thanh ghi nhiệt độ trong vùng nhớ scratchpad và cảm biến trở lại trạng thái nghỉ.

Nếu cảm biến cấp nguồn bên ngoài thì thiết bị chủ có thể phát các khe thời gian đọc sau khi cấp phát lệnh chuyển đổi nhiệt độ (Convert_T) để kiểm tra cảm biến chuyển đổi xong chưa: nếu cảm biến đáp ứng bằng cách phát bit có mức logic 0 thì quá trình chuyển đổi đang xảy ra, nếu phát bit có mức logic 1 thì quá trình chuyển đổi đã kết thúc.

Nếu cảm biến cấp nguồn thông qua chân tín hiệu thì thực hiện kiểm tra theo cách khác.

Dữ liệu nhiệt độ thuộc theo hệ số Celsius, nếu muốn theo hệ số Fahrenheit thì phải thực hiện chuyển đổi. Dữ liệu nhiệt độ lưu trữ ở dạng số bù 2 số có dấu 16 bit trong thanh ghi như hình 7-16.

	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LS BYTE	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	BIT 15	BIT 14	BIT 13	BIT 12	BIT 11	BIT 10	BIT 9	BIT 8
MS BYTE	S	S	S	S	S	2^6	2^5	2^4

S = SIGN

Hình 7-15. Cấu trúc 2 thanh ghi lưu nhiệt độ của cảm biến DS18B20.

Các bit dấu (S) có giá trị là 0 nếu là số dương và bằng 1 nếu là số âm.

Nếu cảm biến cấu hình ở độ phân giải 12 bit thì tất cả các bit trong thanh ghi đều lưu dữ liệu.

Nếu cảm biến cấu hình ở độ phân giải 11 bit thì bỏ bit thứ 0 (BIT 0).

Nếu cảm biến cấu hình ở độ phân giải 10 bit thì bỏ 2 bit thứ 1 và 0 (BIT 1, 0).

Nếu cảm biến cấu hình ở độ phân giải 9 bit thì bỏ 3 bit thứ 2, 1 và 0 (BIT 2, 1, 0).

7.5.5 Hoạt động cảnh báo quá nhiệt của cảm biến DS18B20

Sau khi cảm biến DS18B20 thực hiện xong chuyển đổi nhiệt độ thì giá trị nhiệt độ được so sánh với các giá trị nhiệt độ ngưỡng dưới và ngưỡng trên do người dùng thiết lập trong 2 thanh ghi T_H và T_L – có cấu trúc như hình 7-17.

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
S	2^6	2^5	2^5	2^5	2^2	2^1	2^0

Hình 7-16. Cấu trúc 2 thanh ghi lưu nhiệt độ báo động của cảm biến DS18B20.

Bit S là bit dấu để thiết lập giá trị dương và âm. Giá trị lưu trong 2 thanh ghi T_H và T_L không bị mất khi mất điện do có lưu trong bộ nhớ EEPROM. Có thể đọc giá trị của 2 thanh ghi T_H và T_L chính là byte thứ 2 và thứ 3 trong vùng nhớ scratchpad.

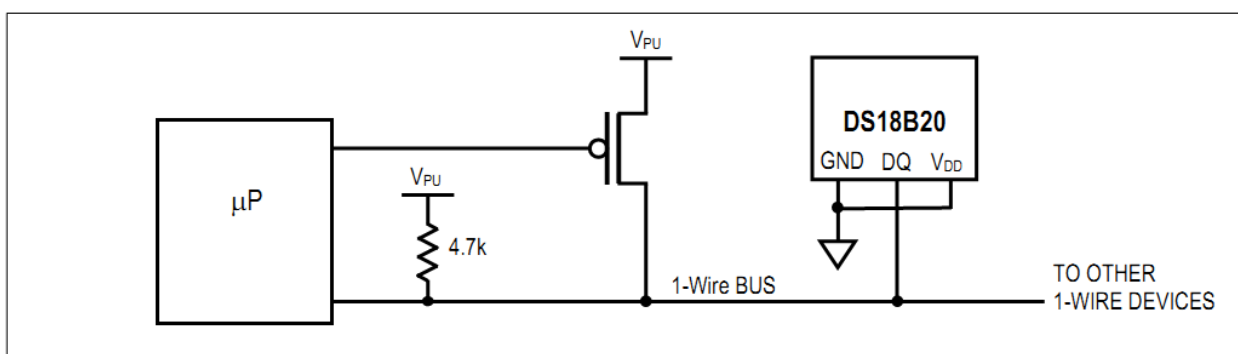
Chỉ có các bit từ bit thứ 11 đến bit thứ 4 (phần nguyên) của 2 thanh ghi nhiệt độ được dùng để so sánh với giá trị lưu trong 2 thanh ghi T_H và T_L .

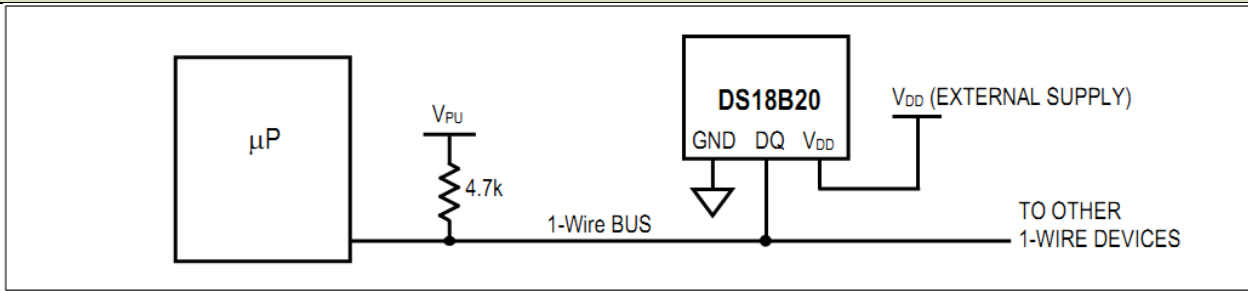
Nếu nhiệt độ đo thấp hơn hoặc bằng giá trị lưu trong thanh ghi T_L hoặc cao hơn hoặc bằng giá trị lưu trong thanh ghi T_H thì cờ báo động quá nhiệt trong cảm biến DS18B20 sẽ được set lên 1. Cờ báo động được cập nhật ở mỗi lần đo nên nếu có báo động ở lần đo này thì cờ sẽ bị xóa sau khi thực hiện lần chuyển đổi tiếp theo.

Thiết bị chủ có thể kiểm tra trạng thái cờ của cảm biến DS18B20 nối với bus bằng cách phát lệnh tìm báo động (Alarm search có mã là ECH). Bất kỳ cảm biến nào có cờ báo động tích cực sẽ đáp ứng lệnh báo động nên thiết bị chủ có thể xác định chính xác cảm biến DS18B20 nào đã xảy ra báo động.

7.5.6 Cấp nguồn cho cảm biến DS18B20

Cảm biến DS18B20 có 2 kiểu cấp nguồn như hình 7-18.





Hình 7-17. Sơ đồ cấp nguồn cho cảm biến DS18B20.

Hình trên thì chân cấp nguồn là chân tín hiệu dùng thêm 1 transistor FET, hình bên dưới sử dụng nguồn cấp đúng chân nguồn V_{DD} .

7.5.7 Mã 64 bit của cảm biến DS18B20

Mỗi cảm biến DS18B20 có mã duy nhất là 64 bit như hình 7-19.

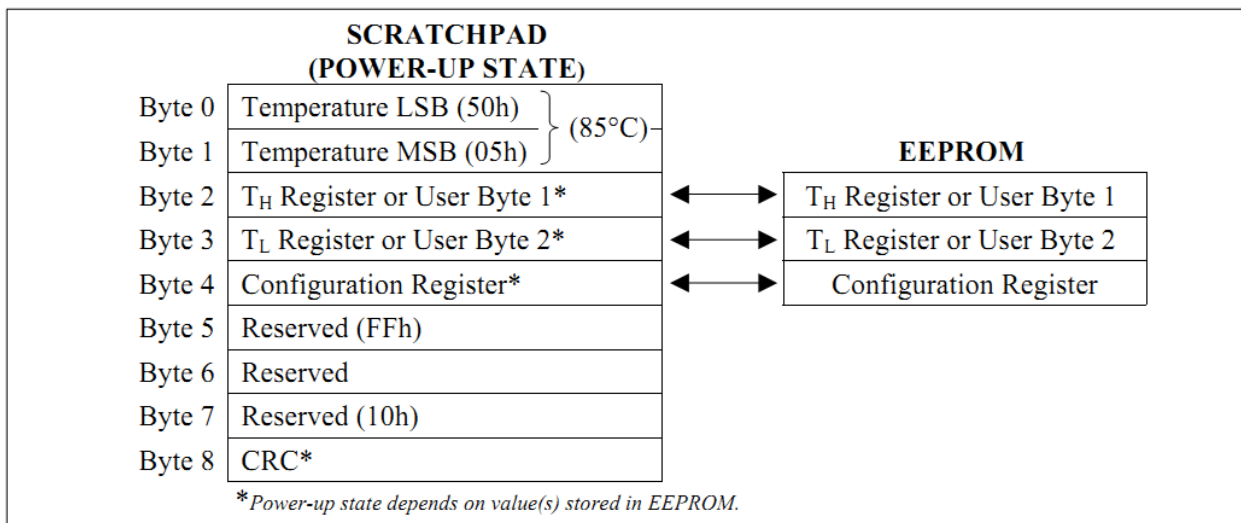
8-BIT CRC		48-BIT SERIAL NUMBER				8-BIT FAMILY CODE (28h)	
MSB	LSB	MSB		LSB	MSB		LSB

Hình 7-18. Cấu trúc mã 64 bit của cảm biến DS18B20.

Trong cấu trúc 64 bit thì 8 bit thấp luôn bằng 28H. 48 bit tiếp theo là mã duy nhất. 8 bit cuối cùng là kiểm tra mã thừa tuần hoàn (CRC cyclic redundancy check) được tính toán từ 56 bit. Bộ mã 64 bit lưu trong ROM và khối điều khiển ROM cho phép cảm biến DS18B20 hoạt động như thiết bị 1 dây.

7.5.8 Bộ nhớ rom của cảm biến DS18B20

Bộ nhớ ROM của cảm biến DS18B20 có tổ chức như hình 7-20.



Hình 7-19. Tổ chức bộ nhớ ROM của cảm biến DS18B20.

Bộ nhớ của cảm biến DS18b20 bao gồm bộ nhớ SRAM scratchpad và bộ nhớ EEPROM có thể xem là các thanh ghi dùng để lưu trữ giá trị nhiệt độ để so sánh báo động quá nhiệt – hai thanh ghi có tên là T_H và T_L , một thanh ghi định cấu hình. Nếu chức năng lưu nhiệt độ báo động không được sử dụng thì 2 thanh ghi này có thể dùng để lưu dữ liệu như các ô nhớ thông dụng.

Byte thứ 0 và thứ 1 dùng để lưu nhiệt độ chuyển đổi và theo thứ tự là byte thấp và byte cao, hai byte này chỉ cho phép đọc.

Byte thứ 2 và thứ 3 dùng để lưu nhiệt độ báo động và theo thứ tự là byte thấp và byte cao. Byte thứ 4 là thanh ghi định cấu hình.

Byte thứ 5, 6, 7 phục vụ cho hoạt động bên trong của cảm biến.

Byte thứ 8 là byte kiểm tra cho tất cả các byte từ thứ 0 đến 7 của vùng nhớ này.

Dữ liệu ghi vào byte 2, 3 và byte 4 của vùng nhớ bằng lệnh Write Scratchpad (4EH), dữ liệu gửi đến DS18B20 theo trình tự bắt đầu với bit có trọng số nhỏ nhất LSB của byte thứ 2 rồi đến byte 3 và byte 4.

Để kiểm tra dữ liệu lưu vào có đúng không thì tiến hành đọc lại bằng lệnh Read Scratchpad (BEH). Khi đọc vùng nhớ Scratchpad thì bit dữ liệu sẽ bắt đầu với bit có trọng số nhỏ nhất của byte thứ 0.

Để truyền giá trị cho thanh ghi T_H , T_L và thanh ghi định cấu hình từ vùng nhớ Scratchpad vào bộ nhớ EEPROM thì phải dùng lệnh Copy Scratchpad (48H).

Dữ liệu trong thanh ghi của vùng nhớ EEPROM không bị mất khi mất điện, khi có điện thì dữ liệu trong bộ nhớ EEPROM sẽ nạp vào vùng nhớ Scratchpad đúng với các giá trị và thanh ghi đã thiết lập.

7.5.9 Thanh ghi định cấu hình của cảm biến DS18B20

Byte thứ 4 của vùng nhớ Scratchpad của cảm biến DS18B20 là thanh ghi định cấu hình có cấu trúc như hình 7-21. Thanh ghi này cho phép thiết lập độ phân giải chuyển đổi của cảm biến bằng 2 bit có tên là R1 và R0. Hai bit sẽ cho phép thiết lập 4 hệ số chuyển đổi 9 bit đến 12 bit và thời gian chuyển đổi tương ứng như bảng 7-2.

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
0	R1	R0	1	1	1	1	1

Hình 7-20. Byte thanh ghi điều khiển của cảm biến DS18B20.

Bảng 7-2. Độ phân giải và thời gian chuyển đổi.

R1	R0	RESOLUTION (BITS)	MAX CONVERSION TIME	
0	0	9	93.75ms	($t_{CONV}/8$)
0	1	10	187.5ms	($t_{CONV}/4$)
1	0	11	375ms	($t_{CONV}/2$)
1	1	12	750ms	(t_{CONV})

7.5.10 Trình tự hoạt động của cảm biến DS18B20

Trình tự để truy xuất cảm biến DS18B20 như sau:

- Bước 1: khởi động.

- Bước 2: lệnh truy xuất ROM.
- Bước 3: lệnh chức năng DS18B20.

Khởi động: tất cả các giao tiếp với bus 1 dây đều phải bắt đầu với trình tự động. Trình tự khởi động bao gồm 1 xung reset được phát bởi thiết bị chủ và tiếp theo là xung của thiết bị tớ hay của cảm biến DS18B20. Chức năng của xung reset có chức năng kiểm tra thiết bị tớ là cảm biến có nối với bus hay không, nếu có thì thiết bị tớ là cảm biến sẽ trả lời bằng cách tạo ra 1 xung tương ứng – quá trình này còn gọi là kiểm tra sự hiện diện của cảm biến.

Thời gian xung kiểm tra của thiết bị chủ và tớ đều phải tuân thủ quy định về thời gian rất chính xác, nếu sai thì mạch sẽ không hoạt động đúng hay dữ liệu nhận về và gửi đi sẽ sai.

7.5.11 Các lệnh hoạt động của cảm biến DS18B20

a. Các lệnh liên quan đến bộ nhớ rom

Sau khi thiết bị chủ kiểm tra sự hiện diện của cảm biến thì tiếp theo là phát các lệnh đọc ROM. Các lệnh này hoạt động dựa trên bộ mã 64 bit duy nhất lưu trong ROM của mỗi thiết bị và cho phép thiết bị chủ truy xuất tới 1 thiết bị tớ chỉ định duy nhất nếu có nhiều thiết bị tớ là cảm biến nối chung với nhau.

Lệnh ROM cũng cho biết có bao nhiêu thiết bị tớ và loại thiết bị hiện diện nối với bus hoặc thiết bị nào đã có báo động quá nhiệt.

Có 5 lệnh ROM và mỗi lệnh dài 8 bit, lưu đồ hoạt động ROM được trình bày như hình 7-22.

1. Lệnh tìm ROM – Search ROM (mã lệnh là F0H)

Khi hệ thống được cấp điện thì thiết bị chủ tiến hành nhận dạng mã ROM của tất cả các thiết bị tớ nối với bus, kết quả của nhận dạng này cho phép thiết bị chủ xác định số lượng thiết bị tớ và loại thiết bị. Thiết bị chủ đọc mã ROM thông qua quá trình thử và loại – quá trình này yêu cầu thiết bị chủ thực hiện chu kỳ lệnh Search ROM nhiều lần để nhận dạng tất cả các thiết bị tớ.

Sau khi thực hiện xong chu kỳ lệnh tìm ROM thì thiết bị chủ phải trở lại bước 1 để khởi tạo lại khi thực hiện truy xuất thiết bị tớ.

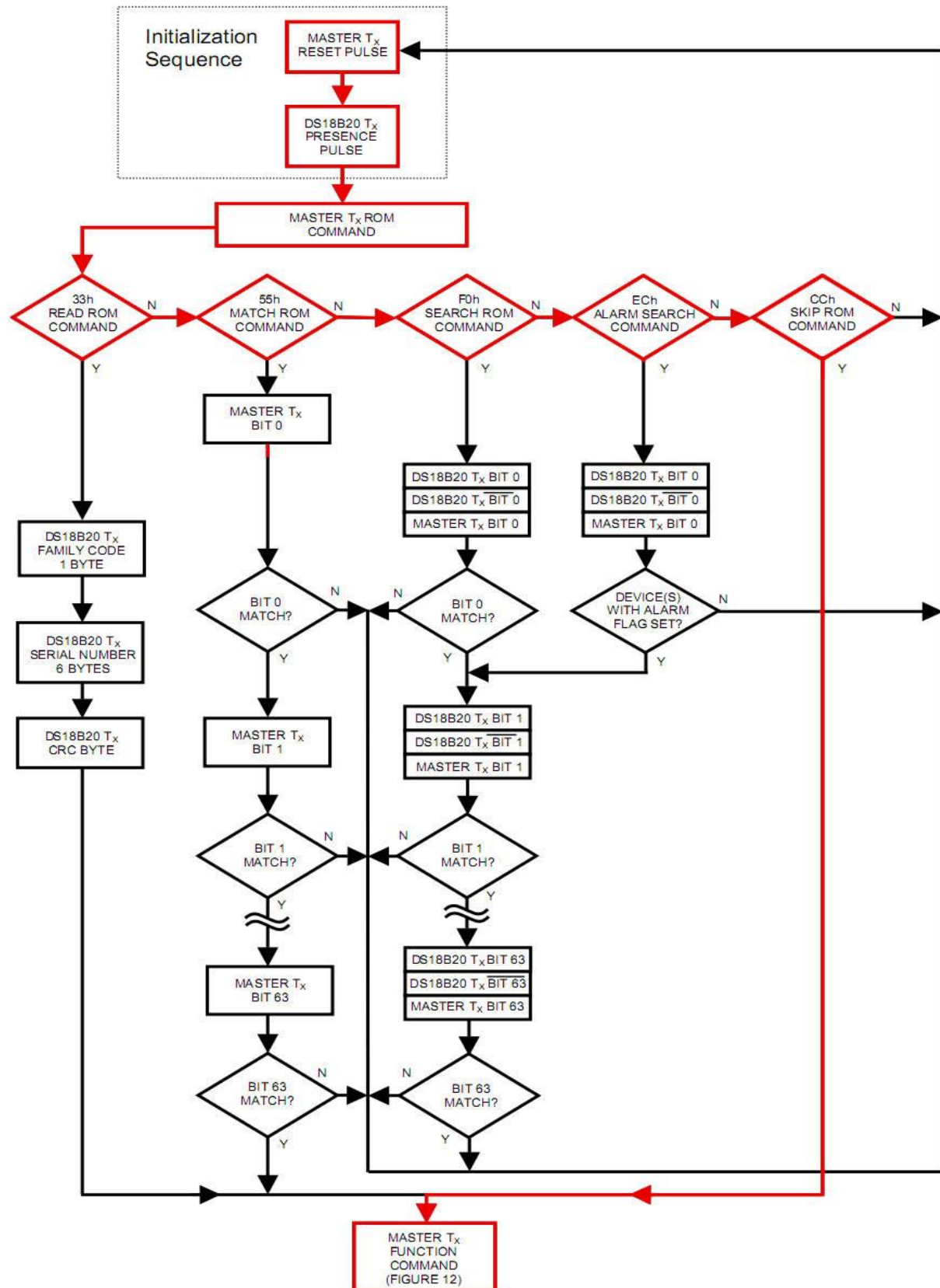
2. Lệnh đọc ROM – Read ROM (mã lệnh là 33H)

Lệnh đọc ROM chỉ có thể được khi chỉ có 1 thiết bị tớ nối với bus. Lệnh này cho phép thiết bị chủ đọc mã ROM 64 bit của thiết bị tớ mà không cần thực hiện lệnh tìm ROM. Nếu lệnh này được sử dụng khi có nhiều thiết bị tớ nối với bus thì dữ liệu sẽ bị sai vì tất cả các thiết bị tớ đều đáp ứng cùng một lúc.

3. Lệnh tương thích ROM – Match ROM (mã lệnh là 55H)

Lệnh tương thích ROM theo sau là mã 64 bit của ROM sẽ cho phép thiết bị chủ định địa chỉ của thiết bị tớ đúng với địa chỉ đó có nối với bus. Chỉ có thiết bị tớ tương thích với địa chỉ 64 bit của ROM sẽ đáp ứng với các lệnh yêu cầu từ thiết bị chủ, tất cả các thiết bị còn lại trên bus sẽ đợi cho đến khi có xung reset tiếp theo.

Figure 11. ROM Commands Flowchart



Hình 7-21. Lưu đồ các lệnh liên quan đến ROM của cảm biến DS18B20.

4. Lệnh bỏ qua đọc ROM – Skip ROM (mã lệnh là CCH)

Thiết bị chủ dùng lệnh skip ROM để định địa chỉ cho tất cả các thiết bị trên bus cùng một lúc mà không cần gửi bất kỳ thông tin nào về code của ROM. Ví dụ thiết bị chủ có thể ra lệnh cho tất cả các thiết bị tở có trên bus thực hiện chuyển đổi nhiệt độ cùng một lúc bằng cách thực hiện lệnh Skip ROM và theo sau là lệnh chuyển đổi Convert T (44H).

5. Lệnh tìm báo động – Alarm Search (mã lệnh là ECH)

Hoạt động của lệnh này giống như hoạt động của lệnh tìm ROM chỉ ngoại trừ là thiết bị tở có cờ báo động tích cực sẽ đáp ứng.

Lệnh này cho phép thiết bị chủ xác định nếu bất kỳ thiết bị tở DS18B20 nào đã xảy ra điều kiện báo động trong quá trình chuyển đổi nhiệt độ gần nhất.

Sau khi thực hiện xong lệnh tìm báo động thì thiết bị chủ phải trở lại bước 1 để khởi động.

Lệnh tương thích ROM theo sau là mã 64 bit của ROM sẽ cho phép thiết bị chủ định địa chỉ của thiết bị tở đúng với địa chỉ có nối với bus. Chỉ có thiết bị tở tương thích với địa chỉ 64 bit của ROM sẽ đáp ứng với các lệnh yêu cầu từ thiết bị chủ, tất cả các thiết bị còn lại trên bus sẽ đợi cho đến khi có xung reset tiếp theo.

b. Lệnh chức năng của DS18B20

Sau khi thiết bị chủ thực hiện lệnh về ROM để địa chỉ hóa cảm biến DS18B20 thì thiết bị chủ có thể thực hiện các lệnh chức năng của DS18B20. Các lệnh này cho phép thiết bị chủ ghi và đọc bộ nhớ Scratchpad của DS18B20, bắt đầu thực hiện quá trình chuyển đổi và xác định chế độ nguồn cung cấp.

Các lệnh chức năng của DS18B20 được tóm tắt ở bảng theo sau và minh họa bằng lưu đồ như hình 7-23.

1. Lệnh chuyển đổi – Convert T (mã lệnh là 44H)

Lệnh này khởi động quá trình chuyển đổi duy nhất. Theo sau lệnh này thì kết quả nhiệt độ sau khi chuyển đổi được lưu vào 2 thanh ghi 2 byte nằm trong bộ nhớ Scratchpad và DS18B20 trở lại trạng thái chờ công suất thấp.

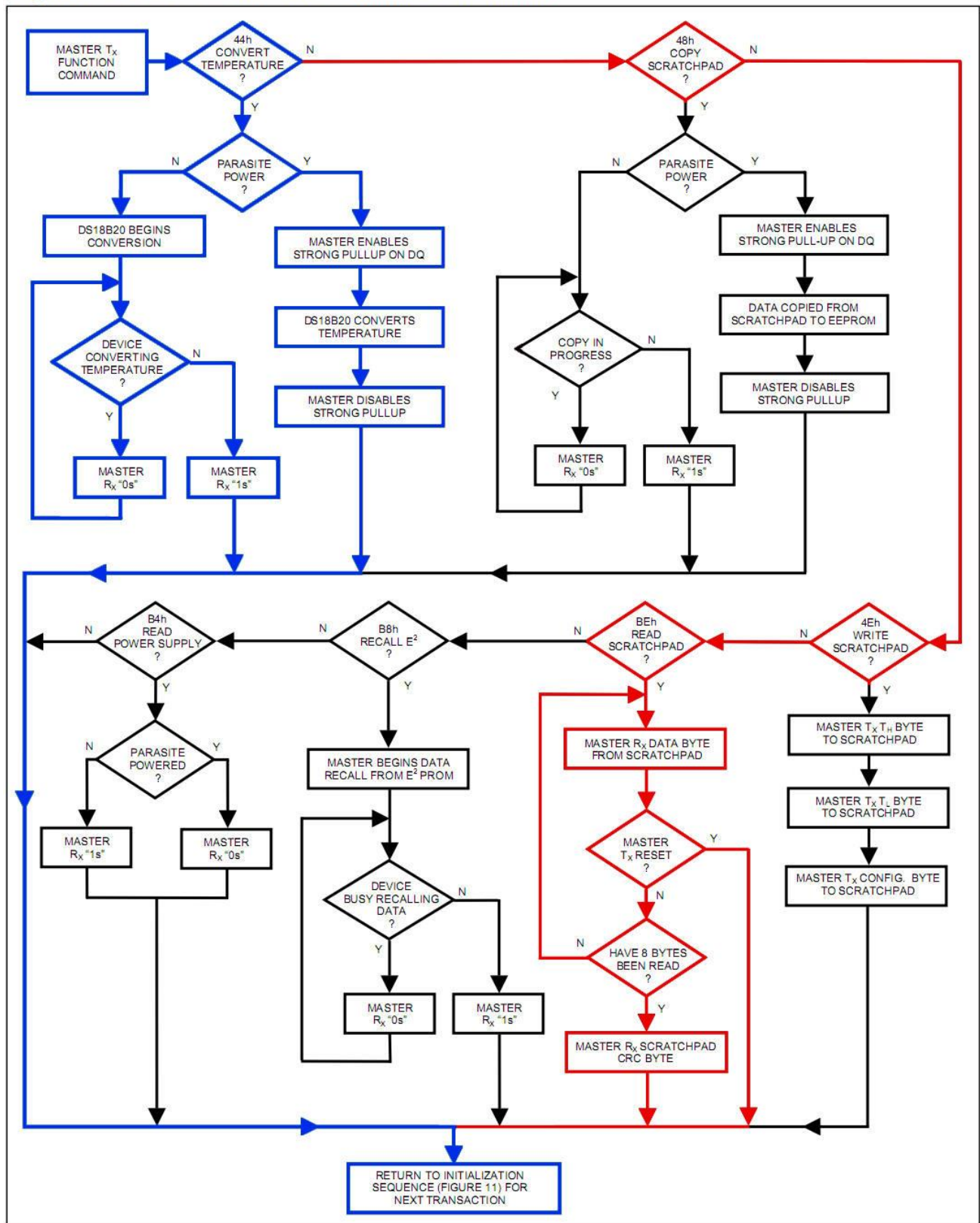
Nếu thiết bị đang sử dụng nguồn cung cấp thông qua chân tín hiệu thì trong khoảng 10 μ s (lớn nhất) sau khi thực hiện lệnh thì thiết bị chủ cho phép kéo lên cho bus nhanh trong khoảng thời gian chuyển đổi trình bày trong phần nguồn cung cấp.

Nếu thiết bị đang sử dụng nguồn cung cấp bên ngoài thì thiết bị chủ có thể thực hiện lệnh đọc theo sau khi thực hiện chuyển đổi Convert T và DS18B20 sẽ đáp ứng bằng cách phát mức logic 0 trong quá trình chuyển đổi và phát mức logic 1 khi quá trình chuyển đổi kết thúc.

2. Lệnh copy vùng nhớ Scratchpad – Copy Scratchpad (mã lệnh là 48H)

Lệnh này copy nội dung các thanh ghi T_H, T_L và thanh ghi định cấu hình của vùng nhớ SRAM Scratchpad (byte thứ 2, 3 và 4) vào bộ nhớ EEPROM.

Figure 12. DS18B20 Function Commands Flowchart



Hình 7- 22. Lưu đồ của các lệnh chức năng của cảm biến DS18B20.

3. Lệnh gọi lại – RECALL E²(mã lệnh là B8H)

Lệnh này nạp lại các giá trị báo động quá nhiệt (T_H và T_L) và dữ liệu định cấu hình từ bộ nhớ EEPROM sang bộ nhớ Scratchpad tương ứng.

Thiết bị chủ có thể cấp phát lệnh đọc theo sau lệnh Recall E². Cảm biến DS18B20 sẽ xác định trạng thái của lệnh Recall bằng cách phát mức logic 0 trong quá trình thực hiện lệnh Recall và phát mức logic 1 khi quá trình thực hiện lệnh recall hoàn tất.

Lệnh recall được thực hiện tự động khi cảm biến được cấp điện nên dữ liệu có hiệu lực trong bộ nhớ Scratchpad ngay khi thiết bị được cấp điện.

4. Lệnh đọc nguồn cung cấp – READ POWER SUPPLY (mã lệnh là B4H)

Thiết bị chủ cấp phát lệnh này theo sau khoảng thời gian đọc để xác định xem có bất kỳ cảm biến DS18B20 nào sử dụng nguồn cung cấp bằng đường tín hiệu.

Trong khoảng thời gian đọc nếu thiết bị dùng nguồn cung cấp là dây tín hiệu thì sẽ kéo bus xuống mức logic 0, nếu sử dụng nguồn bên ngoài thì bus vẫn ở mức logic 1.

5. Lệnh ghi dữ liệu vào bộ nhớ Scratchpad – WRITE SCRATCHPAD (mã lệnh là 4EH)

Lệnh này cho phép thiết bị chủ ghi 3 byte dữ liệu vào vùng nhớ scratchpad. Byte dữ liệu thứ 1 sẽ ghi vào thanh ghi T_H (là byte thứ 2 của vùng nhớ Scratchpad), byte thứ 2 ghi vào thanh ghi T_L (byte 3) và byte thứ 3 ghi vào thanh ghi định cấu hình (byte 4). Khi truyền thì byte có trọng số nhỏ nhất LSB được truyền đi đầu tiên. Tất cả các byte phải được ghi trước khi thiết bị chủ phát lệnh reset hoặc dữ liệu có thể bị ngắt.

6. Lệnh đọc dữ liệu bộ nhớ Scratchpad – READ SCRATCHPAD (mã lệnh là BEH)

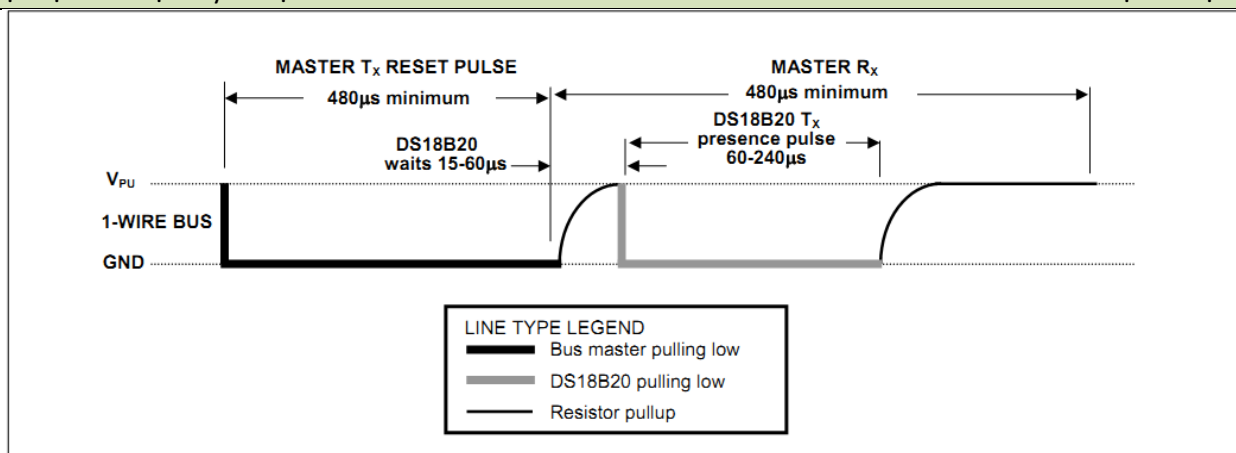
Lệnh này cho phép thiết bị chủ đọc nội dung của vùng nhớ Scratchpad. Khi truyền dữ liệu thì bắt đầu với bit LSB của byte thứ 0 và tiếp tục cho đến byte thứ 9. Thiết bị chủ có thể phát lệnh reset để chấm dứt quá trình đọc bất kỳ lúc nào nếu chỉ đọc một phần dữ liệu của bộ nhớ Scratchpad.

7.5.12 Các dạng tín hiệu của chuẩn 1 dây

Cảm biến DS18B20 dùng chuẩn truyền dữ liệu 1 dây. Có nhiều loại tín hiệu được định nghĩa bởi chuẩn này như sau: xung reset, xung hiện diện, ghi mức logic 0, ghi mức logic 1, đọc mức logic 0, đọc mức logic 1. Thiết bị chủ khởi động tất cả các tín hiệu này sau khi chấp nhận xung hiện diện.

7.5.13 Trình tự khởi động – xung reset và xung hiện diện

Tất cả các giao tiếp với cảm biến DS18B20 bắt đầu với trình tự khởi động bao gồm xung reset từ thiết bị chủ theo sau là xung hiện diện của cảm biến DS18B20. Trình tự này được minh họa như hình 7-24.



Hình 7-23. Dạng sóng của xung reset và xung hiện diện.

Khi cảm biến DS18B20 gọi xung hiện diện để đáp ứng xung reset thì đồng thời xác định với thiết bị chủ là cảm biến sẵn sàng hoạt động.

Trong quá trình khởi động thì thiết bị chủ (hoạt động ở chế độ phát Tx) tạo xung reset bằng cách kéo tín hiệu bus xuống mức thấp trong khoảng thời gian tối thiểu là 470µs. Sau đó thiết bị chủ thả nổi tín hiệu bus và hoạt động ở chế độ nhận (hoạt động ở chế độ nhận Rx) để trao quyền điều khiển cho cảm biến DS18B20.

Khi thả nổi thì điện trở kéo lên sẽ kéo bus về mức logic 1. Khi cảm biến DS18B20 phát hiện cạnh lên của xung thì nó sẽ đợi từ 15µs đến 60µs và sau đó phát đi 1 xung hiện diện bằng cách kéo bus xuống mức logic 0 trong khoảng thời gian từ 60µs đến 240µs.

7.5.14 Các khe thời gian đọc/ghi

Thiết bị chủ ghi dữ liệu vào DS18B20 trong các khe thời gian ghi và đọc dữ liệu từ DS18B20 trong các khe thời gian đọc. Một bit dữ liệu được phát lên bus trong 1 khe thời gian.

Khe thời gian ghi

Có 2 loại khe thời gian ghi là: khe thời gian ghi mức 1 (ghi dữ liệu mức 1 vào cảm biến DS18B20) và khe thời gian ghi mức 0 (ghi dữ liệu mức 0 vào cảm biến DS18B20). Tất cả các khe thời gian có lượng thời gian tối thiểu là 60µs và thời gian tín hiệu khôi phục lại từ mức 0 về mức 1 do điện trở kéo lên tối thiểu là 1 µs.

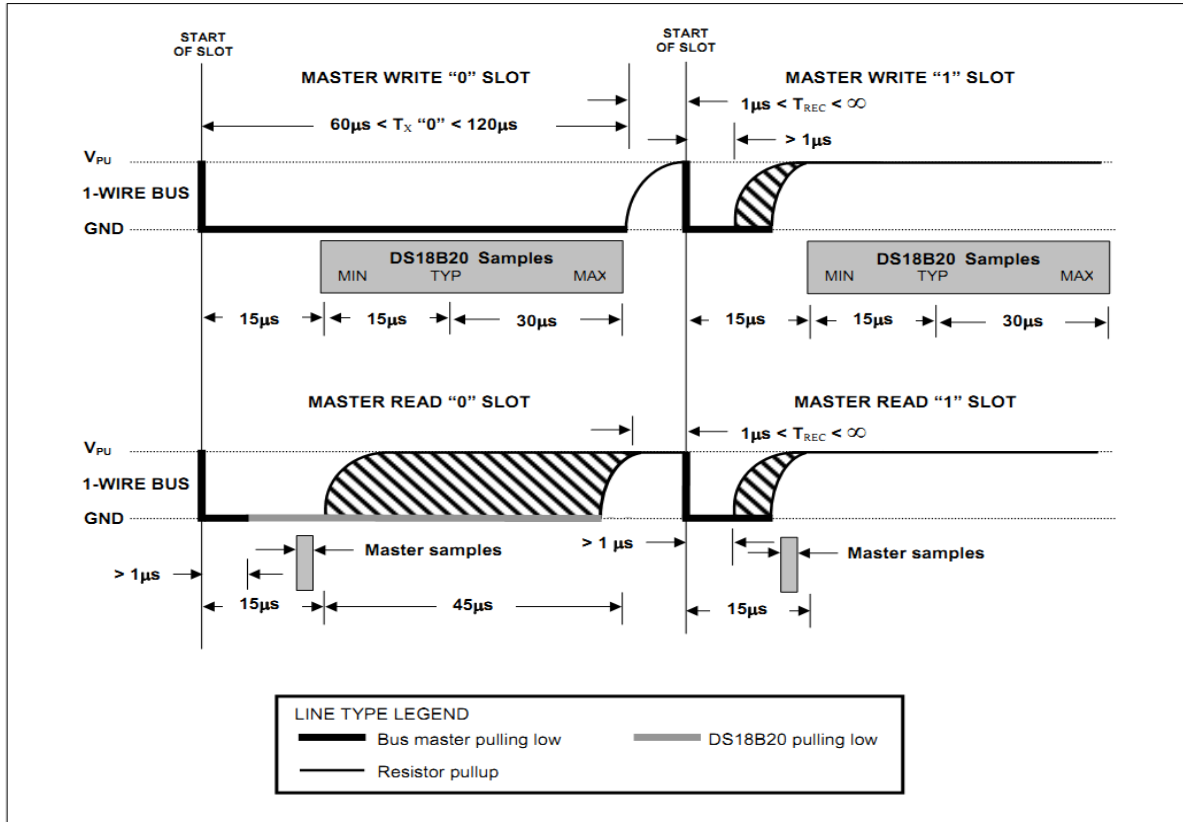
Cả 2 khe thời gian ghi dữ liệu mức logic 1 và 0 đều được khởi động bởi thiết bị chủ kéo bus xuống mức 0 như hình 7-25.

Để tạo ra khe thời gian ghi dữ liệu mức logic 1 thì sau khi phát xung kéo bus xuống mức 0 thì thiết bị chủ phải thả nổi bus trong vòng 15µs. Khi thả nổi bus thì điện trở kéo lên sẽ kéo bus về mức 1.

Để tạo ra khe thời gian ghi dữ liệu mức logic 0 thì sau khi phát xung kéo bus xuống mức 0 thì thiết bị chủ tiếp tục giữ bus ở mức 0 trong suốt khe thời gian (ít nhất là 60µs).

Cảm biến DS18B20 sẽ lấy mẫu từ bus với các khoảng thời gian kéo dài từ 15µs đến 60µs sau khi khởi động khe thời gian ghi.

Nếu bus ở mức cao trong suốt thời gian lấy mẫu thì mức 1 sẽ được ghi vào DS18B20, nếu ở mức 0 thì mức 0 được ghi vào.



Hình 7-24. Dạng sóng khe thời gian đọc/ghi.

Khe thời gian đọc

DS18B20 chỉ có thể truyền dữ liệu đến thiết bị chủ khi thiết bị chủ phát lệnh đọc các khe thời gian. Do đó, thiết bị chủ phải tạo các khe thời gian đọc ngay sau khi phát lệnh đọc bộ nhớ Scratchpad hoặc lệnh đọc nguồn cung cấp [B4H], khi đó DS18B20 có thể cung cấp dữ liệu yêu cầu.

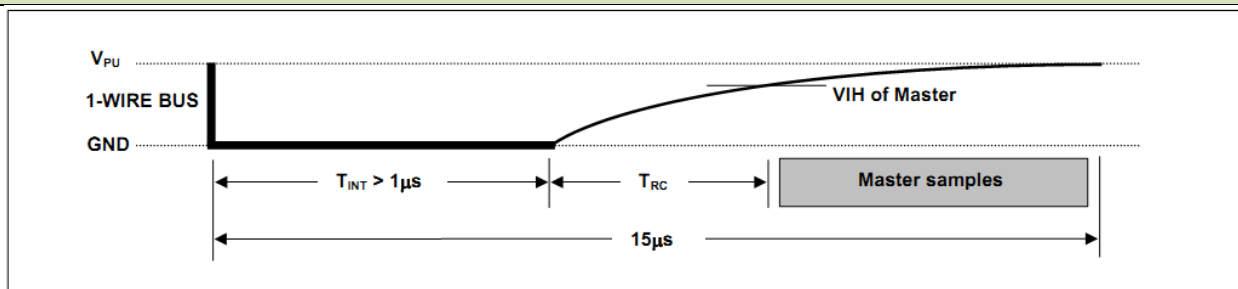
Tất cả các khe thời gian đọc có lượng thời gian tối thiểu là $60\mu s$ và thời gian khôi phục tối thiểu là $1\mu s$ giữa 2 khe thời gian.

Khe thời gian đọc dữ liệu bắt đầu bởi thiết bị chủ bằng cách kéo bus xuống mức 0 trong khoảng thời gian tối thiểu là $1\mu s$ và sau đó là thả nổi bus – xem hình 7-24.

Sau khi thiết bị chủ khởi tạo xong khe thời gian đọc thì cảm biến DS18B20 sẽ bắt đầu truyền mức 1 hoặc mức 0 lên bus.

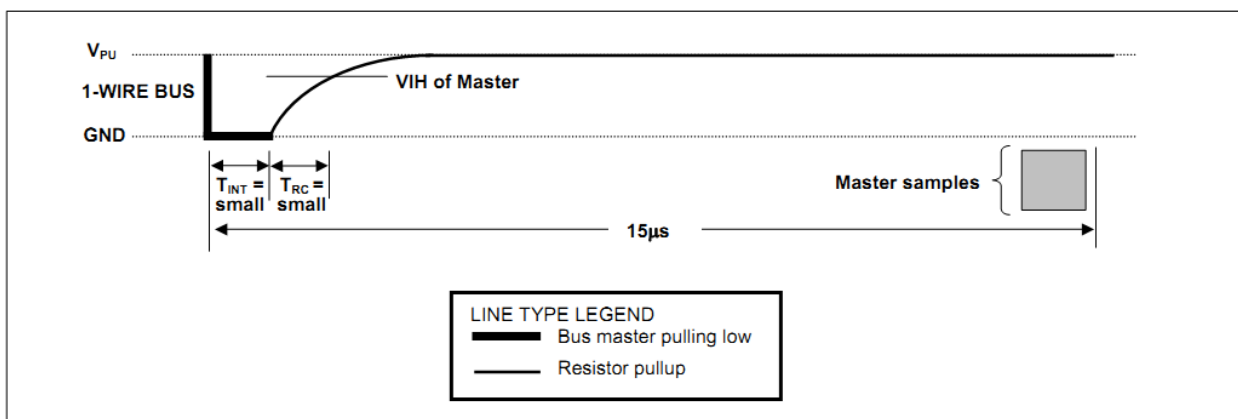
DS18B20 truyền mức 1 bằng cách để bus trở về mức 1 và truyền mức 0 thì kéo bus ở luôn mức 0. Khi đang truyền mức 0, DS18B20 sẽ thả nổi bus khi gần kết thúc khe thời gian và bus sẽ được kéo lên mức 1 bởi điện trở kéo lên. Ngõ ra dữ liệu từ DS18B20 có hiệu lực trong vòng $15\mu s$ tính từ khi có cạnh xuống của xung khởi tạo khe thời gian đọc. Do đó thiết bị chủ phải thả nổi bus và sau đó tiến hành lấy mẫu trạng thái bus trong vòng $15\mu s$ tính từ lúc bắt đầu khe thời gian.

Hình 7-26 minh họa tổng thời gian T_{INIT} , T_{RC} và T_{SAMPLE} phải nhỏ hơn $15\mu s$ đối với khe thời gian đọc.



Hình 7-25. Chi tiết dạng sóng khe thời gian đọc mức 1 của thiết bị chủ.

Hình 7-27 minh họa các giới hạn thời gian lớn nhất bằng cách giữ T_{INIT} và T_{RC} nhỏ nhất có thể và định vị thời gian lấy mẫu của thiết bị chủ trong khe thời gian đọc hướng về điểm gần kết thúc chu kỳ khe thời gian.



Hình 7-26. Thu ngắn các khoảng thời gian.

7.6 CÁC CHƯƠNG TRÌNH ĐO NHIỆT ĐỘ DÙNG CẢM BIẾN DS18B20

7.6.1 Mạch điện giao tiếp vi điều khiển với cảm biến DS18B20

Mạch điện giao tiếp vi điều khiển với cảm biến DS18B20 như hình 7-28.

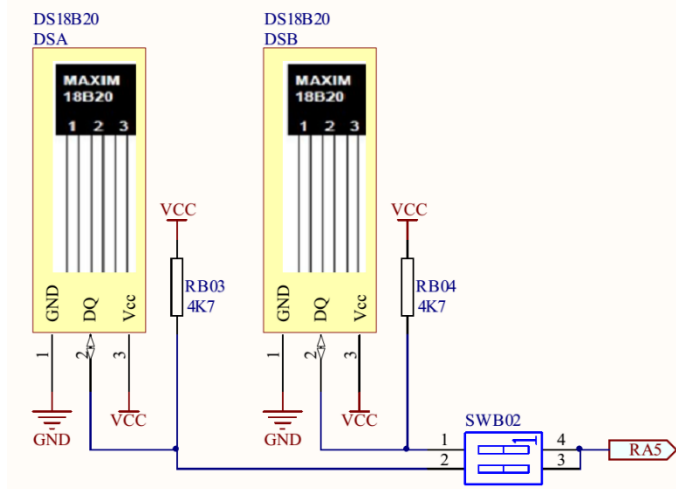
Trong sơ đồ mạch thì có 2 cảm biến: DS18B20A và DS18B20B nối với port RA5 thông qua switch SWB02.

Khi sử dụng thì cảm biến nào thì phải chuyển switch tương ứng sang ON.

Có các bài thực hành cho 2 cảm biến như sau:

Bài thực hành chỉ dùng 1 trong 2 cảm biến: Với bài dạng này thì không cần biết và không cần đọc code của bộ nhớ của cảm biến vì chỉ có 1 cảm biến. Khi dùng thì 1 switch ON, 1 switch còn lại OFF.

Bài thực hành dùng cả 2 cảm biến: Với bài dạng này thì cần phải biết code của bộ nhớ của 2 cảm biến, khi truy xuất cảm biến nào thì phải gởi địa chỉ của cảm biến đó mới truy xuất được. Khi dùng thì 2 switch đều ON. Dạng bài này phức tạp hơn dạng trên.



Hình 7-27. Mạch đo nhiệt độ dùng 2 cảm biến DS18B20.

7.6.2 Đo nhiệt độ dùng cảm biến DS18B20 hiển thị trên led 7 đoạn

Thư viện DS18B20. Chương trình thư viện của cảm biến DS18B20 đã cho trong thư mục của bạn có tên là “TV_DS18B20.c” và “TV_DS18B20.h”.

Bạn có thể sử dụng cho các bài thực hành và nên đọc hiểu.

File “TV_DS18B20.h”

```
#ifndef __TV_DS18B20_H__
#define __TV_DS18B20_H__
#include "touch.c"

typedef struct
{
    unsigned int8 romcode[3][8]; // Biến lưu mã ROM của cảm biến
    unsigned int8 number_of_ds18b20; // Biến lưu số lượng cảm biến
    unsigned int16 temperatureA; // Biến lưu nhiệt độ của cảm biến A
    unsigned int16 temperatureB; // Biến lưu nhiệt độ của cảm biến B
}DS18B20_STRUCT;

DS18B20_STRUCT
ds18b20={{0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},0,0,0};
result ds18b20_read_temperature(void);
result ds18b20_set_resolution(int8 res_9_12);
#endif
```

File “TV_DS18B20.c”

```
#ifndef __TV_DS18B20_C__
#define __TV_DS18B20_C__
#include<TV_DS18B20.h>

/* Hàm SEARCH ROM - Thống kê số lượng cảm biến và mã ROM từng cảm biến
- ds18b20.romcode[?][?]: chứa mã ROM của các cảm biến tìm được
- ds18b20.number_of_ds18b20 : chứa số lượng cảm biến tìm được
- Hàm trả về "Ok" khi có ít nhất 1 cảm biến còn tốt
- Hàm trả về "ERROR" khi không giao tiếp được bất kỳ cảm biến nào
*/
result DS18B20_SEARCH_ROM()
{
    int8
        i,
        last_desc_pos = 0,
```



```

    desc_marker = 0,
    contents[8] = {0,0,0,0,0,0,0,0};
int1
    bitA,
    bitB,
    last_desc = 0;
int1
    firstrom = TRUE,
    done = FALSE;
do
{
    last_desc_pos = desc_marker;
    reset_pulse();
    touch_write_byte(0xF0);
    for(i = 0; i < 64; i++)
    {
        bitA = touch_read_bit();
        bitB = touch_read_bit();
        if((!bitA) && (!bitB))
        {
            desc_marker = i;
            if(i == last_desc_pos)
                last_desc = !last_desc;
            shift_right(contents, 8, last_desc);
            touch_write_bit(last_desc);
        }
        else
        {
            shift_right(contents, 8, bitA);
            touch_write_bit(bitA);
        }
    }
    if(firstrom)
    {
        for(i = 0; i < 8; i++)
            ds18b20.romcode[0][i] =contents[i];
        firstrom = FALSE; ds18b20.number_of_ds18b20++;
    }
    else
    {
        for(i = 0; i < 8; i++)
            ds18b20.romcode[ds18b20.number_of_ds18b20][i] =contents[i];
        ds18b20.number_of_ds18b20++;
        done = TRUE;
        for(i = 0; i < 8; i++)
            if(ds18b20.romcode[0][i] != contents[i])
                done = FALSE;
    }
} while(!done);
ds18b20.number_of_ds18b20--;
if(ds18b20.number_of_ds18b20>0) return OK;
else return ERROR;
}

/* Hàm MATCH ROM – Dùng khi giao tiếp với 1 cảm biến đã biết mã ROM
- *romdata: Là mã ROM của cảm biến muốn giao tiếp

```

```

- Hàm luôn trả về kết quả OK
*/
result DS18B20_MATCH_ROM(int8 *romdata)
{
    int8 i;
    touch_write_byte(0x55);
    for(i = 0; i < 8; i++)
        touch_write_byte(romdata[i]);
    return OK;
}
/* Hàm đọc nhiệt độ 1 cảm biến - Dừng khi hệ thống chỉ có 1 cảm biến
- ds18b20.temperatureA : chứa giá trị nhiệt độ đo được
- Hàm trả về "Ok" khi hệ thống tốt và đo xong
- Hàm trả về "ERROR" khi hệ thống bị lỗi
*/
result DS18B20_SINGLE_READ_TEMP()
{
    unsigned int8 ndh,ndl;
    if(touch_present()) //1. Khởi động
    {
        touch_write_byte(0xCC); //2. SKIP ROM
        touch_write_byte(0x44); //3. Yêu cầu chuyển đổi nhiệt độ
    }
    if(touch_present()) //1. Khởi động
    {
        touch_write_byte(0xCC); //2. SKIP ROM
        touch_write_byte(0xBE); //3. Yêu cầu đọc RAM
        ndl= touch_read_byte(); // Đọc byte thấp của nhiệt độ
        ndh= touch_read_byte(); // Đọc byte cao của nhiệt độ
        ds18b20.temperatureA = make16(ndh,ndl); // Ghép thành số 16 bit
        return OK;
    }
    return ERROR;
}
/* Hàm đọc nhiệt độ nhiều cảm biến - Dừng khi hệ thống có nhiều cảm biến
- ds18b20.temperatureA: Chứa giá trị nhiệt độ đo được của cảm biến A
- ds18b20.temperatureB: Chứa giá trị nhiệt độ đo được của cảm biến B
(Vì trên board chỉ có 2 cảm biến nên chỉ cần dùng 2 biến trên để lưu)
- Hàm trả về "Ok" khi hệ thống tốt và đo xong
- Hàm trả về "ERROR" khi hệ thống bị lỗi
*/
result DS18B20_MULTI_READ_TEMP()
{
    unsigned int8 ndh,ndl;
    static int1 cb=1; cb=!cb;
    if(cb==0)
    {
        if(touch_present()) //1. Khởi động
        {
            touch_write_byte(0xCC); //2. SKIP ROM
            touch_write_byte(0x44); //3. Tất cả chuyển đổi nhiệt độ
        }
    }
    if(touch_present()) //1. Khởi động
    {

```

```

        DS18B20_MATCH_ROM(ds18b20.romcode[cb]); //2. Chọn cảm biến muốn đọc
        touch_write_byte(0xBE); //3. Yêu cầu đọc RAM
        ndl= touch_read_byte(); // Đọc byte thấp
        ndh= touch_read_byte(); // Đọc byte cao
        if(cb==0) ds18b20.temperatureA= make16(ndh,ndl); // Cảm biến A
        else      ds18b20.temperatureB= make16(ndh,ndl); // Cảm biến B
        return OK;
    }
    return ERROR;
}

/* Hàm đọc nhiệt độ DS18B20 - Bắt chấp 1 hay 2 cảm biến
- Nếu hệ thống không có cảm biến hoặc bị lỗi thì hàm sẽ trả về "ERROR"
- Nếu hệ thống có 1 cảm biến thì nhiệt độ đọc được sẽ lưu trong biến
  "ds18b20.temperatureA" và hàm sẽ trả về "OK"
- Nếu hệ thống có 2 cảm biến thì nhiệt độ đọc được sẽ lưu trong 2 biến
  "ds18b20.temperatureA" và "ds18b20.temperatureB" và hàm trả về "OK"
*/
result ds18b20_read_temperature()
{
    static int1 ttsearchrom=0;
    if(ttsearchrom==0){DS18B20_SEARCH_ROM();ttsearchrom=1;}
    if(ds18b20.number_of_ds18b20==1)return DS18B20_SINGLE_READ_TEMP();
    else return DS18B20_MULTI_READ_TEMP();
}

/* Hàm cài đặt độ phân giải cho DS18B20
- res_9_12: giá trị từ 9 đến 12 tương ứng chế độ 9 đến 12 bit
- Hàm trả về "Ok" khi hệ thống tốt và cài đặt xong
- Hàm trả về "ERROR" khi hệ thống bị lỗi
*/
result ds18b20_set_resolution(int8 res_9_12)
{
    const unsigned int8 res[]={0x1f,0x3f,0x5f,0x7f};
    if ((res_9_12 < 9)|| (res_9_12>12)) return ERROR;
    if(touch_present())
    {
        touch_write_byte(0xCC);
        touch_write_byte(0x4e);
        touch_write_byte(0);
        touch_write_byte(0);
        touch_write_byte(res[res_9_12-9]);
        if(touch_present())
        {
            touch_write_byte(0xCC);
            touch_write_byte(0x48);
            return OK;
        }
    }
    return ERROR;
}
#endif

```

Bài mẫu 731. Chương trình đọc nhiệt độ từ cảm biến DS18B20A hiển thị trên led 7 đoạn. Chọn độ phân giải là 0.5 độ và lấy 1 số thập phân.

Lưu tên file là “BAI_731_DS18B20A_D7SEG”.

- Mục đích: biết lập trình đọc nhiệt độ và cài đặt độ phân giải cho DS18B20 và hiển thị trên led 7 đoạn.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_ds18b20.c>
unsigned int16 tp,ng ;
void main()
{
    system_init();
    ds18b20_set_resolution(9);           // Cài đặt độ phân giải 9 bit (0.5độ)
    while(true)
    {
        if(ds18b20_read_temperature()==OK)
        {
            ng = ds18b20.temperatureA>>4 ;           // Tính phần nguyên
            tp = (ds18b20.temperatureA&0x0f)*10/16; // Tính phần thập phân
            d7seg.led[2]= m7d[ng/10%10];
            d7seg.led[1]= m7d[ng%10]-128;             // -128 chấm thập phân
            d7seg.led[0]= m7d[tp];
            d7seg_display();
        }
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: sẽ hiển thị nhiệt độ môi trường và mở đèn để gia tăng nhiệt độ.
- Giải thích chương trình:

Bài mẫu 732. Chương trình đo nhiệt độ 2 cảm biến DS18B20 và hiển thị trên LCD theo font 2x3. Sử dụng độ phân giải 0.125 độ và lấy 2 số thập phân.

Chú ý: Gạt cả 2 công tắc DS18B20 ở vị trí ON trước khi nạp chương trình

Lưu tên file là “BAI_732_2DS18B20_LCD”.

- Mục đích: Biết cách đo nhiệt độ 2 cảm biến DS18B20 và hiển thị LCD
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_ds18b20.c>
#include<tv_lcd.c>
unsigned int16 ng,tp;
void main()
{
    system_init();
    lcd_setup();
    lcd_goto(0,0);
```

```

lcd_data("DS18B20A    DS18B20B");
ds18b20_set_resolution(11); // Cài đặt độ phân giải 11 bit (0.125độ)
while(true)
{
    if(ds18b20_read_temperature()==OK)
    {
        ng = ds18b20.temperatureA>>4; //Phần nguyên DS18B20A
        tp = (ds18b20.temperatureA &0x0f)*100/16; //Thập phân
        lcd_write_2x3_num(ng/10%10,0,1);
        lcd_write_2x3_num(ng/1%10,3,1);
        printf(lcd_data, "%.1d", tp);

        ng = ds18b20.temperatureB>>4; //Phần nguyên DS18B20B
        tp = (ds18b20.temperatureB &0x0f)*100/16; //Thập phân
        lcd_write_2x3_num(ng/10%10,11,1);
        lcd_write_2x3_num(ng/1%10,14,1);
        printf(lcd_data, "%.1d", tp);
    }
}
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: Nếu 1 trong 2 cảm biến hư sẽ hiển thị giá trị là 00.0 độ
- f. Giải thích chương trình:
- g. Giải thích cách tính và hiển thị giá trị phần thập phân

Bài tập 733. Thêm vào bài 732 hai nút nhấn UP và DW để chỉnh độ phân giải từ 9 đến 12 bit. Hiển thị hàng cuối LCD giá trị độ phân giải hiện tại.

Lưu tên file là “BAI_733_2DS18B20_CHINH_DPGIAI”.

Bài tập 734. Chương trình đọc nhiệt độ từ cảm biến DS18B20A hiển thị trên 2 led 7 đoạn bên trái và đo nhiệt độ từ cảm biến LM35_A hiển thị trên 2 led bên phải để so sánh độ chính xác.

Lưu tên file là “BAI_734_DS18B20A_LM35A”.

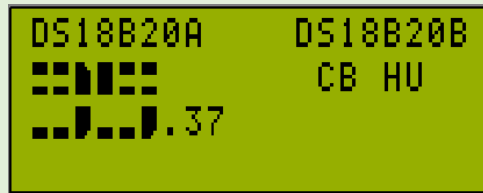
Bài tập 735. Hệ thống vừa đọc nhiệt độ từ cảm biến DS18B20A hiển thị trên 2 led 7 đoạn bên phải vừa đếm sản phẩm dùng T0 hiển thị trên led 7 đoạn bên trái đếm từ 00 đến 99.

Lưu tên file là “BAI_735_DS18B20A_DSP_T1”.

Bài mẫu 736. Chương trình đọc nhiệt độ từ cảm biến hiển thị trên 2 led 7 đoạn, 2 led 7 đoạn còn lại hiển thị giá trị cài đặt trước để báo động, giá trị mặc nhiên thấp nhất là 35 độ, lớn nhất là 60 độ, có thể điều chỉnh bằng 2 nút nhấn UP và DW. Khi giá trị đo lớn hơn giá trị cài thì báo động ngược lại thì tắt báo động. Tự động tắt mở đèn.

Lưu tên file là “BAI_736_DS18B20A_UP_DW_BUZZ”.

Bài tập 737. Thêm vào bài 732 chức năng kiểm tra cảm biến. Nếu cảm biến nào hư thì hiển thị chữ “CB HU” theo mẫu dưới (ví dụ cảm biến DS18B20B bị hư):



Chú ý: Gạt công tắc DS18B20 để giả lập cảm biến hư. Sau đó Reset lại VDK.

(SV có thể tham khảo bài “EXAMPLE_7_DS18B20” về cách kiểm tra cảm biến)

Lưu tên file là “BAI_737_DS18B20_KIEMTRA_CB”.

Chương 8: **THỰC HÀNH**
MODULE 5 – REAL TIME
DS13B07, ADC–DAC PCF8591,
EEPROM NỐI TIẾP AT24C256
THEO CHUẨN I2C VÀ CÁC
CẢM BIẾN

8.1 LÝ THUYẾT I2C

8.1.1 Giới thiệu

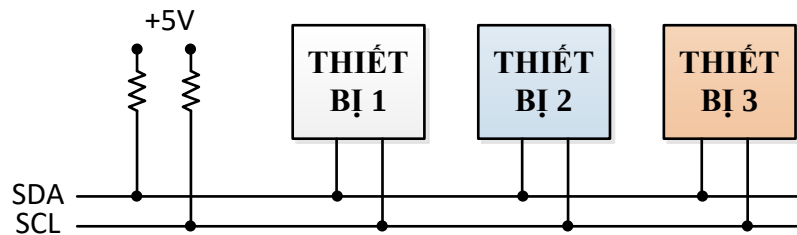
Vi điều khiển PIC18F4550 tích hợp chuẩn truyền dữ liệu I2C để giao tiếp với các thiết bị ngoại vi theo chuẩn I2C.

Có rất nhiều ngoại vi tích chuẩn giao tiếp I2C như bộ nhớ Eeprom nối tiếp, ADC-DAC, Real-time, IC mở rộng ngoại vi, ...

Trong phần này trình bày chuẩn giao tiếp I2C và lập trình cho vi điều khiển PIC18F4550 thực hiện giao tiếp với đồng hồ thời gian thực theo chuẩn I2C.

8.1.2 Tổng quan về truyền dữ liệu chuẩn I2C

I2C, viết tắt của từ Inter-Integrated Circuit, là một chuẩn truyền thông do hãng điện tử Philips Semiconductor sáng lập, cho phép giao tiếp một thiết bị chủ với nhiều thiết bị tớ với nhau như hình 8-1.



Hình 8-1. Hệ thống các thiết bị giao tiếp theo chuẩn I2C.

Chuẩn giao tiếp I2C có 2 đường tín hiệu tên là SDA (serial data) có chức năng truyền tải dữ liệu và tín hiệu SCL (serial clock) truyền tải xung clock để dịch chuyển dữ liệu.

Trong hệ thống truyền dữ liệu I2C, thiết bị nào cung cấp xung clock thì được gọi là chủ (master), thiết bị nhận xung clock được gọi là tớ (slave).

Thiết bị chủ chỉ có 1, thiết bị tớ thì có nhiều, mỗi thiết bị tớ sẽ có 1 địa chỉ độc lập, chuẩn truyền ban đầu dùng địa chỉ 7 bit nên có thể 1 chủ giao tiếp với 128 thiết bị tớ. Các thiết bị sau này tăng thêm số bit địa chỉ nên có thể giao tiếp nhiều hơn.

Địa chỉ của thiết bị tớ thường do nhà chế tạo thiết bị thiết lập sẵn.

8.1.3 Quy trình truyền dữ liệu chuẩn I2C

Quy trình thiết bị chủ giao tiếp với thiết bị tớ để thực hiện việc ghi đọc dữ liệu như sau:

Quá trình thiết bị chủ ghi dữ liệu vào thiết bị tớ

Bước 1: Thiết bị chủ tạo trạng thái START để bắt đầu quá trình truyền dữ liệu – các thiết bị tớ sẽ ở trạng thái sẵn sàng nhận địa chỉ từ thiết bị chủ.

Bước 2: Thiết bị chủ gửi địa chỉ của thiết bị tớ cần giao tiếp – khi đó tất cả các thiết bị tớ đều nhận địa chỉ và so sánh với địa chỉ của mình, các thiết bị tớ sau khi phát hiện không phải địa chỉ của mình thì chờ cho đến khi nào nhận trạng thái START mới.

Trong dữ liệu 8 bit thì có 7 bit địa chỉ và 1 bit điều khiển đọc/ghi (R/W): bit này bằng 0 để báo cho thiết bị tớ sẽ nhận byte tiếp theo.

Bước 3: Thiết bị chủ chờ nhận tín hiệu bắt tay từ thiết bị tớ. Thiết bị tớ nào đúng địa chỉ thì phát 1 tín hiệu trả lời cho chủ biết.

Bước 4: Thiết bị chủ tiến hành gọi địa chỉ của ô nhớ bắt đầu cần ghi dữ liệu, bit R/W ở trạng thái ghi.

Bước 5: Thiết bị chủ chờ nhận tín hiệu trả lời từ thiết bị tớ.

Bước 6: Thiết bị chủ gọi tiến hành gọi dữ liệu để ghi vào thiết bị tớ, mỗi lần ghi 1 byte, sau khi gọi xong thì tiến hành chờ nhận tín hiệu trả lời từ thiết bị tớ, quá trình thực hiện cho đến byte cuối cùng xong rồi thì thiết bị chủ chuyển sang trạng thái STOP để chấm dứt quá trình giao tiếp với thiết bị tớ.

Quá trình thiết bị chủ đọc dữ liệu vào thiết bị tớ

Bước 1: Thiết bị chủ tạo trạng thái START để bắt đầu quá trình truyền dữ liệu, các thiết bị tớ sẽ ở trạng thái sẵn sàng nhận địa chỉ từ thiết bị chủ.

Bước 2: Thiết bị chủ gọi địa chỉ của thiết bị tớ cần giao tiếp, khi đó tất cả các thiết bị tớ đều nhận địa chỉ và so sánh với địa chỉ của mình. Các thiết bị tớ sau khi phát hiện không phải địa chỉ của mình thì chờ cho đến khi nào nhận trạng thái START mới.

Trong dữ liệu 8 bit thì có 7 bit địa chỉ và 1 bit điều khiển đọc/ghi (R/W): bit này bằng 0 để báo cho thiết bị tớ sẽ nhận byte tiếp theo.

Bước 3: Thiết bị chủ chờ nhận tín hiệu bắt tay từ thiết bị tớ. Thiết bị tớ nào đúng địa chỉ thì phát 1 tín hiệu trả lời cho chủ biết.

Bước 4: Thiết bị chủ tiến hành gọi địa chỉ của ô nhớ bắt đầu cần đọc dữ liệu, bit R/W ở trạng thái đọc.

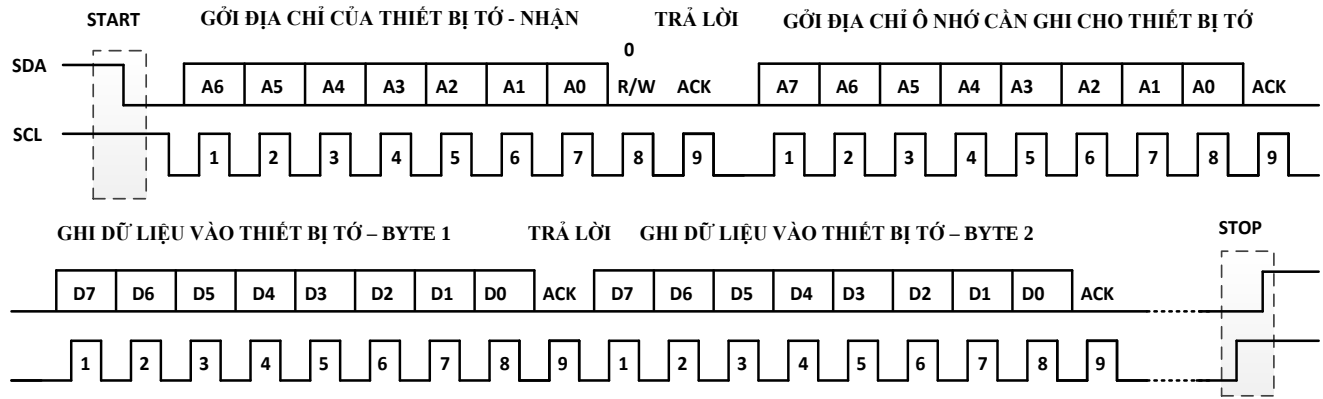
Bước 5: Thiết bị chủ chờ nhận tín hiệu trả lời từ thiết bị tớ.

Bước 6: Thiết bị chủ chuyển sang trạng thái STOP, bắt đầu lại trạng thái START, tiến hành gọi địa chỉ của thiết bị và bit R/W bằng 1 để yêu cầu tớ gọi dữ liệu nội dung ô nhớ của địa chỉ đã nhận.

Bước 7: Thiết bị chủ sau khi nhận sẽ báo tín hiệu trả lời, quá trình này thực hiện cho đến khi nhận hết dữ liệu mong muốn thì thiết bị chủ tạo tín hiệu STOP để chấm dứt.

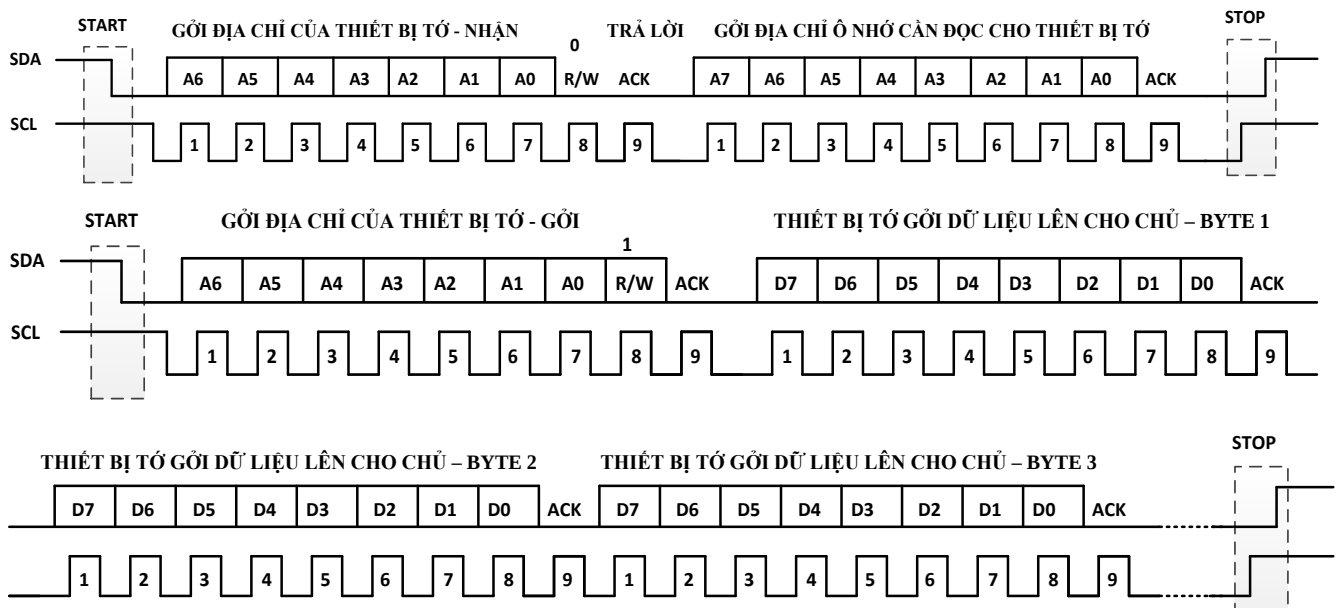
8.1.4 Dạng sóng truyền dữ liệu chuẩn I2C

Dạng sóng truyền dữ liệu I2C của quá trình ghi dữ liệu từ chủ lên tớ như hình 8-2.



Hình 8-2. Quá trình chủ ghi dữ liệu vào tớ.

Dạng sóng truyền dữ liệu I2C của quá trình đọc dữ liệu từ tớ như hình 8-3.



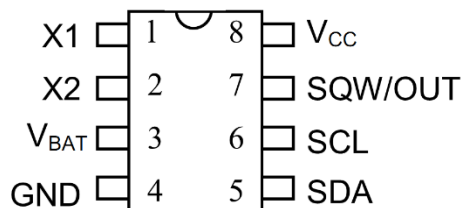
Hình 8-3. Quá trình chủ đọc dữ liệu từ tớ.

8.2 KHẢO SÁT IC REALTIME DS13B07

8.2.1 Khảo sát IC DS1307

DS1307 là chip đồng hồ thời gian thực của Dallas Semiconductor. Chip có 64 ô nhớ, trong đó có 8 ô nhớ 8-bit lưu thời gian: giây, phút, giờ, thứ (trong tuần), ngày, tháng, năm và thanh ghi điều khiển ngõ ra, vùng nhớ còn lại là 56 ô dùng có thể dùng để lưu dữ liệu.

DS1307 giao tiếp theo chuẩn nối tiếp I2C nên sơ đồ chân bên ngoài chỉ có 8 chân.



Hình 8-4. Sơ đồ chân DS1307.

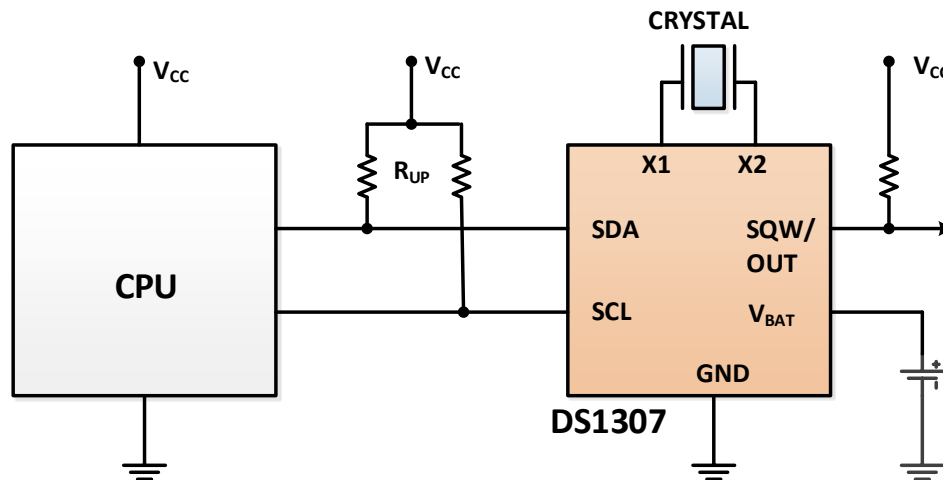
Các chân của DS1307:

- Hai chân **X1** và **X2**: nối với tụ thạch anh có tần số 32768Hz để tạo dao động.
- Chân **V_{BAT}** nối với nguồn dương của pin 3V3 để chip vẫn hoạt động đếm thời gian khi mất nguồn.
- Chân **GND** là chân nối 0V.
- Chân **V_{CC}** nối với nguồn V_{CC} có thể là 5V hoặc 3V3 tùy thuộc vào từng loại chip.
- Chân **SQW/OUT** là ngõ ra tạo xung vuông (Square Wave / Output Driver), tần số có thể lập trình, tích cực mức 0, chế độ nhận dòng.
- Hai chân **SCL**, **SDA** là 2 đường tín hiệu giao tiếp chuẩn I2C.

Tổ chức bộ nhớ bên trong của RT DS1307 như hình 8-6.

Vì 7 thanh ghi đầu tiên là quan trọng nhất trong hoạt động của DS1307 nên ta quan sát chi tiết tổ chức theo từng bit của các thanh ghi này như trong hình 8-7.

Ô nhớ lưu giây (SECONDS): có địa chỉ là 0×00, có chức năng lưu hàng chục giây và hàng đơn vị giây. Bit thứ 7 có tên CH (Clock halt – ngừng đồng hồ), nếu bit này bằng 1 thì bộ dao động trong chip ngừng dao động làm đồng hồ ngừng hoạt động. Nếu muốn đồng hồ hoạt động thì bit này phải bằng 0.



Hình 8-5. Sơ đồ kết nối vi điều khiển với DS1307

ADDRESS

00H	SECONDS
01H	MINUTES
02H	HOURS
03H	DAY
04H	DATE
05H	MONTHS
06H	YEARS
07H	CONTROL
08H	RAM 56 BYTE
3FH	

Hình 8-6. Tổ chức bộ nhớ của DS1307

								BIT0	
00H	CH	10 SECONDS			SECONDS				00 - 59
01H	0	10 MINUTES			MINUTES				00 - 59
02H	0	12 24	10HR A/P	10 HR	HOURS				01 - 12 00 - 23
03H	0	0	0	0	0	DAY			1 - 7
04H	0	0	10 DATE		DATE				1 - 28, 29, 30, 31
05H	0	0	10 MONTH		MONTH				01 - 12
06H	10 YEAR				YEAR				00 - 99
07H	OUT	0	0	SQWE	0	0	RS1	RS0	

Hình 8-7. Tổ chức các thanh ghi thời gian

Ô nhớ lưu phút (MINUTES): có địa chỉ 0x01, có chức năng lưu phút hàng đơn vị và hàng chục, bit 7 luôn bằng 0.

Ô nhớ lưu giờ (HOURS): có địa chỉ 0x02 có chức năng lưu hàng chục giờ và hàng đơn vị giờ ở 2 chế độ 12 giờ và 24 giờ được lựa chọn bởi bit thứ 6 có tên là 12/24.

Nếu bit 12/24 chọn chế độ 24 giờ thì phần hàng chục giờ sử dụng 2 bit thứ 4 và thứ 5 có kí hiệu là 10HR.

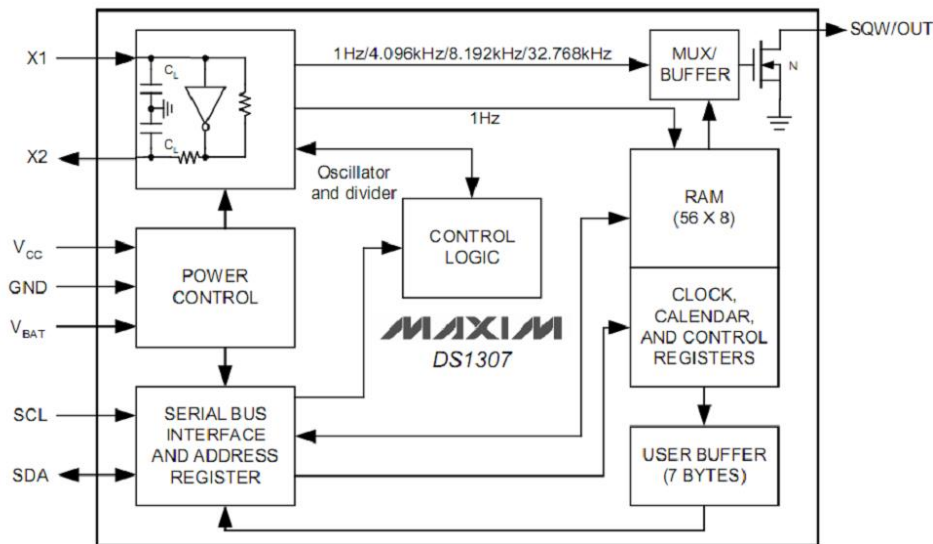
Nếu bit 12/24 chọn chế độ 12 giờ thì phần hàng chục giờ sử dụng bit thứ 4, còn bit thứ 5 sẽ có kí hiệu là A/P tương ứng với 2 chế độ giờ AM và PM.

Bit 7 luôn bằng 0.

Ô nhớ lưu thứ (DAY – ngày trong tuần): có địa chỉ 0x03, có chức năng lưu thứ trong tuần có giá trị từ 1 đến 7 tương ứng từ Chủ nhật đến thứ bảy trong tuần, chỉ sử dụng 3 bit thấp.

Các ô còn lại là ngày tháng năm.

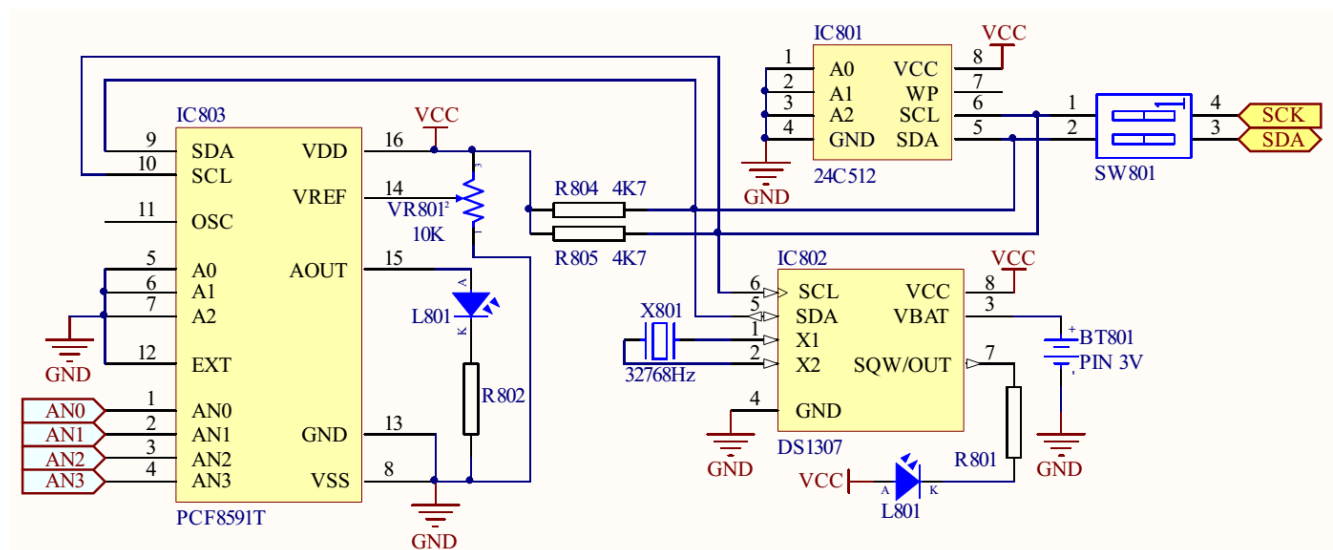
Cấu trúc bên trong của DS1307 như hình 8-8.

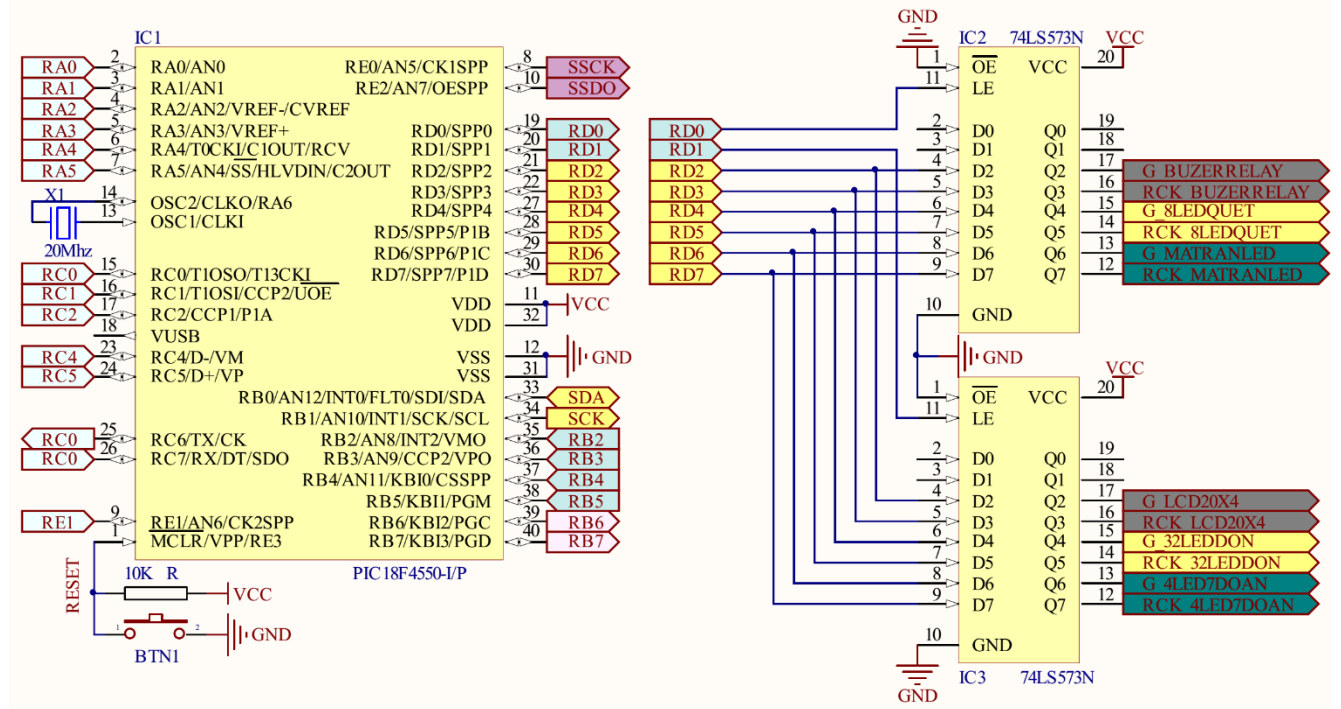


Hình 8-8. Cấu trúc bên trong DS1307

8.2.2 Sơ đồ giao tiếp vi điều khiển với DS1307

Trong phần này sẽ trình bày phần giao tiếp vi điều khiển PIC18F4550 với I2C như hình 8-9:





Hình 8-9. Giao tiếp vi điều khiển các thiết bị theo chuẩn I2C

Khởi dùng chuẩn I2C giao tiếp với vi điều khiển thông qua 2 chân: RB0/SDA và RB1/SCL, khi sử dụng thì chuyển 2 switch con của SW23 sang vị trí ON.

8.2.3 Các chương trình giao tiếp DS13B07

Thư viện I2C của DS1307. Chương trình thư viện của IC thời gian thực đã cho trong thư mục của bạn có tên là “TV_DS1307.c” và “TV_DS1307.h”.

Bạn có thể sử dụng cho các bài thực hành và nên đọc hiểu.

File “TV_DS1307.h”

```
#ifndef __TV_DS1307_H__
#define __TV_DS1307_H__
typedef struct
{
    unsigned int8 second;
    unsigned int8 minute;
    unsigned int8 hour;
    unsigned int8 day_of_week;
    unsigned int8 date;
    unsigned int8 month;
    unsigned int8 year;
}DS1307_STRUCT;
DS1307_STRUCT ds1307={0,0,0,0,0,0,0};
result ds1307_read_time(void);
result ds1307_set_time(DS1307_STRUCT t);
result ds1307_battery_check();
#endif
```

File “TV_DS1307.c”

```

#ifndef __TV_DS1307_C__
#define __TV_DS1307_C__
#include<TV_DS1307.h>
/* Hàm đọc thời gian từ DS1307
   - Thời gian đọc về sẽ lưu trong các biến ds1307.minute, ..., ds1307.year
   - Hàm trả về "Ok" khi đọc hoàn tất.
   - Hàm trả về "ERROR" khi quá trình đọc bị lỗi
*/
result ds1307_read_time()
{
    static int8 old_minute=0xff;
    i2c_start();
    if(i2c_write(0xd0)==ERROR) return ERROR; // Bị lỗi thì trả về ERROR
    i2c_write(0);
    i2c_start();
    i2c_write(0xd1);
    ds1307.second = i2c_read(1);
    if(ds1307.minute!=old_minute) // 1 phút mới đọc 1 lần
    {
        ds1307.minute = i2c_read(1);
        ds1307.hour = i2c_read(1);
        ds1307.day_of_week = i2c_read(1);
        ds1307.date = i2c_read(1);
        ds1307.month = i2c_read(1);
        ds1307.year = i2c_read(0);
        old_minute = ds1307.minute;
    }
    else ds1307.minute = i2c_read(0);
    i2c_stop();
    return OK;
}
/* Hàm cài đặt thời gian cho DS1307
   - DS1307_STRUCT t : là thời gian cài đặt dạng số BCD
     (ds1307.minute = 0x38; có nghĩa là cài đặt phút là 38)
   - Hàm trả về "Ok" khi cài đặt hoàn tất
   - Hàm trả về "ERROR" khi cài đặt bị lỗi
*/
result ds1307_set_time(DS1307_STRUCT t)
{
    i2c_start();
    if(i2c_write(0xd0)==ERROR) return ERROR;
    i2c_write(0);
    i2c_write(t.second);
    i2c_write(t.minute);
    i2c_write(t.hour);
    i2c_write(t.day_of_week);
    i2c_write(t.date);
    i2c_write(t.month);
    i2c_write(t.year);
    i2c_stop();
    return OK;
}

```

```

/* Hàm kiểm tra cục PIN của DS1307 còn hay hết
- Hàm trả về "Ok" khi PIN còn
- Hàm trả về "ERROR" khi PIN hết
*/
result ds1307_battery_check()
{
    if(ds1307_read_time()==OK)
    {
        if(ds1307.second>0x59) return ERROR;
        else return OK;
    }
    return ERROR;
}
#endif

```

Bài mẫu 801. Chương trình đọc giờ, phút, giây hiện tại và hiển thị trên LCD 20×4.

Chú ý: Kiểm tra công tắc SDA, SCL phải ở vị trí ON

Lưu tên file là “BAI_801_DS1307_LCD ”

- Mục đích: biết viết chương trình đọc thời gian thực hiển thị trên LCD.
- Lưu đồ:
- Chương trình:

```

#include<tv_boards.c>
#include<tv_lcd.c>
#include<tv_ds1307.c>
void main()
{
    system_init();
    lcd_setup();
    lcd_command(0x80+4);
    lcd_data("DS1307 CLOCK");
    // Nếu hết PIN hoặc mới chạy lần đầu tiên thì phải cài thời gian lại
    if(ds1307_battery_check()==ERROR)
    {
        ds1307.second=0x00; // Chính cho đúng giây hiện tại
        ds1307.minute=0x00; // Chính cho đúng phút hiện tại
        ds1307.hour =0x00; // Chính cho đúng giờ hiện tại
        ds1307_set_time(ds1307); // Nạp vào DS1307
    }
    while(true)
    {
        if(ds1307_read_time()==OK) // Nếu đọc xong thì hiển thị
        {
            lcd_command(0xc0+6);
            printf(lcd_data,"%d%d:%d%d:%d%d",
                ds1307.hour/16, ds1307.hour%16, // Chia cho 16
                ds1307.minute/16,ds1307.minute%16, // vì là số BCD
                ds1307.second/16,ds1307.second%16);
        }
    }
}

```


- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: Nếu thời gian không đúng hiện tại thì chỉnh lại trong chương.
- f. Giải thích chương trình.

Bài mẫu 802. Chương trình đọc giờ, phút, giây hiện tại và hiển thị trên LCD 20×4, có 3 nút chỉnh thời gian giờ, phút, giây là UP, DW và MOD.

Lưu tên file là “BAI_802_DS1307_LCD_CHINHTG”

- a. Mục đích: biết viết thêm phần chỉnh thời gian thực gồm giờ phút giây.
- b. Lưu đồ:
- c. Chương trình:

```
#include<tv_boards.c>
#include<tv_lcd.c>
#include<tv_ds1307.c>
unsigned int8 mode=0;
// Trường hợp tăng 0x09 lên 1 thành 0x0A nên phải +6 để thành 0x10
// Trường hợp giảm 0x10 đi 1 thành 0x0F nên phải -6 để thành 0x09
unsigned int8 hieuchinhBCD(unsigned int8 x)
{
    if((x&0x0f)==0x0a) x+= 6;
    if((x&0x0f)==0x0f) x-= 6;
    return x;
}
void hieuchinhBCDvaNapthoigian() // Nạp thời gian cài đặt vào DS1307
{
    ds1307.second= hieuchinhbcd(ds1307.second);
    ds1307.minute= hieuchinhbcd(ds1307.minute);
    ds1307.hour = hieuchinhbcd(ds1307.hour);
    ds1307_set_time(ds1307);
}
void mode_display() // Hiển thị dấu"^" dưới cái đang chỉnh
{
    lcd_command(0x94+6);
    lcd_data(" "); // Xóa dấu "^" cũ đi
    if(mode>0)
    {
        lcd_command(0x94+3+mode*3); // Hiển thị dấu "^" tại vị trí mới
        lcd_data("^");
    }
}
void main()
{
    system_init();
    lcd_setup();
    lcd_command(0x80+4);
    lcd_data("DS1307 CLOCK");
    while(true)
    {
        // + mode =0: Không chỉnh
        // + mode=1: Chỉnh giờ
        // + mode=2: Chỉnh phút
    }
}
```

```

// + mode =3: Chỉnh giây
if(inputcd(MOD)==0){mode++; mode%=4;mode_display();}
if(inputcd(UP)==0) // Nếu nhấn nút UP thì tăng thời gian
{
    if((mode==3)&&(ds1307.second<0x59)) ds1307.second++;
    if((mode==2)&&(ds1307.minute<0x59)) ds1307.minute++;
    if((mode==1)&&(ds1307.hour <0x23)) ds1307.hour++;
    hieuchinhBCDvaNapthoigian();
}
if(inputcd(DW)==0) // Nếu nhấn nút DW thì giảm thời gian
{
    if((mode==3)&&(ds1307.second>0)) ds1307.second--;
    if((mode==2)&&(ds1307.minute>0)) ds1307.minute--;
    if((mode==1)&&(ds1307.hour >0)) ds1307.hour--;
    hieuchinhBCDvaNapthoigian();
}
if(ds1307_read_time()==OK) // Đọc thời gian và hiển thị
{
    lcd_command(0xc0+6);
    printf(lcd_data,"%d%d:%d%d:%d%d",
           ds1307.hour/16, ds1307.hour%16,
           ds1307.minute/16,ds1307.minute%16,
           ds1307.second/16,ds1307.second%16);
}
}
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: nếu kết quả không đúng yêu cầu thì kiểm tra lại chương trình.
- f. Giải thích chương trình:

Bài mẫu 803. Thêm vào bài 802 phần hiệu chỉnh ngày, tháng và năm.

Lưu tên file là “BAI_803_DS1307_LCD_CHINH_FULL”

Bài tập 804. Hãy hiệu chỉnh chương trình trên sao cho khi hiệu chỉnh đối tượng nào thì đối tượng đó nhấp nháy thay vì hiển thị dấu “^” bên dưới. Ví dụ chỉnh giờ thì giờ nhấp nháy, giải thuật là cho đối tượng đó sáng $\frac{1}{2}$ giây rồi tắt $\frac{1}{2}$ giây còn lại.

Lưu tên file là “BAI_804_DS1307_LCD_3KEY_NN”

Bài tập 805. Hiệu chỉnh bài 804 sao cho khi phút tăng lên 1 thì buzzer kêu bíp, thời gian mở buzzer là 1 giây hoặc 2 giây.

Lưu tên file là “BAI_805_DS1307_LCD_3KEY_BUZZER”

Bài tập 806. Hiệu chỉnh bài 804 có thêm chức năng hẹn giờ tắt mở thiết bị với yêu cầu như sau: thêm hàng thứ 3 hiển thị thời gian hẹn chỉ có giờ phút giây, sử dụng các phím

MOD, UP và DW để cài đặt thời gian tắt mở thiết bị, khi đúng giờ thì mở buzzer, sau 5 giây thì tắt buzzer.

Lưu tên file là “BAI_806_I2C_DS1307_3KEY_ALARM”

Bài tập 807. Hiệu chỉnh bài 805 có thêm chức năng reng chuông giờ học, với yêu cầu như sau:

Các thông số reng chuông (dùng buzzer thay cho chuông):

07-00-00 reng 1 hồi chuông, 00 giây mở chuông, 03 giây tắt chuông.

07-50-00 reng 1 hồi chuông, 00 giây mở chuông, 03 giây tắt chuông.

08-40-00 reng 1 hồi chuông, 00 giây mở chuông, 03 giây tắt chuông.

Hàng thứ 3 hiển thị thời gian sắp đổ chuông chỉ có giờ phút giây.

Lưu tên file là “BAI_807_I2C_RT_LCD_3KEY_CHUONG”

Chú ý: các thông số thời gian bạn có thể thay đổi thành ngắn để không phải chờ đợi quá lâu như sau:

07-00-00 reng 1 hồi chuông, 00 giây mở chuông, 03 giây tắt chuông.

07-02-00 reng 1 hồi chuông, 00 giây mở chuông, 03 giây tắt chuông.

07-04-00 reng 1 hồi chuông, 00 giây mở chuông, 03 giây tắt chuông.

Bạn có thể thêm nhiều mốc thời gian để reng chuông, có thể thay đổi nhiều hồi reng chuông khác nhau.

Bài tập 808. Hãy viết chương trình hiển thị giờ phút giây bằng số có kích thước lớn đã làm ở chương LCD.

Lưu tên file là “BAI_808_DS1307_LCD_TO”

Bài tập 809. Hãy viết chương trình hiển thị giờ phút giây bằng số có kích thước bình thường và kích thước lớn đã làm ở chương LCD. Có 1 nút để chuyển chế độ hiển thị bình thường và hiển thị kích thước lớn.

Lưu tên file là “BAI_809_DS1307_LCD_TO_NHO”

8.3 KHẢO SÁT IC PCF8591

8.3.1 Giới thiệu IC PCF8591

Ở chương 7 chúng ta đã thực hành các bài chuyển đổi tương tự sang số sử dụng các bộ chuyển đổi tích hợp bên trong vi điều khiển. Ở chương này chúng ta thực tập các bài chuyển đổi

tương tự sang số và số sang tương tự do IC PCF8591 thực hiện, IC này giao tiếp với vi điều khiển theo chuẩn I2C.

IC PCF8591 là một chip CMOS thu thập dữ liệu đơn dùng nguồn cung cấp đơn công suất thấp 8-bit với 4 ngõ vào tương tự và một ngõ ra tương tự giao tiếp nối tiếp I2C.

Ba chân địa chỉ A0, A1 và A2 được sử dụng cho lập trình địa chỉ phần cứng, cho phép mở rộng lên đến 8 thiết bị kết nối với theo chuẩn bus I2C mà không cần thêm phần cứng. Địa chỉ, điều khiển và dữ liệu đến và đi từ thiết bị sẽ được chuyển nối tiếp thông qua 2 dây của bus I2C.

Các chức năng của thiết bị bao gồm ghép kênh các ngõ vào tương tự, chuyển đổi tương tự sang số 8 bit, chức năng theo dõi và giữ. Tốc độ chuyển đổi tối đa được cho bởi tốc độ tối đa của bus I2C.

8.3.2 Cấu hình IC PCF8591

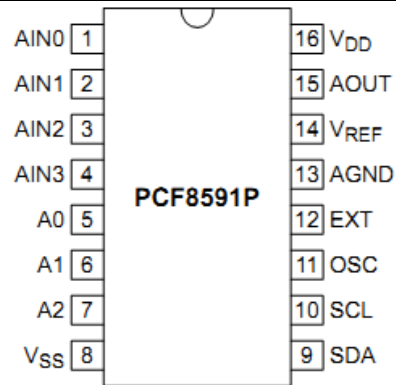
- Nguồn cung cấp là nguồn đơn từ 2,5V đến 6V.
- Dòng điện không sử dụng rất thấp.
- Dữ liệu vào và ra nối tiếp theo chuẩn I2C.
- Lựa chọn địa chỉ I2C bởi 3 chân địa chỉ phần cứng.
- Tốc độ lấy mẫu cao nhất bằng tốc độ I2C.
- Có 4 kênh tương tự ngõ vào có thể cấu hình như ngõ vào đơn cực hoặc vi sai.
- Có thể lựa chọn kênh tự động tăng.
- Điện áp tương tự từ V_{SS} đến V_{DD} .
- Có mạch điện theo dõi và giữ trên chip.
- Chuyển đổi A/D sắp xỉ liên tiếp 8 bit.
- Có 1 bộ DAC với 1 ngõ ra tương tự.

8.3.3 Ứng dụng IC PCF8591

- Dùng để giám sát nguồn.
- Thiết lập các tham chiếu.
- Vòng điều khiển tương tự.

8.3.4 Sơ đồ chân IC PCF8591

Sơ đồ chân IC như hình 8-11:



Hình 8-10. Sơ đồ khối IC PCF8591.

Chức năng từng chân như bảng 8-1.

Bảng 8-1. Tên và chức năng từng chân của IC PCF8591.

TT	Tên chân	Chân	Chức năng
1	AIN0	1	Ngõ vào tương tự thứ 0 (analog inputs A/D converter)
2	AIN1	2	Ngõ vào tương tự thứ 1 (analog inputs A/D converter)
3	AIN2	3	Ngõ vào tương tự thứ 2 (analog inputs A/D converter)
4	AIN3	4	Ngõ vào tương tự thứ 3 (analog inputs A/D converter)
5	A0	5	Ngõ vào thiết lập địa chỉ thiết bị (hardware slave address)
6	A1	6	Ngõ vào thiết lập địa chỉ thiết bị (hardware slave address)
7	A2	7	Ngõ vào thiết lập địa chỉ thiết bị (hardware slave address)
8	VSS	8	Chân nối mass (ground supply voltage)
9	SDA	9	I2C- bus serial data input and output
10	SCL	10	I2C-bus serial clock input
11	OSC	11	Ngõ vào ra của dao động (oscillator input/output)
12	EXT	12	Ngõ vào dao động lựa chọn (external/internal switch for oscillator input)
13	AGND	13	Chân nối mass của tín hiệu tương tự (analog ground supply)
14	VREF	14	Ngõ vào nối điện áp tham chiếu (voltage reference input)
15	AOOUT	15	Ngõ ra tín hiệu tương tự của DA (analog output (D/A converter))

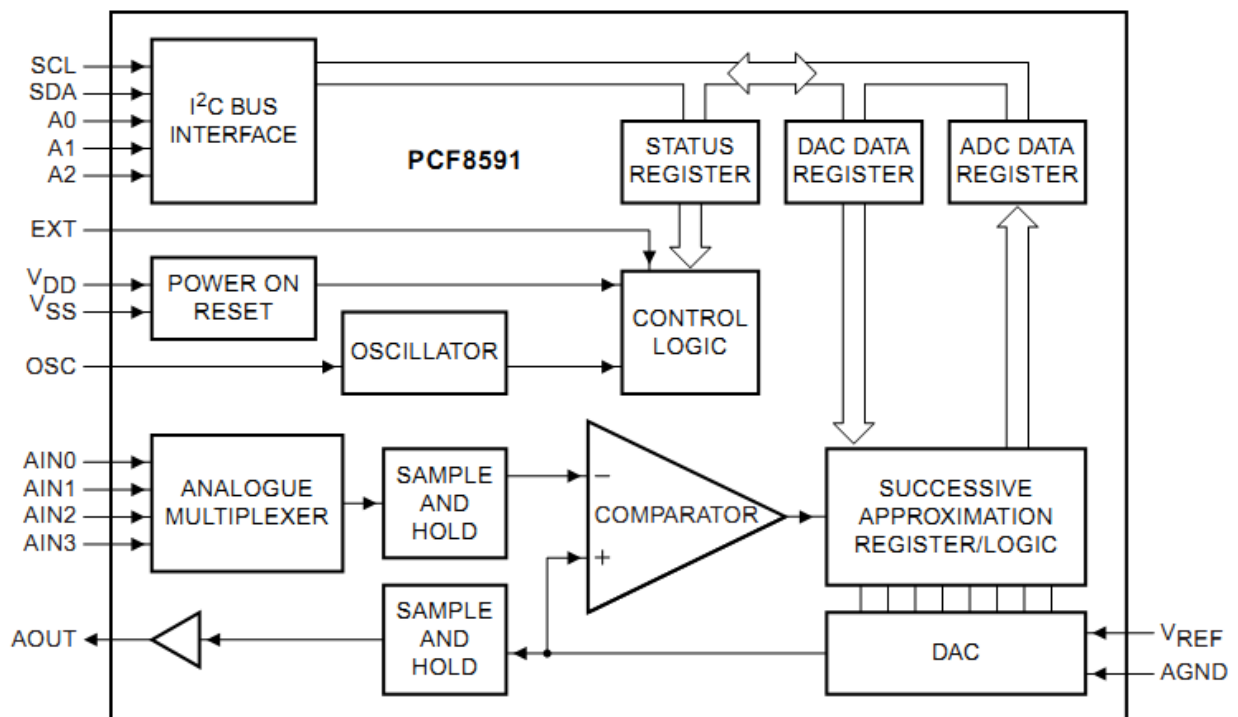
16	VDD	16	Nguồn cung cấp (supply voltage)
----	-----	----	---------------------------------

8.3.5 Sơ đồ khối IC PCF8591

Sơ đồ khối IC như hình 8-12:

Cấu trúc của IC bao gồm các khối:

- Khối giao tiếp chuẩn I2C.
- Khối nguồn cung cấp và reset
- Khối dao động nội
- Khối điều khiển logic
- Thanh ghi trạng thái
- Thanh ghi dữ liệu DAC
- Thanh ghi dữ liệu ADC
- Khối đa hợp 4 kênh tương tự
- Khối ADC-DAC bao gồm hai khối lấy mẫu và giữ cho hai ngõ vào, bộ so sánh, bộ chuyển đổi DAC và thanh ghi xấp xỉ liên tiếp, nguồn điện áp tham chiếu.
- Khối khuếch đại đệm tín hiệu tương tự.



Hình 8-11. Sơ đồ khối IC PCF8591.

8.3.6 Địa chỉ ngoài IC PCF8591

Mỗi thiết bị PCF8591 trong hệ thống bus I2C được kích hoạt bằng cách gửi đúng địa chỉ cho thiết bị. Địa chỉ bao gồm một phần cố định và một phần lập trình. Phần lập trình được thiết lập bởi các chân địa chỉ A0, A1 và A2.

Địa chỉ luôn là byte đầu tiên được gửi sau khi thực hiện xong điều kiện **start** trong giao thức bus I2C. Trong byte địa chỉ thì bit cuối cùng là bit xác nhận đọc / ghi.

8.3.7 Thanh ghi điều khiển của IC PCF8591

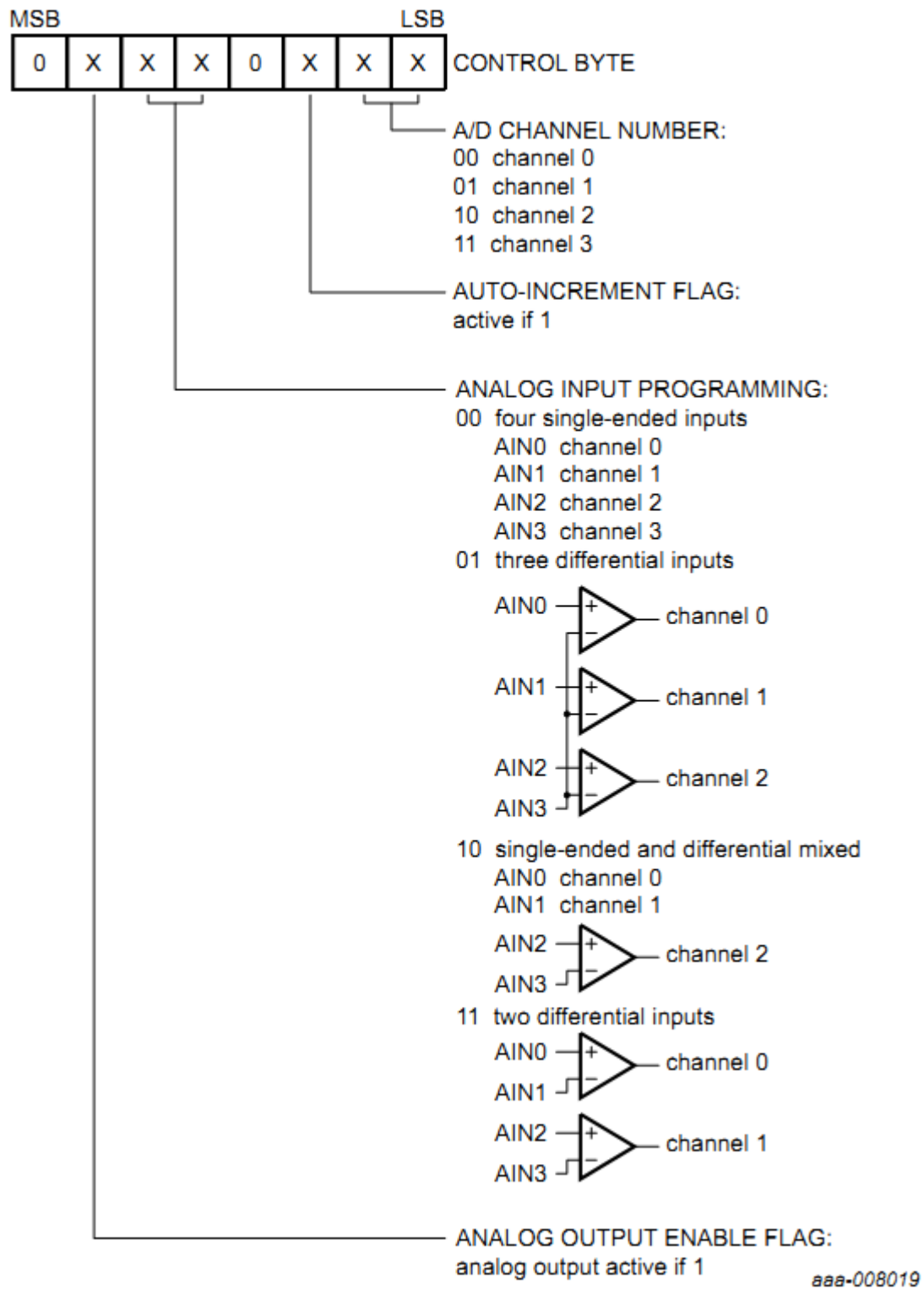
Byte thứ hai gửi đến IC PCF8591 được lưu trữ trong thanh ghi điều khiển (control register) để điều khiển các chức năng của thiết bị như hình 8-13.

Bốn bit cao của thanh ghi điều khiển được sử dụng để thiết lập:

- Bit thứ 7 luôn bằng 0
- Bit thứ 6 dùng để cho phép xuất tín hiệu tương tự ở ngõ ra tương tự - hoạt động DAC
- Bit thứ 5 và 4 dùng để thiết lập cho các chế độ ngõ vào tương tự như chế độ ngõ vào duy nhất hoặc chế độ vi sai ngõ vào.

Bốn bit thấp:

- Bit thứ 3 luôn bằng 0
- Bit thứ 2 là bit cho phép tự động tăng kênh nếu bằng 1, kênh chọn tự động tăng lên sau mỗi lần chuyển đổi xong.
- Bit thứ 1 và 0 dùng để chọn 1 trong 4 kênh khi thực hiện đọc dữ liệu.



Hình 8-12. Cấu trúc thanh ghi điều khiển của IC PCF8591.

Nếu chế độ tự động tăng được yêu cầu trong các ứng dụng thì phải sử dụng bộ dao động nội bộ và cờ cho phép ngõ ra tương tự phải bằng 1 (bit thứ 6 của thanh ghi điều khiển). Điều này cho phép bộ dao động nội bộ chạy liên tục. Cách này sẽ ngăn chặn lỗi chuyển đổi do dao động

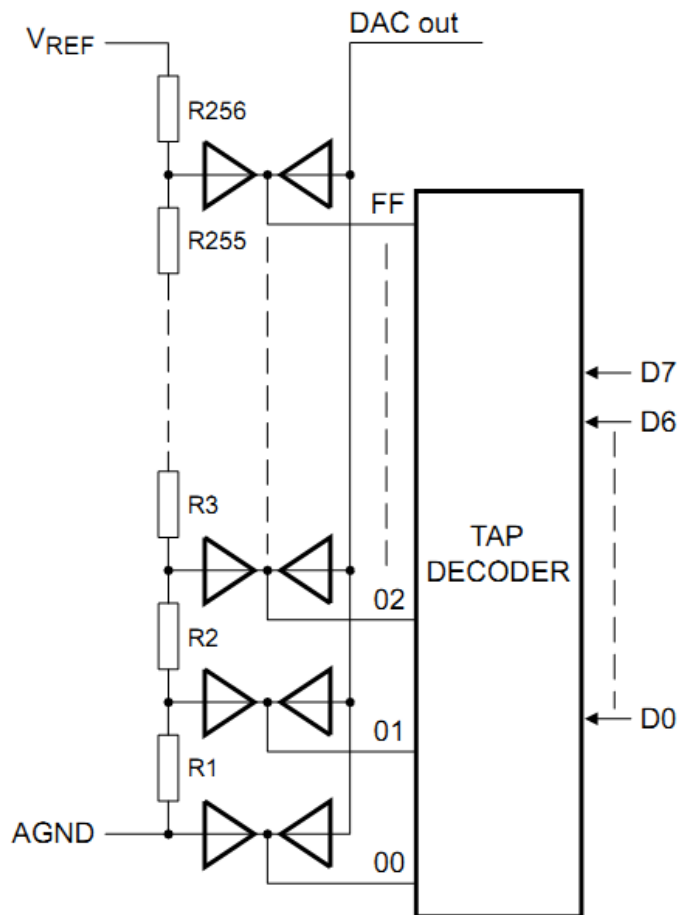
khởi động chậm trễ gây ra. Cờ cho phép ngõ ra tương tự có thể reset ở thời điểm khác để giảm công suất tiêu thụ điện khi không còn chuyển đổi.

Bit có trọng số cao nhất được dành riêng cho chức năng trong tương lai có thể và được thiết lập ở mức logic 0. Sau khi cấp điện thì mạch reset khi có điện POR sẽ reset tất cả các bit của thanh ghi điều khiển và tắt dao động để tiết kiệm năng lượng và ngõ ra tín hiệu tương tự ở trạng thái tổng trở cao.

8.3.8 Chuyển đổi D/A của IC PCF8591

Byte thứ ba được gửi đến IC PCF8591 sẽ lưu trữ trong thanh ghi dữ liệu DAC và được chuyển đổi thành điện áp tương tự bởi bộ chuyển đổi DAC.

Chuyển đổi DA bao gồm một chuỗi cầu điện trở mắc nối tiếp và nối với điện áp tham chiếu bên ngoài tạo thành 256 cấp điện áp khác nhau và switch lựa chọn. Bộ giải mã sẽ chuyển mạch cho một trong 256 cấp điện áp này đến ngõ ra DAC – xem hình 8-14.

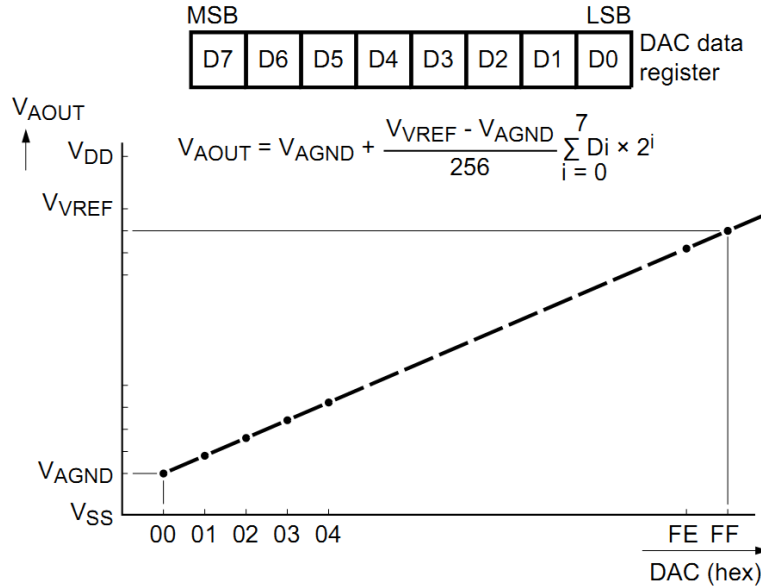


Hình 8-13. Cấu trúc khối DAC của IC PCF8591.

Điện áp đầu ra tương tự được đệm bởi một bộ khuếch đại tự động hệ số KĐ bằng 1. Thiết lập bit cho phép xuất ngõ ra tương tự trong thanh ghi điều khiển để tắt mở bộ đệm này. Trong trạng thái hoạt động, điện áp đầu ra được giữ cho đến khi một byte dữ liệu tiếp tục được gửi đi để chuyển đổi tiếp.

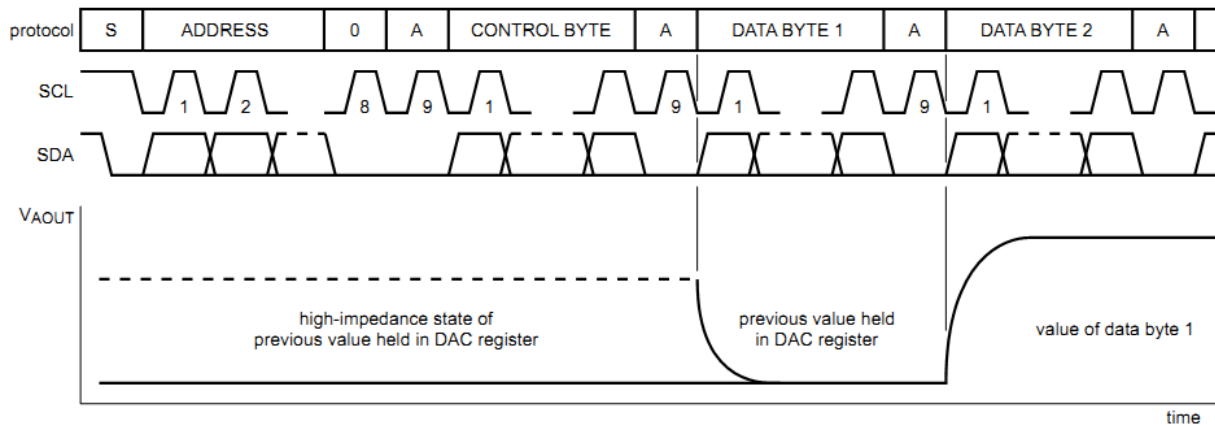
Bộ chuyển đổi DA trên chip cũng được sử dụng để phục vụ cho chuyển đổi AD liên tiếp xấp xỉ. Để khỏi giải thoát khỏi DA phục vụ cho chuyển đổi AD thì mạch khuếch đại có độ lợi bằng 1 cần phải có thêm mạch theo dõi và giữ để giữ điện áp ngõ ra tương tự trong lúc thực hiện chuyển đổi AD.

Công thức cho điện áp ngõ ra được thể hiện trong hình 8-15:



Hình 8-14. Công thức điện áp ra và đặc tính chuyển đổi.

Các dạng sóng của chuyển đổi DA theo trình tự được thể hiện trong hình 8-16.



Hình 8-15. Dạng sóng chuyển đổi D/A.

8.3.9 Chuyển đổi A/D của IC PCF8591

Chuyển đổi A/D sử dụng kỹ thuật chuyển đổi xấp xỉ liên tiếp. Bộ chuyển đổi D/A chuyển đổi và bộ so sánh độ lợi cao được sử dụng tạm thời trong chu kỳ chuyển đổi A/D.

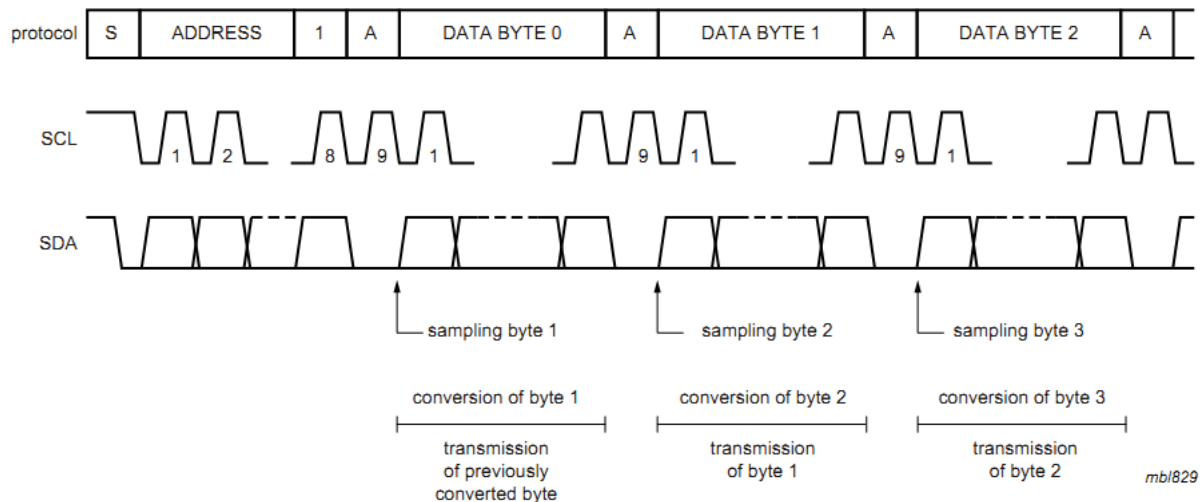
Một chu kỳ chuyển đổi A/D được bắt đầu sau khi nhận địa chỉ hợp lệ ở chế độ đọc gửi đến IC PCF8591. Chu kỳ chuyển đổi A/D được kích hoạt khi xuất hiện cạnh của xung đồng hồ trả lời (acknowledge) và thực hiện chuyển đổi trong khi đó vẫn thực hiện truyền kết quả của chuyển đổi trước đó, xem hình 8-17.

Kết quả chuyển đổi được lưu trong thanh ghi dữ liệu ADC và chờ truyền đi. Nếu cờ tự động tăng cờ được thiết lập, các kênh tiếp theo được chọn. Byte đầu tiên truyền đi trong chu kỳ đọc có chứa mã kết quả chuyển đổi của chu kỳ đọc trước.

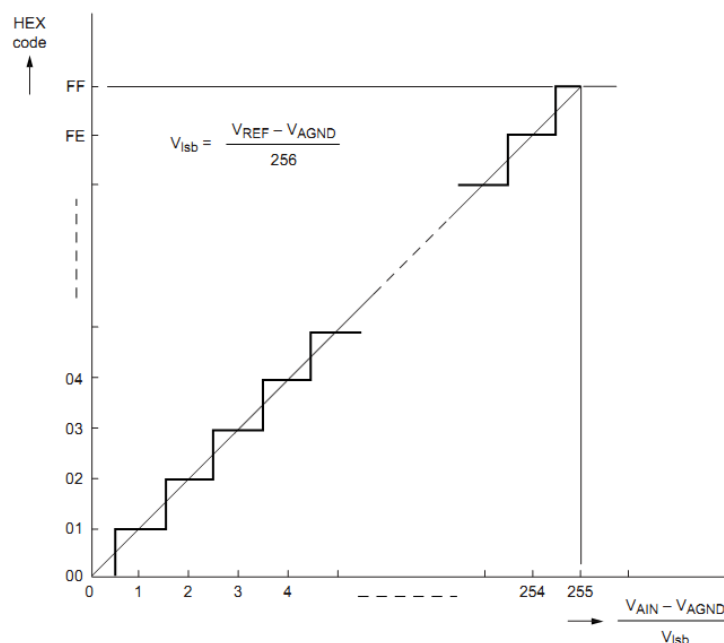
Sau điều kiện POR, các byte đầu tiên đọc có giá trị là 80h.

Tốc độ tối đa của chuyển đổi A/D bằng với tốc độ thực tế của bus I2C.

Dạng sóng kết quả chuyển đổi theo chế độ ngõ vào đơn cực như hình 8-18.

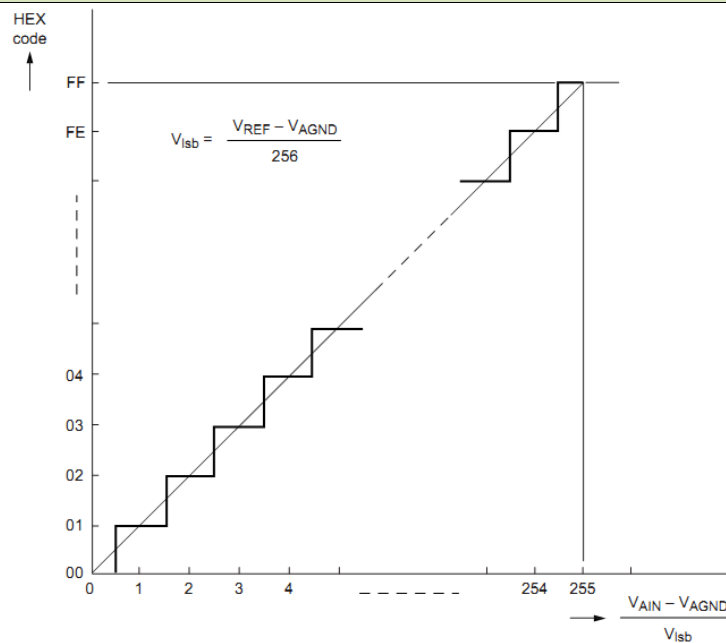


Hình 8-16. Dạng sóng truyền dữ liệu của IC PCF8591.



Hình 8-17. Dạng sóng chuyển đổi A/D ở chế độ đơn cực.

Dạng sóng kết quả chuyển đổi theo chế độ vi sai ngõ vào như hình 8-19:



Hình 8-18. Dạng sóng chuyển đổi A/D ở chế độ ngõ vào vi sai.

8.3.10 Điện áp tham chiếu của IC PCF8591

Cả hai chuyển đổi D/A và A/D đều sử dụng điện áp tham chiếu bên ngoài cấp cho các chân VREF và AGND. Chân AGND được nối với mass tương tự hoặc có thể nối với nguồn điện áp DC để chỉnh điện áp nếu bị lệch.

8.3.11 Bộ dao động của IC PCF8591

Bộ dao động trên chip tạo ra các tín hiệu đồng hồ cần thiết cho chuyển đổi A/D và làm tươi bộ khuếch đại đệm. Khi sử dụng dao động này chân EXT phải nối với VSS (GND). Tần số dao động xuất hiện tại chân OSC.

Nếu chân EXT kết nối với VDD thì ngõ ra dao động OSC sẽ chuyển sang trạng thái trở kháng cao cho phép nhận tín hiệu đồng hồ từ bên ngoài đưa đến chân OSC.

8.3.12 Địa chỉ thanh ghi của IC PCF8591

Byte địa chỉ của IC PCF8591 như hình 8-20:

Bit	Slave address							0
	7	6	5	4	3	2	1	
	MSB							LSB
slave address	1	0	0	1	A2	A1	A0	R/W

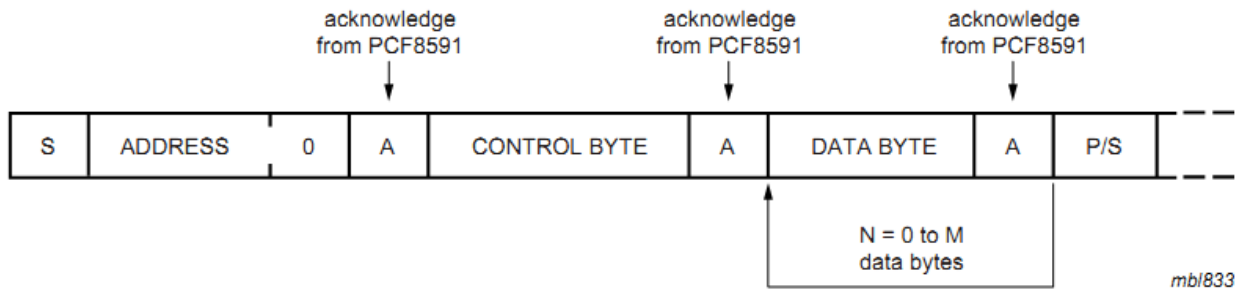
Hình 8-19. Địa chỉ của IC PCF8591.

Tín hiệu đọc/ghi như hình 8-21:

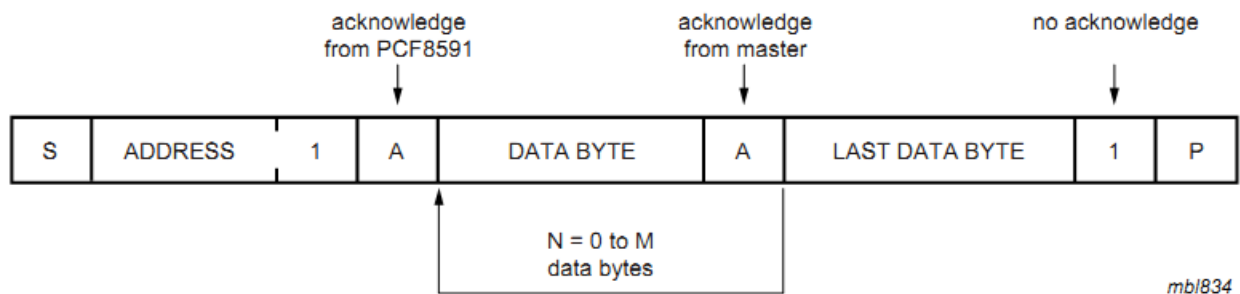
R/W	Description
0	write data
1	read data

Hình 8-20. Tín hiệu đọc ghi.

Chế độ ghi dữ liệu vào IC PCF 8591 như hình 8-22:

**Hình 8-21. Chế độ ghi dữ liệu vào IC PCF8591.**

Chế độ đọc dữ liệu từ IC PCF 8591 như hình 8-23:

**Hình 8-22. Chế độ đọc dữ liệu từ IC PCF8591.**

Về giao tiếp chuẩn truyền dữ liệu I2C thì đã trình bày chi tiết ở phần giao tiếp với IC thời gian thực DS13B07 nên ở đây không trình bày lại.

8.3.13 Giao tiếp vi điều khiển với IC PCF8591

Hình 8-9 ở trên, trình bày kết nối với vi điều khiển theo chuẩn I2C giống các thiết bị khác.

Địa chỉ của IC PCF 8591 trong bộ thực hành là 0x90 đối với lệnh ghi và 0x91 đối với lệnh đọc vì 3 chân thiết lập địa chỉ cứng nối mass.

8.3.14 Các chương trình giao tiếp IC PCF 8591

Độ phân giải của IC PCF8591 được điều chỉnh bằng biến trở bên ngoài và đã hiệu chỉnh với độ phân giải là 10mV để tương thích với cảm biến LM35.

Thư viện PCF8591. Chương trình thư viện điều khiển IC PCF8591 theo chuẩn I2C đã cho trong thư mục của bạn có tên là “TV_PCF8591.c” và “TV_PCF8591.h”

Bạn có thể sử dụng cho các bài thực hành và nên đọc hiểu.

File “TV_PCF8591.h”

```
#ifndef __TV_PCF8591_H__
#define __TV_PCF8591_H__
typedef struct
{
    unsigned int8 adc_value[4];
}
```

```

}PCF8591_STRUCT;
PCF8591_STRUCT pcf8591;
result pcf8591_read_adc(int8 channel_0_3);
result pcf8591_write_dac( int8 value);
#endif

```

File “TV_PCF8591.c”

```

#ifndef __TV_PCF8591_C__
#define __TV_PCF8591_C__
#include <TV_PCF8591.h>
/* Hàm đọc kết quả chuyển đổi ADC
- channel_0_3: là kênh cần chuyển đổi giá trị từ 0 đến 3
- pcf8591_adc_value[x]: Chứa kết quả chuyển đổi của các kênh từ 0 đến 3
- Hàm trả về "OK" khi chuyển đổi xong mà không bị lỗi
- Hàm trả về "ERROR" khi hệ thống bị lỗi
*/
result pcf8591_read_adc(unsigned int8 channel_0_3)
{
    const unsigned int8 k[4]= {0x40,0x41,0x42,0x43};
    i2c_start();
    if(i2c_write(0x90)==ERROR) return ERROR;
    i2c_write(k[channel_0_3]);
    i2c_stop();
    delay_us(10);
    i2c_start();
    if(i2c_write(0x91)==ERROR) return ERROR;
    pcf8591_adc_value[channel_0_3] = i2c_read(0);
    i2c_stop();
    delay_us(100);
    return OK;
}
/* Hàm xuất dữ liệu DAC
- value: giá trị DAC cần xuất - số từ 0 đến 255
- Hàm trả về "OK" khi xuất xong mà không bị lỗi
- Hàm trả về "ERROR" khi hệ thống bị lỗi
*/
result pcf8591_write_dac( int8 value)
{
    i2c_start();
    if(soft_i2c_write(0x90)==ERROR) return ERROR;
    i2c_write(0x40);
    i2c_write(value);
    i2c_stop();
    return OK;
}
#endif

```

Bài mẫu 811. Chương trình đo nhiệt độ từ cảm biến LM35A nối với kênh AN0 của IC ADC PCF 8591, hiển thị kết quả đo trên module 4 led 7 đoạn.

Lưu tên file là “BAI_811_PCF8591_4LED_LM35A.C”

- Mục đích: biết lập trình dùng ADC giao tiếp chuẩn I2C để đo nhiệt độ từ cảm biến LM35 hiển thị trên led 7 đoạn.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_pcf8591.c>
void main()
{
    system_init();
    while(true)
    {
        if(pcf8591_read_adc(0)==OK)    // Đọc kết quả chuyển đổi kênh 0
        {
            d7seg.led[1]=m7d[pcf8591_adc_value[0]/10%10]; // hàng chục
            d7seg.led[0]=m7d[pcf8591_adc_value[0]%10];    // đơn vị
            d7seg_display();
        }
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: hiển thị nhiệt độ của cảm biến LM35A trên 2 led 7 đoạn. Có thể mở đèn gia tăng nhiệt độ để thấy sự thay đổi.
- Giải thích chương trình:

Bài tập 812. Hãy thực hiện giống bài 811 nhưng thêm phần báo chuông khi nhiệt độ quá 40 độ C.

Lưu tên file là “BAI_812_PCF8591_4LED_LM35A_BUZ.C”

Bài tập 813. Hãy thực hiện giống bài 812 nhưng đo kênh AN1 nối với cảm biến LM35B và hiển thị trên 2 led 7 đoạn bên trái.

Lưu tên file là “BAI_813_PCF8591_4LED_LM35B_BUZ.C”

Bài tập 814. Hãy viết chương trình tự động đo và khống chế nhiệt độ trong tầm từ 33 đến 35 độ. Khi nhiệt độ nhỏ hơn 33 thì mở đèn triac để gia tăng nhiệt độ. Khi nhiệt độ quá 35 thì tắt đèn triac và báo động bằng buzzer. Khi nhiệt độ nhỏ hơn 35 thì tắt buzzer.

Lưu tên file là “BAI_814_PCF8591_4LED_LM35A_BUZ_DEN.C”

Bài tập 815. Chương trình đo nhiệt độ từ 2 cảm biến LM35A nối với 2 kênh AN0 và AN1 của IC ADC PCF 8591, hiển thị kết quả đo trên module 4 led 7 đoạn, mỗi kênh 2 led. Có dấu “.” ở giữa ngăn cách kết quả đo của 2 cảm biến.

Lưu tên file là “BAI_815_PCF8591_4LED_LM35AB.C”

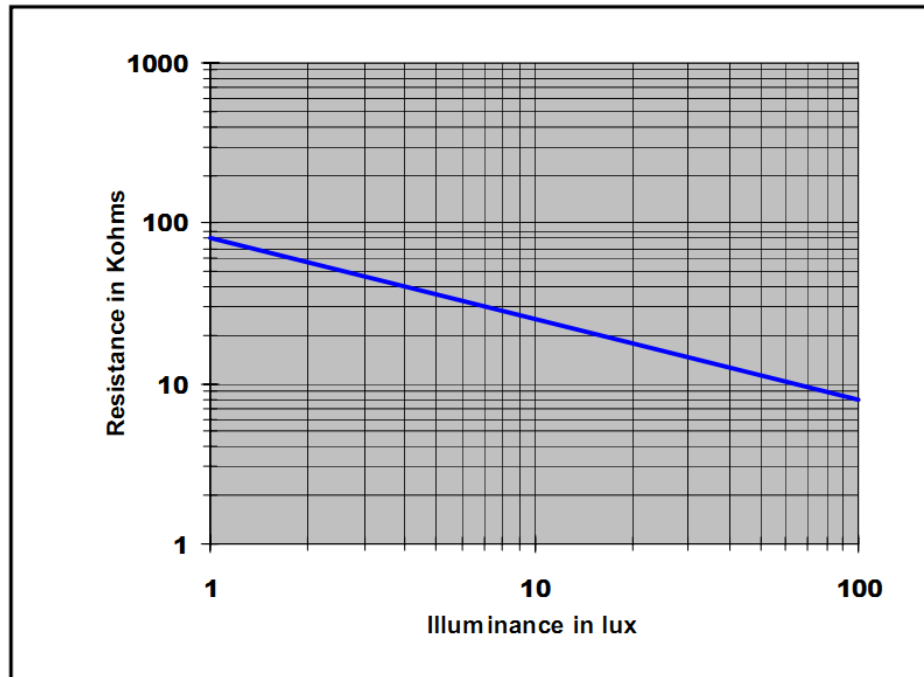
8.4 CẢM BIẾN QUANG TRỞ

8.4.1 Khảo sát quang trở

Trong bộ thực hành có dùng cảm biến quang để chuyển sự thay đổi của ánh sáng thành điện trở và qua cầu điện trở thành sự thay đổi của điện áp.

Mối quan hệ giữa giá trị điện trở và cường độ ánh sáng như hình 8-24.

CELL RESISTANCE VS. ILLUMINANCE



Hình 8-23. Mối quan hệ giữa sự thay đổi ánh sáng và điện trở.

Trong đồ thị ta thấy khi cường độ ánh sáng tăng thì giá trị điện trở giảm. Mối quan hệ theo tỷ lệ nghịch.

Do chúng ta không có thiết bị đo ánh sáng chuẩn nên ta không thể hiển thị giá trị đo ánh sáng được mà chỉ hiển thị giá trị số sau chuyển đổi trên led.

Các ứng dụng thường dùng để điều khiển đèn theo ánh sáng như tự động mở đèn khi trời tối hay tắt đèn khi trời sáng.

8.4.2 Vi điều khiển giao tiếp với quang trở

Trong bộ thực hành thì cảm biến quang trở nối với ngõ vào AN2 của IC ADC PCF8591, xem hình 8-9.

8.4.3 Các chương trình điều khiển dùng quang trở

Bài mẫu 821. Chương trình đo ánh sáng (illuminance in lux) dùng cảm biến photocell CDS PDV-P8103 nối với kênh AN3 của IC ADC PCF 8591, hiển thị kết quả đo trên module 4 led 7 đoạn. Chỉ thấy sự thay đổi.

Lưu tên file là “BAI_821_PCF8591_4LED_CDS.C”

- Mục đích: biết lập trình dùng ADC giao tiếp chuẩn I2C để đọc giá trị thay đổi của quang trở hiển thị trên led 7 đoạn.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_pcf8591.c>
void main()
{
    system_init();
    while(true)
    {
        if(pcf8591_read_adc(3)==OK)    // Đọc kết quả chuyển đổi kênh 3
        {
            d7seg.led[1]=m7d[pcf8591.adc_value[3]/10%10]; // hàng chục
            d7seg.led[0]=m7d[pcf8591.adc_value[3]%10];    // đơn vị
            d7seg_display();
        }
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: sau khi nạp xong thì ta nhìn thấy giá trị số hiển thị trên module led 7 đoạn. Bạn có thể dùng tay che ánh sáng bằng cách để gần quang trở chứ không cần phải chạm vào, khi đó giá trị số hiển thị trên led sẽ thay đổi theo chiều tăng vì chúng tỷ lệ nghịch như trong đồ thị.
- Giải thích chương trình:

Bài tập 822. Chương trình điều khiển đèn tự động sáng hoặc tắt dùng cảm biến photocell CDS PDV-P8103 nối với kênh AN3 của IC ADC PCF 8591, hiển thị kết quả đo trên module 4 led 7 đoạn.

Khi trời tối thì mở đèn sáng, khi trời sáng thì đèn tắt.

Lưu tên file là “BAI_822_PCF8591_4LED_CDS_ON_OFF_DEN.C”

Chương 9: THỰC HÀNH MODULE 6 – ĐỘNG CƠ BƯỚC, ĐỘNG CƠ DC VÀ ENCODER, PWM, ĐIỀU KHIỂN PID

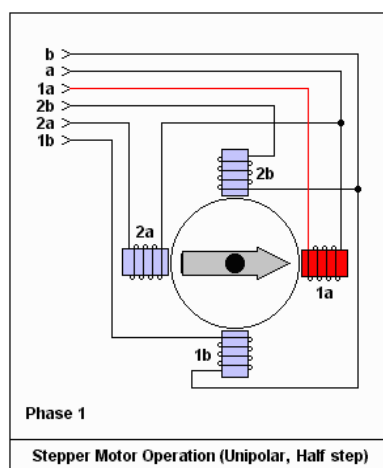
9.1 ĐIỀU KHIỂN ĐỘNG CƠ BƯỚC

9.1.1 Giới thiệu động cơ bước

Động cơ bước có hình ảnh minh họa như hình 9-1 và sơ đồ nguyên lý như hình 9-2.



Hình 9-1. Động cơ bước.



Hình 9-2. Sơ đồ nguyên lý động cơ bước.

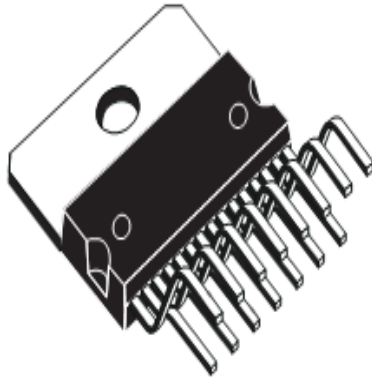
Động cơ bước có nhiều loại và nhiều cách điều khiển tùy thuộc vào các ứng dụng cụ thể, trong kit thực hành có trang bị 1 motor bước để các bạn thực hành.

Trong hình 9-2, động cơ bước có 6 đầu dây: 1a, 2a và 1b, 2b và 2 đầu dây chung là a và b.

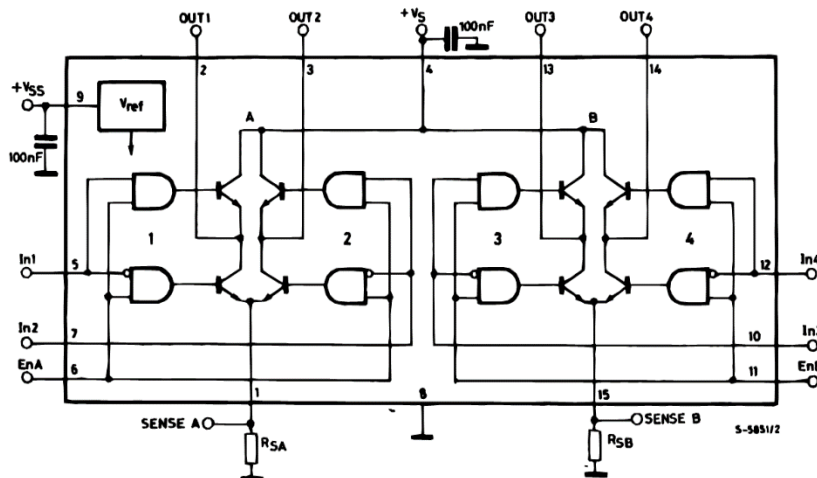
Động cơ bước có nhiều ứng dụng trong thực tế, ví dụ như ổ đĩa máy tính sử dụng động cơ bước để di chuyển đầu đọc từ rãnh này sang rãnh kế, phạm vi di chuyển ngắn, còn động cơ DC quay với tốc độ cao thì rất khó điều khiển. Trong ổ đĩa, động cơ DC quay với tốc độ 5400 vòng trên 1 phút để quay đĩa.

9.1.2 IC giao tiếp công suất điều khiển L298

Các động cơ bước có nhiều chủng loại tùy thuộc vào ứng dụng, trong bộ thí nghiệm sử dụng động cơ bước có dòng là 1A. Vì điều khiển không thể trực tiếp điều khiển được mà phải dùng IC chuyên dùng để điều khiển là L298. Hình ảnh IC và sơ đồ nguyên lý bên trong như các hình sau:



Hình 9-3. Hình dạng IC L298.

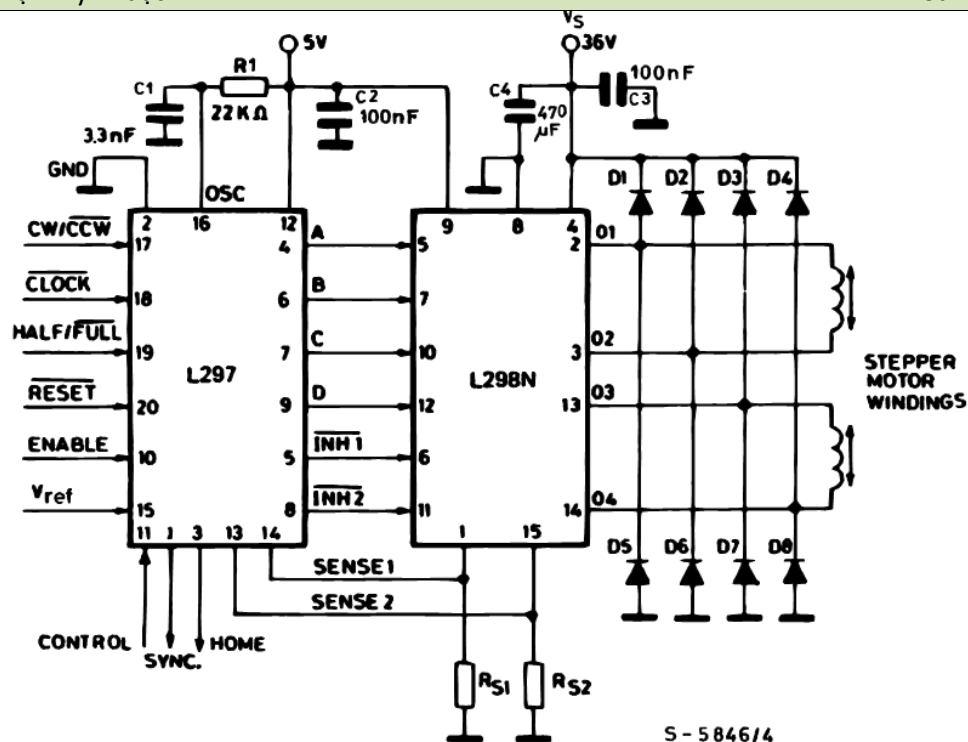


Hình 9-4. Sơ đồ nguyên lý IC L298.

Bên trong IC có các cổng logic và 2 cặp cầu H chuyên dùng để điều khiển động cơ DC và có thể điều khiển được 2 động cơ DC hoặc 1 động cơ bước.

IC L298 có 2 điện áp: điện áp cấp cho các transistor để điều khiển động cơ có thể hoạt động tối đa lên đến 46V. Khi sử dụng động cơ có nguồn cung cấp bao nhiêu thì ta cấp đúng bấy nhiêu, ví dụ động cơ dùng nguồn 24V thì ta cấp nguồn 24V. Điện áp thứ hai là cấp cho các cổng logic theo chuẩn TTL hoạt động bình thường là 5V DC.

Sơ đồ mạch điều khiển 1 động cơ bước như hình sau:



Hình 9-5. IC L298 điều khiển 1 động cơ bước.

Trong hình có 2 IC L298 và L297 dùng để điều khiển động cơ bước. Bạn chỉ quan sát phần IC L298 có 4 ngõ ra điều khiển 4 đầu dây của động cơ bước. 8 diode dùng để bảo vệ các transistor giải phóng năng lượng của cuộn dây về nguồn.

IC L297 chuyên để điều khiển động cơ bước nhưng trong bộ thí nghiệm không sử dụng nên không đề cập.

9.1.3 Mạch giao tiếp điều khiển động cơ bước

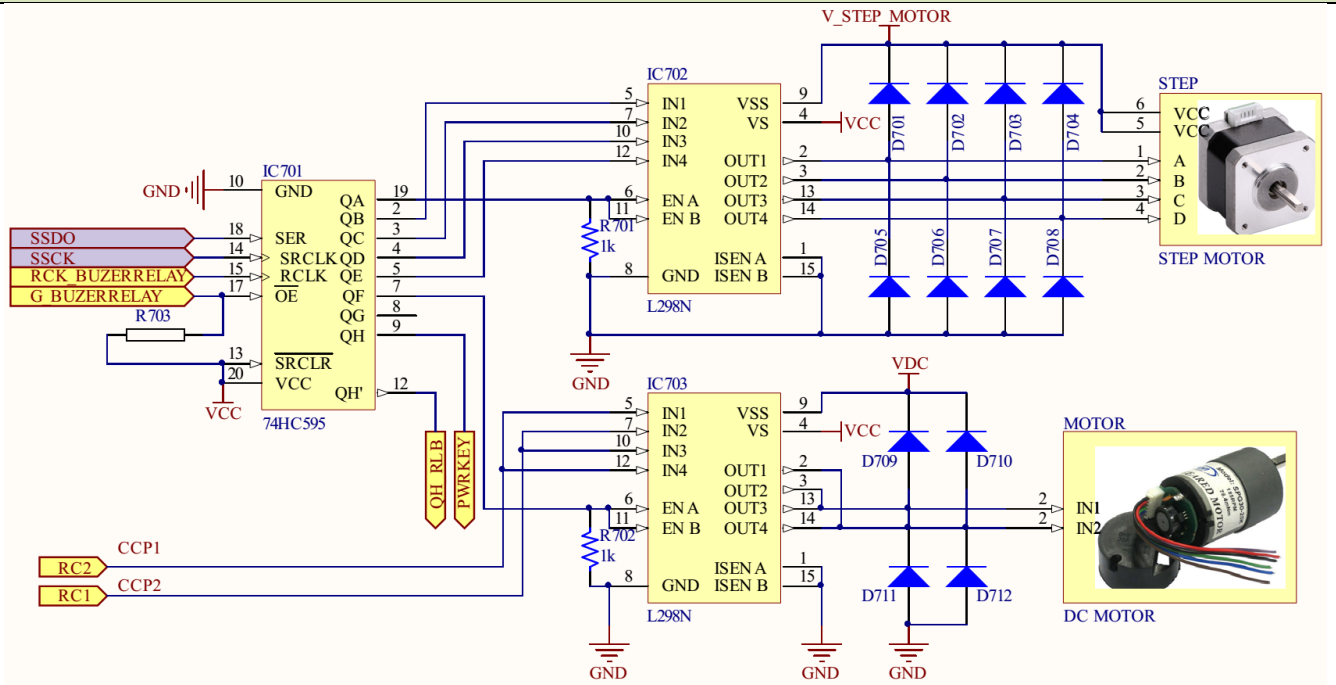
Sơ đồ mạch giao tiếp với động cơ bước và động cơ DC như hình 9-6.

Trong mạch sử dụng 1 IC thanh ghi 74HC595 để mở rộng port điều khiển động cơ bước và DC, 1 thanh ghi dịch điều khiển 2 relay, 2 triac và 1 buzzer.

Phần vi điều khiển PIC giao tiếp với IC thanh ghi dịch

Có 4 tín hiệu điều khiển là:

- SDO dùng để dịch dữ liệu nối tiếp.
- SCK dùng để cấp xung clock.
- RCK_RLB dùng điều khiển nạp dữ liệu song song từ bên trong IC thanh ghi dịch ra bên ngoài để điều khiển.
- G_RLB dùng để cho phép xuất dữ liệu.



Hình 9-6. Mạch giao tiếp động cơ bước và động cơ DC.

Phần IC thanh ghi dịch giao tiếp với động cơ

Năm tín hiệu từ QA đến QE nối với các tín hiệu tương ứng là ENAX, IN1X, IN2X, IN3X, IN4X của IC702 – L298. Bốn ngõ ra OUT1, OUT2, OUT3, OUT4 sẽ nối với 4 đầu dây CA, CB, CC, CD của động cơ bước, hai đầu dây chung nối lên nguồn cấp cho động cơ. Có 8 diode dùng để bảo vệ các transistor bên trong IC L298.

9.1.4 Mã điều khiển động cơ bước

Dữ liệu điều khiển động cơ bước từ vi điều khiển sẽ gửi ra thanh ghi dịch, dữ liệu từ thanh ghi dịch sẽ đưa đến IC công suất L298 để điều khiển động cơ bước.

Có 2 loại dữ liệu điều khiển động cơ bước: điều khiển bước đủ (full step) và nửa bước (half step) được trình bày trong 2 bảng sau.

Bảng 9-1. Các trạng thái điều khiển động cơ bước theo bước đủ.

	SỐ NHỊ PHÂN 8 BIT								SỐ HEX	TRẠNG THÁI ĐỘNG CƠ
	QH	QG	QF	QE	QD	QC	QB	QA		
TT				IN4	IN3	IN2	IN1	ENA		
1				X	X	X	X	0		Không cho phép
2	0	0	0	1	1	1	0	1	0×1D	Động cơ quay 1 bước
3	0	0	0	1	0	1	1	1	0×17	
4	0	0	0	1	1	0	1	1	0×1B	
5	0	0	0	0	1	1	1	1	0×0F	

Khi muốn điều khiển động cơ bước quay thì bit ENA phải bằng 1, 4 bit còn lại sẽ có giá trị như trong bảng cột số hex.

Nếu gởi 4 mã theo thứ tự từ trên xuống thì động cơ bước sẽ quay theo chiều kim đồng hồ, nếu gởi ngược lại thứ tự từ dưới lên thì động cơ sẽ quay ngược chiều kim đồng hồ.

Bảng 9-2. Các trạng thái điều khiển động cơ bước dạng nửa bước.

	SỐ NHỊ PHÂN 8 BIT								SỐ HEX	TRẠNG THÁI ĐỘNG CƠ
	QH	QG	QF	QE	QD	QC	QB	QA		
TT				IN4	IN3	IN2	IN1	ENA		
1				X	X	X	X	0		Không cho phép
2	0	0	0	1	1	1	0	1	0×1D	Động cơ quay ½ bước
3	0	0	0	1	0	1	0	1	0×15	
4	0	0	0	1	0	1	1	1	0×17	
5	0	0	0	1	0	0	1	1	0×13	
6	0	0	0	0	0	0	1	1	0×03	
7	0	0	0	0	1	0	1	1	0×0B	
8	0	0	0	0	1	0	0	1	0×09	
9	0	0	0	0	1	1	0	1	0×0D	

9.1.5 Các biến và hàm điều khiển Relay, Triac, Buzzer, động cơ

Như đã trình bày module này có 2 byte dữ liệu để xuất ra điều khiển nhiều đối tượng khác nhau.

Trong thư viện đã định nghĩa sẵn 1 biến dữ liệu 16 bit và sau đó định nghĩa các bit điều khiển cho từng đối tượng như sau:

```
typedef struct
{
    int1        STEP_MOTOR_ENABLE;
    int1        STEP_MOTOR_IN1;
    int1        STEP_MOTOR_IN2;
    int1        STEP_MOTOR_IN3;
    int1        STEP_MOTOR_IN4;
    int1        DC_MOTOR_ENABLE;
    int1        PWRKEY;
    int1        NC_PIN1;
    int1        BUZZER;
    int1        TRIAC_1;
    int1        TRIAC_2;
    int1        RELAY_1;
    int1        RELAY_2 ;
}
```

```

    int1        NC_PIN2 ;
    int1        NC_PIN3;
    int1        NC_PIN4;
}RBDC_STRUCT;
typedef union
{
    RBDC_STRUCT      bit;
    unsigned int8     xbyte[2];
}RBDC_UNION;

typedef struct
{
    RBDC_UNION        rbdc;
    unsigned int8      lcddata;
    unsigned int8      lcdcontrol;
    unsigned int8      glcddata;
    unsigned int8      glcdcontrol;
}SYSTEM_VAR_STRUCT;
SYSTEM_VAR_STRUCT system_var={0,0,0,0,0};

```

Hàm thứ 901: xuất 2 byte ra module relay, triac, buzzer, động cơ:

```

result private_write_buzzer_relay()
{
    SSPI_WRITE(system_var.rbdc.xbyte[1]);
    SSPI_WRITE(system_var.rbdc.xbyte[0]);
    output_high(rbdc_rck);
    output_low(rbdc_rck);
    return OK;
}

```

Hàm này tách dữ liệu 16 bit thành 2 byte rồi xuất đi.

Hàm thứ 902: Hàm cho phép ICL298 kết nối với động cơ hoạt động

```

result STEPMOTOR_ENABLE()
{
    system_var.rbdc.bit.step_motor_enable=1;
    return OK;
}

```

Hàm thứ 903: Hàm điều khiển động cơ bước

```

/* Hàm điều khiển động cơ bước
- mode: 0 là chế độ halfstep và bằng 1 là chế độ fullstep
- step: bước thứ n trong 1 chu trình điều khiển - giá trị từ 0 đến 7
- Hàm luôn trả về "OK"
*/
result STEPMOTOR_LOWLEVEL_CONTROL(int1 mode, int8 step)
{
    const unsigned int8 Step_data[2][8]={0x0e,0x0a,0x0b,0x08,0x01,0x05,
                                           0x04,0x06,0x0e,0x0b,0x0d,0x07,0x0e,0x0b,0x0d,0x07};
    system_var.rbdc.bit.step_motor_in1=bit_test(Step_data[mode][step],0);
    system_var.rbdc.bit.step_motor_in2=bit_test(Step_data[mode][step],1);
    system_var.rbdc.bit.step_motor_in3=bit_test(Step_data[mode][step],2);
    system_var.rbdc.bit.step_motor_in4=bit_test(Step_data[mode][step],3);
    private_write_buzzer_relay();
    return OK;
}

```


}

9.1.6 Các bài thực hành điều khiển động cơ bước

Thư viện động cơ bước. Chương trình thư viện của động cơ bước đã cho trong thư mục của bạn có tên là “TV_STEP MOTOR.c” và “TV_STEP MOTOR.h”

Bạn có thể sử dụng cho các bài thực hành và nên đọc hiểu.

File “TV_STEP MOTOR.h”

```
#ifndef __TV_STEP MOTOR_H__
#define __TV_STEP MOTOR_H__
typedef enum
{
    stepmotor_mode_fullstep = 1,
    stepmotor_mode_halfstep = 0
}STEP MOTOR_MODE;
typedef enum
{
    stepmotor_direct_forward = 1,
    stepmotor_direct_reverse = 0
}STEP MOTOR_DIRECT;
typedef struct
{
    STEP MOTOR_DIRECT    direct;
    STEP MOTOR_MODE      mode;
    unsigned int8         time_per_step;
    unsigned int32        step;
}STEP MOTOR_STRUCT;
STEP MOTOR_STRUCT stepmotor={1,1,10,0};
result stepmotor_control(void);
#endif
```

File “TV_STEP MOTOR.c”

```
#ifndef __TV_STEP MOTOR_C__
#define __TV_STEP MOTOR_C__
#include <TV_STEP MOTOR.h>
/* Hàm điều khiển động cơ bước
- stepmotor.direct: dùng để điều khiển chiều quay của động cơ
+ stepmotor.direct=stepmotor_direct_forward : động cơ quay thuận
+ stepmotor.direct=stepmotor_direct_reverse : động cơ quay nghịch
- stepmotor.mode: dùng để điều khiển chế độ hoạt động của động cơ
+ stepmotor.mode = stepmotor_mode_fullstep: chế độ fullstep
+ stepmotor.mode = stepmotor_mode_halfstep: chế độ halfstep
- stepmotor.step: chứa số bước động cơ đã quay được
- stepmotor.time_per_step: để điều khiển tốc độ quay của động cơ
- Hàm trả về "BUSY" khi đang trong quá trình chờ chuyển bước tiếp theo
- Hàm trả về "OK" khi vừa mới điều khiển động cơ quay
*/
result stepmotor_control()
{
    static int8 step=0;
    static signed int8 n=1;
    static unsigned int32 t;
```

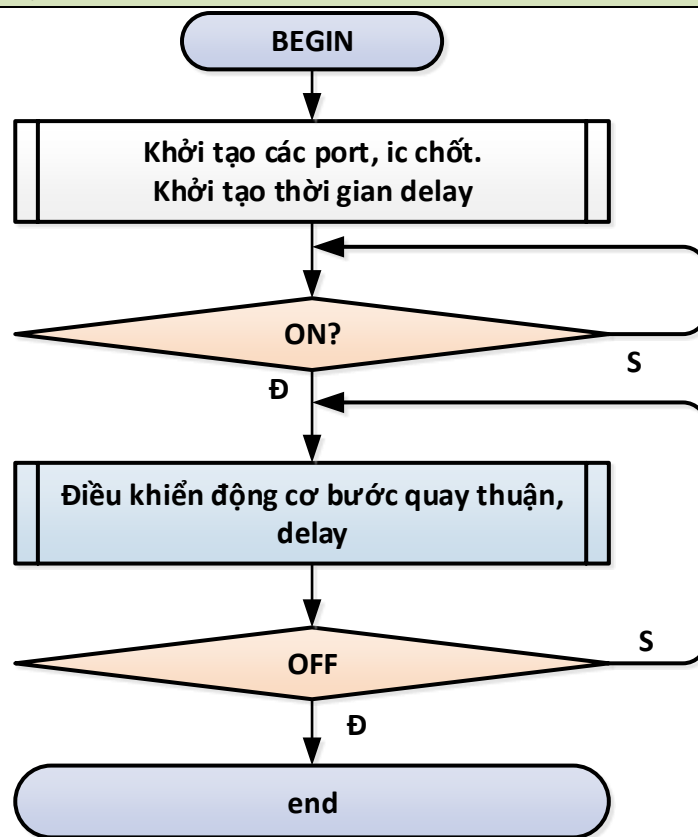
```

switch (step)
{
    case 0:
        if(stepmotor.direct==stepmotor_direct_forward) n=1;
        else n=7;
        STEPMOTOR_ENABLE();
        step =1;
        break;
    case 1:
        t      = delay_start();
        step   = 2;
        break;
    case 2:
        if(delay_stop(t)>=stepmotor.time_per_step)
        {
            STEPMOTOR_LOWLEVEL_CONTROL(stepmotor.mode,n);
            if(stepmotor.direct) n++;
            else n--;
            if(n==8) n =0;
            else if(n== -1) n =7;
            step = 1;
            stepmotor.step++;
            return OK;
        }
        break;
}
return BUSY;
}
#endif

```

Bài mẫu 901. Chương trình điều khiển động cơ bước có 2 nút điều khiển: ON, OFF. Khi nhấn ON thì động cơ quay theo chiều kim đồng hồ, khi nhấn OFF thì ngừng. Lưu tên file là “BAI_901_STEP_ON_OFF”

- a. Mục đích: Biết lập trình điều khiển động cơ bước bằng 2 nút nhấn ON và OFF.
- b. Lưu đồ:



Hình 9-7. Lưu đồ điều khiển động cơ bước quay thuận bằng 2 nút ON, OFF.

c. Chương trình:

```

#include<tv_boards.c>
#include<tv_stepmotor.c>
int1 ttquay=0;
void main()
{
    system_init();
    stepmotor.time_per_step =10;           // 10ms/step
    stepmotor.direct = stepmotor_direct_forward; // Quay thuận
    stepmotor.mode = stepmotor_mode_fullstep; // Chế độ fullstep
    while(true)
    {
        if(input(ON)==0) ttquay=1;
        if(input(OFF)==0) ttquay=0;
        if(ttquay) stepmotor_control(); // Điều khiển động cơ
    }
}
  
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: sau khi nạp xong thì nhấn nút ON động cơ sẽ quay thuận, nhấn OFF sẽ ngừng động cơ. Có thể điều chỉnh thời gian delay giữa 2 lần gọi để thay đổi tốc độ động cơ, nếu thời gian ngắn quá thì động cơ có thể đứng yên vì không đáp ứng được.
- f. Hãy giải thích chương trình và đặt các câu hỏi nếu chưa hiểu.

Bài tập 902. Hãy hiệu chỉnh bài 901 sao cho khi nhấn ON thì động cơ quay ngược chiều kim đồng hồ, nhấn OFF thì ngừng. Thử tự dữ liệu điều khiển động cơ bước giữ nguyên.

Lưu tên file là “BAI_902_STEP_ON_OFF_QN”

Bài mẫu 903. Hãy viết chương trình điều khiển động cơ bước có 3 nút điều khiển: ON, OFF, INV.

Khi nhấn ON thì động cơ quay theo chiều kim đồng hồ ở chế độ halfstep.

Khi nhấn INV thì động cơ đảo chiều đang quay, nếu động cơ ngừng thì phím này không có tác dụng.

Khi nhấn OFF thì động cơ ngừng.

Lưu tên file là “BAI_903_STEP_ON_OFF_INV”

- Mục đích: Biết lập trình điều khiển động cơ bước bằng 3 nút nhấn ON, OFF và INV.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_stepmotor.c>
int1 ttquay=0;
void main()
{
    system_init();
    stepmotor.time_per_step =10;
    stepmotor.direct = stepmotor_direct_forward; // Quay thuận
    stepmotor.mode     = stepmotor_mode_halfstep; // Chế độ halfstep
    while(true)
    {
        if(input(ON)==0)  ttquay=1;
        if(input(OFF)==0) ttquay=0;
        if((inputcd(INV)==0)&&(ttquay)) // Nếu đang quay mà nhấn INV
            {stepmotor.direct= !stepmotor.direct;} // thì đảo chiều
        if(ttquay) stepmotor_control();
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: sau khi nạp xong thì nhấn nút ON động cơ sẽ quay thuận, nhấn INV thì động cơ đảo chiều, nhấn OFF sẽ ngừng động cơ.
- Hãy giải thích chương trình và đặt các câu hỏi nếu chưa hiểu.

Bài tập 904. Hãy viết chương trình điều khiển động cơ bước giống bài 903 nhưng thêm phần hiển thị số bước trên 4 led 7 đoạn. Khi nhấn nút đảo chiều động cơ thì xóa luôn số bước đang hiển thị để bắt đầu lại từ đầu.

Lưu tên file là “BAI_904_STEP_ON_OFF_INV_HT_BUOC”

Làm sao để biết động cơ bước quay 1 vòng thì có bao nhiêu bước?

Bài tập 905. Hãy viết chương trình điều khiển động cơ bước giống bài 904 nhưng 3 led 7 đoạn bên phải hiển thị số bước của 1 vòng, led thứ 4 còn lại hiển thị số vòng.
Lưu tên file là “BAI_905_STEP_ON_OFF_INV_HT_BUOC_VONG”

Bài tập 906. Hãy viết chương trình điều khiển động cơ bước có thể thay đổi tốc độ bằng 3 nút điều khiển: STOP, UP, DW.
Khi nhấn UP thì động cơ quay theo chiều kim đồng hồ tăng tốc.
Khi nhấn DW thì động cơ quay theo chiều kim đồng hồ giảm tốc.
Có 3 led 7 đoạn hiển thị số bước, led thứ tư hiển thị số cấp từ 0 đến 9. Cấp 0 thì ngừng. Cấp 1 đến cấp 9: tương ứng với các khoảng thời gian delay: 45, 40, 35, 30, 25, 20, 15, 10, 5 ms.
Khi nhấn STOP thì động cơ ngừng.
Lưu tên file là “BAI_906_STEP_STOP_UP_DW”

Bài tập 907. Hãy viết thêm vào bài 906 phím có chức năng đảo chiều INV
Lưu tên file là “BAI_907_STEP_STOP_UP_DW_INV”

Bài tập 908. Hãy viết chương trình điều khiển động cơ quay theo số vòng được đặt trước dùng bàn phím ma trận, dùng các phím số từ 0 đến 9 để nhập số vòng, để đơn giản chỉ nhập 1 con số hiển thị ở 1 led 7 đoạn.
Nhấn phím D thì động cơ quay thuận và số vòng giảm dần đến khi bằng 0 thì ngừng.
Lưu tên file là “BAI_908_STEP_KEY_44”

Bài tập 909. Hãy hiệu chỉnh lại bài 906 nhưng điều khiển động cơ quay halfstep.
Lưu tên file là “BAI_909_STEP_STOP_UP_DW_INV_HS”

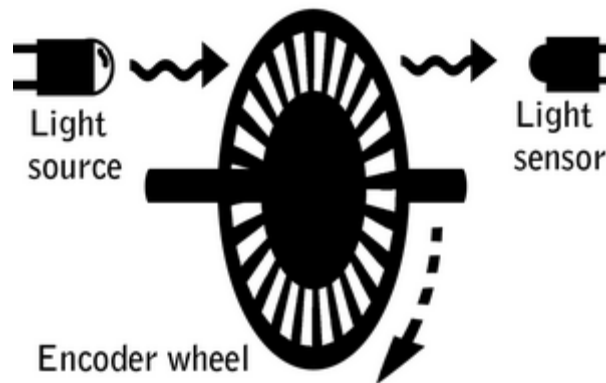
9.2 ĐIỀU KHIỂN ĐỘNG CƠ DC, ENCODER

9.2.1 Giới thiệu động cơ DC, ENCODER

Động cơ DC và encoder có hình ảnh như hình 9-8 và 9-9:



Hình 9- 8Động cơ DC trong BTN.



Hình 9-9. Encoder gắn trong sau động cơ.

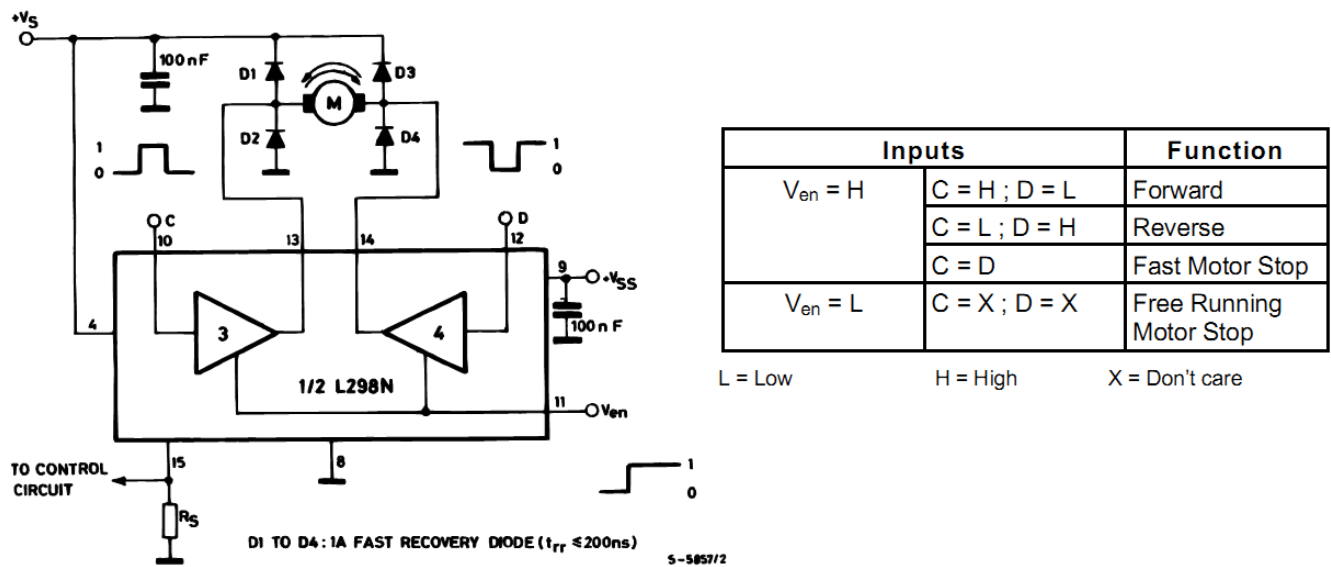
Động cơ DC có nhiều loại. Trong kit thực hành có trang bị 1 động cơ DC dùng nguồn 24V, dòng 1A, có encoder gắn vào trục động cơ tạo ra 45 cho 1 vòng quay để các bạn thực hành.

Loại encoder trong BTN này đơn giản cho các ứng dụng không cần xác định chiều, loại encoder có 2 đầu dây ra A và B dùng để xác định chiều của động cơ, A lên mức logic 1 nhanh hơn B thì động cơ quay thuận, ngược lại khi đảo chiều thì B lên mức 1 nhanh hơn A.

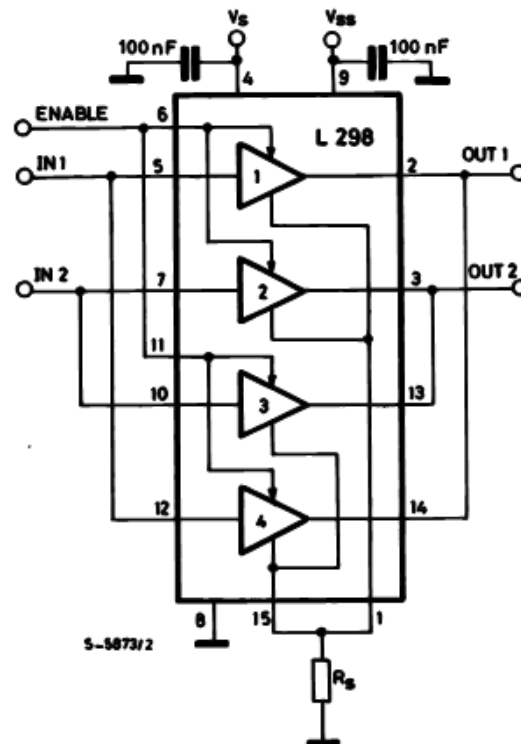
Sơ đồ mạch điều khiển 1 động cơ dùng L298 và các trạng thái điều khiển như hình 9-10.

Hãy quan sát hình và bảng trạng thái, bạn sẽ biết được nguyên lý điều khiển động cơ DC.

Nếu chỉ dùng 1 IC L298 điều khiển 1 động cơ DC thì dư 1 cầu H. Để tránh lãng phí và tăng khả năng cấp dòng ta sử dụng 2 cầu H mắc song song để điều khiển 1 động cơ DC như hình 9-11.



Hình 9-10. IC L298 điều khiển 1 động cơ DC dùng 1 cầu H.



Hình 9-11. IC L298 điều khiển 1 động cơ DC dùng 2 cầu H.

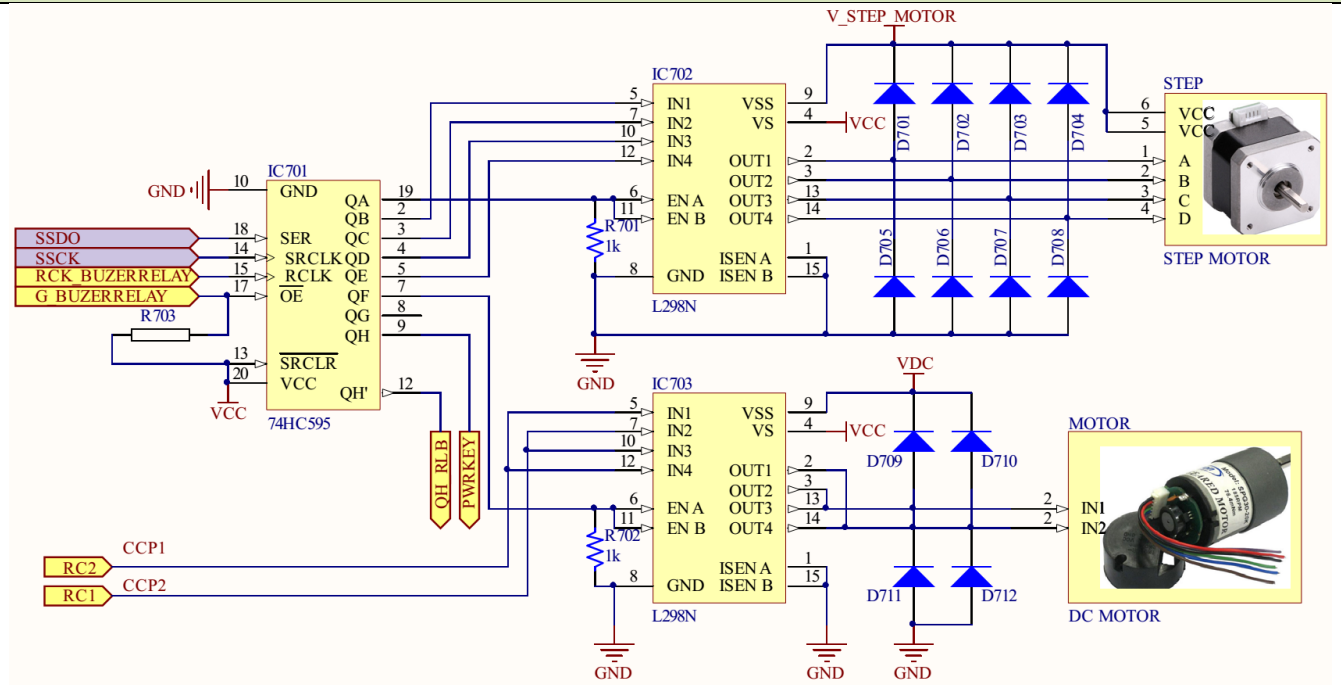
Trong bộ thực hành thì dùng theo cách này.

9.2.2 Mạch giao tiếp điều khiển động cơ DC

Sơ đồ mạch giao tiếp với động cơ DC như hình 9-12.

Trong sơ đồ mạch sử dụng IC703 – L298 để điều khiển động cơ DC qua IC trung gian mở rộng port IC701 – 74HC595D.

Tín hiệu QF nối với tín hiệu ENA của IC703 – L298.



Hình 9-12. Mạch giao tiếp động cơ bước và động cơ DC.

IC L298 có thể điều khiển được 2 động cơ DC nhưng kit thực hành chỉ có 1 động cơ nên 2 mạch điều khiển động cơ được nối song song để tăng khả năng chịu dòng. Trong sơ đồ mạch thì 2 cặp ngõ vào input1, input4 và input2, input3 nối với nhau. Hai cặp tín hiệu ngõ vào này sẽ nối trực tiếp với 2 chân RC1/CCP2 và RC2/CCP1 của vi điều khiển PIC.

Tương tự, ngõ ra cũng nối với nhau, chỉ còn 2 ngõ ra nối với động cơ DC.

9.2.3 Dữ liệu điều khiển động cơ DC

Dữ liệu điều khiển động cơ DC từ vi điều khiển sẽ gửi ra thanh ghi dịch, dữ liệu từ thanh ghi dịch sẽ đưa đến IC công suất L298 để điều khiển tín hiệu cho phép/cấm IC L298.

Bảng 9-3. Các trạng thái điều khiển động cơ DC.

TT	ENA	INPUT1	INPUT2	TRẠNG THÁI ĐỘNG CƠ
1	0	X	X	NGỪNG
2	1	0	0	
3	1	1	1	
4	1	1	0	QUAY THUẬN
5	1	0	1	QUAY NGHỊCH

9.2.4 Các bài thực hành điều khiển động cơ DC

Thư viện động cơ DC. Để tránh lặp lại, ta viết chương trình thư viện cho động cơ DC như sau gồm 2 file “TV_DCMOTOR.c” và “TV_DCMOTOR.h”.

Hàm thứ 904: Cho phép ICL298 điều khiển động cơ hoạt động và cấu hình PWM


```

result DCMOTOR_ENABLE()
{
    system_var.rbdm.bit.DC_MOTOR_ENABLE=1;
    private_write_buzzer_relay();
    setup_timer_2(t2_div_by_16,249,1); // Cấu hình PWM T=0.8ms
    setup_ccp1(ccp_pwm);
    setup_ccp2(ccp_pwm);
    return OK;
}

```

Hàm thứ 905: Hàm điều khiển động cơ DC

```

/* Hàm điều khiển động cơ DC
- duty: dùng để thay đổi tốc độ động cơ - phạm vi từ 0 đến 1000
- direct: là chiều quay
+ direct=1: động cơ quay thuận
+ direct=0: động cơ quay nghịch
- Hàm luôn trả về "OK"
*/
result DCMOTOR_LOWLEVEL_CONTROL(signed int16 duty, int1 direct)
{
    if(duty)
        if(direct){ set_pwm1_duty((int16)0); set_pwm2_duty(duty); }
        else      { set_pwm2_duty((int16)0); set_pwm1_duty(duty); }
    else      { set_pwm1_duty((int16)0); set_pwm2_duty((int16)0); }
    return OK;
}

```

File TV_DCMOTOR.h

```

#ifndef __TV_DCMOTOR_H__
#define __TV_DCMOTOR_H__
#define SPEEDMUL 1333
typedef enum
{
    dcmotor_direct_forward = 1,
    dcmotor_direct_reverse = 0
}DCMOTOR_DIRECT;
typedef struct
{
    signed int16      duty;
    DCMOTOR_DIRECT    direct;
    unsigned int16     speed;
}
DCMOTOR_STRUCT ;
DCMOTOR_STRUCT dcmotor={0,1,0};
DCMOTOR_STRUCT nowdcmotor={2000,1,0};
result dcmotor_control(void);
result dcmotor_read_speed(void);
#endif

```

File TV_DCMOTOR.c

```

#ifndef __TV_DCMOTOR_C__
#define __TV_DCMOTOR_C__
#include <TV_DCMOTOR.h>

```

```

/* Hàm điều khiển động cơ DC thay đổi tốc độ và chiều
- duty : dùng để thay đổi tốc độ - phạm vi từ 0 đến 1000
- direct: là chiều quay
+ direct= dcmotor_direct_forward: động cơ quay thuận
+ direct= dcmotor_direct_reverse: động cơ quay nghịch
-Hàm trả về "BUSY" khi chưa xử lý xong, trả về "OK" khi xử lý xong.
*/
result DCMOTOR_MIDDLELEVEL_CONTROL(signed int16 duty,int1 direct)
{
    static unsigned int32 t=0;
    result tt=BUSY;
    if(nowdcmotor.duty<duty) //Duty hiện tại bé hơn duty cần đặt thì tăng
    {
        if(delay_stop(t)>=20) //Tăng tốc từ từ để tránh hư phần cứng.
        {
            //Tăng 5%duty trong mỗi 20ms
            nowdcmotor.duty+=50;
            if(nowdcmotor.duty>=duty) {nowdcmotor.duty=duty;tt= OK;}
            DCMOTOR_LOWLEVEL_CONTROL(nowdcmotor.duty,direct);
            t = delay_start();
        }
    }
    else if(delay_stop(t)>=20) //Duty hiện tại lớn hơn duty đặt thì giảm
    {
        //Giảm tốc từ từ để tránh hư phần cứng.
        nowdcmotor.duty-=50; //Giảm 5%duty trong mỗi 20ms
        if(nowdcmotor.duty<=duty) {nowdcmotor.duty=duty;tt= OK;}
        DCMOTOR_LOWLEVEL_CONTROL(nowdcmotor.duty,direct);
        t = delay_start();
    }
    return tt;
}

/* Hàm điều khiển động cơ DC đầy đủ các chế độ
- dcmotor.duty : dùng để thay đổi tốc độ - phạm vi từ 0 đến 1000
- dcmotor.direct: là chiều quay
+ dcmotor.direct = dcmotor_direct_forward: động cơ quay thuận
+ dcmotor.direct = dcmotor_direct_reverse: động cơ quay nghịch
-Hàm luôn trả về "OK".
*/
result dcmotor_control()
{
    static int8 step=0;
    switch (step)
    {
        case 0: // Cho phép động cơ,cấu hình PWM
            DCMOTOR_ENABLE();
            DCMOTOR_MIDDLELEVEL_CONTROL(dcmotor.duty,dcmotor.direct);
            step =1;
            break;
        case 1: // Nếu duty thay đổi thì sẽ điều khiển thay đổi tốc độ
            if(nowdcmotor.duty!= dcmotor.duty)
                DCMOTOR_MIDDLELEVEL_CONTROL(dcmotor.duty,dcmotor.direct);
            // Nếu chiều thay đổi thì tiến hành đảo chiều (step=2)
            else if(nowdcmotor.direct!= dcmotor.direct) step=2;
            break;
        case 2: // Trước khi đảo chiều cần phải giảm tốc từ từ về 0

```

```

        if(DCMOTOR_MIDDLELEVEL_CONTROL(0,nowdcmotor.direct)==OK)
            step=3;
        break;
    case 3: // Tiến hành tăng tốc từ từ theo chiều ngược lại
        if(DCMOTOR_MIDDLELEVEL_CONTROL(dcmotor.duty,!nowdcmotor.direct)==OK)
        {
            nowdcmotor.direct = !nowdcmotor.direct;
            step=1;
        }
        break;
    }
    return OK;
}

/* Hàm đo tốc độ động cơ theo đơn vị vòng/phút RMP
- dcmotor.speed: chứa tốc độ động cơ theo đơn vị vòng/phút
-Hàm trả về "BUSY" khi chưa đo xong, trả về "OK" khi đo xong.
*/
result dcmotor_read_speed()
{
    static int1 tt_init=0;
    static int8 step=0;
    static unsigned int32 t;
    if(tt_init==0)
    {
        setup_timer_1(t1_external_sync|t1_div_by_1); // COUNTER1 đọc encoder
        tt_init=1;
    }
    switch (step)
    {
        case 0: // Bắt đầu đo
            set_timer1(0);
            t = delay_start();
            step=1;
            break;
        case 1:
            if(delay_stop(t)>=100) // Đủ 100ms thì đo xong và tính speed
            {
                dcmotor.speed = (int32)SPEEDMUL*get_timer1()/delay_stop(t);
                step=0;
                return OK;
            }
            break;
    }
    return BUSY;
}
#endif

```

Bài mẫu 911. Chương trình điều khiển động cơ DC có 2 nút điều khiển: ON, OFF. Khi nhấn ON thì động cơ quay theo chiều kim đồng hồ, khi nhấn OFF thì ngừng. Có 8 led sáng cho biết động cơ quay, tắt thì động cơ ngừng. Lưu tên file là “BAI_911_DC_MOTOR_ON_OFF”

- Mục đích: biết điều khiển động cơ DC bằng 2 nút nhấn ON và OFF.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_dcmotor.c>
void main()
{
    system_init();
    dcmotor.direct= dcmotor_direct_forward; // Chiều quay thuận
    dcmotor.duty = 0;                       // Động cơ dừng
    while(true)
    {
        if(input(ON)==0) {dcmotor.duty=1000;led32.ledx8[0]=0xff;}
        if(input(OFF)==0) {dcmotor.duty=0; led32.ledx8[0]=0x00;}
        dcmotor_control();
        led32_display();
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: Sau khi nạp xong thì nhấn nút ON động cơ sẽ quay thuận, nhấn OFF sẽ ngừng động cơ. Trong chương trình có gởi dữ liệu điều khiển động cơ ra 32 led đơn để bạn có thể nhìn thấy. Động cơ chạy với tốc độ cực đại.
- Giải thích chương trình: bạn hãy giải thích.

Bài mẫu 912. Chương trình điều khiển động cơ có 3 nút điều khiển: ON, OFF, INV. Khi nhấn INV thì đảo chiều động cơ đang quay, nếu động cơ đang ngừng thì nhấn đảo chiều không có tác dụng.

Lưu tên file là “BAI_912_DC_ON_OFF_INV”.

- Mục đích: biết điều khiển động cơ DC chạy thuận và chạy nghịch.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_dcmotor.c>
void main()
{
    system_init();
    dcmotor.direct= dcmotor_direct_forward; // Chiều quay thuận
    dcmotor.duty = 0;                       // Động cơ dừng
    while(true)
    {
        if(input(ON)==0) dcmotor.duty=1000;
        if(input(OFF)==0) dcmotor.duty=0;
        if((inputcd(INV)==0) && (dcmotor.duty>0)) //Nhấn INV thì đảo
            dcmotor.direct=!dcmotor.direct;
        dcmotor_control();
    }
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: Sau khi nạp xong thì nhấn nút ON động cơ sẽ quay thuận, nhấn OFF sẽ ngừng động cơ, nhấn INV thì động cơ ngừng rồi sau đó đảo chiều.
- f. Giải thích chương trình:

Bài tập 913. Hãy viết lại chương trình bài 912 thêm phần điều khiển 32 led: khi động cơ ngừng thì 32 led tắt, khi động cơ chạy thuận thì 16 led phải sáng, nghịch thì 16 led trái sáng.

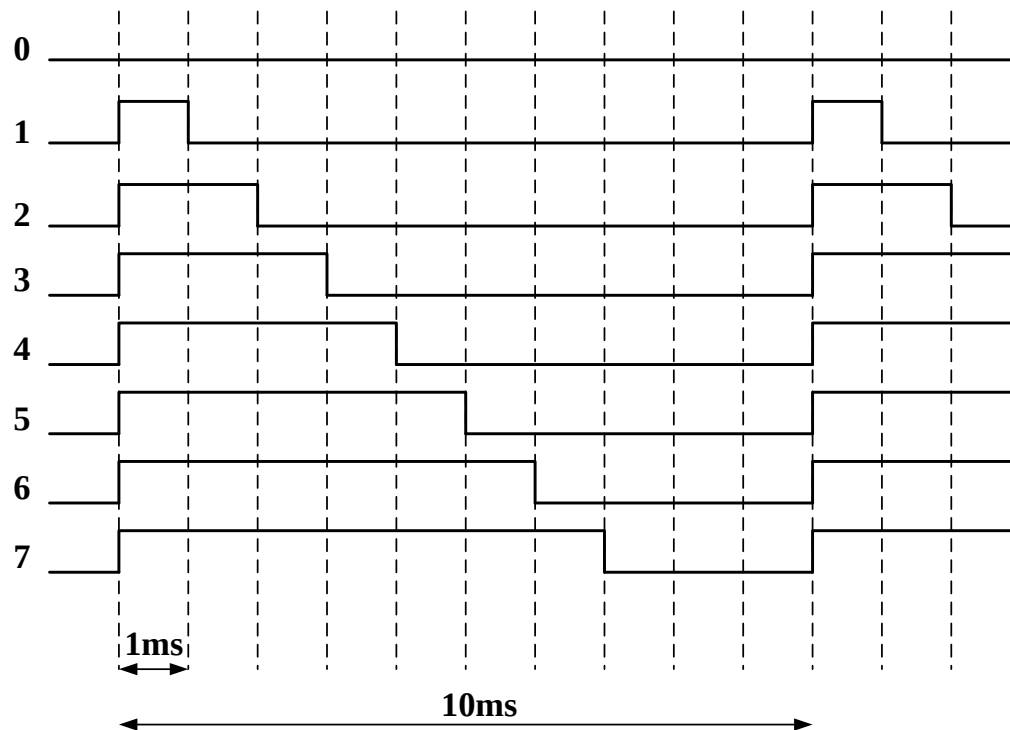
Lưu tên file là “BAI_913_DC_ON_OFF_INV_LED”.

9.3 ĐIỀU KHIỂN THAY ĐỔI TỐC ĐỘ ĐỘNG CƠ DC PWM, ENCODER

9.3.1 Giới thiệu PWM (PULSE WIDTH MODULATION)

Nhắc lại lý thuyết:

Để giúp bạn hiểu PWM là gì thì chúng ta khảo sát dạng sóng điều chế độ rộng xung PWM như hình 9-13.



Hình 9-13. Dạng sóng điều chế độ rộng xung.

Quan sát dạng sóng hình 9-13 chúng ta thấy:

- Chu kỳ tín hiệu là 10ms.
- Ở cấp tốc độ 0 thì tín hiệu bằng 0, động cơ ngừng.
- Ở cấp tốc độ 1 thì tín hiệu điều khiển ở mức 1 chỉ có 1ms, ở mức 0 là 9ms. Giả sử dòng tạo ra là 10 mA khi ở mức 1, khi đó dòng trung bình là $(10 \cdot 1)/10 = 1\text{mA}$. Nếu điều khiển động cơ thì động cơ chạy 1/10 tốc độ định mức.

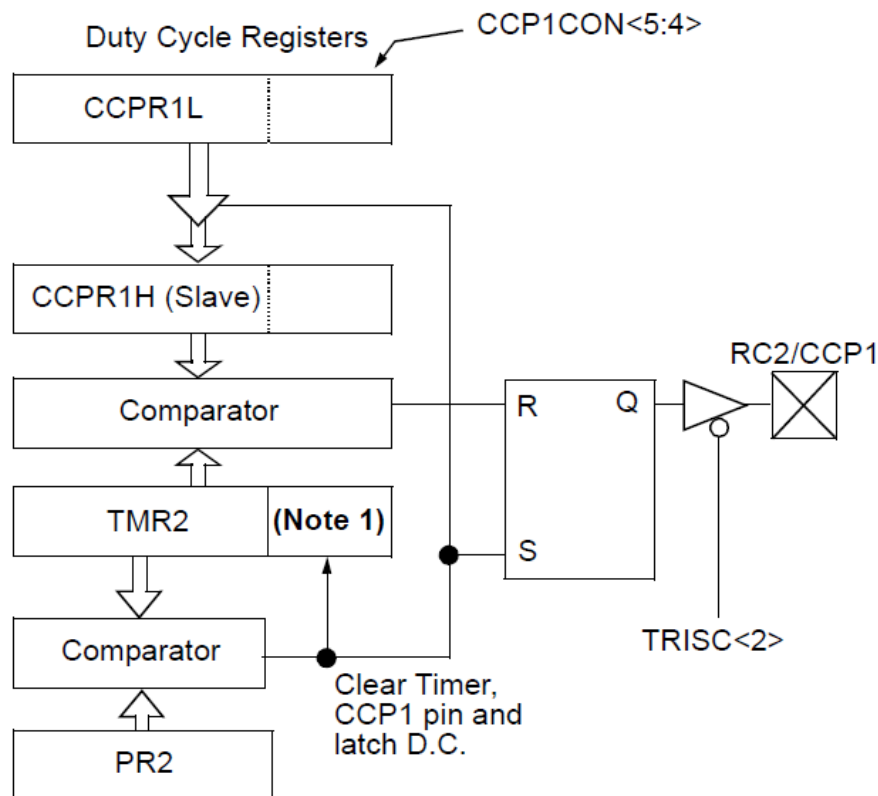
- Ở cấp tốc độ 2 thì tín hiệu điều khiển ở mức 1 chỉ có 2ms, ở mức 0 là 8ms. Giả sử dòng tạo ra là 10 mA khi ở mức 1, khi đó dòng trung bình là $(10 \cdot 2)/10 = 2\text{mA}$. Nếu điều khiển động cơ thì động cơ chạy 2/10 tốc độ định mức.
- Ở cấp tốc độ 5 thì tín hiệu điều khiển ở mức 1 chỉ có 5ms, ở mức 0 là 5ms. Giả sử dòng tạo ra là 10 mA khi ở mức 1, khi đó dòng trung bình là $(10 \cdot 5)/10 = 5\text{mA}$.
- Ở cấp tốc độ 10 thì tín hiệu điều khiển ở mức 1 là 10ms, ở mức 0 là 0ms. Giả sử dòng tạo ra là 10 mA khi ở mức 1, khi đó dòng trung bình là $(10 \cdot 10)/10 = 10\text{mA}$. Tốc độ đạt cực đại.

Khi thay đổi tốc độ thì chỉ thay đổi thời gian xung ở mức 1, còn gọi là hệ số chu kỳ (hay hệ số công tác), còn chu kỳ thì không đổi.

Ứng dụng điều chế độ rộng xung PWM dùng để điều khiển tốc độ động cơ DC, thay đổi độ sáng của đèn, ...

9.3.2 Cấu trúc khối điều chế độ rộng xung PWM của PIC

PIC có tích hợp module điều khiển PWM và có sơ đồ khối như hình 9-14:



Hình 9-14. Sơ đồ khối của PWM PIC.

Khối PWM gồm có 2 mạch so sánh: mạch so sánh 8 bit với mạch so sánh 10 bit.

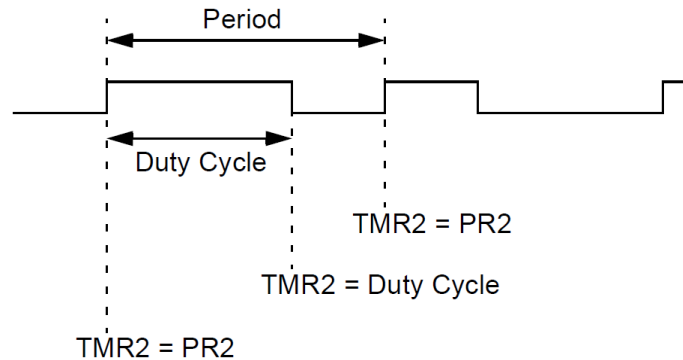
Mạch so sánh 8 bit sẽ so sánh giá trị đếm của timer T2 với giá trị của thanh ghi PR2 (period register), giá trị trong timer T2 tăng từ giá trị đặt trước cho đến khi bằng giá trị của PR2 thì mạch so sánh sẽ set flip flop RS làm ngõ ra CCPx lên mức 1. Đồng thời nạp giá trị 10 bit từ thanh ghi CCPRxL sang thanh ghi CCPRxH, timer T2 bị reset và bắt đầu đếm lại cho đến khi giá trị của

timer T2 bằng giá trị của CCPRxH thì mạch so sánh sẽ reset flip flop RS làm ngõ ra CCPx về mức 0. Quá trình này lặp lại.

Dạng sóng điều chế PWM như hình 9-15.

Chu kỳ không thay đổi, muốn thay đổi thời gian xung ở mức 1 thì ta thay đổi hệ số chu kỳ (duty cycle). Khi hệ số chu kỳ thay đổi thì điện áp hay dòng trung bình thay đổi.

Hệ số chu kỳ càng lớn thì dòng trung bình càng lớn, nếu điều khiển động cơ sẽ làm thay đổi tốc độ.



Hình 9-15. Dạng sóng PWM.

9.3.3 Tính chu kỳ xung PWM

Chu kỳ PWM của vi điều khiển PIC được tính theo công thức:

$$PERIOD_{PWM} = [(PR2) + 1] * 4 * T_{OSC} * PV_{TMR2}$$

Trong đó: T_{OSC} là chu kỳ của tụ thạch anh tạo dao động.

PV_{TMR2} (Prescale Value) giá trị chia trước của timer2.

$PR2$ (period register) thanh ghi chu kỳ.

9.3.4 Tính hệ số chu kỳ xung PWM

Hệ số chu kỳ được thiết lập bởi giá trị lưu trong thanh ghi 10 bit gồm thanh ghi 8 bit CCPRxL và 2 bit còn lại là bit thứ 4 và thứ 5 lưu ở trong thanh ghi CCPxCON – kí hiệu là CCPxCON<5:4>.

Giá trị của hệ số chu kỳ là 10 bit nên có thể thay đổi từ 0 đến 1023 tạo ra 1024 cấp giá trị điều khiển.

Giá trị 10 bit thì 8 bit có trọng số lớn lưu trong thanh ghi CCPRxL và 2 bit còn lại có trọng số thấp thì ở CCPxCON<5:4>.

Hệ số chu kỳ của vi điều khiển PIC được tính theo công thức:

$$DUTY_CYCLE_{PWM} = (CCPRxL : CCPxCON < 5 : 4 >) * T_{OSC} * PV_{TMR2}$$

9.3.5 Các lệnh điều khiển PWM

a. LỆNH ĐỊNH CẤU HÌNH KHỐI CCP

Cú pháp: `setup_ccp1 (mode)` or `setup_ccp1 (mode, pwm)`

Thông số: **mode** là hằng số. Giá trị của mode xem trong file thiết bị, một vài thông số:
 Disable the CCP: CCP_OFF
 Set CCP to PWM mode: CCP_PWM Enable Pulse Width Modulator
 Chức năng: Khởi tạo khối CCP.

b. LỆNH THIẾT LẬP HỆ SỐ CHU KỲ

Cú pháp: set_pwm1_duty (**value**)

Thông số: **value** có thể là hằng số 8 bit hoặc 16 bit.

Chức năng: Ghi giá trị 10 bit vào PWM để thiết lập hệ số chu kỳ.
 Giá trị 10 bit được dùng để xác định lượng thời gian của tín hiệu PWM ở mức 1 trong một chu kỳ.

c. LỆNH ĐỊNH CẤU HÌNH TIMER2 – SETUP_TIMER_2

Cú pháp: Setup_timer_2(**mode, period, postscale**)

Thông số: **mode** có thể là 1 trong các thông số: T2_DISABLED, T2_DIV_BY_1, T2_DIV_BY_4, T2_DIV_BY_16

Period là số nguyên có giá trị từ 0 đến 255 dùng để xác định khi nào giá trị timer bị reset.

postscale là số nguyên có giá trị từ 1 đến 16 dùng để xác định timer tràn bao nhiêu lần trước khi phát sinh tín hiệu ngắt.

Chức năng: Khởi tạo cho TIMER2.

d. LỆNH THIẾT LẬP GIÁ TRỊ BẮT ĐẦU – SET_TIMERx(value)

Cú pháp: set_timerX(**value**) ; x là 0, 1, 2

Thông số: **value** là hằng số nguyên 8 hoặc 16 bit dùng để thiết lập giá trị mới cho timer.

Chức năng: thiết lập giá trị bắt đầu cho TIMER.

9.3.6 Tính toán điều khiển tốc độ động cơ bằng PWM

Phần này trình bày cách chọn chu kỳ, tính toán các thông số cho phù hợp để điều khiển thay đổi tốc độ động cơ DC dựa vào phần cứng đã thiết kế của BTN.

Cho tần số tụ thạch anh BTN đang dùng là 20MHz.

Cho chu kỳ PWM là 0.8ms.

Hãy tính toán các thông số để điều khiển tốc độ của động cơ từ tốc độ thấp nhất đến tốc độ nhanh nhất.

Tính toán:

Tần số thạch anh vi điều khiển sử dụng: $f_{OSC} = 20MHz$

nên chu kỳ là:
$$T_{OSC} = \frac{1}{F_{OSC}} = \frac{1}{20MHz} = 0,5\mu S = 50ns$$

Tính chu kỳ PWM: $PERIOD_{PWM} = [(PR2) + 1] * 4 * T_{OSC} * PV_{TMR2} = 0.8ms = 800,000ns$

Chỉ biết được $T_{OSC} = 50ns$ còn các thông số $PR2, PV_{TMR2}$ thì chưa biết.

Phải chọn 1 thông số và tính thông số còn lại: chú ý $PR2$ có giá trị 8 bit từ 0 đến 255, còn PV_{TMR2} có 3 giá trị là chia 1, chia 4 và chia 16.

Chọn hệ số chia lớn nhất là 16 hay $PV_{TMR2} = 16$

Khi đó tìm giá trị còn lại $PR2$:

$$[(PR2) + 1] = \frac{PERIOD_{PWM}}{4 * T_{OSC} * PV_{TMR2}} = \frac{800,000ns}{4 * 50ns * 16} = 250$$

Vậy $PR2 = 249$.

Gán giá trị 249 và hệ số chia trước cho timer T2 thì ta phải sử dụng Lệnh khởi tạo cho timer2 là:

`setup_timer_2(T2_DIV_BY_16, 249, 1)`

Tính hệ số chu kỳ (duty):

Hệ số chu kỳ thay đổi sẽ làm giá trị trung bình của tín hiệu thay đổi, hệ số chu kỳ nhỏ nhất là bằng 0, khi đó tín hiệu ra CCPx ở mức 0, hệ số chu kỳ tăng làm tín hiệu xuất hiện và giá trị trung bình tăng, hệ số chu kỳ lớn nhất bằng chu kỳ của PWM.

Cho hệ số chu kỳ bằng chu kỳ để tính toán giới hạn hệ số chu kỳ:

$$DUTY_CYCLE_{PWM} = PERIOD_{PWM_MAX}$$

Ta có công thức: $DUTY_CYCLE_{PWM} = (CCPRxL:CCPxCON < 5:4 >) * T_{OSC} * PV_{TMR2}$

Ta tìm giá trị lớn nhất của $CCPRxL:CCPxCON < 5:4 >$.

$$\text{Suy ra: } (CCPRxL:CCPxCON < 5:4 >) = \frac{DUTY_CYCLE_{PWM_MAX}}{T_{OSC} * PV_{TMR2}} = \frac{PERIOD_{PWM}}{T_{OSC} * PV_{TMR2}}$$

$$(CCPRxL:CCPxCON < 5:4 >) = \frac{PERIOD_{PWM}}{T_{OSC} * PV_{TMR2}} = \frac{800,000ns}{50ns * 16} = 1000$$

Vậy giới hạn của hệ số chu kỳ là từ 0 đến 1000 đối với các thông số đã cho.

Nếu số cấp điều khiển là 10 cấp thì giá trị cho mỗi cấp là 100.

Nếu số cấp điều khiển là 20 cấp thì giá trị cho mỗi cấp là 50.

Tùy theo yêu cầu mà bạn tính toán được giá trị thay đổi cho mỗi cấp.

9.3.7 Các bài thực hành thay đổi tốc độ động cơ dc bằng PWM

Bài mẫu 921. Chương trình dùng CCP1 ở chế độ PWM để điều khiển động cơ quay thuận thay đổi tốc độ bằng 3 nút điều khiển: UP, DW và nút STOP.

Khi nhấn UP thì động cơ quay thuận và tăng tốc lên, khi nhấn DW thì động cơ giảm tốc, nhấn STOP thì ngừng.

Số cấp điều khiển là 20 hiển thị trên 2 led 7 đoạn.

Sử dụng các thông số tính toán ở trên

Lưu tên file là “BAI_921_DC_PWM_CCP1_UP_DW_STOP”.

- Mục đích: biết lập trình sử dụng CCP1 điều khiển động cơ DC quay thuận thay đổi tốc độ.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_dcmotor.c>
void main()
{
    system_init();
    dcmotor.direct= dcmotor_direct_forward;
    dcmotor.duty = 0;
    while(true)
    {
        if((inputcd(UP)==0)&&(dcmotor.duty<1000)) // UP tăng tốc
            dcmotor.duty+=50 ;
        if((inputcd(DW)==0)&&(dcmotor.duty>0)) // DW giảm tốc
            dcmotor.duty-=50 ;
        if(input(STOP)==0) // STOP dừng
            dcmotor.duty=0;
        dcmotor_control();
        d7seg.led[1]= m7d[dcmotor.duty/50/10%10]; //Hiển thị cấp tốc độ
        d7seg.led[0]= m7d[dcmotor.duty/50%10];
        d7seg_display();
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: Sau khi nạp xong thì nhấn nút UP động cơ sẽ tăng tốc, nhấn DW thì giảm tốc, nhấn STOP thì động cơ ngừng. Chú ý khi ở cấp tốc độ 1 hoặc 2 thì moment chưa đủ làm động cơ quay nên ta tiếp tục tăng cấp cho đến khi đủ moment quay động cơ. Có led hiển thị các cấp tốc độ.
- Giải thích chương trình:

Bài tập 922. Hãy lấy kết quả tính toán ở trên và lập trình theo yêu cầu.

Nút nhấn BT0 làm nút stop để ngừng động cơ.

Nút nhấn BT1 thì điều khiển động cơ chạy 35% tốc độ tối đa.

Nút nhấn BT2 thì điều khiển động cơ chạy 70% tốc độ tối đa.

Nút nhấn BT3 thì điều khiển động cơ chạy 100% tốc độ tối đa.

Hiển thị số cấp trên module 4 led 7 đoạn.

Lưu tên file là “BAI_922_DC_PWM_CCP1_3_CAP_TOCDO”.

Bài tập 923. Dùng vi điều khiển PIC 18F4550 thay đổi tốc độ động cơ DC bằng 4 nút điều khiển: UP, DW, STOP, INV.

Khi nhấn UP thì động cơ tăng tốc lên, khi nhấn DW thì động cơ giảm tốc, nhấn STOP thì ngừng.

Khi nhấn INV thì động cơ ngừng và đảo chiều, 16 led phải sáng thì DC quay thuận, 16 led trái sáng thì động cơ quay nghịch.

Số cấp điều khiển là 20 hiển thị trên 2 led 7 đoạn.

Lưu tên file là “BAI_923_DC_PWM_2CCP_UP_DW_STOP_INV”.

Bài tập 924. Hãy lập trình điều khiển động DC có 3 nút: UP, DW, STOP.

Khi nhấn UP 1 lần (không cần giữ) thì động cơ tự động tăng tốc lên, khi nhấn DW thì động cơ giảm tốc, nhấn STOP thì ngừng.

Số cấp điều khiển là 20 hiển thị trên 2 led 7 đoạn, thời gian trễ là 1 giây.

Lưu tên file là “BAI_924_DC_UP_DW_STOP_AUTO”.

Bài mẫu 925. Giống bài 921 nhưng có thêm phần đo tốc độ động cơ theo đơn vị RPM và hiển thị trên 8 led quét. Cấp tốc độ hiển thị module 4 led.

Lưu tên file là “BAI_925_DC_UP_DW_STOP_ENCODER_T1”.

- Mục đích: đã biết sử dụng T1 để định thời và bây giờ biết đếm xung từ encode dùng counter T1.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_dcmotor.c>
void main()
{
    system_init();
    dcmotor.direct= dcmotor_direct_forward;
    dcmotor.duty = 0;
    while(true)
    {
        if ( (inputcd(UP)==0) && (dcmotor.duty<1000) )
            dcmotor.duty+=50;
        if ( (inputcd(DW)==0) && (dcmotor.duty>0) )
            dcmotor.duty-=50;
        if (input(STOP)==0)
            dcmotor.duty=0;
        dcmotor_control();
        if(dcmotor_read_speed()==OK)    // Đo tốc độ
        {
            s7seg.led[3]= m7d[dcmotor.speed /1000%10];
            s7seg.led[2]= m7d[dcmotor.speed /100%10];
            s7seg.led[1]= m7d[dcmotor.speed /10%10];
            s7seg.led[0]= m7d[dcmotor.speed %10];
        }
        s7seg_display();
        d7seg.led[1]= m7d[dcmotor.duty/50/10%10];
        d7seg.led[0]= m7d[dcmotor.duty/50%10];
    }
}
```

```

        d7seg_display();
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: bạn tăng tốc độ để nhìn thấy giá trị đếm trên led khi động cơ quay.
- f. Giải thích chương trình:

Bài tập 926. Hiệu chỉnh bài 925 sao cho thêm nút INV để đảo chiều quay của động cơ. Sau khi đảo chiều động cơ vẫn quay ở tốc độ cũ.

Lưu tên file là “BAI_926_DC_UP_DW_STOP_INV_DO_TOCDO”.

Bài tập 927. Hiệu chỉnh bài 926 thêm vào động cơ bước và cũng điều khiển thay đổi 20 cấp tốc độ bằng UP, DW, STOP và INV để đảo chiều. Không cần đo tốc độ động cơ bước.

Lưu tên file là “BAI_927_THEM926_DCBUOC”.

Bài tập 928. Giống bài 925 nhưng thay đổi tốc độ động cơ 10 cấp từ 0 đến 9 được chọn bằng 10 nút nhấn từ 0 đến 9 của bàn phím ma trận. Chọn chiều quay thuận hay nghịch bằng phím A, ngừng là phím B.

Lưu tên file là “BAI_928_DC_UP_DW_STOP_ENCODER_T1_8LED”.

9.4 ĐIỀU KHIỂN KẾT HỢP 2 ĐỘNG CƠ BƯỚC VÀ DC

Bài tập 931. Hãy viết chương trình điều khiển 2 động cơ: DC và bước bằng 4 nút: BT0(ON_MTDC), BT1(OFF_MTDC), BT2(ON_MTB), BT3(OFF_MTB).

Có led đơn báo trạng thái ngừng, quay của động cơ DC và động cơ bước.

Động cơ DC chỉ ON và OFF.

Lưu tên file là “BAI_931_DC_SMT_ON_OFF”.

Bài tập 932. Hãy viết chương trình điều khiển 2 động cơ: DC và bước bằng 3 nút: UP, DW, OFF. UP và DW dùng để thay đổi tốc độ cho cả 2 động cơ, số cấp là 10, hiển thị trên 2 led 7 đoạn. Khi nhấn OFF thì ngừng.

Lưu tên file là “BAI_932_DC_SMT_UP_DW_OFF”.

9.5 ĐIỀU KHIỂN ỔN ĐỊNH TỐC ĐỘ ĐỘNG CƠ DC

9.3.1 Giới thiệu PID

Phần này bạn nên xem lại lý thuyết điều khiển dùng PID.

9.3.2 Điều khiển động cơ dùng PID

Bài mẫu 933. Chương trình điều khiển động cơ DC ổn định tốc độ dùng phương pháp điều khiển PID. Nút UP dùng để tăng tốc độ, nút DW dùng để giảm tốc độ và STOP dùng để ngừng.

Lưu tên file là “BAI_933_DC_UP_DW_STOP_PID”.

- Mục đích: biết điều khiển ổn định tốc độ động cơ dùng PID.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include<tv_boards.c>
#include<tv_dcmotor.c>
#include<tv_lcd.c>
unsigned int32 time;
signed int16 speedset=0,e,ecu=0;
// Tùy hệ thống mà chỉnh lại các hệ số kp,kd,ki cho hệ thống đáp ứng tốt
float kp =0.5,kd =0.002,ki=0.002,tp=0,td=0,ti=0;
int1 step=0;
void main()
{
    system_init();
    lcd_setup();
    lcd_data("SPEED SET : (rpm)");
    lcd_command(0xc0);
    lcd_data("SPEED READ: (rpm)");
    dcmotor.direct= dcmotor_direct_forward;
    dcmotor.duty = 0;
    while(true)
    {
        if((inputcd(UP)==0)&&(dcmotor.duty<1000))
            speedset+=50;
        if((inputcd(DW)==0)&&(speedset>0))
            speedset-=50;
        dcmotor_control();
        dcmotor_read_speed(); // Đo tốc độ

        if(step==0) //Điều chỉnh tốc độ mỗi 100ms
        {
            time = delay_start(); //Nên dùng ngắt sẽ hay hơn
            step=1;
        }
        else if(delay_stop(time)>=100)
        {
            e= speedset-dcmotor.speed;
            tp = e;
            td = (e-ecu)*1000/delay_stop(time);
            ti = ti+e*delay_stop(time)/1000;
            ecu=e;
            dcmotor.duty = dcmotor.duty + kp*tp + kd*td + ki*ti;
            if(dcmotor.duty>1000) dcmotor.duty =1000;
            else if(dcmotor.duty<0) dcmotor.duty=0;
            step=0;
        }
        lcd_goto(11,0);
        printf lcd_data,"%04ld",speedset);
    }
}
```

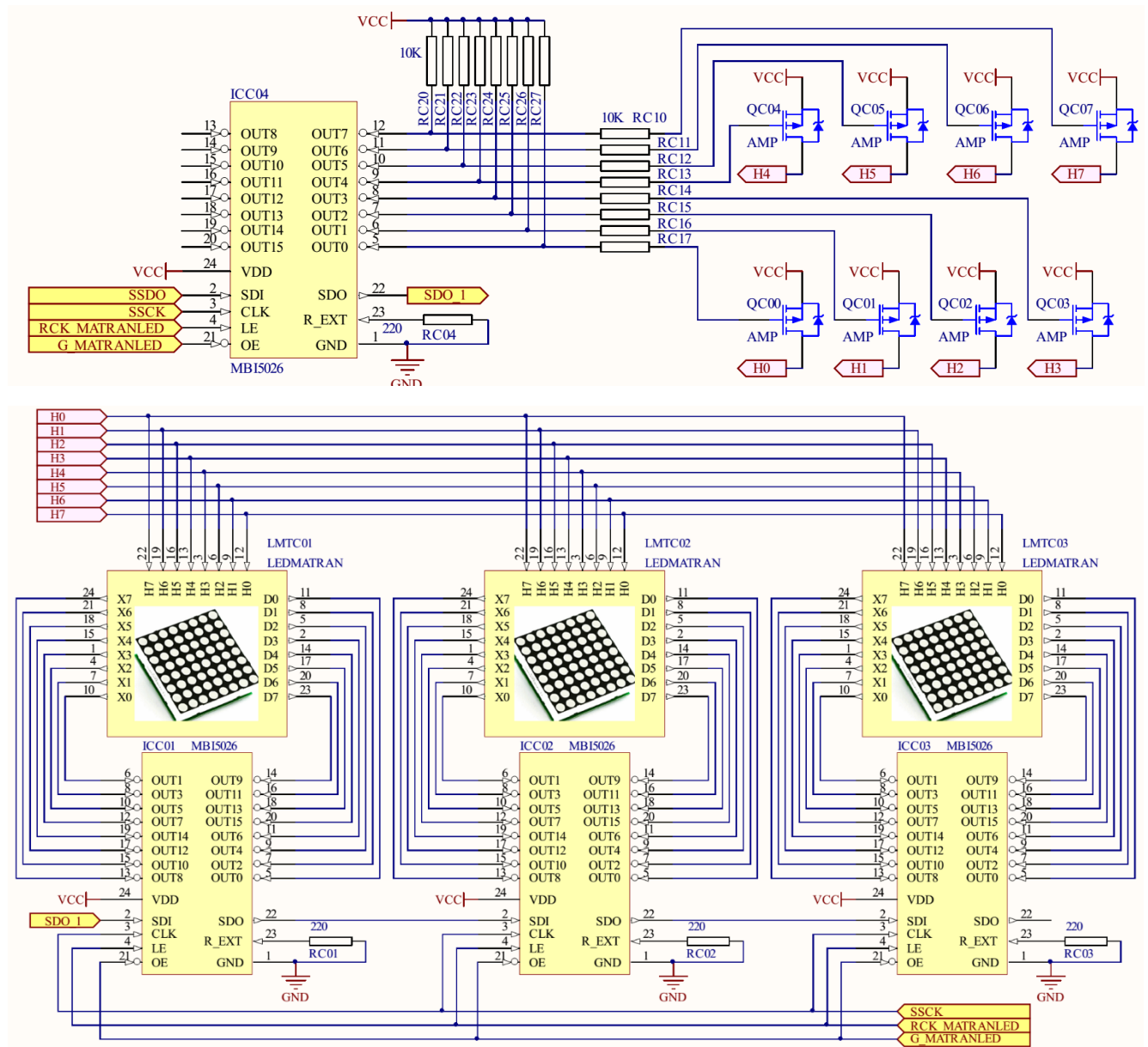
```
        lcd_goto(11,1);  
        printf lcd_data,"%04ld",dcmotor.speed);  
    }  
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: Hàng 1 hiển thị cấp tốc độ, hàng 2 hiển thị tốc độ động cơ, hàng 3 hiển thị số xung cài đặt trước, hàng 4 hiển thị số xung đếm được từ encoder. Sau khi nạp xong chương trình thì nhấn nút UP để tăng giá trị tốc độ cài đặt lên 50 đơn vị, hệ thống sẽ tự động điều chỉnh tốc độ thực gần đúng với tốc độ cài.
- f. Giải thích chương trình: SV tự giải thích. Để thấy được sự ổn định thì phải có tải thay đổi nhưng trong bộ thí nghiệm không có kéo tải nên khó nhìn thấy chức năng của PID.

Chương 10: CÁC BÀI THỰC HÀNH MODULE 7 – ĐIỀU KHIỂN LED MA TRẬN 2 MÀU

10.1 SƠ ĐỒ MẠCH GIAO TIẾP VI ĐIỀU KHIỂN VỚI LED MA TRẬN

Trong phần này sẽ trình bày phần giao tiếp vi điều khiển PIC18F4550 với 3 led ma trận như hình 10-1:



Hình 10-1. Sơ đồ nguyên lý giao tiếp vi điều khiển với 3 led ma trận.

Kit có tích hợp module giao tiếp 3 led ma trận 2 màu xanh và đỏ, sử dụng 4 IC thanh ghi dịch 16 bit để điều khiển hàng và cột.

10.2 MÃ KÍ TỰ CHO LED MA TRẬN DẠNG QUÉT CỘT

Theo sơ đồ mạch điện có 3 led ma trận 8×8 sẽ có 8 hàng và 24 cột, có 2 phương pháp điều khiển led: quét cột và quét hàng.

Có 24 cột thì khi quét sẽ có 1 cột sáng, 23 cột tắt, thời gian sáng cho 1 cột là 1/24, thời gian ngắn nên led sẽ sáng mờ - đây là khuyết điểm. Ưu điểm là lập trình tạo mã hiển thị cho các kí tự đơn giản.

Có 8 hàng thì khi quét sẽ có 1 hàng sáng, 7 hàng tắt, thời gian sáng cho 1 hàng là 1/8, thời gian dài hơn so với quét cột nên led sáng rõ, đẹp – đây là ưu điểm. Khuyết điểm là lập trình tạo mã hiển thị cho các kí tự liên kết khá phức tạp.

a. Mã quét cột:

Mã quét cột cho 1 led gồm 16 bit: 8 bit cho 8 cột đỏ, 8 bit cho 8 cột xanh, khi quét cột thì mỗi một thời điểm chỉ cho 1 cột ở mức logic 1, các cột còn lại ở mức 0.

Do kết nối mạch tối ưu theo vị trí các chân IC thanh ghi dịch MBI 5026 với các led nên thứ tự các chân điều khiển không liên tục với nhau.

Bảng 10-1. Mã quét cột cho led màu đỏ sáng, xanh tắt.

T T	bit	15	14	13	12	11	10	9	8		7	6	5	4	3	2	1	0		
	Cột	Đ 3	X 3	Đ 2	X 2	Đ 1	X 1	Đ 0	X 0	B H	X 4	Đ 4	X 5	Đ 5	X 6	Đ 6	X 7	Đ 7	B L	2 BYT E
1								1		02									00	0200
2						1				08									00	0800
3				1						20									00	2000
4		1								80									00	8000
5										00		1							40	0040
6										00			1						10	0010
7										00						1			04	0004
8										00							1		01	0001

{0x0200,0x0800,0x2000,0x8000,0x0040,0x0010,0x0004,0x0001}

Chú ý: các ô còn lại đều bằng 0.

Bảng 10-2. Mã quét cột cho led màu xanh sáng, đỏ tắt.

T T	bit	15	14	13	12	11	10	9	8		7	6	5	4	3	2	1	0		
	Cột	Đ 3	X 3	Đ 2	X 2	Đ 1	X 1	Đ 0	X 0	B H	X 4	Đ 4	X 5	Đ 5	X 6	Đ 6	X 7	Đ 7	B L	2 BYT E
1									1	01									00	0100
2							1			04									00	0400
3					1					10									00	1000
4			1							40									00	4000
5										00	1								80	0080
6										00			1						20	0020
7										00					1				08	0008

8									00							1		02	0002
---	--	--	--	--	--	--	--	--	----	--	--	--	--	--	--	---	--	----	------

{0x0100,0x0400,0x1000,0x4000,0x0080,0x0020,0x0008,0x0002}

Bảng 10-3. Mã quét cột cho led màu xanh và đỏ đều sáng thành màu cam.

T	bit	15	14	13	12	11	10	9	8		7	6	5	4	3	2	1	0		
T	Cột	Đ 3	X 3	Đ 2	X 2	Đ 1	X 1	Đ 0	X 0	B H	X 4	Đ 4	X 5	Đ 5	X 6	Đ 6	X 7	Đ 7	B L	2 BYTE
1								1	1	03									00	0300
2						1	1			0C									00	0C00
3				1	1					30									00	3000
4		1	1							C0									00	C000
5										00	1	1							C0	00C0
6										00			1	1					30	0030
7										00					1	1			0C	000C
8										00							1	1	03	0003

{0x0300,0x0C00,0x3000,0xC000,0x00C0,0x0030,0x000C,0x0003}

Mã điều khiển cả 2 màu xanh đỏ có thể OR 2 mã của riêng từng led với nhau.

b. Mã kí tự cho kiểu quét cột:

Dữ liệu điều khiển hàng: mức 1 thì led sáng, mức 0 thì led tắt: mỗi kí tự dùng ma trận 5×7 và một cột khoảng cách giữa 2 kí tự nên tổng số kí tự là 6.

Bảng 10-4. Mã các kí tự mẫu A, B, C, D.

	CHỮ A						CHỮ B						CHỮ C						CHỮ D					
Bit	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
B7(8)																								
B6(4)			X				X	X	X	X				X	X	X			X	X	X	X		
B5(2)		X		X			X				X		X				X		X				X	
B4(1)	X				X		X				X		X						X				X	
B3(8)	X				X		X	X	X	X			X						X				X	
B2(4)	X	X	X	X	X		X				X		X						X				X	
B1(2)	X				X		X				X		X				X		X				X	
B0(1)	X				X		X	X	X	X				X	X	X			X	X	X	X		
MÃ	1F	24	44	24	1F	00	7F	49	49	49	36	00	3E	41	41	41	22	00	7F	41	41	41	3E	00

Bảng 10-5. Bảng mã ma trận của các kí tự.

TT	Kí tự	Mã ma trận
----	-------	------------

1	A	0×1F,0×24,0×44,0×24,0×1F,0×00
2	B	0×7F,0×49,0×49,0×49,0×36,0×00
3	C	0×3E,0×41,0×41,0×41,0×22,0×00
4	D	0×7F,0×41,0×41,0×41,0×3E,0×00
5	E	0×3E,0×49,0×49,0×41,0×22,0×00
6	F	0×7F,0×48,0×48,0×48,0×40,0×00
7	G	0×3E,0×41,0×49,0×49,0×2E,0×00
8	H	0×7F,0×08,0×08,0×08,0×7F,0×00
9	I	0x41,0x41,0x7F,0x41,0x41,0x00
10	J	0x42,0x41,0x7E,0x40,0x40,0x00
11	K	0x7F,0x08,0x14,0x22,0x41,0x00
12	L	0x7F,0x01,0x01,0x01,0x02,0x00
13	M	0x7F,0x20,0x10,0x20,0x7F,0x00
14	N	0x7F,0x20,0x10,0x08,0x7F,0x00
15	O	0x3E,0x41,0x41,0x41,0x3E,0x00
16	P	0x3F,0x48,0x48,0x48,0x30,0x00
17	Q	0x3E,0x41,0x45,0x43,0x3E,0x0
18	R	0x3F,0x48,0x4C,0x4A,0x31,0x00
19	S	0x32,0x49,0x49,0x49,0x26,0x00
20	T	0x60,0x40,0x7F,0x40,0x60,0x00
21	U	0x7E,0x01,0x01,0x01,0x7E,0x00
22	V	0x7C,0x02,0x01,0x02,0x7C,0x00
23	W	0x7E,0x01,0x06,0x01,0x7E,0x00
24	X	0x41,0x22,0x1C,0x22,0x41,0x00
25	Y	0x70,0x08,0x07,0x08,0x70,0x00
26	Z	0x63,0x45,0x49,0x51,0x63,0x00
27	0	0x3E,0x41,0x41,0x41,0x3E,0x00

28	1	0x11,0x21,0x7F,0x01,0x01,0x00
29	2	0x27,0x49,0x49,0x49,0x31,0x00
30	3	0x22,0x41,0x49,0x49,0x36,0x00
31	4	0x0C,0x14,0x24,0x7F,0x04,0x00
32	5	0x72,0x51,0x51,0x51,0x4E,0x00
33	6	0x3E,0x49,0x49,0x49,0x26,0x00
34	7	0x41,0x42,0x44,0x48,0x70,0x00
35	8	0x36,0x49,0x49,0x49,0x36,0x00
36	9	0x32,0x49,0x49,0x49,0x3E,0x00

10.3 CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN LED MA TRẬN QUÉT CỘT

Thư viện điều khiển led ma trận theo kiểu quét cột. Chương trình thư viện có tên là: “TV_PICKIT2_SHIFT_MATRAN_QUET_COT.c”.

Đã có trong thư mục của bạn, bạn có thể sử dụng cho các bài thực hành led ma trận theo kiểu quét cột và nên đọc hiểu.

```
#include <tv_pickit2_shift_font.c>
//du lieu ma quet cot
const unsigned int16 ma_quet_cot_do[8]={
0x0200,0x0800,0x2000,0x8000,0x0040,0x0010,0x0004,0x0001};
const unsigned int16 ma_quet_cot_xanh[]={
0x0100,0x0400,0x1000,0x4000,0x0080,0x0020,0x0008,0x0002};
const unsigned int16 ma_quet_cot_cam[]={
0x0300,0x0c00,0x3000,0xc000,0x00c0,0x0030,0x000c,0x0003};

unsigned int8    dl_hienthi[6*20];
unsigned int8    slkt,mtb0,mtb1,mtb2;
unsigned int16   ma_hang, mqc_led0, mqc_led1, mqc_led2;
unsigned int32   tt_qc=1;
#define  tg_delay    100

unsigned int8 doi_1_bit_1_sang_thutu(unsigned int8 bx)
{
    unsigned int8 ttcot=0;
    bx= bx>>1;
    while(bx!=0)
    {
        ttcot++;
        bx= bx>>1;
    }
    return(ttcot);
}
```

```

}

void lay_ma_ky_tu(unsigned int8 slkt_x)
{
    unsigned int8 kytu,i,n,j;
    n=0;
    for (i=0; i<slkt_x; i++)
    {
        kytu = chuoi_hienthi[i];
        if ((kytu>=0x41)&&(kytu<=0x5a))          kytu = kytu - 0x41;
        else if ((kytu>=0x30)&&(kytu<=0x39))      kytu = (kytu - 0x30) + 26;
        for(j=0;j<6 ;j++,n++)
            dl_hienthi[n]=hieu_chinh_4bit_cao(matran_kytu[kytu][j]);
    }
}

void lay_ma_quet_24_cot_xanh()
{
    mtb0 = tt_qc;
    mtb1 = tt_qc>>8;
    mtb2 = tt_qc>>16;
    if (mtb0!=0)
    {
        mtb0=doi_1_bit_1_sang_thutu(mtb0);
        mqc_led0 = ma_quet_cot_xanh[mtb0];
        mqc_led1 = 0;
        mqc_led2 = 0;
    }
    else if (mtb1!=0)
    {
        mtb1=doi_1_bit_1_sang_thutu(mtb1);
        mqc_led0 = 0;
        mqc_led1 = ma_quet_cot_xanh[mtb1];
        mqc_led2 = 0;
    }
    else if (mtb2!=0)
    {
        mtb2=doi_1_bit_1_sang_thutu(mtb2);
        mqc_led0 = 0;
        mqc_led1 = 0;
        mqc_led2 = ma_quet_cot_xanh[mtb2];
    }
}

void lay_ma_quet_24_cot_do()
{
    mtb0 = tt_qc;
    mtb1 = tt_qc>>8;
    mtb2 = tt_qc>>16;
    if (mtb0!=0)
    {
        mtb0=doi_1_bit_1_sang_thutu(mtb0);
        mqc_led0 = ma_quet_cot_do[mtb0];
        mqc_led1 = 0;
        mqc_led2 = 0;
    }
}

```

```

    }
    else if (mtb1!=0)
    {
        mtb1=doi_1_bit_1_sang_thutu(mtb1);
        mqc_led0 = 0;
        mqc_led1 = ma_quet_cot_do[mtb1];
        mqc_led2 = 0;
    }
    else if (mtb2!=0)
    {
        mtb2=doi_1_bit_1_sang_thutu(mtb2);
        mqc_led0 = 0;
        mqc_led1 = 0;
        mqc_led2 = ma_quet_cot_do[mtb2];
    }
}

```

```

void lay_ma_quet_24_cot_cam()
{
    mtb0 = tt_qc;
    mtb1 = tt_qc>>8;
    mtb2 = tt_qc>>16;
    if (mtb0!=0)
    {
        mtb0=doi_1_bit_1_sang_thutu(mtb0);
        mqc_led0 = ma_quet_cot_cam[mtb0];
        mqc_led1 = 0;
        mqc_led2 = 0;
    }
    else if (mtb1!=0)
    {
        mtb1=doi_1_bit_1_sang_thutu(mtb1);
        mqc_led0 = 0;
        mqc_led1 = ma_quet_cot_cam[mtb1];
        mqc_led2 = 0;
    }
    else if (mtb2!=0)
    {
        mtb2=doi_1_bit_1_sang_thutu(mtb2);
        mqc_led0 = 0;
        mqc_led1 = 0;
        mqc_led2 = ma_quet_cot_cam[mtb2];
    }
}

```

```

void lay_so_luong_ky_tu_chuoi()
{
    slkt=0;
    while(chuoi_hienthi[slkt])
    {
        slkt++;
    }
    xuat_4led_7doan_giaima_xoa_so0(slkt);
}

```

```

void quet_4kytu_mau_xanh(unsigned int8 sck, unsigned int8 vitri)

```

```

{
    unsigned int8 ck,j;
    for (ck=0;ck<sck;ck++)
    {
        tt_qc = 1;
        for(j=vitri;j<vitri+24;j++)
        {
            lay_ma_quet_24_cot_xanh();
            ma_hang = dl_hienthi[j];
            xuất_matranled(ma_hang,mqc_led2,mqc_led1,mqc_led0);
            delay_us(tg_delay);
            xuất_matranled(0,0,0,0);
            tt_qc = tt_qc << 1;
        }
    }
}

void quet_4kytu_mau_do(unsigned int8 sck, unsigned int8 vitri)
{
    unsigned int8 ck,j;
    for (ck=0;ck<sck;ck++)
    {
        tt_qc = 1;
        for(j=vitri;j<vitri+24;j++)
        {
            lay_ma_quet_24_cot_do();
            ma_hang = dl_hienthi[j];
            xuất_matranled(ma_hang,mqc_led2,mqc_led1,mqc_led0);
            delay_us(tg_delay);
            xuất_matranled(0,0,0,0);
            tt_qc = tt_qc <<1;
        }
    }
}

void quet_4kytu_mau_cam(unsigned int8 sck, unsigned int8 vitri)
{
    unsigned int8 ck, j;
    for (ck=0;ck<sck;ck++)
    {
        tt_qc = 1;
        for(j=vitri;j<vitri+24;j++)
        {
            lay_ma_quet_24_cot_cam();
            ma_hang = dl_hienthi[j];
            xuất_matranled(ma_hang,mqc_led2,mqc_led1,mqc_led0);
            delay_us(tg_delay);
            xuất_matranled(0,0,0,0);
            tt_qc = tt_qc <<1;
        }
    }
}

```

Bài mẫu 1001. Chương trình điều khiển LED ma trận hiển thị 4 ký tự chữ ABCD màu đỏ.

Lưu tên file là “BAI_1001_MT_ABCD_DO”

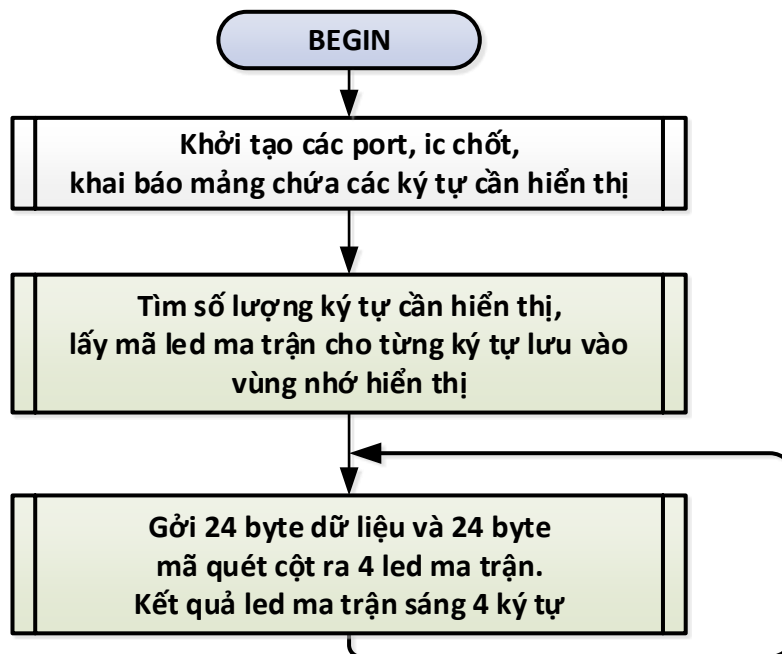
- Mục đích: biết mã ma trận, chuyển đổi mã ASCII sang mã led ma trận, xử lý đúng thứ tự và quét led ma trận theo cột.
- Lưu đồ: như hình 10-2.
- Giải thích lưu đồ :

Phần đầu thì khởi tạo port và IC chốt giống như các chương trình trước, khai báo mảng chứa các ký tự cần hiển thị.

Phần tiếp theo thì tìm số lượng ký tự trong mảng cần hiển thị là bao nhiêu, tối đa không quá 20 ký tự vì vùng nhớ đệm khai báo trong thư viện tối đa là 20.

Sau đó lấy mã led ma trận của từng ký tự có trong vùng nhớ đã tạo sẵn trong thư viện, mỗi ký tự 6 byte, 5 byte và 1 byte khoảng cách giữa 2 ký tự lưu vào vùng nhớ để hiển thị.

Tiến hành gửi dữ liệu của vùng nhớ hiển thị ra ma trận led.



Hình 10-2. Lưu đồ điều khiển hiển thị 4 ký tự trên 3 led ma trận.

d. Chương trình:

```

#include <tv_pickit2_shift_1.c>
const unsigned char  chuoi_hienthi[]={ "abcd" };
#include <tv_pickit2_shift_matran_quet_cot.c>
void main()
{
    set_up_port_ic_chot();
    lay_so_luong_ky_tu_chuoi();
    lay_ma_ky_tu(slkt);
    while(true)
    {

```



```

    quet_4kytu_mau_do(50,0);
}
}

```

- e. Tiến hành biên dịch và nạp.
- f. Quan sát kết quả: sau khi nạp xong thì LED sáng chữ ABCD. Bạn có thể thay đổi sang các kí tự khác.
- g. Giải thích chương trình: các chương trình xử lý: lấy số lượng chuỗi cần hiển thị, chuyển mã ASCII sang mã led ma trận lưu vào vùng nhớ hiển thị rồi hiển thị ra led.

Bài tập 1002. Hãy điều chỉnh trình bài 1001 để hiển thị “0123” màu xanh.

Lưu tên file là “BAI_1002_MT_0123_XANH”

Bài tập 1003. Hãy điều chỉnh trình bài 1001 để hiển thị “ILOY” màu cam.

Lưu tên file là “BAI_1003_MT_ILOY_CAM ”

Bài tập 1004. Chương trình điều khiển LED ma trận hiển thị “ILOY” màu xanh, màu đỏ rồi sau đó hiển thị màu cam và lặp lại.

Lưu tên file là “BAI_1004_MT_ILOY_XANH_DO_CAM”

Bài tập 1005. Chương trình điều khiển LED ma trận hiển thị chuỗi “DAIHOCSUPHAMKYTHUAT” màu xanh.

Lưu tên file là “BAI_1005_MT_CHUOI_XANH_DICH_TRAI”

- a. Mục đích: biết hiển thị dịch chuỗi.
- b. Lưu đồ: sinh viên hãy tự viết.
- c. Chương trình:

```

#include <tv_pickit2_shift_1.c>
const unsigned char chuoi_hienthi[]={"abcdefghjikl"};
#include <tv_pickit2_shift_matran_quet_cot.c>
unsigned int16 vt;
void main()
{
    set_up_port_ic_chot();
    lay_so_luong_ky_tu_chuoi();
    lay_ma_ky_tu(slkt);
    while(true)
    {
        for (vt=0; vt < (slkt-4)*6; vt++)
            quet_4kytu_mau_xanh(30,vt);
    }
}

```

- d. Tiến hành biên dịch và nạp.

- e. Quan sát kết quả: sau khi nạp xong thì led ma trận hiển thị 4 kí tự màu xanh, sau đó dịch sang trái.
- f. Giải thích chương trình:

Bài tập 1006. Hãy lập trình giống bài 1005 nhưng hiển thị lần lượt 3 màu xanh, đỏ, cam.

Lưu tên file là “BAI_1006_MT_CHUOI_XANH_DO_CAM_DICH_TRAI”

Bài tập 1007. Hãy lập trình đếm từ 0 đến 9 hiển thị trên led ma trận màu xanh.

Thời gian delay 1s gần đúng hoặc chính xác thì dùng timer T1.

Lưu tên file là “BAI_1007_MT_DEM_1SO_XANH”

Bài tập 1008. Hãy lập trình đếm từ 00 đến 99 hiển thị trên led ma trận màu xanh.

Thời gian delay 1s gần đúng hoặc chính xác thì dùng timer T1.

Lưu tên file là “BAI_1008_MT_DEM_2SO_XANH”

Bài tập 1009. Hãy lập trình đếm từ 000 đến 999 hiển thị trên led ma trận màu xanh.

Thời gian delay 1s gần đúng hoặc chính xác thì dùng timer T1.

Lưu tên file là “BAI_1009_MT_DEM_3SO_XANH”

10.4 MÃ KÍ TỰ CHO LED MA TRẬN DẠNG QUÉT HÀNG

Ở các bài thực hành led ma trận theo phương pháp quét cột thì led sáng không rõ với nguyên nhân đã trình bày, phần này sẽ trình bày cách quét hàng. Khi quét hàng thì mỗi một thời điểm chỉ có 1 hàng sáng, muốn led nào sáng thì cột tương ứng sẽ ở mức 0. Mỗi một cột chỉ có 1 led sáng duy nhất nên ngõ vào của IC dễ dàng đáp ứng nên led sẽ sáng rõ, lý do thứ 2 là chỉ có 8 hàng nên thời gian sáng là $1/8$ so với quét cột là $1/24$ – đây chính là ưu điểm của quét hàng.

Khuyết điểm của quét hàng là phải xử lý mã kí tự cần hiển thị: một kí tự gồm 5 byte:

Ở quét cột thì lấy byte thứ 1 gởi ra hàng – cho cột thứ 1 sáng, tương tự cho đến byte thứ 5 và cột thứ 5 sáng.

Ở quét hàng thì lấy bit thứ 0 của byte thứ 1 gởi ra cột thứ 1, lấy bit thứ 0 của byte thứ 2 gởi ra cột thứ 2, lấy bit thứ 0 của byte thứ 3 gởi ra cột thứ 3, lấy bit thứ 0 của byte thứ 4 gởi ra cột thứ 4, lấy bit thứ 0 của byte thứ 5 gởi ra cột thứ 5 – cho hàng thứ 0 sáng. Tại thời điểm này thì hàng thứ 0 của kí tự sáng.

Tiếp tục: lấy bit thứ 1 của byte thứ 1 gởi ra cột thứ 1, lấy bit thứ 1 của byte thứ 2 gởi ra cột thứ 2, lấy bit thứ 1 của byte thứ 3 gởi ra cột thứ 3, lấy bit thứ 1 của byte thứ 4 gởi ra cột thứ 4, lấy bit thứ 1 của byte thứ 5 gởi ra cột thứ 5 – cho hàng thứ 1 sáng. Tại thời điểm này thì hàng thứ 1 của kí tự sáng.

Tương tự cho bit thứ 2 của các byte – hàng thứ 2 sáng, bit thứ 3 của các byte – hàng thứ 3 sáng, bit thứ 4 của các byte – hàng thứ 4 sáng, ..., bit thứ 7 của các byte – hàng thứ 7 sáng.

Do có 3 led 24 cột thì phải thực hiện 24 byte.

Mã quét hàng: mỗi một thời điểm chỉ cho 1 hàng ở mức logic 1, các hàng còn lại bằng 0.

Bảng 10-6. Mã quét hàng.

Bit	0	1	2	3	4	5	6	7	Số hex
B4(1)				1					10
B5(2)			1						20
B6(4)		1							40
B7(8)	1								80
B3(8)					1				08
B2(4)						1			04
B1(2)							1		02
B0(1)								1	01

Mã quét hàng theo sơ đồ mạch: {0×01,0×02,0×04,0×08,0×80,0×40,0×20,0×10}

Do có 3 led 24 cột xanh và 24 cột đỏ nên tổng số cột là 48 cột, số byte cần để lưu dữ liệu cho 48 cột là 48 byte từ byte thứ 0 cho cột thứ 0 đến byte thứ 47 cho cột thứ 47.

Trong các chương trình điều khiển quét hàng ta phải khai báo mảng có 48 byte để chứa dữ liệu dịch ra cột: 24 byte cho cột màu đỏ, 24 byte cho cột màu xanh.

Do kết nối phân cứng thuận cho việc kết nối dây nên các cột không theo thứ tự liên tục mà chúng theo thứ tự như bảng sau:

Bảng 10-7. Bảng thứ tự các cột màu đỏ.

TT 48 BYTE GỎI RA CỘT	00	01	02	03	04	05	06	07
CỘT LED1	CD3		CD2		CD1		CD0	
TT 48 BYTE GỎI RA CỘT	08	09	10	11	12	13	14	15
CỘT LED1		CD4		CD5		CD6		CD7
TT 48 BYTE GỎI RA CỘT	16	17	18	19	20	21	22	23
CỘT LED2	CD11		CD10		CD9		CD8	
TT 48 BYTE GỎI RA CỘT	24	25	26	27	28	29	30	31
CỘT LED2		CD12		CD13		CD14		CD15
TT 48 BYTE GỎI RA CỘT	32	33	34	35	36	37	38	39

CỘT LED3	CD19		CD18		CD17		CD16	
TT 48 BYTE GỒI RA CỘT	40	41	42	43	44	45	46	47
CỘT LED3		CD20		CD21		CD22		CD23

Giải thích cho các thông tin trong bảng:

Dữ liệu của byte thứ 0 (00) khi dịch ra cột thì sẽ làm cột thứ 3 của led đỏ sáng (CD3).

Dữ liệu của byte thứ 2 (02) khi dịch ra cột thì sẽ làm cột thứ 2 của led đỏ sáng (CD2).

Dữ liệu của byte thứ 4 (04) khi dịch ra cột thì sẽ làm cột thứ 1 của led đỏ sáng (CD1).

Dữ liệu của byte thứ 6 (06) khi dịch ra cột thì sẽ làm cột thứ 0 của led đỏ sáng (CD0).

Dữ liệu của byte thứ 9 (09) khi dịch ra cột thì sẽ làm cột thứ 4 của led đỏ sáng (CD4).

Tương tự cho các cột đỏ còn lại, các ô trống là của 24 cột màu xanh – xem bảng tiếp theo.

Bảng 10-8. Bảng thứ tự các cột màu xanh.

TT 48 BYTE GỒI RA CỘT	00	01	02	03	04	05	06	07
LED1		CX3		CX2		CX1		CX0
TT 48 BYTE GỒI RA CỘT	08	09	10	11	12	13	14	15
LED1	CX4		CX5		CX6		CX7	
TT 48 BYTE GỒI RA CỘT	16	17	18	19	20	21	22	23
LED2		CX11		CX10		CX9		CX8
TT 48 BYTE GỒI RA CỘT	24	25	26	27	28	29	30	31
LED2	CX12		CX13		CX14		CX15	
TT 48 BYTE GỒI RA CỘT	32	33	34	35	36	37	38	39
LED3		CX19		CX18		CX17		CX16
TT 48 BYTE GỒI RA CỘT	40	41	42	43	44	45	46	47
LED3	CX20		CX21		CX22		CX23	

Ví dụ 10-1: Yêu cầu muốn hiển thị một chuỗi 4 kí tự “ABCD” trên 3 led ma trận 8×8, sáng màu đỏ. Mỗi kí tự có 5 byte và 1 byte khoảng cách giữa 2 kí tự liên tiếp nên tổng là 6 byte, 4 kí tự thì tổng là 24 byte.

Chuỗi mã của 4 kí tự như sau:

0×1F,0×24,0×44,0×24,0×1F,0×00: sẽ hiển thị từ cột thứ 0 đến cột thứ 5

0×7F,0×49,0×49,0×49,0×36,0×00: sẽ hiển thị từ cột thứ 6 đến cột thứ 11

0×3E,0×41,0×41,0×41,0×22,0×00: sẽ hiển thị từ cột thứ 12 đến cột thứ 17

0×7F,0×41,0×41,0×41,0×3E,0×00: sẽ hiển thị từ cột thứ 18 đến cột thứ 23

24 byte của 4 kí tự trên sẽ theo thứ tự tương ứng với 24 cột màu đỏ nhưng do thứ tự kết nối không liên tục nên cần phải copy chuỗi dữ liệu 24 byte của 4 kí tự ABCD ở trên tương ứng với thứ tự 48 byte gởi ra cột như sau:

Byte thứ 0 của kí tự A là 0x1F sẽ gán cho byte thứ 06 của mảng 48 byte, xem bảng 10-7.

Byte thứ 1 của kí tự A là 0x24 sẽ gán cho byte thứ 04 của mảng 48 byte, xem bảng 10-7.

Tương tự cho các byte còn lại.

Thư viện điều khiển led ma trận theo kiểu quét hàng. Chương trình thư viện có tên là:

“TV_PICKIT2_SHIFT_MATRAN_QUET_HANG.c”.

Đã có trong thư mục của bạn, bạn có thể sử dụng cho các bài thực hành led ma trận theo kiểu quét hàng và nên đọc hiểu.

```
#include <tv_pickit2_shift_font.c>
const unsigned int8
ma_quet_hang[8]={0x01,0x02,0x04,0x08,0x80,0x40,0x20,0x10};
#define slkt_hienthi 40
unsigned int8 dl_hienthi[6*slkt_hienthi];
unsigned int8 slkt;

unsigned int8 dl_cot_cd[48] ={};
unsigned int8 dl_cot_dich[48] ={};
unsigned int8 dl_cot_cd_do[24] ={};
unsigned int8 dl_cot_cd_xanh[24] ={};
unsigned int8 dl_cot_dich_d[24] ={};
unsigned int8 dl_cot_dich_x[24] ={};
unsigned int16 vtk1;

unsigned int8 doi_1_bit_1_sang_thutu(unsigned int8 bx)
{
    unsigned int8 ttcot=0;
    bx= bx>>1;
    while(bx!=0)
    {
        ttcot++;
        bx= bx>>1;
    }
    return(ttcot);
}

void xoa_dl_hienthi()
{
    unsigned int8 j;
    for(j=0;j<6*slkt_hienthi;j++)
        dl_hienthi[j]=0;
}

void lay_ma_1_so_matran(unsigned int8 so, unsigned int8 vitri_luu)
{

```

```

    unsigned int8  n,j;
    n=0;
    so = so + 26;
    for(j=0;j<6 ;j++,n++)
        dl_hienthi[n+vitri_luu*6] = matran_kytu[so][j];
}

void lay_so_luong_ky_tu_chuoi()
{
    slkt=0;
    while(chuoi_hienthi[slkt])
    {
        slkt++;
    }
    xuat_4led_7doan_giaima_xoa_so0(slkt);
}

void lay_ma_ky_tu(unsigned int8 slkt_x)
{
    unsigned int8  kytu,i,n,j;
    n=0;
    for (i=0; i<slkt_x; i++)
    {
        kytu = chuoi_hienthi[i];
        if ((kytu>=0x41)&&(kytu<=0x5a))          kytu = kytu - 0x41;
        else if ((kytu>=0x30)&&(kytu<=0x39))      kytu = (kytu - 0x30) + 26;
        for(j=0;j<6 ;j++,n++)
            dl_hienthi[n]=matran_kytu[kytu][j];
    }
}

void tinh_vi_tri_ky_tu(unsigned int16 k)
{
    unsigned int8 y;
    y=k/6;
    vtktl=(y*6)+5;
}
//sap xep cac ky tu mau xanh 1,2,3,4
void ky_tu_xanh_so_1(unsigned int8 k)
{
    dl_cot_cd[1]=dl_hienthi[k+3];    dl_cot_cd[3]=dl_hienthi[k+2];
    dl_cot_cd[5]=dl_hienthi[k+1];    dl_cot_cd[7]=dl_hienthi[k+0];
    dl_cot_cd[8]=dl_hienthi[k+4];    dl_cot_cd[10]=dl_hienthi[k+5];
}

void ky_tu_xanh_so_2(unsigned int8 k)
{
    dl_cot_cd[12]=dl_hienthi[k+6];    dl_cot_cd[14]=dl_hienthi[k+7];
    dl_cot_cd[17]=dl_hienthi[k+11];   dl_cot_cd[19]=dl_hienthi[k+10];
    dl_cot_cd[21]=dl_hienthi[k+9];    dl_cot_cd[23]=dl_hienthi[k+8];
}

void ky_tu_xanh_so_3(unsigned int8 k)
{
    dl_cot_cd[24]=dl_hienthi[k+12];   dl_cot_cd[26]=dl_hienthi[k+13];
    dl_cot_cd[28]=dl_hienthi[k+14];   dl_cot_cd[30]=dl_hienthi[k+15];
    dl_cot_cd[37]=dl_hienthi[k+17];   dl_cot_cd[39]=dl_hienthi[k+16];
}

```

```

}
void ky_tu_xanh_so_4(unsigned int8 k)
{
    dl_cot_cd[33]=dl_hienthi[k+19];    dl_cot_cd[35]=dl_hienthi[k+18];
    dl_cot_cd[40]=dl_hienthi[k+20];    dl_cot_cd[42]=dl_hienthi[k+21];
    dl_cot_cd[44]=dl_hienthi[k+22];    dl_cot_cd[46]=dl_hienthi[k+23];
}

//sap xep cac ky tu mau do 1,2,3,4
void ky_tu_do_so_1(unsigned int8 k)
{
    dl_cot_cd[0]=dl_hienthi[k+3];    dl_cot_cd[2]=dl_hienthi[k+2];
    dl_cot_cd[4]=dl_hienthi[k+1];    dl_cot_cd[6]=dl_hienthi[k+0];
    dl_cot_cd[9]=dl_hienthi[k+4];    dl_cot_cd[11]=dl_hienthi[k+5];
}
void ky_tu_do_so_2(unsigned int8 k)
{
    dl_cot_cd[13]=dl_hienthi[k+6];    dl_cot_cd[15]=dl_hienthi[k+7];
    dl_cot_cd[16]=dl_hienthi[k+11];    dl_cot_cd[18]=dl_hienthi[k+10];
    dl_cot_cd[20]=dl_hienthi[k+9];    dl_cot_cd[22]=dl_hienthi[k+8];
}
void ky_tu_do_so_3(unsigned int8 k)
{
    dl_cot_cd[25]=dl_hienthi[k+12];    dl_cot_cd[27]=dl_hienthi[k+13];
    dl_cot_cd[29]=dl_hienthi[k+14];    dl_cot_cd[31]=dl_hienthi[k+15];
    dl_cot_cd[36]=dl_hienthi[k+17];    dl_cot_cd[38]=dl_hienthi[k+16];
}
void ky_tu_do_so_4(unsigned int8 k)
{
    dl_cot_cd[32]=dl_hienthi[k+19];    dl_cot_cd[34]=dl_hienthi[k+18];
    dl_cot_cd[41]=dl_hienthi[k+20];    dl_cot_cd[43]=dl_hienthi[k+21];
    dl_cot_cd[45]=dl_hienthi[k+22];    dl_cot_cd[47]=dl_hienthi[k+23];
}
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
void ky_tu_xanh_hoac_do_so_1(unsigned int8 k)
{
    tinh_vi_tri_ky_tu(k);
    if (dl_hienthi[vtk1]==0)
    {
        dl_cot_cd[1]=dl_hienthi[k+3];    dl_cot_cd[3]=dl_hienthi[k+2];
        dl_cot_cd[5]=dl_hienthi[k+1];    dl_cot_cd[7]=dl_hienthi[k+0];
        dl_cot_cd[8]=dl_hienthi[k+4];    dl_cot_cd[10]=dl_hienthi[k+5];
    }
    else
    {
        dl_cot_cd[0]=dl_hienthi[k+3];    dl_cot_cd[2]=dl_hienthi[k+2];
        dl_cot_cd[4]=dl_hienthi[k+1];    dl_cot_cd[6]=dl_hienthi[k+0];
        dl_cot_cd[9]=dl_hienthi[k+4];    dl_cot_cd[11]=dl_hienthi[k+5];
    }
}
void ky_tu_xanh_hoac_do_so_2(unsigned int8 k)
{
    if (dl_hienthi[vtk1+6]==0)
    {
        dl_cot_cd[12]=dl_hienthi[k+6];    dl_cot_cd[14]=dl_hienthi[k+7];

```

```

        dl_cot_cd[17]=dl_hienthi[k+11];    dl_cot_cd[19]=dl_hienthi[k+10];
        dl_cot_cd[21]=dl_hienthi[k+9];     dl_cot_cd[23]=dl_hienthi[k+8];
    }
    else
    {
        dl_cot_cd[13]=dl_hienthi[k+6];    dl_cot_cd[15]=dl_hienthi[k+7];
        dl_cot_cd[16]=dl_hienthi[k+11];   dl_cot_cd[18]=dl_hienthi[k+10];
        dl_cot_cd[20]=dl_hienthi[k+9];    dl_cot_cd[22]=dl_hienthi[k+8];
    }
}
void ky_tu_xanh_hoac_do_so_3(unsigned int8 k)
{
    if (dl_hienthi[vtk1+12]==0)
    {
        dl_cot_cd[24]=dl_hienthi[k+12];    dl_cot_cd[26]=dl_hienthi[k+13];
        dl_cot_cd[28]=dl_hienthi[k+14];    dl_cot_cd[30]=dl_hienthi[k+15];
        dl_cot_cd[37]=dl_hienthi[k+17];    dl_cot_cd[39]=dl_hienthi[k+16];
    }
    else
    {
        dl_cot_cd[25]=dl_hienthi[k+12];    dl_cot_cd[27]=dl_hienthi[k+13];
        dl_cot_cd[29]=dl_hienthi[k+14];    dl_cot_cd[31]=dl_hienthi[k+15];
        dl_cot_cd[36]=dl_hienthi[k+17];    dl_cot_cd[38]=dl_hienthi[k+16];
    }
}
void ky_tu_xanh_hoac_do_so_4(unsigned int8 k)
{
    if (dl_hienthi[vtk1+18]==0)
    {
        dl_cot_cd[33]=dl_hienthi[k+19];    dl_cot_cd[35]=dl_hienthi[k+18];
        dl_cot_cd[40]=dl_hienthi[k+20];    dl_cot_cd[42]=dl_hienthi[k+21];
        dl_cot_cd[44]=dl_hienthi[k+22];    dl_cot_cd[46]=dl_hienthi[k+23];
    }
    else
    {
        dl_cot_cd[32]=dl_hienthi[k+19];    dl_cot_cd[34]=dl_hienthi[k+18];
        dl_cot_cd[41]=dl_hienthi[k+20];    dl_cot_cd[43]=dl_hienthi[k+21];
        dl_cot_cd[45]=dl_hienthi[k+22];    dl_cot_cd[47]=dl_hienthi[k+23];
    }
}
void sapxep_vitri_48byte_mau_theo_kytu(unsigned int8 k1)
{
    ky_tu_xanh_hoac_do_so_1(k1);
    ky_tu_xanh_hoac_do_so_2(k1);
    ky_tu_xanh_hoac_do_so_3(k1);
    ky_tu_xanh_hoac_do_so_4(k1);
}
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
void sapxep_vitri_48byte_cot_xanh(unsigned int8 k1)
{
    ky_tu_xanh_so_1(k1);    ky_tu_xanh_so_2(k1);
    ky_tu_xanh_so_3(k1);    ky_tu_xanh_so_4(k1);
}
void sapxep_vitri_48byte_cot_do(unsigned int8 k1)

```



```

{
    ky_tu_do_so_1(k1);    ky_tu_do_so_2(k1);
    ky_tu_do_so_3(k1);    ky_tu_do_so_4(k1);
}

void sapxep_vitri_xanh_do_xanh_do(unsigned int8 k1)
{
    ky_tu_xanh_so_1(k1);  ky_tu_do_so_2(k1);
    ky_tu_xanh_so_3(k1);  ky_tu_do_so_4(k1);
}
void sapxep_vitri_do_xanh_do_xanh(unsigned int8 k1)
{
    ky_tu_do_so_1(k1);    ky_tu_xanh_so_2(k1);
    ky_tu_do_so_3(k1);    ky_tu_xanh_so_4(k1);
}

void sapxep_vitri_xanh_xanh_do_do(unsigned int8 k1)
{
    ky_tu_xanh_so_1(k1);  ky_tu_xanh_so_2(k1);
    ky_tu_do_so_3(k1);    ky_tu_do_so_4(k1);
}
void sapxep_vitri_do_do_xanh_xanh(unsigned int8 k1)
{
    ky_tu_do_so_1(k1);    ky_tu_do_so_2(k1);
    ky_tu_xanh_so_3(k1);  ky_tu_xanh_so_4(k1);
}

void xoa_dl_48_cot()
{
    unsigned int8 j1;
    for (j1=0;j1<48;j1++) dl_cot_cd[j1]=0;
}
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
void copy_48_cot_hienthi()
{
    unsigned int8 i;
    for (i=0;i<48;i++)    dl_cot_dich[i]= dl_cot_cd[i];
}

void xuất_lbit_cua_48_cot_hienthi()
{
    unsigned int8 i;
    for (i=0;i<48;i++)    dl_cot_dich[i]= xuất_lbit(dl_cot_dich[i]);
}
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
void copy_24_cot_hienthi_xanh(unsigned int8 k1)
{
    unsigned int8 i;
    for (i=0;i<24;i++)
        dl_cot_cd_xanh[i+k1]= dl_hienthi[i+k1];
}
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
void copy_24_cot_hienthi_do(unsigned int8 k1)
{
    unsigned int8 i;

```

```

    for (i=0;i<24;i++)
        dl_cot_cd_do[i+k1]= dl_hienthi[i+k1];
}
//xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
void copy_24_cot_hienthi_xanh_do_de_dich()
{
    unsigned int8 i;
    for (i=0;i<48;i++)
    {
        dl_cot_dich_d[i]= dl_cot_cd_do[i];
        dl_cot_dich_x[i]= dl_cot_cd_xanh[i];
    }
}
void xoa_dl_24_cot_xanh_do()
{
    unsigned int8 j1;
    for (j1=0;j1<24;j1++)
    {
        dl_cot_cd_do[j1]=0;
        dl_cot_cd_xanh[j1]=0;
    }
}
void xuat_dulieu_hienthi_matran()
{
    mo_ic_74573_b_thong_dl();
    output_high(rck_matranled);
    output_low(rck_matranled);
    mo_led_matran;
    chot_ic_74573_b_goi_du_lieu;
}

void xuat_1bit_cua_48_cot_hienthi_c2()
{
    signed ix;
    for(ix=3;ix>-1;ix--)
    {
        dl_cot_dich_d[ix]=xuat_1bit(dl_cot_dich_d[ix]);
        dl_cot_dich_x[ix]=xuat_1bit(dl_cot_dich_x[ix]);
    }
    for(ix=4;ix<8;ix++)
    {
        dl_cot_dich_d[ix]=xuat_1bit(dl_cot_dich_d[ix]);
        dl_cot_dich_x[ix]=xuat_1bit(dl_cot_dich_x[ix]);
    }
    for(ix=11;ix>7;ix--)
    {
        dl_cot_dich_d[ix]=xuat_1bit(dl_cot_dich_d[ix]);
        dl_cot_dich_x[ix]=xuat_1bit(dl_cot_dich_x[ix]);
    }
    for(ix=12;ix<16;ix++)
    {
        dl_cot_dich_d[ix]=xuat_1bit(dl_cot_dich_d[ix]);
        dl_cot_dich_x[ix]=xuat_1bit(dl_cot_dich_x[ix]);
    }
}

```

```

    for (ix=19; ix>15; ix--)
    {
        dl_cot_dich_d[ix]=xuat_lbit(dl_cot_dich_d[ix]);
        dl_cot_dich_x[ix]=xuat_lbit(dl_cot_dich_x[ix]);
    }
    for (ix=20; ix<24; ix++)
    {
        dl_cot_dich_d[ix]=xuat_lbit(dl_cot_dich_d[ix]);
        dl_cot_dich_x[ix]=xuat_lbit(dl_cot_dich_x[ix]);
    }
}

void hien_thi_ma_tran_quet_hang(unsigned int8 ck)
{
    unsigned int8 j,i;
    for (i=0; i<ck; i++)
    {
        copy_48_cot_hienthi();
        for (j=0; j<8; j++)
        {
            xuat_lbit_cua_48_cot_hienthi();
            xuat_2byte(ma_quet_hang[j]);
            xuat_dulieu_hienthi_matran();
            delay_us(500);
            xuat_matranled(0,0,0,0);
        }
    }
}

```

Với 3 led ma trận 8×8 có 24 cột, mỗi kí tự 6 byte nên có thể hiển thị được 4 kí tự, việc copy dữ liệu sẽ được chia ra cho từng kí tự độc lập để dễ điều khiển.

4 kí tự màu xanh và 4 kí tự màu đỏ, mỗi kí tự là 1 chương trình con màu xanh hoặc màu đỏ, các chương trình con sắp xếp lại cho cả 4 kí có thể thực hiện các màu như sau :

- 4 kí tự màu xanh.
- 4 kí tự màu đỏ.
- 4 kí tự: xanh xanh đỏ đỏ.
- 4 kí tự: đỏ đỏ xanh xanh.
- 4 kí tự xen kẽ: đỏ xanh đỏ xanh.
- 4 kí tự xen kẽ: xanh đỏ xanh đỏ.

Trong các chương trình con copy để sắp xếp lại đúng thứ tự cột thì có thêm tham số K với mục đích để điều khiển dịch chuỗi trên led ma trận:

Khi K = 0 thì sẽ copy dữ liệu từ byte thứ 0 đến byte thứ 47 đem gởi ra led hiển thị.

Khi K = 1 thì sẽ copy dữ liệu từ byte thứ 1 đến byte thứ 48 đem gởi ra led hiển thị - so với K = 0 thì xem như chuỗi hiển thị dịch đi 1 cột.

Khi K = 2 thì sẽ copy dữ liệu từ byte thứ 2 đến byte thứ 49 đem gởi ra led hiển thị - so với K = 1 thì xem như chuỗi hiển thị dịch đi thêm 1 cột nữa.

Tương tự cho đến khi hết chuỗi cần hiển thị.

Các bài hiển thị chuỗi đứng yên thì cho tham số K = 0.

10.5 CÁC CHƯƠNG TRÌNH ĐIỀU KHIỂN LED MA TRẬN QUÉT HÀNG

Bài mẫu 1011. Chương trình điều khiển LED ma trận hiển thị chữ ABCD màu xanh theo phương pháp quét hàng.

Lưu tên file là “BAI_1011_MTQH_ABCD_XANH”

- Mục đích: biết cách điều khiển led quét hàng.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include <tv_pickit2_shift_1.c>
const unsigned char chuoi_hienthi[]={"abcd"};
#include <tv_pickit2_shift_matran_quet_hang.c>

void main()
{
    set_up_port_ic_chot();
    lay_so_luong_ky_tu_chuoi();
    lay_ma_ky_tu(slkt);
    xoa_dl_48_cot();
    sapxep_vitri_48byte_cot_xanh(0);
    while(true)
    {
        hien_thi_ma_tran_quet_hang(50);
    }
}
```

- Tiến hành biên dịch và nạp.
- Quan sát kết quả: sau khi nạp xong thì LED sáng chữ ABCD rõ đẹp hơn so với quét cột.
- Giải thích chương trình:

Bài tập 1012. Hãy hiệu chỉnh chương trình bài 1011 để 4 kí tự sáng màu đỏ.

Lưu tên file là “BAI_1012_MTQH_ABCD_DO”

GỢI Ý :

Thay sapxep_vitri_48byte_cot_xanh(0); bằng ham sapxep_vitri_48byte_cot_do(0);

Bài mẫu 1013. Chương trình điều khiển LED ma trận hiển thị chữ ABCD màu xanh rồi màu đỏ.

Lưu tên file là “BAI_1013_MTQH_ABCD_XANH_DO”

- Mục đích: biết cách điều khiển led quét hàng hiển thị 2 màu.
- Lưu đồ: sinh viên hãy tự viết.
- Chương trình:

```
#include <tv_pickit2_shift_1.c>
const unsigned char chuoi_hienthi[]={"abcd"};
#include <tv_pickit2_shift_matran_quet_hang.c>
void main()
```

```

{
    set_up_port_ic_chot();
    lay_so_luong_ky_tu_chuoi();
    lay_ma_ky_tu(slkt);

    while(true)
    {
        xoa_dl_48_cot();
        sapxep_vitri_48byte_cot_xanh(0);
        hien_thi_ma_tran_quet_hang(50);

        xoa_dl_48_cot();
        sapxep_vitri_48byte_cot_do(0);
        hien_thi_ma_tran_quet_hang(50);
    }
}

```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: sau khi nạp xong thì LED sáng chữ ABCD màu xanh rồi đến màu đỏ.
- f. Giải thích chương trình:

Bài mẫu 1014. Chương trình điều khiển LED ma trận hiển thị chuỗi màu xanh dịch từ phải sang trái.

Lưu tên file là “BAI_1014_MTQH_CHUOI_XANH_DICH”

- a. Mục đích: biết cách điều khiển led quét hàng hiển thị chuỗi dịch trái.
- b. Lưu đồ: sinh viên hãy tự viết.
- c. Chương trình:

```

#include <tv_pickit2_shift_1.c>
const unsigned char chuoi_hienthi[]={"abcdefghijklm"};
#include <tv_pickit2_shift_matran_quet_hang.c>
unsigned int8 vitri;
void matran_qh_dich_mau_xanh(unsigned int8 i)
{
    xoa_dl_48_cot();
    sapxep_vitri_48byte_cot_xanh(i);
    hien_thi_ma_tran_quet_hang(50);
}
void main()
{
    set_up_port_ic_chot();
    lay_so_luong_ky_tu_chuoi();
    lay_ma_ky_tu(slkt);

    while(true)
    {
        vitri=0;
        for(vitri=0; vitri<(slkt-4)*6; vitri++)
            matran_qh_dich_mau_xanh(vitri);
    }
}

```

}

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: sau khi nạp xong thì LED sáng chuỗi sẽ dịch chuyển.
- f. Giải thích chương trình:

Bài tập 1015. Chương trình điều khiển LED ma trận hiển thị chữ ABCD dịch từ phải sang trái màu xanh rồi đến màu đỏ.

Lưu tên file là “BAI_1015_MTQH_ABCD_XANH_DO_DICH”

Bài tập 1016. Chương trình điều khiển LED ma trận hiển thị chữ ABCD dịch từ phải sang trái màu xanh rồi đến màu đỏ và màu cam.

Lưu tên file là “BAI_1016_MTQH_ABCD_XANH_DO_CAM_DICH”

Bài mẫu 1017. Chương trình điều khiển LED ma trận đếm 1 số từ 0 đến 9.

Lưu tên file là “BAI_1017_MTQH_DEM_1SO_XANH”

- a. Mục đích: biết cách điều khiển led quét hàng hiển thị giá trị đếm 1 số.
- b. Lưu đồ: sinh viên hãy tự viết.
- c. Chương trình:

```
#include <tv_pickit2_shift_1.c>
const unsigned char chuoi_hienthi[]={"1234"};
#include <tv_pickit2_shift_matran_quet_hang.c>
unsigned int8 i;
void main()
{
    set_up_port_ic_chot();
    //lay_so_luong_ky_tu_chuoi();
    xoa_dl_hienthi();
    slkt=1;
    while(true)
    {
        for(i=0; i<10; i++)
        {
            lay_ma_1_so_matran(i,3);
            xoa_dl_48_cot();
            sapxep_vitri_48byte_cot_xanh(0);
            hien_thi_ma_tran_quet_hang(100);
        }
    }
}
```

- d. Tiến hành biên dịch và nạp.
- e. Quan sát kết quả: sau khi nạp xong thì LED hiển thị giá trị đếm từ 0 đến 9.
- f. Giải thích chương trình:

Bài tập 1018. Hãy viết chương trình đếm 2 số từ 00 đến 99.

Lưu tên file là “BAI_1018_MTQH_DEM_2SO_XANH”

Bài tập 1019. Hãy viết chương trình đếm 3 số từ 000 đến 999.

Lưu tên file là “BAI_1019_MTQH_DEM_3SO_XANH”

Bài tập 1020. Hãy viết chương trình đếm sản phẩm dùng timer T0 hiển thị giá trị đếm trên led ma trận, giới hạn đếm từ 00 đến 99.

Lưu tên file là “BAI_1020_MTQH_DEM_SP_T0”

Bài tập 1021. Hãy viết chương trình đo nhiệt độ hiển thị trên 2 led ma trận.

Lưu tên file là “BAI_1021_MTQH_ADC_LM35”

TÀI LIỆU THAM KHẢO

- [1]. MICROCHIP, PIC 16F87X DATASHEET, 1997.
- [2]. MICROCHIP, PIC 18F4550 DATASHEET, 2009.
- [3]. Nebojsa Matic, PIC microcontrollers for beginner, too.

- [4]. ITead – Studio Ultrasonic ranging module: HC-SR04, 2010.
- [5]. ATMEL – AT24C512 DATASHEET, 2007.
- [6]. SHARP – GP2D12 optoelectronic Device DATASHEET, 2007.
- [7]. MICROCHIP – PICmicro Mid- Range MCU family reference Manual, 1997
- [8]. Nguyễn Đình Phú, Vi xử lý - PIC, Đại Học Sư Phạm Kỹ Thuật TP HCM, 2017.